



软件源代码缺陷检测 TDengine 检测报告

苏州棱镜七彩信息科技有限公司

2026-04-17 21:26:36



2004

2010

2012

2006

CHART SEGMENT TITLE

CHART SEGMENT TITLE

报告声明

棱镜七彩是苏州棱镜七彩信息科技有限公司的简称，PrismSAST 是棱镜七彩具有自主知识产权的软件源代码安全缺陷检测平台的简称。

本检测报告由棱镜七彩软件源代码安全缺陷检测平台（PrismSAST）采用智能检测引擎进行自动化检测，并对检测结果经过人工复核后产生的检测报告。棱镜七彩保留对检测报告中各款项的解读权利，但并不意味着对检测结果中的缺陷或安全漏洞可能引起的任何直接经济损失或任何间接损失承担责任。

本检测报告的最终解释权归属棱镜七彩。

16 C/C++缺陷类型-返回值问题

16.1 分项汇总表

序号	C/C++缺陷类型-返回值问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4729	有返回值的函数必须通过返回语句返回	21	严重
2	WK4732	函数返回值的类型必须与定义一致	54	严重
3	WK4733	具有返回值的函数,其返回值如果不被使用,调用是应有void说明	401	严重

16.2 缺陷/安全漏洞详解

16.2.1 严重类

16.2.1.1 WK4729 有返回值的函数必须通过返回语句返回

缺陷数量： 21 个		
缺陷描述： 有返回值的函数必须通过返回语句返回。		
修复意见： 无		
1	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbUtil.c	行号： 571
步骤描述：		

	有返回值的函数必须通过返回语句返回	
	代码片段: 569: } 570: 571: int32_t tsdbFidLevel(int32_t fid, STsdbKeepCfg *pKeepCfg, int64_t nowSec) { 572: int32_t aFid[3]; 573: TSKEY key;	
	污染路径: 无	
2	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchUtil.c	719
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 717: } 718: 719: int convertStringToDatatype(char *type, int length, void* ctx) { 720: // compare with length 721: if (0 == strcmp(type, "binary", length)) {	
3	污染路径: 无	
	缺陷发生位置:	行号:
	/source/client/src/clientMain.c	1020
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1018: } 1019: 1020: const char *taos_errstr(TAOS_RES *res) { 1021: if (res == NULL TD_RES_TMQ_RAW(res) TD_RES_TMQ_META(res) TD_RES_TMQ_BATCH_META(res)) { 1022: if (*(taosGetErrMsg()) == 0) {	
4	污染路径: 无	
	缺陷发生位置:	行号:
	/source/common/src/tdatablock.c	25
	步骤描述: 有返回值的函数必须通过返回语句返回	

	代码片段: 23: #define MALLOC_ALIGN_BYTES 32 24: 25: int32_t colDataGetLength(const SColumnInfoData* pColumnInfoData, int32_t numRows) { 26: if (IS_VAR_DATA_TYPE(pColumnInfoData->info.type)) { 27: if (pColumnInfoData->reassigned) {	
	污染路径: 无	
5	缺陷发生位置: /source/util/src/tcompare.c	行号: 283
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 281: } 282: } 283: int32_t compareBlobVal(const void *pLeft, const void *pRight) { 284: int32_t len1 = blobDataLen(pLeft); 285: int32_t len2 = blobDataLen(pRight);	
	污染路径: 无	
6	缺陷发生位置: /source/os/src/osThread.c	行号: 261
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 259: } 260: 261: int32_t taosThreadCondTimedWait(TdThreadCond *cond, TdThreadMutex *mutex, const struct timespec *abstime) { 262: if (!abstime) return 0; 263: OS_PARAM_CHECK(cond);	
	污染路径: 无	
7	缺陷发生位置: /source/dnode/mnode/impl/src/mndSync.c	行号: 455
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	453: } 454: 455: static int32_t mndApplyQueueItems(const SSyncFSM *pFsm) { 456: SMnode *pMnode = pFsm->data; 457:	
	污染路径: 无	
8	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 106
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 104: * Dictionary training functions 105: *****/ 106: static unsigned ZDICT_NbCommonBytes (size_t val) 107: { 108: if (MEM_isLittleEndian()) {	
	污染路径: 无	
9	缺陷发生位置: /source/os/src/osString.c	行号: 219
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 217: } 218: 219: int32_t taosStr2int8(const char *str, int8_t *val) { 220: OS_PARAM_CHECK(str); 221: OS_PARAM_CHECK(val);	
	污染路径: 无	
10	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 3201
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 3199: } 3200:	

	3201: int32_t mndBuildAlterVgroupAction(SMnode *pMnode, STrans *pTrans, SDbObj *pOldDb, SDbObj *pNewDb, SVgObj *pVgroup, 3202: SArray *pArray, SVgObj *pNewVgroup) { 3203: int32_t code = 0;	
	污染路径: 无	
11	缺陷发生位置: /source/client/src/clientMain.c	行号: 1494
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1492: } 1493: 1494: int taos_fetch_block_s(TAOS_RES *res, int *numOfRows, TAOS_ROW *rows) { 1495: if (res == NULL TD_RES_TMQ_RAW(res) TD_RES_TMQ_META(res) TD_RES_TMQ_BATCH_META(res)) { 1496: return 0;	
	污染路径: 无	
12	缺陷发生位置: /source/util/src/thashutil.c	行号: 190
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 188: } 189: } 190: uint32_t taosDoubleHash(const char *key, uint32_t UNUSED_PARAM(len)) { 191: double f = GET_DOUBLE_VAL(key); 192: if (isnan(f)) {	
	污染路径: 无	
13	缺陷发生位置: /source/libs/transport/src/transComm.c	行号: 677
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段:	

	675: return 0; 676: } 677: int32_t transDoCrcCheck(char* buf, int32_t len) { 678: STransMsgHead* pHead = (STransMsgHead*)buf; 679: uint32_t checkSum = ntohl(pHead->magicNum);	
	污染路径: 无	
14	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 30
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 28: static double tlog(double v) { return log(v); } 29: 30: static double tlog2(double v, double base) { 31: double a = log(v); 32: double b = log(base);	
	污染路径: 无	
15	缺陷发生位置: /source/client/src/clientTmq.c	行号: 2873
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 2871: } 2872: 2873: const char* tmq_get_topic_name(TAOS_RES* res) { 2874: if (res == NULL) { 2875: return NULL;	
	污染路径: 无	
16	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtil.c	行号: 34
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 32: #endif 33: 34: int mqttt__check_heartbeat(struct mqttt *ttq) { 35: time_t next_msg_out;	

	36: time_t last_msg_in;	
	污染路径: 无	
17	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleRead.c	行号: 21
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 19: } 20: 21: static int ttq_handle_pubback(struct tmqtt *ttq) { 22: uint8_t reason_code = 0; 23: uint16_t mid;	
	污染路径: 无	
18	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 1454
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 1452: } 1453: 1454: static int32_t nameComparFn(const void* p1, const void* p2) { 1455: const char* pName1 = *(const char**)p1; 1456: const char* pName2 = *(const char**)p2;	
	污染路径: 无	
19	缺陷发生位置: /source/client/inc/clientSml.h	行号: 281
	步骤描述: 有返回值的函数必须通过返回语句返回	
	代码片段: 279: } 280: 281: static inline int8_t smlGetTsTypeByLen(int32_t len) { 282: if (len == TSDB_TIME_PRECISION_SEC_DIGITS) { 283: return TSDB_TIME_PRECISION_SECONDS;	
	污染路径:	

	无	
20	缺陷发生位置： /source/common/src/ttime.c	行号： 1890
	步骤描述： 有返回值的函数必须通过返回语句返回	
	代码片段： 1888: } 1889: 1890: static bool needMoreDigits(SArray* formats, int32_t curIdx) { 1891: if (curIdx == taosArrayGetSize(formats) - 1) return false; 1892: TSFormatNode* pNextNode = taosArrayGet(formats, curIdx + 1);	
	污染路径： 无	
21	缺陷发生位置： /source/libs/function/src/thistogram.c	行号： 307
	步骤描述： 有返回值的函数必须通过返回语句返回	
	代码片段： 305: } 306: 307: int32_t histoBinarySearch(SHistBin* pEntry, int32_t len, double val) { 308: int32_t end = len - 1; 309: int32_t start = 0;	
	污染路径： 无	

16.2.1.2 WK4732 函数返回值的类型必须与定义一致

缺陷数量： 54 个		
缺陷描述： 函数返回值的类型必须与定义一致。		
修复意见： 无		
1	缺陷发生位置：	行号：

	/source/os/src/osSocket.c		208
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int8_t'.		
	代码片段: 206: if (pSocket == NULL) { 207: TAOS_SKIP_ERROR(taosCloseSocketNoCheck1(fd)); 208: return 0; 209: } 210: pSocket->refId = 0;		
	污染路径: 无		
2	缺陷发生位置:		行号:
	/tools/rocks-reader/rreader.cpp		109
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.		
	代码片段: 107: pLastCol->colVal.value.pData = (uint8_t *) (&pLastColV0[1]); 108: } 109: return sizeof(SLastColV0) + pLastColV0->colVal.value.nData; 110: } else if (pLastCol->colVal.value.type == TSDB_DATA_TYPE_DECIMAL) { 111: pLastCol->colVal.value.nData = pLastColV0->colVal.value.nData;		
3	污染路径: 无		
	缺陷发生位置:		行号:
	/tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c		675
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'char'.		
	代码片段: 673: hol_entry_first_short (const struct hol_entry *entry) 674: { 675: return hol_entry_short_iterate (entry, until_short, 676: entry->argp->argp_domain, 0); 677: }		
	污染路径: 无		

4	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 432
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint32_t'.	
	代码片段: 430: uint32_t bkdrHash(const char *str) { 431: if (str == NULL) { 432: return 0; 433: } 434: uint32_t seed = 131;	
	污染路径: 无	
5	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 228
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 226: // GCC8: generates a bit worse cmp based code 227: // although it generates the xor based one for 228: // _int128. 228: return (x.lo == y.lo) & (x.hi == y.hi); 229: } 230:	
	污染路径: 无	
6	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 440
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'unsigned long'，定义类型为'int'.	
	代码片段: 438: } else { 439: pLastColV0->colVal.value.val = pLastCol->colVal.value.val; 440: return sizeof(SLastColV0); 441: } 442:	
	污染路径: 无	
7	缺陷发生位置: /source/util/src/tutil.c	行号: 98

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 96: if (stop > 0) { 97: z[stop - delta] = 0; 98: return (stop - delta); 99: } else if (j != i) { 100: z[i] = 0;	
	污染路径: 无	
8	缺陷发生位置:	行号:
	/source/libs/decimal/src/detail/intx/int128.hpp	234
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 232: { 233: // Analogous to ==, but == not used directly, because that confuses GCC 8-9. 234: return (x.lo != y.lo) (x.hi != y.hi); 235: } 236:	
9	污染路径: 无	
	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/batchprepare.c	392
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
10	代码片段: 390: 391: if (pBind[i].buffer_type == dataType) { 392: return true; 393: } 394:	
	污染路径: expanded from macro 'true'	
	缺陷发生位置:	行号:
10	/source/libs/decimal/test/decimalTest.cpp	274
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.	
	代码片段:	

	272: DEFINE_REAL_OP(-); 273: DEFINE_REAL_OP(*); 274: DEFINE_REAL_OP(/); 275: 276: template <int BitNum2>	
	污染路径: expanded from macro 'DEFINE_REAL_OP'	
11	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 2351
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint32_t'.	
	代码片段: 2349: errorPrint("taos_stmt_bind_param_batch() failed! reason: %s\n", 2350: taos_stmt_errstr(stmt)); 2351: return 0; 2352: } 2353: *delay2 += toolsGetTimestampUs() - start;	
	污染路径: 无	
12	缺陷发生位置: /source/os/src/osMemory.c	行号: 442
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'size_t'，定义类型为'int'.	
	代码片段: 440: return 0; // TD_ASTRATODO N/A 441: #else 442: return malloc_usable_size(ptr); 443: #endif 444: #endif	
	污染路径: 无	
13	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 962
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.	
	代码片段: 960: size_t ZSTDMT_sizeof_CCtx(ZSTDMT_CCtx* mtctx) 961: { 962: if (mtctx == NULL) return 0; /* supports sizeof NULL	

	*/ 963: return sizeof(*mtctx) 964: + POOL_sizeof(mtctx->factory)	
	污染路径: 无	
14	缺陷发生位置: /source/util/src/tjson.c	行号: 402
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'int *'.	
	代码片段: 400: } 401: 402: Sjson* tjsonParse(const char* pStr) { return cJSON_Parse(pStr); } 403: 404: bool tjsonValidateJson(const char* jIn) {	
	污染路径: 无	
15	缺陷发生位置: /source/common/src/dmRepair.c	行号: 12
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'_Bool'.	
	代码片段: 10: 11: TD_WEAK bool dmRepairFlowEnabled() { return false; } 12: TD_WEAK bool dmRepairNodeTypelsVnode() { return false; } 13: TD_WEAK bool dmRepairModelsForce() { return false; } 14: TD_WEAK bool dmRepairHasBackupPath() { return false; }	
	污染路径: expanded from macro 'false'	
16	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 6068
	步骤描述: 函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'uint8_t'.	
	代码片段: 6066: return ((SSetOperator*)pStmt)->precision; 6067: } 6068: return 0; 6069: }	

	6070:	
	污染路径: 无	
17	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 261
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.	
	代码片段: 259: while (l->name != NULL) 260: if (name != NULL && strcmp (l->name, name) == 0) 261: return l - long_options; 262: else 263: l++;	
	污染路径: 无	
18	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 119
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 117: size_t ZSTD_freeCctx(ZSTD_Cctx* cctx) 118: { 119: if (cctx==NULL) return 0; /* support free on NULL */ 120: if (cctx->staticSize) return ERROR(memory_allocation); /* not compatible with static Cctx */ 121: ZSTD_freeCctxContent(cctx);	
	污染路径: 无	
19	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 108
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'long'.	
	代码片段: 106: case 'h': return num * 60 * 60; 107: case 'd': return num * 60 * 60 * 24; 108: default : return -1; 109: } 110: }	

	污染路径: 无	
20	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 108
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'unsigned int'.	
	代码片段: 106: constexpr unsigned num_bits(const T&) noexcept 107: { 108: return sizeof(T) * 8; 109: } 110:	
	污染路径: 无	
21	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 147
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 145: } } 146: 147: return 0; 148: } 149:	
	污染路径: 无	
22	缺陷发生位置: /tools/src/pub.c	行号: 84
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int8_t'.	
	代码片段: 82: // compare 83: if (strcasecmp(arg, STR_NATIVE) == 0 strcasecmp(arg, "0") == 0) { 84: return CONN_MODE_NATIVE; 85: } else if (strcasecmp(arg, STR_WEBSOCKET) == 0 strcasecmp(arg, "1") == 0) { 86: return CONN_MODE_WEBSOCKET;	

	污染路径: expanded from macro 'CONN_MODE_NATIVE'	
23	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 281
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 279: unsigned ZSTD_isFrame(const void* buffer, size_t size) 280: { 281: if (size < ZSTD_frameIdSize) return 0; 282: { U32 const magic = MEM_readLE32(buffer); 283: if (magic == ZSTD_MAGICNUMBER) return 1;	
	污染路径: 无	
24	缺陷发生位置: /source/libs/index/src/index.c	行号: 865
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.	
	代码片段: 863: SERIALIZE_STR_MEM_TO_BUF(buf, key, colName, key->nColName); 864: } 865: return buf - p; 866: }	
	污染路径: 无	
25	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 519
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: 517: if (pFile == NULL) { 518: (void)fprintf(stderr, "failed to open file: %s\r\n", fullname); 519: return -1; 520: } 521:	
	污染路径: 无	
26	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchUtil.c		781
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.		
	代码片段: 779: 780: int compare(const void *a, const void *b) { 781: return *(int64_t *)a - *(int64_t *)b; 782: } 783:		
	污染路径: 无		
27	缺陷发生位置:		行号:
	/source/libs/decimal/test/decimalTest.cpp		271
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'double'.		
	代码片段: 269: return 0; 270: } 271: DEFINE_REAL_OP(+); 272: DEFINE_REAL_OP(-); 273: DEFINE_REAL_OP(*);		
28	缺陷发生位置:		行号:
	/tools/taos-tools/src/benchInsertMix.c		374
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段: 372: int64_t* buf = mix->buf[type]; 373: if(buf == NULL){ 374: return false; 375: } 376:		
29	缺陷发生位置:		行号:
	/source/libs/executor/test/sortTests.cpp		51
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'double', 定义类型为'float'.		
	代码片段:		

	49: r &= 0x007fffff; 50: r = 0x40800000; 51: return *(float*)&r - 6.0; 52: } 53:	
	污染路径: 无	
30	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 169
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 167: uint64_t gid = qStreamGetGroupIDFromOrigin(sStreamReaderInfo, uid); 168: taosRUnLockLatch(&sStreamReaderInfo->lock); 169: if (gid == (uint64_t)-1) return true; 170: } 171: return false; </pre>	
	污染路径: expanded from macro 'true'	
31	缺陷发生位置: /tools/taos-tools/src/toolsString.c	行号: 70
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 68: int len = strlen(input); 69: if (0 == len) { 70: return false; 71: } 72: </pre>	
	污染路径: expanded from macro 'false'	
32	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 469
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: <pre> 467: 468: for (s=0; s<=maxSymbolValue; s++) { 469: if (count[s] == total) return 0; /* rle special case </pre>	

	<pre> */ 470: if (count[s] == 0) { normalizedCounter[s]=0; continue; } 471: if (count[s] <= lowThreshold) { </pre>	
	污染路径: 无	
33	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 76
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 74: // websocket error 75: if (code >= TSDB_CODE_WBS_NETWORK_BEGIN && code <= TSDB_CODE_WBS_NETWORK_END) { 76: return true; 77: } 78: </pre>	
	污染路径: expanded from macro 'true'	
34	缺陷发生位置: /source/common/src/dmRepair.c	行号: 13
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 11: TD_WEAK bool dmRepairFlowEnabled() { return false; } 12: TD_WEAK bool dmRepairNodeTypelsVnode() { return false; } } 13: TD_WEAK bool dmRepairModelsForce() { return false; } 14: TD_WEAK bool dmRepairHasBackupPath() { return false; } 15: TD_WEAK const char *dmRepairBackupPath() { return ""; } </pre>	
	污染路径: expanded from macro 'false'	
35	缺陷发生位置: /source/util/src/tcompression.c	行号: 244
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int32_t'.	
	代码片段: <pre> 242: output[0] = 1; 243: 244: return len + 1; </pre>	

	245: } 246: int32_t l2DecompressImpl_zstd(const char *const input, const int32_t compressedSize, char *const output,	
	污染路径: 无	
36	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 799
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'int'.	
	代码片段: 797: while (*p != 0 && *p++ != c) { 798: } 799: return p - a; 800: } 801: static int idxCacheJsonTermCompareImpl(char* a, char* b) {	
	污染路径: 无	
37	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 402
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long long', 定义类型为'int64_t'.	
	代码片段: 400: struct timeval systemTime; 401: taosGetTimeOfDay(&systemTime); 402: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 403: } 404:	
	污染路径: 无	
38	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 99
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 97: if (dictSize < 8) return 0; 98: if (MEM_readLE32(dictBuffer) != ZSTD_MAGIC_DICTIONARY) return 0; 99: return MEM_readLE32((const char*)dictBuffer + 4);	

	100: } 101:	
	污染路径: 无	
39	缺陷发生位置: /source/os/src/osTime.c	行号: 483
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'struct tm *'.	
	代码片段: 481: return (code == 0) ? result : NULL; 482: #else 483: return gmtime_r(timep, result); 484: #endif 485: }	
	污染路径: 无	
40	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqProperty.c	行号: 297
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	
	代码片段: 295: return 0; 296: } 297: return 0; 298: } 299:	
	污染路径: 无	
41	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 124
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long', 定义类型为'size_t'.	
	代码片段: 122: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) 123: _argp_fmtstream_update (__fs); 124: return __fs->point_col >= 0 ? __fs->point_col : 0; 125: } 126:	

	污染路径: 无	
42	缺陷发生位置: /contrib/TSZ/zstd/common/pool.c	行号: 335
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: 333: 334: size_t POOL_sizeof(POOL_ctx* ctx) { 335: if (ctx==NULL) return 0; /* supports sizeof NULL */ 336: assert(ctx == &g_ctx); 337: return sizeof(*ctx);	
	污染路径: 无	
43	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 311
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: 309: return 0; 310: } 311: DEFINE_OPERATOR_T(bool); 312: DEFINE_OPERATOR_T(int8_t); 313: DEFINE_OPERATOR_T(uint8_t);	
	污染路径: expanded from macro 'DEFINE_OPERATOR_T'	
44	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 119
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'long long', 定义类型为'int64_t'.	
	代码片段: 117: struct timeval systemTime; 118: rtGetTimeOfDay(&systemTime); 119: return (int64_t)systemTime.tv_sec * 1000000LL + (int64_t)systemTime.tv_usec; 120: } 121:	
	污染路径: 无	
45	缺陷发生位置:	行号:

	/tools/taos-tools/src/benchData.c		2689
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.		
	代码片段: 2687: stbInfo->lenOfTags, 2688: cnt)) { 2689: return false; 2690: } 2691: } else {		
	污染路径: expanded from macro 'false'		
46	缺陷发生位置:		行号:
	/contrib/TSZ/zstd/legacy/zstd_v05.c		889
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.		
	代码片段: 887: MEM_STATIC unsigned BITv05_endOfDStream(const BITv05_DStream_t* DStream) 888: { 889: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 890: } 891:		
47	缺陷发生位置:		行号:
	/contrib/TSZ/zstd/decompress/zstd_decompress.c		289
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.		
	代码片段: 287: if (ZSTD_isLegacy(buffer, size)) return 1; 288: #endif 289: return 0; 290: } 291:		
48	缺陷发生位置:		行号:
	/tools/taos-tools/src/toolstime.c		651
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.		
	代码片段: 287: if (ZSTD_isLegacy(buffer, size)) return 1; 288: #endif 289: return 0; 290: } 291:		
49	缺陷发生位置:		行号:
	/contrib/TSZ/zstd/decompress/zstd_decompress.c		289
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.		
	代码片段: 287: if (ZSTD_isLegacy(buffer, size)) return 1; 288: #endif 289: return 0; 290: } 291:		

	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 649: c--; 650: } 651: return false; 652: } 653: </pre>	
	污染路径: expanded from macro 'false'	
49	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 623
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'int64_t'.	
	代码片段: <pre> 621: int64_t mtime = 0; 622: if (path == NULL path[0] == 0) return -1; 623: if (taosStatFile(path, &fsize, &mtime, NULL) < 0) return -1; 624: return mtime; 625: } </pre>	
	污染路径: 无	
50	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 156
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'unsigned long', 定义类型为'int'.	
	代码片段: <pre> 154: INLINE int getLeftMovingSteps(size_t k, unsigned char resiBitLength) 155: { 156: return 8 - k%8 - resiBitLength; 157: } 158: </pre>	
	污染路径: 无	
51	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 490
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'unsigned int'.	

	代码片段: <pre> 488: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 489: { 490: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream- >bitContainer)*8)); 491: } 492: </pre>	
	污染路径: 无	
52	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 258
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'size_t'.	
	代码片段: <pre> 256: #endif 257: ZSTD_free(dctx, cMem); 258: return 0; 259: } 260: } </pre>	
	污染路径: 无	
53	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 144
	步骤描述: 函数返回值类型必须与定义一致, 实际返回为'int', 定义类型为'_Bool'.	
	代码片段: <pre> 142: int64_t size = taosHashGetSize(g_slidigGrpMemList.pSlidingGrpList); 143: if (size == 0) { 144: return true; 145: } 146: if (g_slidigGrpMemList.waitMoveMemSize > DS_MEM_SIZE_RESERVED) { </pre>	
	污染路径: expanded from macro 'true'	
54	缺陷发生位置: /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号: 246
	步骤描述:	

	函数返回值类型必须与定义一致，实际返回为'int'，定义类型为'size_t'.
	代码片段: 244: size_t ZSTD_freeDCTX(ZSTD_DCTX* dctx) 245: { 246: if (dctx==NULL) return 0; /* support free on NULL */ 247: if (dctx->staticSize) return ERROR(memory_allocation); /* not compatible with static DCTX */ 248: { ZSTD_customMem const cMem = dctx-> customMem;
	污染路径: 无

16.2.1.3 WK4733 具有返回值的函数,其返回值如果不被使用,

调用是应有 void 说明

缺陷数量： 401 个		
缺陷描述： 具有返回值的函数,其返回值如果不被使用,调用时应有 void 说明。		
修复意见： 无		
1	缺陷发生位置：	行号：
	/source/dnode/mnode/impl/src/mndBnode.c	181
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 179: mError("bnode:%d, failed to serialize create drop bnode request since %s", createReq.dnodeId, terrstr()); 180: 181: taosMemoryFree(pReq); 182: TAOS_RETURN(code); 183: }	
2	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置：	行号：
	/source/os/src/osSysinfo.c	518

	步骤描述: 不使用返回值的函数'TAOS_SKIP_ERROR'应使用'void'说明.	
	代码片段: 516: TAOS_SKIP_ERROR(taosGetCpuCores(&tsNumOfCores, false)); 517: TAOS_SKIP_ERROR(taosGetTotalMemory(&tsTotalMemoryKB)); 518: TAOS_SKIP_ERROR(taosGetCpuUsage(NULL, NULL)); 519: TAOS_SKIP_ERROR(taosGetCpuInstructions(&tsSSE42Supported, &tsAVXSupported, &tsAVX2Supported, &tsFMASupported, &tsAVX512Supported)); 520: #endif	
	污染路径: 函数'TAOS_SKIP_ERROR'的定义点.	
3	缺陷发生位置: /source/util/src/tref.c	行号: 104
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 102: } else { 103: rsetId = TSDB_CODE_REF_FULL; 104: taosMemoryFree(nodeList); 105: taosMemoryFree(lockedBy); 106: uTrace("run out of Ref ID, maximum:%d refSetNum:%d", TSDB_REF_OBJECTS, tsRefSetNum);	
	污染路径: 函数'taosMemoryFree'的定义点.	
4	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 57
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 55: taos_free_result(result); 56: 57: usleep(100000); 58: taos_select_db(taos, "test"); 59:	
	污染路径: 无	

5	缺陷发生位置: /source/libs/scalar/src/filter.c	行号: 5203
	步骤描述: 不使用返回值的函数'FLT_RET'应使用'void'说明.	
	代码片段: 5201: FLT_ERR_RET(terrno); 5202: } 5203: FLT_RET(0); 5204: } 5205:	
	污染路径: 函数'FLT_RET'的定义点.	
6	缺陷发生位置: /source/common/src/msg/xnode.c	行号: 218
	步骤描述: 不使用返回值的函数'FREESQL'应使用'void'说明.	
	代码片段: 216: taosMemoryFreeClear(pReq->pass); 217: xFreeCowStr(&pReq->token); 218: FREESQL(); 219: } 220: void xFreeTaskSource(xTaskSource *source) {	
	污染路径: 函数'FREESQL'的定义点.	
7	缺陷发生位置: /utils/test/c/tmq_td33798.c	行号: 55
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 53: } 54: 55: tmq_free_json_meta(result); 56: } 57:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
8	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1066
	步骤描述: 不使用返回值的函数'prepareColData'应使用'void'说明.	

	代码片段: 1064: for (int b = 0; b < bindNum; b++) { 1065: for (int c = 0; c < gCurCase->bindColNum; ++c) { 1066: prepareColData(true, BP_BIND_COL, data, b*gCurCase->bindColNum+c, b*gCurCase->bindRowNum, c); 1067: } 1068: }	
	污染路径: 函数'prepareColData'的定义点.	
9	缺陷发生位置: /utils/test/c/varbinary_test.c	行号: 234
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 232: pRes = taos_query(taos, "select * from stb where c2 > '\x7F8290"); 233: GET_ROW_NUM 234: ASSERT(numRows == 1); 235: taos_free_result(pRes); 236:	
10	污染路径: 函数'ASSERT'的定义点.	
	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	行号: 240
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
11	代码片段: 238: printf("syncPingPrint2 len:%d %s %s \n", (int32_t)strlen(serialized), s, serialized); 239: fflush(NULL); 240: taosMemoryFree(serialized); 241: } 242:	
	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 617
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	615: shellPrintString(hexBuf, width > 0 ? width : 0); 616: 617: taosMemoryFree(hexBuf); 618: } 619:	
	污染路径: 函数'taosMemoryFree'的定义点.	
12	缺陷发生位置: /source/libs/new-stream/src/streamCheckpoint.c	行号: 192
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 190: if (count <= 0) { 191: stError("failed to decrypt checkpoint for streamId:%" PRIx64 ", dataLen:%" PRIu64, streamId, originalLen); 192: taosMemoryFree(plainContent); 193: code = terrno; 194: goto end;	
	污染路径: 函数'taosMemoryFree'的定义点.	
13	缺陷发生位置: /source/libs/parser/src/parInsertUtil.c	行号: 1226
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 1224: snprintf(buf, sizeof(buf), "column type not equal, name:%s, schema type:%s, data type:%s", pColSchema->name, 1225: tDataTypes[pColSchema->type].name, tDataTypes[*fields].name); 1226: uError("checkSchema %s", buf); 1227: } 1228: return TSDB_CODE_INVALID_PARA;	
	污染路径: 函数'uError'的定义点.	
14	缺陷发生位置: /source/dnode/mnode/impl/src/mndProfile.c	行号: 104
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段:	

	<pre> 102: if (pMgmt->connCache == NULL) { 103: code = TSDB_CODE_OUT_OF_MEMORY; 104: mError("failed to alloc profile cache since %s", terrstr()); 105: TAOS_RETURN(code); 106: }</pre>	
	污染路径: 函数'mError'的定义点.	
15	缺陷发生位置:	行号:
	/source/util/src/tmempool.c	510
	步骤描述: 不使用返回值的函数'uInfo'应使用'void'说明.	
	代码片段: <pre> 508: int32_t remainNum = taosHashGetSize(pStat- >remainHash); 509: if (remainNum <= 0) { 510: uInfo("no alloc remaining memory"); 511: return; 512: }</pre>	
	污染路径: 函数'uInfo'的定义点.	
16	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmt2-blob-performance.c	138
	步骤描述: 不使用返回值的函数'clock_gettime'应使用'void'说明.	
	代码片段: <pre> 136: bind_time += ((double)(end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9); 137: 138: clock_gettime(CLOCK_MONOTONIC, &start); // 获取开 始时间 139: // exec 140: if (taos_stmt2_exec(stmt, NULL)) {</pre>	
	污染路径: 无	
17	缺陷发生位置:	行号:
	/source/libs/monitor/src/monFramework.c	104
	步骤描述: 不使用返回值的函数'taos_collector_registry_default_init'应使用'void'说明.	
	代码片段:	

	102: 103: void monInitMonitorFW(){ 104: taos_collector_registry_default_init(); 105: 106: tsMonitor.metrics = taosHashInit(16, taosGetDefaultHashFunction(TSDB_DATA_TYPE_BINARY), true, HASH_ENTRY_LOCK);	
	污染路径: 函数'taos_collector_registry_default_init'的定义点.	
18	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 498
	步骤描述: 不使用返回值的函数'checkNull'应使用'void'说明.	
	代码片段: 496: 497: columnInfoData.hasNull = false; 498: checkNull(0, false); 499: checkNull(1, false); 500: checkNull(2, false);	
	污染路径: 函数'checkNull'的定义点.	
19	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSubs.c	行号: 218
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 216: HASH_DELETE(hh, subhier->shared, shared); 217: ttq_free(shared->name); 218: ttq_free(shared); 219: } 220: return rc;	
	污染路径: 函数'ttq_free'的定义点.	
20	缺陷发生位置: /source/os/src/osTimezone.c	行号: 1211
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 1209: char str1[TD_TIMEZONE_LEN] = {0}; 1210: if (taosStrfTime(str1, sizeof(str1), "%Z", &tm1) == 0)	

	<pre> { 1211: uError("failed to get timezone name"); 1212: return TSDB_CODE_TIME_ERROR; 1213: }</pre>	
	污染路径: 函数'uError'的定义点.	
21	缺陷发生位置: /source/libs/qworker/src/qworker.c	行号: 422
	步骤描述: 不使用返回值的函数'QW_SINK_DISABLE_MEMPOOL'应使用'void'说明.	
	代码片段: <pre> 420: QW_SINK_ENABLE_MEMPOOL(ctx); 421: dsGetDataLength(ctx->sinkHandle, &len, &rawLen, &queryEnd); 422: QW_SINK_DISABLE_MEMPOOL(); 423: 424: if (len <= 0 len != sizeof(SDeleterRes)) {</pre>	
	污染路径: 函数'QW_SINK_DISABLE_MEMPOOL'的定义点.	
22	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 277
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: <pre> 275: } 276: 277: TAOS_RETURN(0); 278: } 279:</pre>	
	污染路径: 函数'TAOS_RETURN'的定义点.	
23	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1624
	步骤描述: 不使用返回值的函数'FSE_buildDTable'应使用'void'说明.	
	代码片段: <pre> 1622: if (LLlog > LLFSELog) return ERROR(corruption_detected); 1623: ip += headerSize; 1624: FSE_buildDTable(DTableLL, norm, max, LLlog);</pre>	

	1625: } } 1626:	
	污染路径: 函数'FSE_buildDTable'的定义点.	
24	缺陷发生位置: /source/dnode/mnode/impl/src/mndSsMigrate.c	行号: 561
	步骤描述: 不使用返回值的函数'TAOS_RETURN'应使用'void'说明.	
	代码片段: 559: } 560: 561: TAOS_RETURN(0); 562: } 563:	
	污染路径: 函数'TAOS_RETURN'的定义点.	
25	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 333
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 331: tsdbTFileObjRemoveLC(fobj); 332: (void)taosThreadMutexDestroy(&fobj->mutex); 333: taosMemoryFree(fobj); 334: } 335: return 0;	
	污染路径: 函数'taosMemoryFree'的定义点.	
26	缺陷发生位置: /source/dnode/vnode/src/tq/tqOffset.c	行号: 106
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 104: tqInfo("tq: offset restore from file to tdb, size:%d, hash size:%d subkey:%s", total, taosHashGetSize(pTq->pOffset), 105: offset.subKey); 106: taosMemoryFree(pMemBuf); 107: pMemBuf = NULL; 108: }	

	函数'atomic_add_fetch_64'的定义点.	
30	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 46
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 44: TAOS_RES* result = taos_query(taos, "drop database if exists test;"); 45: taos_free_result(result); 46: usleep(100000); 47: result = taos_query(taos, "create database test;"); 48:	
	污染路径: 无	
31	缺陷发生位置: /docs/examples/c-ws/stmt_insert_demo.c	行号: 88
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 86: sprintf(table_name, "d_bind_%d", i); 87: char location[20]; 88: sprintf(location, "location_%d", i); 89: 90: // set table name and tags	
	污染路径: 无	
32	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 226
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 224: taosMemoryFree(v->ts); 225: taosMemoryFree(v->br); 226: taosMemoryFree(v->nr); 227: taosMemoryFree(v); 228: taosMemoryFree(lb);	
	污染路径: 函数'taosMemoryFree'的定义点.	
33	缺陷发生位置: /utils/tsim/src/simEntry.c	行号: 50

	步骤描述: 不使用返回值的函数'simSystemInit'应使用'void'说明.	
	代码片段: 48: simInfo("simulator is running ..."); 49: 50: simSystemInit(); 51: taosSetSignal(SIGINT, simHandleSignal); 52:	
	污染路径: 函数'simSystemInit'的定义点.	
34	缺陷发生位置: /docs/examples/c-ws-new/stmt_insert_demo.c	行号: 73
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 71: for (int i = 1; i <= num_of_sub_table; i++) { 72: char table_name[20]; 73: sprintf(table_name, "d_bind_%d", i); 74: char location[20]; 75: sprintf(location, "location_%d", i);	
	污染路径: 无	
35	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet2.c	行号: 40
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 38: void ttqNetBrokerInit(void) { 39: spare_sock = socket(AF_INET, SOCK_STREAM, 0); 40: UNUSED(net__init()); 41: #ifdef WITH_TLS 42: net__init_tls();	
	污染路径: 函数'UNUSED'的定义点.	
36	缺陷发生位置: /source/dnode/mnode/impl/src/mndRetention.c	行号: 983
	步骤描述: 不使用返回值的函数'mTrace'应使用'void'说明.	
	代码片段: 981:	

	982: static int32_t mndProcessQueryRetentionTimer(SRpcMsg *pReq) { 983: mTrace("start to process query trim timer"); 984: mndRetentionPullup(pReq->info.node); 985: return 0;	
	污染路径: 函数'mTrace'的定义点.	
37	缺陷发生位置: /source/util/src/tskiplist.c	行号: 394
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 392: if (pSkipList->lock) { 393: if (taosThreadRwlockWrlock(pSkipList->lock) != 0) { 394: uError("failed to lock skip list"); 395: } 396: }	
	污染路径: 函数'uError'的定义点.	
38	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 762
	步骤描述: 不使用返回值的函数'colName'应使用'void'说明.	
	代码片段: 760: for (int i = 0; i < numOfTable; i++) { 761: tColVal[i % colValSize] = 'a' + i % 26; 762: SearchOne(colName, tColVal); 763: } 764: return 0;	
	污染路径: 函数'colName'的定义点.	
39	缺陷发生位置: /source/libs/scalar/test/filter/filterTests.cpp	行号: 1495
	步骤描述: 不使用返回值的函数'fp'应使用'void'说明.	
	代码片段: 1493: break; 1494: case TSDB_DATA_TYPE_UINT: 1495: doCompareWithValueRange_SignedWithUnsigned<LType,	

	uint32_t>(fp); 1496: break; 1497: case TSDB_DATA_TYPE_UBIGINT:	
	污染路径: 无	
40	缺陷发生位置: /source/libs/index/src/indexCache.c	行号: 373
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 371: _return: 372: taosMemoryFree(pCt); 373: taosMemoryFree(exBuf); 374: TAOS_UNUSED(tSkipListDestroyIter(iter)); 375:	
	污染路径: 函数'taosMemoryFree'的定义点.	
41	缺陷发生位置: /source/dnode/mnode/impl/src/mndIndex.c	行号: 202
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 200: action.msgType = TDMT_VND_DROP_INDEX; 201: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 202: taosMemoryFree(pReq); 203: sdbCancelFetch(pSdb, pIter); 204: sdbRelease(pSdb, pVgroup);	
	污染路径: 函数'taosMemoryFree'的定义点.	
42	缺陷发生位置: /source/dnode/vnode/src/tq/tqRead.c	行号: 866
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 864: code = colDataSetVal(pColumnInfoData, idx, val, false); 865: 866: taosMemoryFree(val); 867: } else {	

	868: colDataSetNULL(pColumnInfoData, idx);	
	污染路径: 函数'taosMemoryFree'的定义点.	
43	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSttFileRW.c	行号: 210
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 208: 0, pEncryptData); 209: if (code) { 210: taosMemoryFree(data); 211: return code; 212: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
44	缺陷发生位置: /source/util/src/tsched.c	行号: 149
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 147: char name[16] = {0}; 148: snprintf(name, tListLen(name), "%s-taskQ", pSched->label); 149: setThreadName(name); 150: int64_t pid = taosGetSelfPthreadId(); 151: uInfo("scheduler %s is started, thread:%" PRIu64, name, pid);	
	污染路径: 函数'setThreadName'的定义点.	
45	缺陷发生位置: /source/dnode/mnode/impl/src/mndSnode.c	行号: 352
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 350: 351: if ((code = mndTransAppendUndoAction(pTrans, &action)) != 0) { 352: taosMemoryFree(pReq); 353: TAOS_RETURN(code); 354: }	

	污染路径: 函数'taosMemoryFree'的定义点.	
46	缺陷发生位置: /source/common/src/ttime.c	行号: 632
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 630: int32_t ret = taosParseTime(newColData, timeVal, len, (int32_t)timePrec, tz); 631: if (ret != TSDB_CODE_SUCCESS) { 632: taosMemoryFree(newColData); 633: TAOS_RETURN(ret); 634: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
47	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 25
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 23: char* token = strtok_r(tmp, separator, &context); 24: if (token) { 25: snprintf(host, len, "%s", token); 26: } 27:	
	污染路径: 无	
48	缺陷发生位置: /source/os/src/osLocale.c	行号: 146
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 144: tstrncpy(outCharset, revisedCharset, TD_CHARSET_LEN); 145: 146: taosMemoryFree(revisedCharset); 147: } 148: // printf("charset not configured, set to system default: %s", outCharset);	

	污染路径: 函数'taosMemoryFree'的定义点.	
49	缺陷发生位置: /source/dnode/mnode/impl/src/mndStream.c	行号: 65
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 63: 64: void mndCleanupStream(SMnode *pMnode) { 65: mDebug("try to clean up stream"); 66: 67: msmHandleBecomeNotLeader(pMnode);	
	污染路径: 函数'mDebug'的定义点.	
50	缺陷发生位置: /source/dnode/mgmt/exe/dmMain.c	行号: 163
	步骤描述: 不使用返回值的函数'dWarn'应使用'void'说明.	
	代码片段: 161: } 162: if (taosIgnSignal(SIGHUP) != 0) { 163: dWarn("failed to ignore signal SIGHUP"); 164: } 165: if (taosIgnSignal(SIGINT) != 0) {	
	污染路径: 函数'dWarn'的定义点.	
51	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 500
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 498: (void)taosThreadRwlockDestroy(&pBse->rwlock); 499: 500: taosMemoryFree(pBse); 501: return; 502: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
52	缺陷发生位置: /source/libs/parser/src/parlInsertSql.c	行号: 608

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 606: outputBytes = tbase64_decode_len(inputBytes); 607: if (outputBytes + VARSTR_HEADER_SIZE > bytes) { 608: taosMemoryFree(tmpTokenBuf); 609: 610: return generateSyntaxErrMsg(&pCxt->msg, TSDB_CODE_PAR_VALUE_TOO_LONG, pSchema->name);	
	污染路径: 函数'taosMemoryFree'的定义点.	
53	缺陷发生位置: /source/libs/transport/test/http_test.c	行号: 67
	步骤描述: 不使用返回值的函数'taosCloseLog'应使用'void'说明.	
	代码片段: 65: } 66: 67: taosCloseLog(); 68: 69: return 0;	
	污染路径: 函数'taosCloseLog'的定义点.	
54	缺陷发生位置: /source/os/src/osTimer.c	行号: 150
	步骤描述: 不使用返回值的函数'taosThreadCleanupPop'应使用'void'说明.	
	代码片段: 148: } while (0); 149: 150: taosThreadCleanupPop(1); 151: 152: return NULL;	
	污染路径: 函数'taosThreadCleanupPop'的定义点.	
55	缺陷发生位置: /source/libs/executor/src/executil.c	行号: 847
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	845: } 846: taosArrayDestroy(pIdWithVal); 847: taosMemoryFree(payload); 848: return code; 849: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
56	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 3550
	步骤描述: 不使用返回值的函数'parserError'应使用'void'说明.	
	代码片段: 3548: nodesDestroyNode((SNode*)pCol); 3549: } 3550: parserError("%s failed to rewrite count(tbname), code: %d", __func__, code); 3551: return code; 3552: }	
	污染路径: 函数'parserError'的定义点.	
57	缺陷发生位置: /contrib/TSZ/sz/src/sz_double.c	行号: 120
	步骤描述: 不使用返回值的函数'int64ToBytes_bigEndian'应使用'void'说明.	
	代码片段: 118: 119: unsigned char preDataBytes[8]; 120: int64ToBytes_bigEndian(preDataBytes, 0); 121: 122: int reqBytesLength = reqLength/8;	
	污染路径: 函数'int64ToBytes_bigEndian'的定义点.	
58	缺陷发生位置: /test/new_test_framework/script/api/apitest.c	行号: 24
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 22: result = taos_query(taos, "drop database if exists test;"); 23: taos_free_result(result); 24: usleep(100000);	

	25: result = taos_query(taos, "create database test precision 'us';"); 26: taos_free_result(result);	
	污染路径: 无	
59	缺陷发生位置: /test/new_test_framework/script/api/stmt2-insert-dupkeys.c	行号: 178
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 176: TAOS_STMT2_BIND **paramv, **tags; 177: 178: INIT(tbs, ts, ts_len, b, b_len, tags, paramv); 179: 180: for (int i = 0; i < CTB_NUMS; i++) {	
	污染路径: expanded from macro 'INIT'	
60	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmWorker.c	行号: 54
	步骤描述: 不使用返回值的函数'taosMsleep'应使用'void'说明.	
	代码片段: 52: setThreadName("dnode-config"); 53: while (1) { 54: taosMsleep(50); 55: if (pMgmt->pData->dropped pMgmt->pData->stopped tsConfigInited) break; 56:	
	污染路径: 函数'taosMsleep'的定义点.	
61	缺陷发生位置: /source/dnode/mnode/impl/src/mndDnode.c	行号: 176
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 174: if (machineld) { 175: (void)memcpy(dnodeObj.machineld, machineld, TSDB_MACHINE_ID_LEN); 176: taosMemoryFreeClear(machineld); 177: } else {	

	178: #if defined(TD_ENTERPRISE) && !defined(GRANTS_CFG)	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
62	缺陷发生位置: /source/libs/qworker/src/qwDbg.c	行号: 337
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 335: if (0 == strcasecmp(option, "lock")) { 336: gQWDebug.lockEnable = true; 337: qError("qw lock debug enabled"); 338: return TSDB_CODE_SUCCESS; 339: }	
	污染路径: 函数'qError'的定义点.	
63	缺陷发生位置: /source/libs/monitor/src/monMain.c	行号: 129
	步骤描述: 不使用返回值的函数'monInitMonitorFW'应使用'void'说明.	
	代码片段: 127: (void)taosThreadMutexInit(&tsMonitor.lock, NULL); 128: 129: monInitMonitorFW(); 130: 131: return 0;	
	污染路径: 函数'monInitMonitorFW'的定义点.	
64	缺陷发生位置: /source/libs/wal/test/walMetaTest.cpp	行号: 490
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 488: int i = 0; 489: char newStr[100]; 490: sprintf(newStr, "%s-%d", ranStr, i); 491: int len = strlen(newStr); 492: int64_t offset = walGetCurFileOffset(pWal);	
	污染路径: 无	

65	缺陷发生位置: /source/libs/command/src/command.c	行号: 736
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 734: parseTagDatatoJson(pTag, &pJson, charsetCxt); 735: if (NULL == pJson) { 736: qError("failed to parse tag to json, pJson is NULL"); 737: return terrno; 738: }	
	污染路径: 函数'qError'的定义点.	
66	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncBatch.c	行号: 166
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 164: printf("syncClientRequestBatchPrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 165: fflush(NULL); 166: taosMemoryFree(serialized); 167: } 168:	
	污染路径: 函数'taosMemoryFree'的定义点.	
67	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 98
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 96: 97: if (!xFree) { 98: tdbError("tdb/page-destroy: null xFree."); 99: return; 100: }	
	污染路径: 函数'tdbError'的定义点.	
68	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 27
	步骤描述:	

	不使用返回值的函数'tsdbTrace'应使用'void'说明.	
	代码片段: <pre> 25: void tsdbLRUCacheRelease(SLRUCache *cache, LRUHandle *handle, bool eraselfLastRef) { 26: if (!taosLRUCacheRelease(cache, handle, eraselfLastRef)) { 27: tsdbTrace(" release lru cache failed"); 28: } 29: }</pre>	
	污染路径: 函数'tsdbTrace'的定义点.	
69	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 36
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 34: char *serialized = syncIndexMgr2Str(pObj); 35: sTrace("syncIndexMgrLog len:%" PRIu64 " %s", (uint64_t)strlen(serialized), serialized); 36: taosMemoryFree(serialized); 37: } 38:</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
70	缺陷发生位置: /source/common/src/tglobal.c	行号: 2304
	步骤描述: 不使用返回值的函数'cfgCleanup'应使用'void'说明.	
	代码片段: <pre> 2302: } 2303: 2304: cfgCleanup(pCfg); 2305: TAOS_RETURN(code); 2306: }</pre>	
	污染路径: 函数'cfgCleanup'的定义点.	
71	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable2.c	行号: 293
	步骤描述: 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	

	代码片段: <pre> 291: if (tdbTbGet(pMeta->pNameIdx, pReq->name, strlen(pReq->name) + 1, &value, &valueSize) == 0) { 292: pReq->uid = *(int64_t *)value; 293: tdbFreeClear(value); 294: 295: if (metaGetInfo(pMeta, pReq->uid, &info, NULL) != 0) { </pre>	
	污染路径: 函数'tdbFreeClear'的定义点.	
72	缺陷发生位置: /docs/examples/c/stmt_insert_demo.c	行号: 172
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 170: printf("Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 171: taos_errstr(NULL)); 172: taos_cleanup(); 173: exit(EXIT_FAILURE); 174: } </pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
73	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 141
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: <pre> 139: tstrncpy(g_stConflInfo.vnodeWalPath, argv[++i], sizeof(g_stConflInfo.vnodeWalPath)); 140: } else if (strcmp(argv[i], "-f") == 0) { 141: tstrncpy(g_stConflInfo.resultFileName, argv[++i], sizeof(g_stConflInfo.resultFileName)); 142: } else if (strcmp(argv[i], "-t") == 0) { 143: g_stConflInfo.numOfThreads = atoi(argv[++i]); </pre>	
	污染路径: 函数'tstrncpy'的定义点.	
74	缺陷发生位置: /utils/test/c/tmq_ts5466.c	行号: 57
	步骤描述:	

	不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 55: int32_t ret = tmq_write_raw(pConn, raw); 56: printf("write raw data: %s\n", tmq_err2str(ret)); 57: ASSERT(ret == 0); 58: 59: tmq_free_raw(raw);	
	污染路径: 函数'ASSERT'的定义点.	
75	缺陷发生位置: /source/libs/tmqtt/mgmt/src/tmqttMgmt.c	行号: 126
	步骤描述: 不使用返回值的函数'bndInfo'应使用'void'说明.	
	代码片段: 124: tstrncpy(taosFqdnEnvItem, "TAOS_FQDN=", len); 125: TAOS_STRNCAT(taosFqdnEnvItem, taosFqdn, subLen); 126: bndInfo("[MQTTD]Success to set TAOS_FQDN:%s", taosFqdn); 127: } else { 128: bndError("[MQTTD]Failed to allocate memory for TAOS_FQDN");	
	污染路径: 函数'bndInfo'的定义点.	
76	缺陷发生位置: /source/libs/catalog/src/catalog.c	行号: 802
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 800: qError("clear cache enqueue failed, error:%s", tstrerror(code)); 801: if (taosTmrReset(ctgProcessTimerEvent, CTG_DEFAULT_CACHE_MON_MSEC, NULL, gCtgMgmt.timer, &gCtgMgmt.cacheTimer)) { 802: qError("reset catalog cache monitor timer error, timer stoppped"); 803: } 804: }	
	污染路径: 函数'qError'的定义点.	

77	缺陷发生位置： /source/dnode/mnode/impl/src/mndMnode.c	行号： 360
	步骤描述： 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段： 358: code = tSerializeSDCreateMnodeReq(pReq, contLen, pCreateReq); 359: if (code < 0) { 360: taosMemoryFree(pReq); 361: TAOS_RETURN(code); 362: }	
	污染路径： 函数'taosMemoryFree'的定义点.	
78	缺陷发生位置： /contrib/TSZ/zstd/decompress/zstd_decompress.c	行号： 240
	步骤描述： 不使用返回值的函数'DEBUGLOG'应使用'void'说明.	
	代码片段： 238: ZSTD_DCtx* ZSTD_createDCtx(void) 239: { 240: DEBUGLOG(3, "ZSTD_createDCtx"); 241: return ZSTD_createDCtx_advanced(ZSTD_defaultCMem); 242: }	
	污染路径： 函数'DEBUGLOG'的定义点.	
79	缺陷发生位置： /source/dnode/vnode/src/meta/metaEntry2.c	行号： 111
	步骤描述： 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	
	代码片段： 109: tstrerror(code)); 110: tDecoderClear(&decoder); 111: tdbFreeClear(value); 112: return code; 113: }	
	污染路径： 函数'tdbFreeClear'的定义点.	
80	缺陷发生位置： /source/libs/qworker/src/qworker.c	行号： 67

	步骤描述: 不使用返回值的函数'QW_SINK_DISABLE_MEMPOOL'应使用'void'说明.	
	代码片段: <pre> 65: QW_SINK_ENABLE_MEMPOOL(ctx); 66: dsGetDataLength(ctx->sinkHandle, &ctx- >affectedRows, NULL, NULL); 67: QW_SINK_DISABLE_MEMPOOL(); 68: } 69: }</pre>	
	污染路径: 函数'QW_SINK_DISABLE_MEMPOOL'的定义点.	
81	缺陷发生位置: /docs/examples/c-ws/create_db_demo.c	行号: 26
	步骤描述: 不使用返回值的函数'ws_enable_log'应使用'void'说明.	
	代码片段: <pre> 24: 25: static int DemoCreateDB() { 26: ws_enable_log("debug"); 27: // ANCHOR: create_db_and_table 28: int code = 0;</pre>	
82	污染路径: 函数'ws_enable_log'的定义点.	
	缺陷发生位置: /source/util/src/tcompression.c	行号: 140
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: <pre> 138: const int32_t decompressed_size = LZ4_decompress_safe(input + 1, output, compressedSize - 1, outputSize); 139: if (decompressed_size < 0) { 140: uError("Failed to decompress string with LZ4 algorithm, decompressed size:%d", decompressed_size); 141: return TSDB_CODE_THIRDPARTY_ERROR; 142: }</pre>	
83	污染路径: 函数'uError'的定义点.	
	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 5726

	步骤描述: 不使用返回值的函数'nodesError'应使用'void'说明.	
	代码片段: <pre> 5724: } 5725: if (TSDB_CODE_SUCCESS != code) { 5726: nodesError("msgToSpecificNode error node = %s", nodesNodeName(nodeType pObj)); 5727: } 5728: return code; </pre>	
	污染路径: 函数'nodesError'的定义点.	
84	缺陷发生位置: /source/util/test/arrayTest.cpp	行号: 168
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 166: for (int32_t j = 0; j < i; j++) { 167: char *v = (char *)taosMemoryCalloc(100, sizeof(char)); 168: sprintf(v, format, i); 169: //printf(" needfree put i=%d v=%s\n", i, v); 170: taosArrayPush(pa, &v); </pre>	
	污染路径: 无	
85	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 279
	步骤描述: 不使用返回值的函数'benchArrayPushNoFree'应使用'void'说明.	
	代码片段: <pre> 277: if (ret == 0) { 278: // only succ add delay list 279: benchArrayPushNoFree(pThreadInfo-> query_delay_list, &delay); 280: pThreadInfo->total_delay += delay; 281: } </pre>	
	污染路径: 函数'benchArrayPushNoFree'的定义点.	
86	缺陷发生位置: /source/libs/index/src/indexFstFile.c	行号: 36
	步骤描述:	

	不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 34: TAOS_UNUSED(key); 35: TAOS_UNUSED(keyLen); 36: taosMemoryFree(value); 37: } 38:	
	污染路径: 函数'taosMemoryFree'的定义点.	
87	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtil.c	行号: 65
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 63: state = tmqtt__get_state(ttq); 64: if (state == ttq_cs_active && ttq->ping_t == 0) { 65: UNUSED(send__pingreq(ttq)); 66: /* Reset last msg times to give the server time to send a pingresp */ 67: ttq_pthread_mutex_lock(&ttq->msgtime_mutex);	
	污染路径: 函数'UNUSED'的定义点.	
88	缺陷发生位置: /source/util/src/tmempool.c	行号: 234
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 232: pPool->name = taosStrdup(poolName); 233: if (NULL == pPool->name) { 234: uError("calloc memory pool name %s failed", poolName); 235: MP_ERR_RET(terrno); 236: }	
	污染路径: 函数'uError'的定义点.	
89	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 3376
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段:	

	3374: _OVER: 3375: if (code != 0) mError("failed to retrieve at line:%d, since %s", lino, tstrerror(code)); 3376: taosMemoryFree(buf); 3377: } 3378:	
	污染路径: 函数'taosMemoryFree'的定义点.	
90	缺陷发生位置: /source/dnode/mnode/impl/src/mndSync.c	行号: 314
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 312: mndSetRestored(pMnode, true); 313: } else { 314: mInfo("vgId:1, sync restore finished, repeat call"); 315: } 316: } else {	
	污染路径: 函数'mInfo'的定义点.	
91	缺陷发生位置: /source/libs/function/src/functionMgt.c	行号: 399
	步骤描述: 不使用返回值的函数'taosHashCleanup'应使用'void'说明.	
	代码片段: 397: void* m = gFunMgtService.pFuncNameHashTable; 398: if (m != NULL && atomic_val_compare_exchange_ptr((void**)&gFunMgtService.p FuncNameHashTable, m, 0) == m) { 399: taosHashCleanup(m); 400: } 401: }	
	污染路径: 函数'taosHashCleanup'的定义点.	
92	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 949
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 947: if (tDeserializeSVStbRefDbsReq(pMsg->pCont, pMsg-	

	<pre>>contLen, &req)) { 948: code = terrno; 949: qError("tDeserializeSVSubTablesReq failed"); 950: goto _return; 951: }</pre>	
	污染路径: 函数'qError'的定义点.	
93	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 87
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: <pre>85: } 86: } 87: tmq_free_json_meta(result); 88: } 89:</pre>	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
94	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 258
	步骤描述: 不使用返回值的函数'qStreamDestroyTableList'应使用'void'说明.	
	代码片段: <pre>256: end: 257: nodesDestroyList(groupNew); 258: qStreamDestroyTableList(pTableListInfo); 259: STREAM_PRINT_LOG_END(code, lino); 260: return code;</pre>	
	污染路径: 函数'qStreamDestroyTableList'的定义点.	
95	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_lazy.c	行号: 175
	步骤描述: 不使用返回值的函数'assert'应使用'void'说明.	
	代码片段: <pre>173: 174: (void)dictMode; 175: assert(dictMode == ZSTD_dictMatchState); 176:</pre>	

	177: while (nbCompares-- && (dictMatchIndex > dictLowLimit)) {	
	污染路径: 函数'assert'的定义点.	
96	缺陷发生位置: /source/client/wrapper/src/clientJniConnector.c	行号: 129
	步骤描述: 不使用返回值的函数'atomic_store_32'应使用'void'说明.	
	代码片段: 127: (*env)->DeleteLocalRef(env, consumerClass); 128: 129: atomic_store_32(&__init, 2); 130: jniDebug("native method register finished"); 131: }	
	污染路径: 函数'atomic_store_32'的定义点.	
97	缺陷发生位置: /source/dnode/mnode/sdb/src/sdb.c	行号: 69
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 67: pSdb->encrypted = false; 68: (void)taosThreadMutexInit(&pSdb->filelock, NULL); 69: mInfo("sdb init success"); 70: return pSdb; 71: }	
	污染路径: 函数'mInfo'的定义点.	
98	缺陷发生位置: /source/libs/catalog/src/catalog.c	行号: 681
	步骤描述: 不使用返回值的函数'ctgError'应使用'void'说明.	
	代码片段: 679: int32_t ctgGetTbsHashVgId(SCatalog* pCtg, SRequestConnInfo* pConn, int32_t acctId, const char* pDb, const char* pTbs[], int32_t tbNum, int32_t* vgId) { 680: if (IS_SYS_DBNAME(pDb)) { 681: ctgError("db:%s, no valid vgInfo for db", pDb); 682: CTG_ERR_RET(TSDB_CODE_CTG_INVALID_INPUT); 683: }	

	污染路径: 函数'ctgError'的定义点.	
99	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 387
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 385: (void)taosThreadMutexDestroy(&(*async)->mutex); 386: (void)taosThreadCondDestroy(&(*async)->hasTask); 387: taosMemoryFree(*async); 388: return ret; 389: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
100	缺陷发生位置: /source/dnode/mgmt/mgmt_vnode/src/vmlnt.c	行号: 604
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 602: 603: dInfo("thread:%d, start to open or destroy %d vnodes", pThread->threadIndex, pThread->vnodeNum); 604: setThreadName("open-vnodes"); 605: 606: for (int32_t v = 0; v < pThread->vnodeNum; ++v) {	
	污染路径: 函数'setThreadName'的定义点.	
101	缺陷发生位置: /test/new_test_framework/script/api/insert_stb.c	行号: 180
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 178: free(qstr); 179: taos_close(taos); 180: taos_cleanup(); 181: } 182:	
	污染路径: 函数'taos_cleanup'的定义点.	

102	缺陷发生位置： /source/dnode/mnode/impl/src/mndGrant.c	行号： 52
	步骤描述： 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段： 50: GRANT_ITEM_SHOW("limited"); 51: GRANT_ITEM_SHOW("false"); 52: GRANT_ITEM_SHOW("ungranted"); 53: 54: for (int32_t i = 0; i < nCols - 5; ++i) {	
	污染路径： expanded from macro 'GRANT_ITEM_SHOW' 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
103	缺陷发生位置： /source/dnode/mgmt/node_mgmt/src/dmMonitor.c	行号： 146
	步骤描述： 不使用返回值的函数'auditSendRecordsInBatch'应使用'void'说明.	
	代码片段： 144: 145: // Todo: put this in seperate file in the future 146: void dmSendAuditRecords() { auditSendRecordsInBatch(); } 147: 148: void dmGetVnodeLoads(SMonVloadInfo *pInfo) {	
	污染路径： 函数'auditSendRecordsInBatch'的定义点.	
104	缺陷发生位置： /source/dnode/vnode/src/vnd/vnodeSvr.c	行号： 96
	步骤描述： 不使用返回值的函数'TSDB_CHECK_CODE'应使用'void'说明.	
	代码片段： 94: if (tStartDecode(pCoder) < 0) { 95: code = TSDB_CODE_INVALID_MSG; 96: TSDB_CHECK_CODE(code, lino, _exit); 97: } 98:	
	污染路径： 函数'TSDB_CHECK_CODE'的定义点.	
105	缺陷发生位置： /source/client/src/clientImpl.c	行号： 157

	步骤描述: 不使用返回值的函数'TSC_ERR_RET'应使用'void'说明.	
	代码片段: <pre> 155: int32_t taos_connect_by_auth(const char* ip, const char* user, const char* auth, const char* totp, const char* db, 156: uint16_t port, int connType, STscObj** pObj) { 157: TSC_ERR_RET(taos_init()); 158: 159: if (user == NULL) { </pre>	
	污染路径: 函数'TSC_ERR_RET'的定义点.	
106	缺陷发生位置: /source/libs/function/src/detail/tavgfunction.c	行号: 473
	步骤描述: 不使用返回值的函数'AVG_RES_SET_INPUT_SCALE'应使用'void'说明.	
	代码片段: <pre> 471: CHECK_OVERFLOW_SUM_UNSIGNED_BIG(pOutput, (overflow ? SUM_RES_GET_DSUM(&AVG_RES_GET_SUM(pInput)) : SUM_RES_GET_USUM(&AVG_RES_GET_SUM(pInput))), overflow); 472: } else if (IS_DECIMAL_TYPE(type)) { 473: AVG_RES_SET_INPUT_SCALE(pOutput, AVG_RES_GET_INPUT_SCALE(pInput)); 474: bool overflow = false; 475: SUM_RES_INC_DECIMAL_SUM(&AVG_RES_GET_DECIMAL_SUM(p Output), &AVG_RES_GET_DECIMAL_SUM(pInput), TSDB_DATA_TYPE_DECIMAL); </pre>	
	污染路径: 函数'AVG_RES_SET_INPUT_SCALE'的定义点.	
107	缺陷发生位置: /test/new_test_framework/script/api/stmt_function.c	行号: 80
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: <pre> 78: stmt = taos_stmt_init(taos); 79: assert(stmt != NULL); 80: sprintf(stmt_sql, "insert into ? values (?,?,?,?,?,?,?,?,?,?)"); </pre>	

	81: assert(taos_stmt_prepare(stmt, stmt_sql, 0) == 0); 82: assert(taos_stmt_close(stmt) == 0);	
	污染路径: 无	
108	缺陷发生位置: /source/util/src/tcompression.c	行号: 350
	步骤描述: 不使用返回值的函数'tdszExit'应使用'void'说明.	
	代码片段: 348: } 349: // exit call 350: void tsCompressExit() { tdszExit(); } 351: 352: /*	
	污染路径: 函数'tdszExit'的定义点.	
109	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	行号: 210
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 208: s = syncUtilPrintBin((char*)(pMsg->data), pMsg->dataLen); 209: cJSON_AddStringToObject(pRoot, "data", s); 210: taosMemoryFree(s); 211: s = syncUtilPrintBin2((char*)(pMsg->data), pMsg->dataLen); 212: cJSON_AddStringToObject(pRoot, "data2", s);	
	污染路径: 函数'taosMemoryFree'的定义点.	
110	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 131
	步骤描述: 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	
	代码片段: 129: } 130: int64_t uid = *(int64_t *)value; 131: tdbFreeClear(value); 132: 133: code = metaFetchEntryByUid(pMeta, uid, ppEntry);	

	污染路径: 函数'tdbFreeClear'的定义点.	
111	缺陷发生位置: /utils/test/c/tmq_ts5776.c	行号: 237
	步骤描述: 不使用返回值的函数'create_topic'应使用'void'说明.	
	代码片段: 235: return -1; 236: } 237: create_topic(); 238: 239: tmq_t* tmq = build_consumer();	
	污染路径: 函数'create_topic'的定义点.	
112	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqPacket.c	行号: 107
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 105: 106: if (db.config->max_queued_messages > 0 && ttq->out_packet_count >= db.config->max_queued_messages) { 107: ttq_free(packet); 108: if (ttq->is_dropping == false) { 109: ttq->is_dropping = true;	
	污染路径: 函数'ttq_free'的定义点.	
113	缺陷发生位置: /source/os/src/osThread.c	行号: 370
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 368: int32_t taosThreadGetSchedParam(TdThread thread, int32_t *policy, struct sched_param *param) { 369: OS_PARAM_CHECK(policy); 370: OS_PARAM_CHECK(param); 371: int32_t code = pthread_getschedparam(thread, policy, param); 372: if (code) {	

	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
114	缺陷发生位置: /source/dnode/vnode/src/tq/tqSnapshot.c	行号: 105
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 103: tqError("%s failed at %d, vnode tq snapshot read data failed since %s", __func__, lino, tsterror(code)); 104: } 105: tdbFree(pKey); 106: tdbFree(pVal); 107: return code;	
	污染路径: 函数'tdbFree'的定义点.	
115	缺陷发生位置: /source/libs/index/src/index.c	行号: 489
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 487: 488: if (taosThreadMutexLock(&sldx->mtx) != 0) { 489: indexError("failed to lock index mutex"); 490: } 491:	
	污染路径: 函数'indexError'的定义点.	
116	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 670
	步骤描述: 不使用返回值的函数'symTransform_4bytes'应使用'void'说明.	
	代码片段: 668: while(1) 669: { 670: symTransform_4bytes(p); 671: i+=sizeof(unsigned int); 672: if(i<size)	
	污染路径: 函数'symTransform_4bytes'的定义点.	
117	缺陷发生位置:	行号:

	/docs/examples/c/stmt_example.c	136
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 134: insertData(taos); 135: taos_close(taos); 136: taos_cleanup(); 137: } 138:	
	污染路径: 函数'taos_cleanup'的定义点.	
118	缺陷发生位置: /source/client/src/clientMonitor.c	行号: 134
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 132: buf = taosMemoryMalloc(tlen); 133: if (buf == NULL) { 134: tscError("sendReport failed, out of memory, len:%d", tlen); 135: code = terrno; 136: goto FAILED;	
	污染路径: 函数'tscError'的定义点.	
119	缺陷发生位置: /source/libs/index/src/indexFilter.c	行号: 235
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 233: SHashObj *pObj = taosHashInit(256, taosGetDefaultHashFunction(type), true, false); 234: if (NULL == pObj) { 235: indexError("taosHashInit failed, size:%d", 256); 236: return terrno; 237: }	
	污染路径: 函数'indexError'的定义点.	
120	缺陷发生位置: /docs/examples/c/async_query_example.c	行号: 180
	步骤描述:	

	不使用返回值的函数'sleep'应使用'void'说明.	
	代码片段: <pre> 178: // here, because we want to close it after getting data. 179: taos_query_a(taos, "SELECT * FROM meters", select_callback, taos); 180: sleep(1); 181: } 182: </pre>	
	污染路径: 函数'sleep'的定义点.	
121	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 132
	步骤描述: 不使用返回值的函数'tdbOsFree'应使用'void'说明.	
	代码片段: <pre> 130: tdbError("failed to write page since %s", terrstr()); 131: tdbAbort(pEnv, txn); 132: tdbOsFree(pBt); 133: return ret; 134: } </pre>	
	污染路径: 函数'tdbOsFree'的定义点.	
122	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeAsync.c	行号: 371
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 369: (void)taosThreadMutexDestroy(&(*async)->mutex); 370: (void)taosThreadCondDestroy(&(*async)->hasTask); 371: taosMemoryFree(*async); 372: return ret; 373: } </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
123	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 101
	步骤描述: 不使用返回值的函数'dmReportStartup'应使用'void'说明.	
	代码片段: <pre> 99: } </pre>	

	100: 101: dmReportStartup("dnode-transport", "initialized"); 102: dDebug("dnode is created, ptr:%p", pDnode); 103: code = 0;	
	污染路径: 函数'dmReportStartup'的定义点.	
124	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 626
	步骤描述: 不使用返回值的函数'planError'应使用'void'说明.	
	代码片段: 624: _return: 625: if (TSDB_CODE_SUCCESS != code) { 626: planError("%s failed since %s", __func__, tstrerror(code)); 627: } 628: return code;	
	污染路径: 函数'planError'的定义点.	
125	缺陷发生位置: /utlis/tsim/src/simExec.c	行号: 421
	步骤描述: 不使用返回值的函数'simInfo'应使用'void'说明.	
	代码片段: 419: } 420: 421: simInfo("system cmd is %s", buf); 422: return replaced; 423: }	
	污染路径: 函数'simInfo'的定义点.	
126	缺陷发生位置: /source/dnode/vnode/src/meta/metaCache.c	行号: 159
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 157: STagConds** p = param; 158: taosHashCleanup((*p)->set); 159: taosMemoryFreeClear(*p); 160: }	

	161:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
127	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1883
	步骤描述: 不使用返回值的函数'HUFv07_decodeStreamX2'应使用'void'说明.	
	代码片段: 1881: if (HUFv07_isError(errorCode)) return errorCode; } 1882: 1883: HUFv07_decodeStreamX2(op, &bitD, oend, dt, dtLog); 1884: 1885: /* check */	
	污染路径: 函数'HUFv07_decodeStreamX2'的定义点.	
128	缺陷发生位置: /source/dnode/mnode/impl/src/mndStb.c	行号: 1571
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1569: alterReq.numOfFields = taosArrayGetSize(alterReq.pFields); 1570: if (alterReq.numOfFields == 0) { 1571: mError("no valid alter field found"); 1572: goto END; 1573: }	
	污染路径: 函数'mError'的定义点.	
129	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 395
	步骤描述: 不使用返回值的函数'taosGetTimeOfDay'应使用'void'说明.	
	代码片段: 393: static int64_t taosGetTimestampMs() { 394: struct timeval systemTime; 395: taosGetTimeOfDay(&systemTime); 396: return (int64_t)systemTime.tv_sec * 1000LL + (int64_t)systemTime.tv_usec / 1000; 397: }	

	污染路径: 函数'taosGetTimeOfDay'的定义点.	
130	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 141
	步骤描述: 不使用返回值的函数'kill'应使用'void'说明.	
	代码片段: 139: 140: /* Send signal. */ 141: kill (pid, SIGKILL); 142: 143: /* Wait for it to terminate. */	
	污染路径: 无	
131	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmNodes.c	行号: 122
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 120: } 121: 122: dlInfo("The daemon initialized successfully"); 123: dmReportStartup("The daemon", "initialized successfully"); 124: return 0;	
	污染路径: 函数'dlInfo'的定义点.	
132	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeModule.c	行号: 39
	步骤描述: 不使用返回值的函数'walCleanUp'应使用'void'说明.	
	代码片段: 37: if (atomic_val_compare_exchange_32(&VINIT, 1, 0) == 0) return; 38: vnodeAsyncClose(); 39: walCleanUp(); 40: }	
	污染路径:	

	函数'walCleanUp'的定义点.	
133	缺陷发生位置: /source/libs/planner/src/planner.c	行号: 40
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 38: if (TSDB_CODE_SUCCESS == code) { 39: planDebugL("QID:0x%" PRIx64 " , Query Plan, JsonPlan: %s", pPlan->queryId, pStr); 40: taosMemoryFree(pStr); 41: } 42: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
134	缺陷发生位置: /contrib/test/cos/main.c	行号: 106
	步骤描述: 不使用返回值的函数'cos_warn_log'应使用'void'说明.	
	代码片段: 104: cos_get_hmac_sha1_hexdigest(sign_key, secret_key, sizeof(secret_key) - 1, time_str, sizeof(time_str) - 1); 105: char *pstr = apr_pstrndup(p, (char *)sign_key, sizeof(sign_key)); 106: cos_warn_log("sign_key: %s", pstr); 107: 108: // method	
	污染路径: 函数'cos_warn_log'的定义点.	
135	缺陷发生位置: /docs/examples/c-ws/tmq_demo.c	行号: 69
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 67: char buf[200] = {0}; 68: i++; 69: snprintf(70: buf, sizeof(buf), 71: "INSERT INTO power.d1001 USING power.meters TAGS(2,'California.SanFrancisco') VALUES (NOW + %da, 10.30000, "	

	污染路径: 无	
136	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleSub.c	行号: 28
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 26: rc = packet__alloc(packet); 27: if (rc) { 28: ttq_free(packet); 29: return rc; 30: }	
	污染路径: 函数'ttq_free'的定义点.	
137	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 1117
	步骤描述: 不使用返回值的函数'pthread_join'应使用'void'说明.	
	代码片段: 1115: 1116: for (int i = 0; i < threadCnt; i++) { 1117: pthread_join(pids[i], NULL); 1118: } 1119:	
	污染路径: 无	
138	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 240
	步骤描述: 不使用返回值的函数'r'应使用'void'说明.	
	代码片段: 238: DEFINE_OPERATOR(+, OP_TYPE_ADD); 239: DEFINE_OPERATOR(-, OP_TYPE_SUB); 240: DEFINE_OPERATOR(*, OP_TYPE_MULTI); 241: DEFINE_OPERATOR(/, OP_TYPE_DIV); 242: DEFINE_OPERATOR(%, OP_TYPE_REM);	
	污染路径: expanded from macro 'DEFINE_OPERATOR'	
139	缺陷发生位置: /source/libs/scheduler/src/schTask.c	行号: 1188

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 1186: if (NULL == pParent) { 1187: qDebug("job refld 0x%" PRIx64 " already not exist", pCtx->jobRid); 1188: taosMemoryFree(param); 1189: SCH_RET(TSDB_CODE_SCH_JOB_IS_DROPPING); 1190: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
140	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 888
	步骤描述: 不使用返回值的函数'ST_TASK_DLOG'应使用'void'说明.	
	代码片段: 886: STREAM_CHECK_RET_GOTO(qStreamModifyTableList(sStreamReaderInfo, tableList, uidList)); 887: 888: ST_TASK_DLOG("%s stream reader scan alter table tag val", __func__); 889: 890: end:	
	污染路径: 函数'ST_TASK_DLOG'的定义点.	
141	缺陷发生位置: /utils/test/c/sml_test.c	行号: 212
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 210: } 211: 212: ASSERT(code == 0); 213: 214: taos_close(taos);	
	污染路径: 函数'ASSERT'的定义点.	
142	缺陷发生位置: /source/libs/tdb/src/db/tdbTable.c	行号: 286

	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 284: if (ret < 0) break; 285: } 286: tdbFree(pKey); 287: tdbFree(pValue); 288: tdbTbcClose(pCur);	
	污染路径: 函数'tdbFree'的定义点.	
143	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 2263
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 2261: if (TSDB_CODE_SUCCESS == code) { 2262: (void)printf("%s\n", pStr); 2263: taosMemoryFree(pStr); 2264: } 2265: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
144	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 203
	步骤描述: 不使用返回值的函数'system'应使用'void'说明.	
	代码片段: 201: char cmd[128]; 202: snprintf(cmd, sizeof(cmd), "rm -rf ./%s", dir); 203: system(cmd); 204: snprintf(cmd, sizeof(cmd), "mkdir -p ./%s", dir); 205: system(cmd);	
	污染路径: 无	
145	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmWorker.c	行号: 585
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 583: tSingleWorkerCleanup(&pMgmt->auditWorker);	

	584: 585: dDebug("mnode workers are closed"); 586: }	
	污染路径: 函数'dDebug'的定义点.	
146	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 534
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 532: code = TSDB_CODE_SYN_RETURN_VALUE_NULL; 533: if (terrno != 0) code = terrno; 534: sError("sync end snapshot error"); 535: TAOS_RETURN(code); 536: }	
	污染路径: 函数'sError'的定义点.	
147	缺陷发生位置: /source/util/src/tdecompressavx.c	行号: 600
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 598: return opos; 599: #else 600: uError("unable run %s without avx512 instructions", __func__); 601: return -1; 602: #endif	
	污染路径: 函数'uError'的定义点.	
148	缺陷发生位置: /docs/examples/c-ws-new/tmq_demo.c	行号: 60
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 58: char buf[200] = {0}; 59: i++; 60: snprintf(61: buf, sizeof(buf), 62: "INSERT INTO power.d1001 USING power.meters	

	TAGS(2,'California.SanFrancisco') VALUES (NOW + %da, 10.30000, "	
	污染路径: 无	
149	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 60
	步骤描述: 不使用返回值的函数'fnError'应使用'void'说明.	
	代码片段: 58: ret = blockDataEnsureCapacity(pBlock, 1024); 59: if (ret != 0) { 60: fnError("failed to ensure capacity"); 61: return -1; 62: }	
	污染路径: 函数'fnError'的定义点.	
150	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 1094
	步骤描述: 不使用返回值的函数'argp_fmtstream_puts'应使用'void'说明.	
	代码片段: 1092: argp_fmtstream_set_lmargin (pest->stream, uparams.header_col); 1093: argp_fmtstream_set_wmargin (pest->stream, uparams.header_col); 1094: argp_fmtstream_puts (pest->stream, fstr); 1095: argp_fmtstream_set_lmargin (pest->stream, 0); 1096: argp_fmtstream_putc (pest->stream, '\n');	
	污染路径: 函数'argp_fmtstream_puts'的定义点.	
151	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 896
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 894: pInfo->pCurrSortHandle = NULL; 895: 896: taosMemoryFreeClear(param); 897: }	

	898:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
152	缺陷发生位置: /source/libs/qcom/src/querymsg.c	行号: 112
	步骤描述: 不使用返回值的函数'QUERY_PARAM_CHECK'应使用'void'说明.	
	代码片段: 110: int32_t queryBuildUseDbMsg(void *input, char **msg, int32_t msgSize, int32_t *msgLen, void *(*mallocFp)(int64_t), 111: void (*freeFp)(void *)) { 112: QUERY_PARAM_CHECK(input); 113: QUERY_PARAM_CHECK(msg); 114: QUERY_PARAM_CHECK(msgLen);	
	污染路径: 函数'QUERY_PARAM_CHECK'的定义点.	
153	缺陷发生位置: /source/libs/monitorfw/src/taos_map.c	行号: 89
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: 87: if (r) { 88: if (taos_map_destroy(self) != 0) { 89: TAOS_LOG("TAOS_MAP_DESTROY_ERROR"); 90: } 91: return NULL;	
	污染路径: 函数'TAOS_LOG'的定义点.	
154	缺陷发生位置: /source/util/src/tlruccache.c	行号: 145
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 143: taosLRUEntryTableApply(table, taosLRUEntryTableFree, 0, 1 << table->lengthBits); 144: 145: taosMemoryFree(table->list); 146: } 147:	

	污染路径: 函数'taosMemoryFree'的定义点.	
155	缺陷发生位置: /source/dnode/mnode/impl/src/mndConsumer.c	行号: 915
	步骤描述: 不使用返回值的函数'MND_TMQ_NULL_CHECK'应使用'void'说明.	
	代码片段: <pre> 913: MND_TMQ_NULL_CHECK(tmp); 914: char *pNewTopic = taosStrdup(tmp); 915: MND_TMQ_NULL_CHECK(pNewTopic); 916: removeFromTopicList(pOldConsumer->rebNewTopics, pNewTopic, pOldConsumer->consumerId, "new"); 917: bool existing = existInCurrentTopicList(pOldConsumer, pNewTopic); </pre>	
	污染路径: 函数'MND_TMQ_NULL_CHECK'的定义点.	
156	缺陷发生位置: /source/dnode/vnode/src/tq/tqMeta.c	行号: 220
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: <pre> 218: return terrno; 219: } 220: tdbFree(data); 221: 222: *pOffset = taosHashGet(pTq->pOffset, subkey, strlen(subkey)); </pre>	
	污染路径: 函数'tdbFree'的定义点.	
157	缺陷发生位置: /source/dnode/vnode/src/tq/tqUtil.c	行号: 242
	步骤描述: 不使用返回值的函数'TQ_NULL_GO_TO_END'应使用'void'说明.	
	代码片段: <pre> 240: int32_t tLen = sizeof(SMqRspHead) + len; 241: void* tBuf = taosMemoryCalloc(1, tLen); 242: TQ_NULL_GO_TO_END(tBuf); 243: void* metaBuff = POINTER_SHIFT(tBuf, sizeof(SMqRspHead)); 244: SEncoder encoder = {0}; </pre>	

	污染路径: 函数'TQ_NULL_GO_TO_END'的定义点.	
158	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIO.c	行号: 178
	步骤描述: 不使用返回值的函数'rpcInit'应使用'void'说明.	
	代码片段: 176: taosBlockSIGPIPE(); 177: 178: rpcInit(); 179: 180: // client rpc init	
	污染路径: 函数'rpcInit'的定义点.	
159	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 181
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 179: 180: for (int i = 1; i <= nData; i++) { 181: sprintf(key, "key%d", i); 182: sprintf(val, "value%d", i); 183:	
	污染路径: 无	
160	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 83
	步骤描述: 不使用返回值的函数'tdbFreeClear'应使用'void'说明.	
	代码片段: 81: .uid = uid, 82: }; 83: tdbFreeClear(value); 84: 85: code = tdbTbGet(pMeta->pTbDb, &key, sizeof(key), &value, &valueSize);	
	污染路径: 函数'tdbFreeClear'的定义点.	
161	缺陷发生位置:	行号:

	/source/dnode/mnode/impl/src/mndRetentionDetail.c	43
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 41: } 42: 43: void mndCleanupRetentionDetail(SMnode *pMnode) { mDebug("mnd retention detail cleanup"); } 44: 45: int32_t mndRetrieveRetentionDetail(SRpcMsg *pReq, SShowObj *pShow, SSDataBlock *pBlock, int32_t rows) {	
	污染路径: 函数'mDebug'的定义点.	
162	缺陷发生位置: /utils/test/c/sml_test.c	行号: 142
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 140: printf("%s result:%s\n", __FUNCTION__, taos_errstr(pRes)); 141: code = taos_errno(pRes); 142: ASSERT(code == 0); 143: taos_free_result(pRes); 144:	
	污染路径: 函数'ASSERT'的定义点.	
163	缺陷发生位置: /docs/examples/c/json_protocol_example.c	行号: 48
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 46: taos_free_result(res); 47: taos_close(taos); 48: taos_cleanup(); 49: } 50: // output:	
	污染路径: 函数'taos_cleanup'的定义点.	
164	缺陷发生位置: /utils/test/c/varbinary_test.c	行号: 244

	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 242: pRes = taos_query(taos, "select * from stb where c2 not in ('\\x00000000')"); 243: GET_ROW_NUM 244: ASSERT(numRows == 4); 245: taos_free_result(pRes); 246:	
	污染路径: 函数'ASSERT'的定义点.	
165	缺陷发生位置: /source/dnode/mnode/impl/src/mndCompact.c	行号: 404
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 402: if ((ret = tSerializeSVKillCompactReq((char *)pReq + sizeof(SMsgHead), contLen, &req)) < 0) { 403: terrno = ret; 404: taosMemoryFreeClear(pReq); 405: return NULL; 406: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
166	缺陷发生位置: /source/libs/tmqtt/tools/src/topic-producer.c	行号: 176
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 174: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 175: taos_errstr(NULL)); 176: taos_cleanup(); 177: return NULL; 178: }	
	污染路径: 函数'taos_cleanup'的定义点.	
167	缺陷发生位置: /source/libs/wal/src/walMgmt.c	行号: 417
	步骤描述:	

	不使用返回值的函数'wDebug'应使用'void'说明.	
	代码片段: 415: } 416: 417: wDebug("wal thread is stopped"); 418: }	
	污染路径: 函数'wDebug'的定义点.	
168	缺陷发生位置: /source/dnode/vnode/src/tq/tqSink.c	行号: 120
	步骤描述: 不使用返回值的函数'tqError'应使用'void'说明.	
	代码片段: 118: pCreateTableReq->ctb.stbName = taosStrdup((char*)tNameGetTableName(&name)); 119: if (pCreateTableReq->ctb.stbName == NULL) { // ignore this error code 120: tqError("failed to duplicate the stb name:%s, failed to init create-table msg and create req table", stbFullName); 121: code = terrno; 122: }	
	污染路径: 函数'tqError'的定义点.	
169	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmTransport.c	行号: 514
	步骤描述: 不使用返回值的函数'dDebug'应使用'void'说明.	
	代码片段: 512: } 513: 514: dDebug("dnode rpc client is initialized"); 515: return 0; 516: }	
	污染路径: 函数'dDebug'的定义点.	
170	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 48
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段:	

	46: gHbDataRefId = taosOpenRef(200, (RefFp)syncHbTimerDataFree); 47: if (gHbDataRefId < 0) { 48: sError("failed to init hbdata rset"); 49: syncCleanUp(); 50: return TSDB_CODE_SYN_WRONG_REF;	
	污染路径: 函数'sError'的定义点.	
171	缺陷发生位置: /source/dnode/mnode/impl/src/mndSubscribe.c	行号: 1224
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1222: code = buildRebOutput(pMnode, &rebInput, &rebOutput); 1223: if (code != 0) { 1224: mError("mq rebalance buildRebOutput, msg:%s", tstrerror(code)) 1225: } 1226:	
	污染路径: 函数'mError'的定义点.	
172	缺陷发生位置: /source/libs/parser/src/parUtil.c	行号: 758
	步骤描述: 不使用返回值的函数'qWarn'应使用'void'说明.	
	代码片段: 756: const char* pBeginQuote = strchr(pStart, '`'); 757:  if (!pBeginQuote) { 758:    qWarn("failed to get string from auth string, %s, should be quoted with `", pStart); 759: return TSDB_CODE_INVALID_PARA; 760: }	
	污染路径: 函数'qWarn'的定义点.	
173	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 30
	步骤描述: 不使用返回值的函数'shellExit'应使用'void'说明.	
	代码片段:	

	28: if (regcomp(®ex, reg, cflags) != 0) { 29: fprintf(stderr, "Fail to compile regex"); 30: shellExit(); 31: } 32:	
	污染路径: 函数'shellExit'的定义点.	
174	缺陷发生位置: /source/libs/new-stream/src/streamCheckpoint.c	行号: 14
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: 12: static int32_t getFilePath(char* filepath) { 13: if (snprintf(filepath, PATH_MAX, "%s%ssnode %scheckpoint", tsDataDir, TD_DIRSEP, TD_DIRSEP) < 0) { 14: stError("failed to generate checkpoint path for"); 15: return TSDB_CODE_FAILED; 16: }	
	污染路径: 函数'stError'的定义点.	
175	缺陷发生位置: /utlis/test/c/sml_test.c	行号: 182
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 180: taosMemoryFree(sql1[i]); 181: } 182: ASSERT(code == 0); 183: 184:	
	污染路径: 函数'ASSERT'的定义点.	
176	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 106
	步骤描述: 不使用返回值的函数'TAOS_STRNCAT'应使用'void'说明.	
	代码片段: 104: } 105: TAOS_STRNCAT(*ppKey, pCol->dbName, TSDB_DB_NAME_LEN);	

	106: TAOS_STRNCAT(*ppKey, ".", 2); 107: TAOS_STRNCAT(*ppKey, pCol->tableAlias, TSDB_TABLE_NAME_LEN); 108: TAOS_STRNCAT(*ppKey, ".", 2);	
	污染路径: 函数'TAOS_STRNCAT'的定义点.	
177	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable2.c	行号: 76
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	
	代码片段: 74: if (r == 0) { // name exists, check uid and type 75: int64_t uid = *(tb_uid_t *)value; 76: tdbFree(value); 77: 78: if (pReq->suid != uid) {	
	污染路径: 函数'tdbFree'的定义点.	
178	缺陷发生位置: /docs/examples/c/tmq_demo.c	行号: 74
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 72: char buf[200] = {0}; 73: i++; 74: snprintf(75: buf, sizeof(buf), 76: "INSERT INTO power.d1001 USING power.meters TAGS(2,'California.SanFrancisco') VALUES (NOW + %da, 10.30000, "	
	污染路径: 无	
179	缺陷发生位置: /source/os/src/osSocket.c	行号: 216
	步骤描述: 不使用返回值的函数'TAOS_SKIP_ERROR'应使用'void'说明.	
	代码片段: 214: reuse = 1; 215: if (taosSetSockOpt(pSocket, SOL_SOCKET, SO_REUSEADDR, (void *)&reuse, sizeof(reuse)) < 0) {	

	216: TAOS_SKIP_ERROR(taosCloseSocket(&pSocket)); 217: return 0; 218: }	
	污染路径: 函数'TAOS_SKIP_ERROR'的定义点.	
180	缺陷发生位置: /source/dnode/mnode/impl/src/mndGrant.c	行号: 46
	步骤描述: 不使用返回值的函数'STR_WITH_MAXSIZE_TO_VARSTR'应使用'void'说明.	
	代码片段: 44: SColumnInfoData *pColInfo = taosArrayGet(pBlock->pDataBlock, cols); 45: const char *src = TD_PRODUCT_NAME; 46: STR_WITH_MAXSIZE_TO_VARSTR(tmp, src, 32); 47: COL_DATA_SET_VAL_GOTO(tmp, false, NULL, NULL, _exit); 48:	
	污染路径: 函数'STR_WITH_MAXSIZE_TO_VARSTR'的定义点.	
181	缺陷发生位置: /source/client/wrapper/src/wrapperDriver.c	行号: 291
	步骤描述: 不使用返回值的函数'taosDriverCleanup'应使用'void'说明.	
	代码片段: 289: printf("failed to load function %s from %s since %s [0x%X]\r\n", funcName, driverPath, terrstr(), terrno); 290: showWrapperHint(terrno); 291: taosDriverCleanup(); 292: } 293:	
	污染路径: 函数'taosDriverCleanup'的定义点.	
182	缺陷发生位置: /source/dnode/vnode/src/meta/metaOpen.c	行号: 555
	步骤描述: 不使用返回值的函数'taosRemoveDir'应使用'void'说明.	
	代码片段: 553: } 554: 555: taosRemoveDir(tmpDir);	

	556: return 0; 557: }	
	污染路径: 函数'taosRemoveDir'的定义点.	
183	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttKeepalive.c	行号: 24
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 22: /* FIXME - this is the prototype for the future tree/trie based keepalive check implementation. */ 23: int ttqKeepaliveAdd(struct tmqtt *context) { 24: UNUSED(context); 25: 26: return TTQ_ERR_SUCCESS;	
	污染路径: 函数'UNUSED'的定义点.	
184	缺陷发生位置: /source/util/src/tlog.c	行号: 335
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 333: (void)taosCloseFile(&tsLogObj.slowHandle->pFile); 334: taosMemoryFreeClear(tsLogObj.slowHandle->buffer); 335: taosMemoryFreeClear(tsLogObj.slowHandle); 336: } 337:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
185	缺陷发生位置: /source/libs/index/src/indexFst.c	行号: 959
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 957: TAOS_UNUSED(taosDecodeFixedU32(buf + len, &checksum)); 958: if (taosCheckChecksum(buf, len, checksum)) { 959: indexError("index file is corrupted"); 960: // verify fst 961: return NULL;	

	污染路径: 函数'indexError'的定义点.	
186	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 790
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 788: TAOS_CHECK_RETURN(taosCloseFile(&pFile)); 789: tjsonDelete(json); 790: taosMemoryFree(pCont); 791: 792: mInfo("dump sdb info success");	
	污染路径: 函数'taosMemoryFree'的定义点.	
187	缺陷发生位置: /source/dnode/mgmt/mgmt_dnode/src/dmHandle.c	行号: 129
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 127: dDebug("disable time-white-list on dnode ver: %" PRId64 " , status ver: %" PRId64, pMgmt->pData->timeWhiteVer, ver); 128: if (rpcSetIpWhite(pMgmt->msgCb.serverRpc, NULL) != 0) { 129: dError("failed to disable time white list on dnode"); 130: } 131: }	
	污染路径: 函数'dError'的定义点.	
188	缺陷发生位置: /contrib/TSZ/sz/src/sz.c	行号: 192
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 190: { 191: totalCost = 0; 192: gettimeofday(&costStart, NULL); 193: } 194:	

	污染路径: 无	
189	缺陷发生位置: /source/libs/nodes/src/nodesMsgFuncs.c	行号: 85
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 83: } 84: 85: static void clearTlvEncoder(STlvEncoder* pEncoder) { taosMemoryFree(pEncoder->pBuf); } 86: 87: static void endTlvEncode(STlvEncoder* pEncoder, char** pMsg, int32_t* pLen) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
190	缺陷发生位置: /source/libs/catalog/src/ctgDbg.c	行号: 334
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 332: 333: if (NULL == param NULL == option) { 334: taosMemoryFree(dup); 335: CTG_RET(TSDB_CODE_INVALID_PARA); 336: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
191	缺陷发生位置: /utils/test/c/tmq_get_meta_json.c	行号: 138
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 136: int code = taos_errno(pRes); 137: printf("%s result0:%s\n", __FUNCTION__, taos_errstr(pRes)); 138: ASSERT(code == 0); 139: taos_free_result(pRes); 140:	
	污染路径:	

	函数'ASSERT'的定义点.	
192	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleSub.c	行号: 329
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 327: rc = packet__alloc(packet); 328: if (rc) { 329: ttq_free(packet); 330: return rc; 331: }	
	污染路径: 函数'ttq_free'的定义点.	
193	缺陷发生位置: /source/dnode/mnode/sdb/src/sdb.c	行号: 27
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 25: if (pSdb == NULL) { 26: terrno = TSDB_CODE_OUT_OF_MEMORY; 27: mError("failed to init sdb since %s", terrstr()); 28: return NULL; 29: }	
	污染路径: 函数'mError'的定义点.	
194	缺陷发生位置: /source/libs/new-stream/src/stream.c	行号: 64
	步骤描述: 不使用返回值的函数'stInfo'应使用'void'说明.	
	代码片段: 62: } 63: 64: stInfo("stream cleanup start"); 65: stTriggerTaskEnvCleanup(); 66: streamTimerCleanUp();	
	污染路径: 函数'stInfo'的定义点.	
195	缺陷发生位置: /source/libs/executor/src/exchangeoperator.c	行号: 453
	步骤描述:	

	不使用返回值的函数'qDebug'应使用'void'说明.	
	代码片段: 451: 452: if (taosArrayGetSize(pExchangeInfo->pResultBlockList) == 0) { 453: qDebug("empty resultBlockList"); 454: return NULL; 455: } else {	
196	污染路径: 函数'qDebug'的定义点.	
	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 285
	步骤描述: 不使用返回值的函数'ttqNetBrokerInit'应使用'void'说明.	
	代码片段: 283: 284: ttq_rand_init(); 285: ttqNetBrokerInit(); 286: 287: ttqConfigInit(config);	
197	污染路径: 函数'ttqNetBrokerInit'的定义点.	
	缺陷发生位置: /contrib/lemon/lemon.c	行号: 1078
	步骤描述: 不使用返回值的函数'State_insert'应使用'void'说明.	
	代码片段: 1076: stp->statenum = lemp->nstate++; /* Every state gets a sequence number */ 1077: stp->ap = 0; /* No actions, yet. */ 1078: State_insert(stp,stp->bp); /* Add to the state table */ 1079: buildshifts(lemp,stp); /* Recursively compute successor states */ 1080: }	
198	污染路径: 函数'State_insert'的定义点.	
	缺陷发生位置: /source/dnode/mnode/impl/src/mndScan.c	行号: 402
	步骤描述:	

	不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 400: int32_t ret = 0; 401: if ((ret = tSerializeSVKillScanReq((char *)pReq + sizeof(SMsgHead), contLen, &req)) < 0) { 402: taosMemoryFreeClear(pReq); 403: terrno = ret; 404: return NULL; </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
199	缺陷发生位置: /source/libs/new-stream/src/streamTriggerTask.c	行号: 2340
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: <pre> 2338: } 2339: if (infoBuf != NULL) { 2340: taosMemoryFreeClear(infoBuf); 2341: } 2342: if (code != TSDB_CODE_SUCCESS) { </pre>	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
200	缺陷发生位置: /source/libs/transport/src/transCli.c	行号: 967
	步骤描述: 不使用返回值的函数'TAOS_UNUSED'应使用'void'说明.	
	代码片段: <pre> 965: transGetRefCount(conn)); 966: conn->broken = true; 967: TAOS_UNUSED(transUnrefCliHandle(conn)); 968: } 969: return; </pre>	
	污染路径: 函数'TAOS_UNUSED'的定义点.	
201	缺陷发生位置: /source/libs/new-stream/src/dataSinkCache.c	行号: 58
	步骤描述: 不使用返回值的函数'stError'应使用'void'说明.	
	代码片段: <pre> 56: _end: </pre>	

	57: if (code != TSDB_CODE_SUCCESS) { 58: stError("failed to decode data since %s, lineno:%d", tstrerror(code), lino); 59: if (pBlock) { 60: blockDataDestroy(pBlock);	
	污染路径: 函数'stError'的定义点.	
202	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttSocket.c	行号: 325
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 323: 324: static void ttq_cleanup(void) { 325: UNUSED(ttqMuxCleanup()); 326: 327: ttq_log_stopping();	
	污染路径: 函数'UNUSED'的定义点.	
203	缺陷发生位置: /docs/examples/c-ws-new/sml_insert_demo.c	行号: 124
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 122: // close & clean 123: taos_close(taos); 124: taos_cleanup(); 125: return 0; 126: // ANCHOR_END: schemaless	
	污染路径: 函数'taos_cleanup'的定义点.	
204	缺陷发生位置: /docs/examples/c-ws-new/async_demo.c	行号: 54
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 52: taos_free_result(pSql); 53: taos_close(taos); 54: taos_cleanup(); 55: exit(EXIT_FAILURE);	

	函数'smUndeployAllTasks'的定义点.	
208	缺陷发生位置: /source/os/src/osSocket.c	行号: 251
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 249: int32_t taosGetIpv4FromFqdn(const char *fqdn, uint32_t *ip) { 250: int32_t code = 0; 251: OS_PARAM_CHECK(fqdn); 252: OS_PARAM_CHECK(ip); 253: int64_t limitMs = 1000;	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
209	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttDaemon.c	行号: 195
	步骤描述: 不使用返回值的函数'TAOS_MQTT_MGMT_CHECK_PTR_RCODE'应使用'void'说明.	
	代码片段: 193: 194: int32_t mqttFillMqttInfoFromMNode(void *clientRpc, char *mqttName, SMqtt *mqtt) { 195: TAOS_MQTT_MGMT_CHECK_PTR_RCODE(clientRpc, mqttName, mqtt); 196: SRetrieveFuncReq retrieveReq = {0}; 197: retrieveReq.numOfFuncs = 1;	
	污染路径: 函数'TAOS_MQTT_MGMT_CHECK_PTR_RCODE'的定义点.	
210	缺陷发生位置: /source/os/src/osDir.c	行号: 354
	步骤描述: 不使用返回值的函数'TAOS_UNUSED'应使用'void'说明.	
	代码片段: 352: 353: TAOS_UNUSED(taosCloseDir(&pDir)); 354: TAOS_UNUSED(rmdir(dirname)); 355: } 356:	
	污染路径:	

	函数'TAOS_UNUSED'的定义点.	
211	缺陷发生位置: /docs/examples/c/query_example.c	行号: 139
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 137: taos_free_result(res); 138: taos_close(taos); 139: taos_cleanup(); 140: } 141:	
	污染路径: 函数'taos_cleanup'的定义点.	
212	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 127
	步骤描述: 不使用返回值的函数'streamCleanup'应使用'void'说明.	
	代码片段: 125: dmClearVars(pDnode); 126: rpcCleanup(); 127: streamCleanup(); 128: indexCleanup(); 129: taosConvDestroy();	
	污染路径: 函数'streamCleanup'的定义点.	
213	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 2527
	步骤描述: 不使用返回值的函数'qDebugL'应使用'void'说明.	
	代码片段: 2525: if (TSDB_CODE_SUCCESS == code) { 2526: if (NULL == pRuleName) { 2527: qDebugL("before split, JsonPlan: %s", pStr); 2528: } else { 2529: qDebugL("apply split %s rule, JsonPlan: %s", pRuleName, pStr);	
	污染路径: 函数'qDebugL'的定义点.	
214	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncMessageDebug.c	行号: 233

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 231: printf("syncPingPrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 232: fflush(NULL); 233: taosMemoryFree(serialized); 234: } 235:	
	污染路径: 函数'taosMemoryFree'的定义点.	
215	缺陷发生位置:	行号:
	/tools/shell/src/shellAuto.c	1518
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 1516: if (p) { 1517: tstrncpy(p, str, strLen + 1); 1518: tstrncpy(p + strLen, ";", 1 + 1); 1519: lastWordBytes += 1; 1520: shellInsertStr(cmd, (char*)p, strLen + 1);	
	污染路径: 函数'tstrncpy'的定义点.	
216	缺陷发生位置:	行号:
	/source/libs/transport/src/trans.c	201
	步骤描述: 不使用返回值的函数'tlInfo'应使用'void'说明.	
	代码片段: 199: transRemoveExHandle(transGetInstMgt(), (int64_t)arg); 200: transReleaseExHandle(transGetInstMgt(), (int64_t)arg); 201: tlInfo("end to close rpc"); 202: return; 203: }	
	污染路径: 函数'tlInfo'的定义点.	
217	缺陷发生位置:	行号:
	/source/dnode/mgmt/mgmt_vnode/src/vmHandle.c	164
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段:	

	<pre> 162: r = taos_counter_get_vgroup_ids(tslInsertCounter, &keys, &vgroup_ids, &list_size); 163: if (r) { 164: dError("failed to get vgroup ids"); 165: return; 166: }</pre>	
	污染路径: 函数'dError'的定义点.	
218	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1093
	步骤描述: 不使用返回值的函数'argp_fmtstream_set_wmargin'应使用'void'说明.	
	代码片段: <pre> 1091: indent_to (pest->stream, uparams.header_col); 1092: argp_fmtstream_set_lmargin (pest->stream, uparams.header_col); 1093: argp_fmtstream_set_wmargin (pest->stream, uparams.header_col); 1094: argp_fmtstream_puts (pest->stream, fstr); 1095: argp_fmtstream_set_lmargin (pest->stream, 0);</pre>	
	污染路径: 函数'argp_fmtstream_set_wmargin'的定义点.	
219	缺陷发生位置: /source/libs/metrics/src/metrics.c	行号: 77
	步骤描述: 不使用返回值的函数'taos_collector_registry_default_init'应使用'void'说明.	
	代码片段: <pre> 75: 76: int32_t initMetricsManager() { 77: taos_collector_registry_default_init(); 78: 79: // Initialize global shared write metrics counters</pre>	
	污染路径: 函数'taos_collector_registry_default_init'的定义点.	
220	缺陷发生位置: /source/common/src/tglobal.c	行号: 2318
	步骤描述: 不使用返回值的函数'TAOS_CHECK_RETURN'应使用'void'说明.	
	代码片段:	

	2316: 2317: SConfig *pCfg = NULL; 2318: TAOS_CHECK_RETURN(cfgInit(&pCfg)); 2319: TAOS_CHECK_EXIT(2320: cfgAddDir(pCfg, "dataDir", tsDataDir, CFG_SCOPE_SERVER, CFG_DYN_SERVER, CFG_CATEGORY_LOCAL, CFG_PRIV_SYSTEM));	
	污染路径: 函数'TAOS_CHECK_RETURN'的定义点.	
221	缺陷发生位置: /source/libs/planner/src/planPhysiCreator.c	行号: 92
	步骤描述: 不使用返回值的函数'TAOS_STRNCAT'应使用'void'说明.	
	代码片段: 90: } 91: TAOS_STRNCAT(*ppKey, pCol->refDbName, TSDB_DB_NAME_LEN); 92: TAOS_STRNCAT(*ppKey, ".", 2); 93: TAOS_STRNCAT(*ppKey, pCol->refTableName, TSDB_TABLE_NAME_LEN); 94: TAOS_STRNCAT(*ppKey, ".", 2);	
	污染路径: 函数'TAOS_STRNCAT'的定义点.	
222	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 212
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 210: 211: if (!doValidateSchema(&pSchema[numOfCols], numOfTags, TSDB_MAX_TAGS_LEN)) { 212: qError("validate schema tags failed"); 213: return false; 214: }	
	污染路径: 函数'qError'的定义点.	
223	缺陷发生位置: /source/libs/tdb/test/tdbExOVFLTest.cpp	行号: 397
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	

	代码片段: 395: for (int iData = 1; iData <= nData; iData++) { 396: sprintf(key, "key0"); 397: sprintf(val, "value%d", iData); 398: 399: // ret = tdbTbInsert(pDb, key, strlen(key), val, strlen(val), &txn);	
	污染路径: 无	
224	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmlnt.c	行号: 124
	步骤描述: 不使用返回值的函数'dlInfo'应使用'void'说明.	
	代码片段: 122: mmBuildOptionForDeploy(pMgmt, pInput, &option); 123: } else { 124: dlInfo("mnode start to open"); 125: mmBuildOptionForOpen(pMgmt, &option); 126: }	
	污染路径: 函数'dlInfo'的定义点.	
225	缺陷发生位置: /source/util/src/tsched.c	行号: 46
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 44: pSched->queue = (SSchedMsg *)taosMemoryCalloc(sizeof(SSchedMsg), queueSize); 45: if (pSched->queue == NULL) { 46: uError("%s: no enough memory for queue", label); 47: taosCleanUpScheduler(pSched); 48: if (schedMalloted) {	
	污染路径: 函数'uError'的定义点.	
226	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 134
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 132: }	

	133: shell.exit = true; 134: taos_cleanup(); 135: } 136:	
	污染路径: 函数'taos_cleanup'的定义点.	
227	缺陷发生位置: /source/libs/qcom/src/querymsg.c	行号: 144
	步骤描述: 不使用返回值的函数'QUERY_PARAM_CHECK'应使用'void'说明.	
	代码片段: 142: int32_t queryBuildQnodeListMsg(void *input, char **msg, int32_t msgSize, int32_t *msgLen, void *(*mallocFp)(int64_t), 143: void (*freeFp)(void *)) { 144: QUERY_PARAM_CHECK(msg); 145: QUERY_PARAM_CHECK(msgLen); 146:	
	污染路径: 函数'QUERY_PARAM_CHECK'的定义点.	
228	缺陷发生位置: /source/libs/monitorfw/src/taos_metric.c	行号: 177
	步骤描述: 不使用返回值的函数'taos_free'应使用'void'说明.	
	代码片段: 175: r = taos_map_set(self->samples, l_value, sample); 176: if (r) { 177: taos_free((void *)l_value); 178: TAOS_METRIC_SAMPLE_FROM_LABELS_HANDLE_UNLOCK(); 179: }	
	污染路径: 函数'taos_free'的定义点.	
229	缺陷发生位置: /source/libs/executor/src/tlinearhash.c	行号: 341
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 339: } 340: 341: taosMemoryFreeClear(pHashObj->pBucket);	

	342: taosMemoryFreeClear(pHashObj); 343: return NULL;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
230	缺陷发生位置: /source/libs/sync/src/syncMain.c	行号: 558
	步骤描述: 不使用返回值的函数'sError'应使用'void'说明.	
	代码片段: 556: if (pSyncNode == NULL) { 557: errno = TSDB_CODE_SYN_INTERNAL_ERROR; 558: sError("sync ready for read error"); 559: return false; 560: }	
	污染路径: 函数'sError'的定义点.	
231	缺陷发生位置: /source/dnode/mnode/impl/src/mndScan.c	行号: 62
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 60: } 61: 62: void mndCleanupScan(SMnode *pMnode) { mDebug("mnd scan cleanup"); } 63: 64: void tFreeScanObj(SScanObj *pScan) {}	
	污染路径: 函数'mDebug'的定义点.	
232	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeOpen.c	行号: 616
	步骤描述: 不使用返回值的函数'vInfo'应使用'void'说明.	
	代码片段: 614: void vnodeClose(SVnode *pVnode) { 615: if (pVnode) { 616: vInfo("start to close vnode"); 617: vnodeAWait(&pVnode->commitTask2); 618: vnodeAWait(&pVnode->commitTask);	

	污染路径: 函数'vInfo'的定义点.	
233	缺陷发生位置: /utils/test/c/tmq_write_raw_with_blob_test.c	行号: 384
	步骤描述: 不使用返回值的函数'create_topic'应使用'void'说明.	
	代码片段: 382: return -1; 383: } 384: create_topic(); 385: 386: tmq_t* tmq = build_consumer();	
	污染路径: 函数'create_topic'的定义点.	
234	缺陷发生位置: /source/dnode/mnode/impl/src/mndCompactDetail.c	行号: 38
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
	代码片段: 36: } 37: 38: void mndCleanupCompactDetail(SMnode *pMnode) { mDebug("mnd compact detail cleanup"); } 39: 40: int32_t mndRetrieveCompactDetail(SRpcMsg *pReq, SShowObj *pShow, SSDataBlock *pBlock, int32_t rows) {	
	污染路径: 函数'mDebug'的定义点.	
235	缺陷发生位置: /docs/examples/c/taos_connect_with_example.c	行号: 30
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 28: 29: taos_close(taos); 30: taos_cleanup(); 31: }	
	污染路径: 函数'taos_cleanup'的定义点.	
236	缺陷发生位置:	行号:

	/test/new_test_framework/script/api/stmt2-geometry-test.c	106
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 104: 105: taos_close(taos); 106: taos_cleanup(); 107: }	
	污染路径: 函数'taos_cleanup'的定义点.	
237	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 1257
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 1255: 1256: if (pInfo->base.dataReader != NULL) { 1257: qError("%s tsdb reader should be null", idStr); 1258: return TSDB_CODE_QRY_EXECUTOR_INTERNAL_ERROR; 1259: }	
	污染路径: 函数'qError'的定义点.	
238	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqSessionExpiry.c	行号: 87
	步骤描述: 不使用返回值的函数'DL_INSERT_INORDER'应使用'void'说明.	
	代码片段: 85: context->expiry_list_item = item; 86: 87: DL_INSERT_INORDER(expiry_list, item, session_expiry__cmp); 88: 89: return TTQ_ERR_SUCCESS;	
	污染路径: 函数'DL_INSERT_INORDER'的定义点.	
239	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 71
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	

	代码片段: 69: } 70: if (p) { 71: taosMemoryFreeClear(p); 72: } 73: return code;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
240	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqLoop.c	行号: 142
	步骤描述: 不使用返回值的函数'ttqCxtFreeDisused'应使用'void'说明.	
	代码片段: 140: while (run) { 141: queue_plugin_msgs(); 142: ttqCxtFreeDisused(); 143: ttqKeepaliveCheck(); 144:	
	污染路径: 函数'ttqCxtFreeDisused'的定义点.	
241	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFile2.c	行号: 291
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 289: } 290: (void)taosThreadMutexDestroy(&fobj->mutex); 291: taosMemoryFree(fobj); 292: } 293:	
	污染路径: 函数'taosMemoryFree'的定义点.	
242	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeQuery.c	行号: 581
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 579: if (tDeserializeSBatchReq(pMsg->pCont, pMsg->contLen, &batchReq)) { 580: code = terrno;	

	581: qError("tDeserializeSBatchReq failed"); 582: goto _exit; 583: }	
	污染路径: 函数'qError'的定义点.	
243	缺陷发生位置: /source/libs/index/src/indexTfile.c	行号: 434
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 432: ret = tfileReaderLoadTableIds((TFileReader*)reader, offset, tr->total); 433: if (ret != 0) { 434: indexError("failed to search json since %s", tstrerror(ret)); 435: taosMemoryFree(p); 436: fstSliceDestroy(&key);	
	污染路径: 函数'indexError'的定义点.	
244	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 177
	步骤描述: 不使用返回值的函数'taosSeedRand'应使用'void'说明.	
	代码片段: 175: 176: int main(int argc, char **argv) { 177: taosSeedRand(time(NULL)); 178: 179: exe_name = argv[0];	
	污染路径: 函数'taosSeedRand'的定义点.	
245	缺陷发生位置: /contrib/test/sqlite/sqliteTest.c	行号: 69
	步骤描述: 不使用返回值的函数'sqlite3_free'应使用'void'说明.	
	代码片段: 67: fprintf(stderr, "SQL error: %s\n", err_msg); 68: 69: sqlite3_free(err_msg); 70: sqlite3_close(db);	

	71:	
	污染路径: 函数'sqlite3_free'的定义点.	
246	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 272
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 270: printf("done\n"); 271: taos_close(taos); 272: taos_cleanup(); 273: }	
	污染路径: 函数'taos_cleanup'的定义点.	
247	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 207
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 205: 206: if (!doValidateSchema(pSchema, numOfCols, isVirtual ? TSDB_MAX_BYTES_PER_ROW_VIRTUAL : TSDB_MAX_BYTES_PER_ROW)) { 207: qError("validate schema columns failed"); 208: return false; 209: }	
	污染路径: 函数'qError'的定义点.	
248	缺陷发生位置: /source/client/src/clientEnv.c	行号: 329
	步骤描述: 不使用返回值的函数'rpcClose'应使用'void'说明.	
	代码片段: 327: 328: tscDebug("free transporter:%p in app inst %p", pTransporter, pAppInfo); 329: rpcClose(pTransporter); 330: } 331:	

	污染路径: 函数'rpcClose'的定义点.	
249	缺陷发生位置: /utils/test/c/tmq_ts5466.c	行号: 49
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 47: char* result = tmq_get_json_meta(msg); 48: printf("meta result: %s\n", result); 49: tmq_free_json_meta(result); 50: } 51:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	
250	缺陷发生位置: /source/libs/sync/src/syncEnv.c	行号: 41
	步骤描述: 不使用返回值的函数'syncCleanUp'应使用'void'说明.	
	代码片段: 39: if (gNodeRefId < 0) { 40: sError("failed to init node rset"); 41: syncCleanUp(); 42: return TSDB_CODE_SYN_WRONG_REF; 43: }	
	污染路径: 函数'syncCleanUp'的定义点.	
251	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 67
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 65: { 66: static char version[40]; 67: snprintf(version, sizeof(version), "%i.%i.%i", CJSON_VERSION_MAJOR, CJSON_VERSION_MINOR, CJSON_VERSION_PATCH); 68: 69: return version;	
	污染路径: 无	

252	缺陷发生位置: /source/dnode/mgmt/mgmt_qnode/src/qmlnt.c	行号: 64
	步骤描述: 不使用返回值的函数'tmsgReportStartup'应使用'void'说明.	
	代码片段: 62: return code; 63: } 64: tmsgReportStartup("qnode-impl", "initialized"); 65: 66: if ((code = udfcOpen()) != 0) {	
	污染路径: 函数'tmsgReportStartup'的定义点.	
253	缺陷发生位置: /source/util/src/tworker.c	行号: 257
	步骤描述: 不使用返回值的函数'closeThreadNotificationConn'应使用'void'说明.	
	代码片段: 255: } 256: DestoryThreadLocalRegComp(); 257: closeThreadNotificationConn(); 258: 259: return NULL;	
	污染路径: 函数'closeThreadNotificationConn'的定义点.	
254	缺陷发生位置: /source/client/src/clientMonitorSql.c	行号: 70
	步骤描述: 不使用返回值的函数'tscLog'应使用'void'说明.	
	代码片段: 68: (void)releaseTscObj(rid); 69: } else { 70: tscLog("sqlReqLog, not found rid"); 71: } 72: }	
	污染路径: 函数'tscLog'的定义点.	
255	缺陷发生位置: /source/libs/executor/src/mergejoinoperator.c	行号: 272
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	

	代码片段: 270: pBlock->info.rows = maxRows; 271: if (pBitmap != NULL) { 272: taosMemoryFree(pBitmap); 273: } 274:	
	污染路径: 函数'taosMemoryFree'的定义点.	
256	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshotRAW.c	行号: 211
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 209: _exit: 210: if (code) { 211: taosMemoryFree(pBuf); 212: pBuf = NULL; 213: TSDB_ERROR_LOG(TD_VID(reader->tsdb->pVnode), code, lino);	
	污染路径: 函数'taosMemoryFree'的定义点.	
257	缺陷发生位置: /tools/shell/src/shellMain.c	行号: 148
	步骤描述: 不使用返回值的函数'shellAutoExit'应使用'void'说明.	
	代码片段: 146: shellAutoInit(); 147: int32_t ret = shellExecute(argc, argv); 148: shellAutoExit(); 149: 150: return ret;	
	污染路径: 函数'shellAutoExit'的定义点.	
258	缺陷发生位置: /source/libs/executor/src/sortoperator.c	行号: 598
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 596: if (pCalc) { 597: taosArrayDestroy(pCalc->pSortColsArr);	

	598: taosMemoryFree(pCalc->keyBuf); 599: taosMemoryFree(pCalc); 600: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
259	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 58
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 56: static int32_t sdbUpgradeData(SSdb *pSdb, int32_t version) { 57: int32_t code = 0; 58: mInfo("start to upgrade sdb"); 59: 60: for (int32_t i = SDB_MAX - 1; i >= 0; --i) {	
	污染路径: 函数'mInfo'的定义点.	
260	缺陷发生位置: /source/libs/catalog/src/ctgDbg.c	行号: 342
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 340: int32_t code = ctgdEnableDebug(option, enable); 341: 342: taosMemoryFree(dup); 343: 344: CTG_RET(code);	
	污染路径: 函数'taosMemoryFree'的定义点.	
261	缺陷发生位置: /source/libs/planner/src/planner.c	行号: 25
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 23: if (TSDB_CODE_SUCCESS == code) { 24: (void)printf("%s\n", pStr); 25: taosMemoryFree(pStr); 26: } 27: return code;	

	污染路径: 函数'taosMemoryFree'的定义点.	
262	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbHash.c	行号: 209
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 207: if (code != 0) { 208: if (taosHashRemove(hash, pRow->pObj, keySize) != 0) { 209: mError("failed to remove row from hash"); 210: } 211: sdbFreeRow(pSdb, pRow, false);	
	污染路径: 函数'mError'的定义点.	
263	缺陷发生位置: /source/libs/monitorfw/src/taos_metric.c	行号: 141
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: 139: taos_metric_t *self = (taos_metric_t *)item; 140: if (taos_metric_destroy(self) != 0) { 141: TAOS_LOG("taos_metric_destroy failed"); 142: } 143: }	
	污染路径: 函数'TAOS_LOG'的定义点.	
264	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp	行号: 510
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 508: 509: for (int iData = 0; iData < nData; ++iData) { 510: sprintf(key, "key%d", iData); 511: sprintf(val, "value%d", iData); 512:	
	污染路径: 无	

265	缺陷发生位置： /source/util/src/tcompare.c	行号： 1342
	步骤描述： 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段： 1340: bool stopped = taosTmrReset(checkRegexCache, REGEX_CACHE_CLEAR_TIME * 1000, param, sRegexCache.regexCacheTmr, &tmrId); 1341: if (stopped) { 1342: uError("failed to reset regex cache timer"); 1343: goto _exit; 1344: }	
	污染路径： 函数'uError'的定义点.	
266	缺陷发生位置： /test/new_test_framework/script/api/stmt-insert-dupkeys.c	行号： 216
	步骤描述： 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段： 214: 215: taos_close(taos); 216: taos_cleanup(); 217: 218: // final results	
	污染路径： 函数'taos_cleanup'的定义点.	
267	缺陷发生位置： /docs/examples/c/create_db_demo.c	行号： 38
	步骤描述： 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段： 36: fprintf(stderr, "Failed to connect to %s:%hu, ErrCode: 0x %x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 37: taos_errstr(NULL)); 38: taos_cleanup(); 39: return -1; 40: }	
	污染路径： 函数'taos_cleanup'的定义点.	
268	缺陷发生位置：	行号：

	/source/client/src/clientTmq.c	340
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 338: } 339: if (conf->pass) { 340: taosMemoryFree(conf->pass); 341: } 342: if (conf->token) { </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
269	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 130
	步骤描述: 不使用返回值的函数'raft_free'应使用'void'说明.	
	代码片段: <pre> 128: 129: void putValueCb(struct raft_apply *req, int status, void *result) { 130: raft_free(req); 131: struct raft *r = req->data; 132: if (status != 0) { </pre>	
	污染路径: 函数'raft_free'的定义点.	
270	缺陷发生位置: /source/dnode/mnode/impl/src/mndConfig.c	行号: 314
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 312: STrans *pTrans = mndTransCreate(pMnode, TRN_POLICY_RETRY, TRN_CONFLICT_NOHING, NULL, "init- write-config"); 313: if (pTrans == NULL) { 314: mError("failed to init write cfg in create trans, since %s", terrstr()); 315: goto _OVER; 316: } </pre>	
	污染路径: 函数'mError'的定义点.	
271	缺陷发生位置:	行号:

	/source/dnode/mnode/impl/src/mndIndex.c	124
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 122: action.msgType = TDMT_VND_CREATE_INDEX; 123: if ((code = mndTransAppendRedoAction(pTrans, &action)) != 0) { 124: taosMemoryFree(pReq); 125: sdbCancelFetch(pSdb, plter); 126: sdbRelease(pSdb, pVgroup);	
272	污染路径: 函数'taosMemoryFree'的定义点.	
	缺陷发生位置: /examples/c/demo.c	行号: 73
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
273	代码片段: 71: } 72: taos_close(taos); 73: taos_cleanup(); 74: } 75: void Test(TAOS *taos, char *qstr, int index) {	
	污染路径: 函数'taos_cleanup'的定义点.	
	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 77
274	步骤描述: 不使用返回值的函数'fnError'应使用'void'说明.	
	代码片段: 75: ret = doCallUdfScalarFunc(handle, &input, 1, &output); 76: if (ret != 0) { 77: fnError("failed to call udf scalar function"); 78: return -1; 79: }	
	污染路径: 函数'fnError'的定义点.	
274	缺陷发生位置: /source/libs/executor/src/operator.c	行号: 970
	步骤描述: 不使用返回值的函数'recordOpExecBeforeDownstream'应使用'void'说明.	

	<pre>%x, ErrorMessage: %s.\n", host, port, taos_errno(NULL), 19: taos_errstr(NULL)); 20: taos_cleanup(); 21: return -1; 22: }</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
281	缺陷发生位置: /source/libs/command/src/explain.c	行号: 287
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: <pre>285: SExplainResNode *resNode = taosMemoryCalloc(1, sizeof(SExplainResNode)); 286: if (NULL == resNode) { 287: qError("calloc SPhysiNodeExplainRes failed"); 288: return terno; 289: }</pre>	
	污染路径: 函数'qError'的定义点.	
282	缺陷发生位置: /source/libs/tfs/src/tfs.c	行号: 319
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre>317: fError("%s failed at line %d since %s", __func__, lino, tstrerror(code)); 318: } 319: taosMemoryFree(s); 320: taosMemoryFree(dir); 321: TAOS_RETURN(code);</pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
283	缺陷发生位置: /source/libs/txnnode/src/txnnodeMgmt.c	行号: 86
	步骤描述: 不使用返回值的函数'xndError'应使用'void'说明.	
	代码片段: <pre>84: pData->isStopped); 85: if (pData == NULL) {</pre>	

	86: xndError("xnoded process data is NULL"); 87: return; 88: }	
	污染路径: 函数'xndError'的定义点.	
284	缺陷发生位置: /source/dnode/mnode/impl/src/mndMnode.c	行号: 391
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 389: code = tSerializeSDCreateMnodeReq(pReq, contLen, pAlterMnodeTypeReq); 390: if (code < 0) { 391: taosMemoryFree(pReq); 392: TAOS_RETURN(code); 393: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
285	缺陷发生位置: /source/libs/monitorfw/src/taos_collector_registry.c	行号: 139
	步骤描述: 不使用返回值的函数'TAOS_TEST_PARA_NULL'应使用'void'说明.	
	代码片段: 137: 138: taos_metric_t *taos_collector_registry_get_metric(char* metric_name){ 139: TAOS_TEST_PARA_NULL(metric_name != NULL); 140: 141: taos_collector_t *default_collector =	
	污染路径: 函数'TAOS_TEST_PARA_NULL'的定义点.	
286	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 445
	步骤描述: 不使用返回值的函数'mFatal'应使用'void'说明.	
	代码片段: 443: } else { 444: terrno = TSDB_CODE_APP_ERROR; 445: mFatal("db:%s, failed to acquire db since %s", db, terrstr());	

	446: } 447: }	
	污染路径: 函数'mFatal'的定义点.	
287	缺陷发生位置: /source/common/src/tglobal.c	行号: 2571
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 2569: uError("failed to read stype config file:%s since %s", filename, tstrerror(code)); 2570: } 2571: taosMemoryFree(buf); 2572: TAOS_RETURN(code); 2573: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
288	缺陷发生位置: /source/libs/index/src/indexFilter.c	行号: 125
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 123: 124: taosArrayDestroy(param->result); 125: taosMemoryFree(param->condValue); 126: param->condValue = NULL; 127: taosHashCleanup(param->pFilter);	
	污染路径: 函数'taosMemoryFree'的定义点.	
289	缺陷发生位置: /source/dnode/mnode/impl/src/mndSync.c	行号: 317
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 315: } 316: } else { 317: mInfo("vgld:1, sync restore finished"); 318: } 319: int32_t code = mndRefreshUserIpWhiteList(pMnode);	

	污染路径: 函数'mInfo'的定义点.	
290	缺陷发生位置: /source/client/test/session.cpp	行号: 451
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 449: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 450: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c", 0xE5, 0x9B, 0xBD); 451: snprintf(zhongguoGbk, sizeof(zhongguoGbk), "%c%c%c%c", 0xD6, 0xD0, 0xB9, 0xFA); 452: snprintf(zhongGbk, sizeof(zhongGbk), "%c%c", 0xD6, 0xD0); 453:	
	污染路径: 无	
291	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 290
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 288: 289: seclen = (int)strlen(s); 290: sprintf(keym, "%s:", s); 291: 292: for (j=0 ; j<d->size ; j++) {	
	污染路径: 函数'sprintf'的定义点.	
292	缺陷发生位置: /source/libs/new-stream/src/stream.c	行号: 65
	步骤描述: 不使用返回值的函数'stTriggerTaskEnvCleanup'应使用'void'说明.	
	代码片段: 63: 64: stInfo("stream cleanup start"); 65: stTriggerTaskEnvCleanup(); 66: streamTimerCleanUp(); 67: smUndeployAllTasks();	

	污染路径: 函数'stTriggerTaskEnvCleanup'的定义点.	
293	缺陷发生位置: /source/client/src/clientEnv.c	行号: 245
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	
	代码片段: 243: static void deregisterRequest(SRequestObj *pRequest) { 244: if (pRequest == NULL) { 245: tscError("pRequest == NULL"); 246: return; 247: }	
	污染路径: 函数'tscError'的定义点.	
294	缺陷发生位置: /source/client/src/clientTmq.c	行号: 337
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 335: } 336: if (conf->user) { 337: taosMemoryFree(conf->user); 338: } 339: if (conf->pass) {	
	污染路径: 函数'taosMemoryFree'的定义点.	
295	缺陷发生位置: /source/libs/new-stream/src/streamUtil.c	行号: 422
	步骤描述: 不使用返回值的函数'cJSON_free'应使用'void'说明.	
	代码片段: 420: } 421: if (temp) { 422: cJSON_free(temp); 423: } 424: return code;	
	污染路径: 函数'cJSON_free'的定义点.	
296	缺陷发生位置: /source/libs/command/src/explain.c	行号: 185

	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 183: SExplainCtx *ctx = taosMemoryCalloc(1, sizeof(SExplainCtx)); 184: if (NULL == ctx) { 185: qError("calloc SExplainCtx failed"); 186: QRY_ERR_JRET(terrno); 187: }	
	污染路径: 函数'qError'的定义点.	
297	缺陷发生位置: /utils/test/c/tmq_write_raw_test.c	行号: 372
	步骤描述: 不使用返回值的函数'create_topic'应使用'void'说明.	
	代码片段: 370: return -1; 371: } 372: create_topic(); 373: 374: tmq_t* tmq = build_consumer();	
	污染路径: 函数'create_topic'的定义点.	
298	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 1283
	步骤描述: 不使用返回值的函数'pthread_join'应使用'void'说明.	
	代码片段: 1281: 1282: infoPrint("pthread_join %zu ...\\n", i); 1283: pthread_join(pids[i], NULL); 1284: } 1285:	
	污染路径: 无	
299	缺陷发生位置: /source/libs/qworker/src/qwMem.c	行号: 124
	步骤描述: 不使用返回值的函数'QW_ERR_RET'应使用'void'说明.	
	代码片段:	

	122: 123: if (NULL == gQueryMgmt.pJobInfo) { 124: QW_ERR_RET(qwInitJobHash()); 125: } 126:	
	污染路径: 函数'QW_ERR_RET'的定义点.	
300	缺陷发生位置: /source/dnode/mnode/impl/src/mndStb.c	行号: 1530
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 1528: alterReq.pFields = taosArrayInit(2, sizeof(SField)); 1529: if (NULL == alterReq.pFields) { 1530: mError("failed to init alter fields array"); 1531: goto END; 1532: }	
	污染路径: 函数'mError'的定义点.	
301	缺陷发生位置: /source/client/test/session.cpp	行号: 448
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 446: char zhongguoGbk[32] = {0}; 447: snprintf(fenUtf8, sizeof(fenUtf8), "%c%c%c", 0xE8, 0x8A, 0xAC); 448: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 449: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c %c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 450: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c", 0xE5, 0x9B, 0xBD);	
	污染路径: 无	
302	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 147
	步骤描述: 不使用返回值的函数'ctgDebug'应使用'void'说明.	
	代码片段: 145: *pCache = NULL;	

	146: CTG_CACHE_NHIT_INC(CTG_CI_DB, 1); 147: ctgDebug("db:%s, db is removing from cache", dbFName); 148: return TSDB_CODE_SUCCESS; 149: }	
	污染路径: 函数'ctgDebug'的定义点.	
303	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 602
	步骤描述: 不使用返回值的函数'sscanf'应使用'void'说明.	
	代码片段: 600: } else if (line[0]=='[' && line[len-1]==']') { 601: /* Section name */ 602: sscanf(line, "[%^]", section); 603: strcpy(section, stripslashes(section)); 604: strcpy(section, strlwc(section));	
	污染路径: 函数'sscanf'的定义点.	
304	缺陷发生位置: /utlis/test/c/tmqDemo.c	行号: 133
	步骤描述: 不使用返回值的函数'tstrncpy'应使用'void'说明.	
	代码片段: 131: exit(0); 132: } else if (strcmp(argv[i], "-d") == 0) { 133: tstrncpy(g_stConflInfo.dbName, argv[++i], sizeof(g_stConflInfo.dbName)); 134: } else if (strcmp(argv[i], "-c") == 0) { 135: tstrncpy(configDir, argv[++i], PATH_MAX);	
	污染路径: 函数'tstrncpy'的定义点.	
305	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 129
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 127: printf("logStorePrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 128: fflush(NULL);	

	129: taosMemoryFree(serialized); 130: } 131:	
	污染路径: 函数'taosMemoryFree'的定义点.	
306	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 75
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 73: taos_free_result(result); 74: // super tables subscription 75: usleep(1000000); 76: } 77:	
	污染路径: 无	
307	缺陷发生位置: /source/client/wrapper/jni/windows/win32/bridge/AccessBridgeCalls.c	行号: 74
	步骤描述: 不使用返回值的函数'PrintDebugString'应使用'void'说明.	
	代码片段: 72: #endif 73: if (theAccessBridgeInstance != 0) { 74: LOAD_FP(Windows_run, Windows_runFP, "Windows_run"); 75: 76: LOAD_FP(SetJavaShutdown, SetJavaShutdownFP, "setJavaShutdownFP");	
	污染路径: expanded from macro 'LOAD_FP' 函数'PrintDebugString'的定义点. expanded from macro 'LOAD_FP'	
308	缺陷发生位置: /source/util/src/tlruccache.c	行号: 711
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 709: if (TSDB_CODE_SUCCESS != 710: taosLRUCacheShardInit(&cache->shards[i],	

	perShard, strictCapacity, highPriPoolRatio, 32 - numShardBits)) { 711: taosMemoryFree(cache->shards); 712: taosMemoryFree(cache); 713: return NULL;	
	污染路径: 函数'taosMemoryFree'的定义点.	
309	缺陷发生位置: /source/libs/scalar/src/scalar.c	行号: 160
	步骤描述: 不使用返回值的函数'sclError'应使用'void'说明.	
	代码片段: 158: pObj = taosHashInit(rows, taosGetDefaultHashFunction(type), true, false); 159: if (NULL == pObj) { 160: sclError("taosHashInit failed, size:%d", 256); 161: SCL_ERR_RET(terrno); 162: }	
	污染路径: 函数'sclError'的定义点.	
310	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 333
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 331: // char const *key = "key123456789"; 332: char key[32] = {0}; 333: sprintf(key, "key-%d", i); 334: ret = tdbTbInsert(pDb, key, strlen(key) + 1, val, valLen, txn); 335: GTEST_ASSERT_EQ(ret, 0);	
	污染路径: 无	
311	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqLoop.c	行号: 143
	步骤描述: 不使用返回值的函数'ttqKeepaliveCheck'应使用'void'说明.	
	代码片段: 141: queue_plugin_msgs(); 142: ttqCxtFreeDisused();	

	143: ttqKeepaliveCheck(); 144: 145: rc = ttqMuxHandle(listensock, listensock_count);	
	污染路径: 函数'ttqKeepaliveCheck'的定义点.	
312	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 23
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 21: printf("syncIndexMgrPrint len:%" PRIu64 " %s \n", (uint64_t)strlen(serialized), serialized); 22: fflush(NULL); 23: taosMemoryFree(serialized); 24: } 25:	
	污染路径: 函数'taosMemoryFree'的定义点.	
313	缺陷发生位置: /contrib/libmqt/ttq/src/ttqHandleConnect.c	行号: 74
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 72: ttqDbMsgStoreRefDec(&msg_tail->store); 73: tmqtt_property_free_all(&msg_tail->properties); 74: ttq_free(msg_tail); 75: } 76: }	
	污染路径: 函数'ttq_free'的定义点.	
314	缺陷发生位置: /examples/c/schemaless.c	行号: 17
	步骤描述: 不使用返回值的函数'gettimeofday'应使用'void'说明.	
	代码片段: 15: static int64_t getTimeInUs() { 16: struct timeval systemTime; 17: gettimeofday(&systemTime, NULL); 18: return (int64_t)systemTime.tv_sec * 1000000L + (int64_t)systemTime.tv_usec;	

	19: }	
	污染路径: 无	
315	缺陷发生位置: /utils/test/c/blob_test.c	行号: 55
	步骤描述: 不使用返回值的函数'taosMemFree'应使用'void'说明.	
	代码片段: 53: } 54: 55: taosMemFree(*table_name); 56: taosMemFree(*tags); 57: taosMemFree(*params);	
	污染路径: 函数'taosMemFree'的定义点.	
316	缺陷发生位置: /utils/test/c/sml_test.c	行号: 134
	步骤描述: 不使用返回值的函数'ASSERT'应使用'void'说明.	
	代码片段: 132: printf("%s result:%s\n", __FUNCTION__, taos_errstr(pRes)); 133: code = taos_errno(pRes); 134: ASSERT(code == 0); 135: taos_free_result(pRes); 136:	
	污染路径: 函数'ASSERT'的定义点.	
317	缺陷发生位置: /source/libs/nodes/src/nodesUtilFuncs.c	行号: 201
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 199: int32_t code = callocNodeChunk(*pAllocator, NULL); 200: if (TSDB_CODE_SUCCESS != code) { 201: taosMemoryFreeClear(*pAllocator); 202: return code; 203: }	
	污染路径:	

	函数'taosMemoryFreeClear'的定义点.	
318	缺陷发生位置: /source/libs/tmqt/mqtt/src/tmqtLogging.c	行号: 117
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 115: } 116: if (log_destinations & MQTT3_LOG_STDOUT) { 117: UNUSED(setvbuf(stdout, NULL, _IOLBF, 0)); 118: } 119: #ifdef WITH_DLT	
	污染路径: 函数'UNUSED'的定义点.	
319	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqDb.c	行号: 247
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 245: tmqtt_property_free_all(&store->properties); 246: ttq_free(store->payload); 247: ttq_free(store); 248: } 249:	
	污染路径: 函数'ttq_free'的定义点.	
320	缺陷发生位置: /source/libs/decimal/src/detail/wideInteger.cpp	行号: 36
	步骤描述: 不使用返回值的函数'operator-='应使用'void'说明.	
	代码片段: 34: intx::uint128 *pX = (intx::uint128*)pLeft; 35: const intx::uint128 *pY = (const intx::uint128*)pRight; 36: *pX -= *pY; 37: } 38: void uInt128Multiply(UInt128* pLeft, const UInt128* pRight) {	
	污染路径: 函数'operator-='的定义点.	
321	缺陷发生位置: /source/util/src/tlosertree.c	行号: 24

	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 22: static int32_t tMergeTreeInit(SMultiwayMergeTreeInfo* pTree) { 23: if (!((pTree->totalSources & 0x01) == 0 && (pTree->numOfSources << 1 == pTree->totalSources))) { 24: uError("losertree failed at: %s:%d", __func__, __LINE__); 25: return TSDB_CODE_INVALID_PARA; 26: }	
	污染路径: 函数'uError'的定义点.	
322	缺陷发生位置: /source/client/src/clientImpl.c	行号: 267
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 265: } else { 266: code = taosThreadMutexUnlock(&appInfo.mutex); 267: taosMemoryFreeClear(key); 268: if (TSDB_CODE_SUCCESS != code) { 269: tscError("failed to unlock app info, code:%s", tstrerror(TAOS_SYSTEM_ERROR(code)));	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
323	缺陷发生位置: /source/dnode/mnode/impl/src/mndConfig.c	行号: 249
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 247: code = tDeserializeSConfigReq(pReq->pCont, pReq->contLen, &configReq); 248: if (code != 0) { 249: mError("failed to deserialize config req, since %s", terrstr()); 250: goto _OVER; 251: }	
	污染路径: 函数'mError'的定义点.	
324	缺陷发生位置:	行号:

	/source/util/src/talgo.c	464
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 462: tInsertSort(src, size, start, end, param, comparFn, buf); 463: } 464: taosMemoryFreeClear(buf); 465: 466: if (numOfElem > THRESHOLD_SIZE) {	
325	污染路径: 函数'taosMemoryFreeClear'的定义点.	
	缺陷发生位置: /source/dnode/mnode/impl/src/mndCompact.c	行号: 53
	步骤描述: 不使用返回值的函数'mDebug'应使用'void'说明.	
326	代码片段: 51: } 52: 53: void mndCleanupCompact(SMnode *pMnode) { mDebug("mnd compact cleanup"); } 54: 55: void tFreeCompactObj(SCompactObj *pCompact) {}	
	污染路径: 函数'mDebug'的定义点.	
	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 241
327	步骤描述: 不使用返回值的函数'r'应使用'void'说明.	
	代码片段: 239: DEFINE_OPERATOR(-, OP_TYPE_SUB); 240: DEFINE_OPERATOR(*, OP_TYPE_MULTI); 241: DEFINE_OPERATOR(/, OP_TYPE_DIV); 242: DEFINE_OPERATOR(% , OP_TYPE_REM); 243:	
	污染路径: expanded from macro 'DEFINE_OPERATOR'	
327	缺陷发生位置: /source/libs/txnode/src/txnodeMgmt.c	行号: 111
	步骤描述:	

	144: STagFilterResEntry** p = param; 145: taosHashCleanup((*p)->set); 146: taosMemoryFreeClear(*p); 147: } 148:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
331	缺陷发生位置: /source/libs/tdb/src/db/tdbPCache.c	行号: 49
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 47: static void tdbPCacheInitLock(SPCache *pCache) { 48: if (tdbMutexInit(&(pCache->mutex), NULL) != 0) { 49: tdbError("tdb/pcache: mutex init failed."); 50: } 51: }	
	污染路径: 函数'tdbError'的定义点.	
332	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 540
	步骤描述: 不使用返回值的函数'system'应使用'void'说明.	
	代码片段: 538: tdbClose(pEnv); 539: 540: system("ls -l ./tdb"); 541: } 542:	
	污染路径: 无	
333	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 681
	步骤描述: 不使用返回值的函数'OS_PARAM_CHECK'应使用'void'说明.	
	代码片段: 679: 680: int32_t taosGetCpuInfo(char *cpuModel, int32_t maxLen, float *numOfCores) { 681: OS_PARAM_CHECK(cpuModel);	

	682: OS_PARAM_CHECK(numOfCores); 683: #ifdef WINDOWS	
	污染路径: 函数'OS_PARAM_CHECK'的定义点.	
334	缺陷发生位置: /source/util/test/trefTest.c	行号: 77
	步骤描述: 不使用返回值的函数'taosUsleep'应使用'void'说明.	
	代码片段: <pre> 75: void *p = taosAcquireRef(pSpace->rsetId, (int64_t)pSpace->p[id]); 76: if (p) { 77: taosUsleep(id % 5 + 1); 78: taosReleaseRef(pSpace->rsetId, (int64_t)pSpace- >p[id]); 79: }</pre>	
	污染路径: 函数'taosUsleep'的定义点.	
335	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 232
	步骤描述: 不使用返回值的函数'ctgTrace'应使用'void'说明.	
	代码片段: <pre> 230: CTG_ERR_JRET(ctgAcquireDBCache(pCtg, dbFName, &dbCache)); 231: if (NULL == dbCache) { 232: ctgTrace("db:%s, db not in cache", dbFName); 233: goto _return; 234: }</pre>	
	污染路径: 函数'ctgTrace'的定义点.	
336	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMisc.c	行号: 49
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: <pre> 47: struct passwd pw, *result; 48: 49: UNUSED(getpwuid_r(getuid(), &pw, buf, sizeof(buf), &result));</pre>	

	50: if (result) { 51: ttq_log(NULL, TTQ_LOG_WARNING,	
	污染路径: 函数'UNUSED'的定义点.	
337	缺陷发生位置: /examples/c/schemaless.c	行号: 43
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 41: result = taos_query(taos, "create database db precision 'ms';"); 42: taos_free_result(result); 43: usleep(100000); 44: 45: (void)taos_select_db(taos, "db");	
	污染路径: 无	
338	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet2.c	行号: 58
	步骤描述: 不使用返回值的函数'ttq_log'应使用'void'说明.	
	代码片段: 56: 57: buf = strerror(errno); 58: ttq_log(NULL, log, format_str, buf); 59: } 60:	
	污染路径: 函数'ttq_log'的定义点.	
339	缺陷发生位置: /test/new_test_framework/script/api/passwdTest.c	行号: 73
	步骤描述: 不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: 71: Sleep(ms); 72: #else 73: usleep(ms * 1000); 74: #endif 75: }	

	污染路径: 无	
340	缺陷发生位置: /docs/examples/c/query_data_demo.c	行号: 50
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 48: taos_errstr(result)); 49: taos_close(taos); 50: taos_cleanup(); 51: return -1; 52: }	
	污染路径: 函数'taos_cleanup'的定义点.	
341	缺陷发生位置: /source/dnode/mnode/impl/src/mndRetention.c	行号: 384
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 382: int32_t ret = 0; 383: if ((ret = tSerializeSVKillCompactReq((char *)pReq + sizeof(SMsgHead), contLen, &req)) < 0) { 384: taosMemoryFreeClear(pReq); 385: terrno = ret; 386: return NULL;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
342	缺陷发生位置: /source/dnode/vnode/src/meta/metaTable.c	行号: 590
	步骤描述: 不使用返回值的函数'metaError'应使用'void'说明.	
	代码片段: 588: 589: if (ret != 0) { 590: metaError("ttl failed to find expired table, ret:%d", ret); 591: } 592:	
	污染路径: 函数'metaError'的定义点.	

343	缺陷发生位置: /source/dnode/mnode/impl/src/mndInstance.c	行号: 242
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: <pre> 240: int32_t code = tDeserializeSInstanceRegisterReq(pReq->pCont, pReq- >contLen, &req); 241: if (code != TSDB_CODE_SUCCESS) { 242: mError("instance register, failed to decode since %s", tstrerror(code)); 243: return code; 244: }</pre>	
	污染路径: 函数'mError'的定义点.	
344	缺陷发生位置: /contrib/test/sqlite/sqliteTest.c	行号: 42
	步骤描述: 不使用返回值的函数'sqlite3_free'应使用'void'说明.	
	代码片段: <pre> 40: fprintf(stderr, "SQL error: %s\n", err_msg); 41: 42: sqlite3_free(err_msg); 43: sqlite3_close(db); 44:</pre>	
	污染路径: 函数'sqlite3_free'的定义点.	
345	缺陷发生位置: /docs/examples/c-ws-new/insert_data_demo.c	行号: 60
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: <pre> 58: 59: taos_close(taos); 60: taos_cleanup(); 61: return 0; 62: // ANCHOR_END: insert_data</pre>	
	污染路径: 函数'taos_cleanup'的定义点.	
346	缺陷发生位置:	行号:

	/source/util/src/tlruCache.c	100
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 98: } 99: 100: taosMemoryFree(entry); 101: } 102:	
	污染路径: 函数'taosMemoryFree'的定义点.	
347	缺陷发生位置: /utils/test/c/createTable.c	行号: 88
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 86: taos_free_result(pRes); 87: 88: sprintf(qstr, "create table if not exists %s (ts timestamp, i int) tags (j bigint)", stbName); 89: pRes = taos_query(con, qstr); 90: code = taos_errno(pRes);	
	污染路径: 函数'sprintf'的定义点.	
348	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMux.c	行号: 110
	步骤描述: 不使用返回值的函数'UNUSED'应使用'void'说明.	
	代码片段: 108: int ttqMuxRemoveOut(struct tmqtt *context) { 109: #ifndef WITH_EPOLL 110: UNUSED(context); 111: return -1; 112: #else	
	污染路径: 函数'UNUSED'的定义点.	
349	缺陷发生位置: /source/dnode/vnode/src/tq/tqMeta.c	行号: 211
	步骤描述: 不使用返回值的函数'tdbFree'应使用'void'说明.	

	代码片段: 209: STqOffset offset = {0}; 210: if (tqMetaDecodeOffsetInfo(&offset, data, vLen >= 0 ? vLen : 0) != TDB_CODE_SUCCESS) { 211: tdbFree(data); 212: return TSDB_CODE_OUT_OF_MEMORY; 213: }	
	污染路径: 函数'tdbFree'的定义点.	
350	缺陷发生位置: /source/dnode/mnode/impl/src/mndTrans.c	行号: 951
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 949: if (array != NULL) { 950: if (taosHashPut(pTrans->redoGroupActions, &pAction->groupId, sizeof(int32_t), &array, sizeof(SArray *)) < 0) { 951: mInfo("failed put action into redo group actions"); 952: return TSDB_CODE_INTERNAL_ERROR; 953: }	
	污染路径: 函数'mInfo'的定义点.	
351	缺陷发生位置: /test/new_test_framework/script/api/dbTableRoute.c	行号: 45
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 43: taos_free_result(pSql); 44: taos_close(taos); 45: taos_cleanup(); 46: exit(EXIT_FAILURE); 47: }	
	污染路径: 函数'taos_cleanup'的定义点.	
352	缺陷发生位置: /source/dnode/mnode/impl/src/mndDump.c	行号: 747
	步骤描述: 不使用返回值的函数'TAOS_CHECK_RETURN'应使用'void'说明.	
	代码片段: 745:	

	746: TAOS_CHECK_RETURN(wallInit(NULL)); 747: TAOS_CHECK_RETURN(syncInit()); 748: 749: SMnodeOpt opt = {.msgCb = msgCb};	
	污染路径: 函数'TAOS_CHECK_RETURN'的定义点.	
353	缺陷发生位置: /source/libs/tdb/src/db/tdbPage.c	行号: 70
	步骤描述: 不使用返回值的函数'tdbError'应使用'void'说明.	
	代码片段: 68: if (code) { 69: // TODO: memory leak here as we don't have xFree to free the allocated memory 70: tdbError("tdb/page-create: init page lock failed."); 71: return code; 72: }	
	污染路径: 函数'tdbError'的定义点.	
354	缺陷发生位置: /source/libs/planner/src/planOptimizer.c	行号: 1970
	步骤描述: 不使用返回值的函数'planError'应使用'void'说明.	
	代码片段: 1968: 1969: if (code) { 1970: planError("%s failed since %s", __func__, tstrerror(code)); 1971: NODES_DESTROY_NODE(*ppRewrittenCond); 1972: }	
	污染路径: 函数'planError'的定义点.	
355	缺陷发生位置: /docs/examples/c/stmt_insert_demo.c	行号: 183
	步骤描述: 不使用返回值的函数'taos_cleanup'应使用'void'说明.	
	代码片段: 181: insertData(taos); 182: taos_close(taos); 183: taos_cleanup();	

	184: }	
	污染路径: 函数'taos_cleanup'的定义点.	
356	缺陷发生位置: /source/dnode/mnode/sdb/src/sdbFile.c	行号: 52
	步骤描述: 不使用返回值的函数'mInfo'应使用'void'说明.	
	代码片段: 50: } 51: 52: mInfo("sdb deploy success"); 53: return 0; 54: }	
	污染路径: 函数'mInfo'的定义点.	
357	缺陷发生位置: /source/util/src/tutil.c	行号: 494
	步骤描述: 不使用返回值的函数'SET_ERRNO'应使用'void'说明.	
	代码片段: 492: float val; 493: char *endPtr; 494: SET_ERRNO(0); 495: val = taosStr2Float(str, &endPtr); 496: if (str == endPtr ERRNO == ERANGE isnan(val)) {	
	污染路径: 函数'SET_ERRNO'的定义点.	
358	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 660
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 658: } 659: 660: taosMemoryFreeClear(pSup->pDataBlockInfo); 661: } 662: }	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	

359	缺陷发生位置: /source/libs/scheduler/src/schRemote.c	行号: 107
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 105: SCH_ERR_JRET(schLaunchFetchTask(pJob)); 106: 107: taosMemoryFreeClear(msg); 108: 109: return TSDB_CODE_SUCCESS;	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
360	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSttFileRW.c	行号: 112
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 110: TARRAY2_DESTROY(reader[0]->statisBlkArray, NULL); 111: TARRAY2_DESTROY(reader[0]->sttBlkArray, NULL); 112: taosMemoryFree(reader[0]); 113: reader[0] = NULL; 114: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
361	缺陷发生位置: /tools/shell/src/shellMain.c	行号: 77
	步骤描述: 不使用返回值的函数'shellPrintVersion'应使用'void'说明.	
	代码片段: 75: 76: if (shell.args.is_version) { 77: shellPrintVersion(); 78: return 0; 79: }	
	污染路径: 函数'shellPrintVersion'的定义点.	
362	缺陷发生位置: /source/client/src/clientImpl.c	行号: 299
	步骤描述: 不使用返回值的函数'tscError'应使用'void'说明.	

	代码片段: 297: if (param == NULL) return; 298: if (TSDB_CODE_SUCCESS != tsem_destroy(¶m->sem)) { 299: tscError("failed to destroy semaphore in freeQueryParam"); 300: } 301: taosMemoryFree(param);	
	污染路径: 函数'tscError'的定义点.	
363	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 883
	步骤描述: 不使用返回值的函数'taosCloseRef'应使用'void'说明.	
	代码片段: 881: taosDestroyHttpChan(httpDefaultChanId); 882: httpDefaultChanId = -1; 883: taosCloseRef(httpRecvRefMgt); 884: } 885:	
	污染路径: 函数'taosCloseRef'的定义点.	
364	缺陷发生位置: /source/libs/catalog/src/catalog.c	行号: 812
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 810: qTrace("reset catalog timer"); 811: if (taosTmrReset(ctgProcessTimerEvent, CTG_DEFAULT_CACHE_MON_MSEC, NULL, gCtgMgmt.timer, &gCtgMgmt.cacheTimer)) { 812: qError("reset catalog cache monitor timer error, timer stoppped"); 813: } 814:	
	污染路径: 函数'qError'的定义点.	
365	缺陷发生位置: /test/new_test_framework/script/api/api_with_reqid_test.c	行号: 19
	步骤描述:	

	不使用返回值的函数'usleep'应使用'void'说明.	
	代码片段: <pre> 17: result = taos_query_with_reqid(taos, "drop database if exists test;", genReqid()); 18: taos_free_result(result); 19: usleep(100000); 20: result = taos_query_with_reqid(taos, "create database test precision 'us';", genReqid()); 21: taos_free_result(result); </pre>	
	污染路径: 无	
366	缺陷发生位置: /source/libs/monitorfw/src/taos_collector_registry.c	行号: 62
	步骤描述: 不使用返回值的函数'TAOS_LOG'应使用'void'说明.	
	代码片段: <pre> 60: r = pthread_rwlock_init(self->lock, NULL); 61: if (r) { 62: TAOS_LOG("failed to initialize rwlock"); 63: taos_free(self); 64: return NULL; </pre>	
	污染路径: 函数'TAOS_LOG'的定义点.	
	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 241
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
367	代码片段: <pre> 239: code = tNameGetDbName(&sn, varDataVal(dbname)); 240: if (code != TSDB_CODE_SUCCESS) { 241: qError("%s failed at line %d since %s", __func__, __LINE__, tstrerror(code)); 242: return code; 243: } </pre>	
	污染路径: 函数'qError'的定义点.	
368	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 909
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	

	代码片段: 907: sprintf(buf + pos, "%.06d", ms); 908: } else { 909: sprintf(buf + pos, "%.03d", ms); 910: } 911:	
	污染路径: 无	
369	缺陷发生位置: /source/libs/function/test/runUdf.c	行号: 33
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 31: static int32_t initLog() { 32: char logName[12] = {0}; 33: snprintf(logName, sizeof(logName), "%slog", "udfc"); 34: return taosCreateLog(logName, 1, configDir, NULL, NULL, NULL, NULL, 0); 35: }	
	污染路径: 函数'snprintf'的定义点.	
370	缺陷发生位置: /source/libs/scheduler/src/scheduler.c	行号: 225
	步骤描述: 不使用返回值的函数'schCleanClusterHb'应使用'void'说明.	
	代码片段: 223: } 224: 225: schCleanClusterHb(pTrans); 226: } 227:	
	污染路径: 函数'schCleanClusterHb'的定义点.	
371	缺陷发生位置: /source/dnode/mnode/impl/src/mndInfoSchema.c	行号: 90
	步骤描述: 不使用返回值的函数'mError'应使用'void'说明.	
	代码片段: 88: 89: if (NULL == pMeta) {	

	90: mError("invalid information schema table name:%s", tbName); 91: code = TSDB_CODE_PAR_TABLE_NOT_EXIST; 92: TAOS_RETURN(code);	
	污染路径: 函数'mError'的定义点.	
372	缺陷发生位置: /test/new_test_framework/script/api/stmt2-insert-dupkeys.c	行号: 141
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 139: TAOS_STMT2_BIND **paramv, **tags; 140: 141: INIT(tbs, ts, ts_len, b, b_len, tags, paramv); 142: 143: insert(taos, tbs, tags, paramv, sql);	
	污染路径: expanded from macro 'INIT'	
373	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttContext.c	行号: 152
	步骤描述: 不使用返回值的函数'ttq_free'应使用'void'说明.	
	代码片段: 150: 151: if (force_free) { 152: ttq_free(context); 153: } 154: }	
	污染路径: 函数'ttq_free'的定义点.	
374	缺陷发生位置: /source/util/src/theap.c	行号: 205
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 203: pq->container = taosArrayInit(1, sizeof(PriorityQueueNode)); 204: if (pq->container == NULL) { 205: taosMemoryFree(pq); 206: return NULL;	

	207: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
375	缺陷发生位置: /source/util/test/utilTests.cpp	行号: 499
	步骤描述: 不使用返回值的函数'checkNull'应使用'void'说明.	
	代码片段: 497: columnInfoData.hasNull = false; 498: checkNull(0, false); 499: checkNull(1, false); 500: checkNull(2, false); 501: checkNull(totalRows - 2, false);	
	污染路径: 函数'checkNull'的定义点.	
376	缺陷发生位置: /source/libs/executor/src/timewindowoperator.c	行号: 2588
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 2586: destroyIntervalOperatorInfo(&miaInfo->intervalAggOperatorInfo); 2587: 2588: taosMemoryFreeClear(param); 2589: } 2590:	
	污染路径: 函数'taosMemoryFreeClear'的定义点.	
377	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 1282
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 1280: if (pStatisBlock) { 1281: tStatisBlockDestroy(pStatisBlock); 1282: taosMemoryFreeClear(pStatisBlock); 1283: } 1284: return code;	
	污染路径:	

	函数'taosMemoryFreeClear'的定义点.	
378	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncSnapshotDebug.c	行号: 39
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 37: s = syncUtilPrintBin((char*)(pSender->pCurrentBlock), pSender->blockLen); 38: cJSON_AddStringToObject(pRoot, "pCurrentBlock", s); 39: taosMemoryFree(s); 40: s = syncUtilPrintBin2((char*)(pSender->pCurrentBlock), pSender->blockLen); 41: cJSON_AddStringToObject(pRoot, "pCurrentBlock2", s);	
	污染路径: 函数'taosMemoryFree'的定义点.	
379	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmWorker.c	行号: 303
	步骤描述: 不使用返回值的函数'ST_TASK_DLOG'应使用'void'说明.	
	代码片段: 301: STREAM_CHECK_NULL_GOTO(sStreamReaderCalcInfo, terrno); 302: void* pTask = sStreamReaderCalcInfo->pTask; 303: ST_TASK_DLOG("mnode %s start", __func__); 304: 305: if (req.reset sStreamReaderCalcInfo->pTaskInfo == NULL) {	
	污染路径: 函数'ST_TASK_DLOG'的定义点.	
380	缺陷发生位置: /source/libs/qcom/src/queryUtil.c	行号: 249
	步骤描述: 不使用返回值的函数'qError'应使用'void'说明.	
	代码片段: 247: taskQueue.pTaskQueue = tQueryAutoQWorkerAllocQueue(&taskQueue.wrokerPool, NULL, (FItem)processTaskQueue); 248: if (NULL == taskQueue.pTaskQueue) { 249: qError("failed to init task queue"); 250: return -1; 251: }	

	污染路径: 函数'qError'的定义点.	
381	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncIndexMgrDebug.c	行号: 65
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 63: } 64: cJSON *pIndex = cJSON_CreateIntArray(arr, pSyncIndexMgr->replicaNum); 65: taosMemoryFree(arr); 66: cJSON_AddItemToObject(pRoot, "index", pIndex); 67: }	
	污染路径: 函数'taosMemoryFree'的定义点.	
382	缺陷发生位置: /source/client/test/session.cpp	行号: 449
	步骤描述: 不使用返回值的函数'snprintf'应使用'void'说明.	
	代码片段: 447: snprintf(fenUtf8, sizeof(fenUtf8), "%c%c%c", 0xE8, 0x8A, 0xAC); 448: snprintf(fenGbk, sizeof(fenGbk), "%c%c", 0xB7, 0xD2); 449: snprintf(zhongguoUtf8, sizeof(zhongguoUtf8), "%c%c%c%c", 0xE4, 0xB8, 0xAD, 0xE5, 0x9B, 0xBD); 450: snprintf(guoUtf8, sizeof(guoUtf8), "%c%c%c", 0xE5, 0x9B, 0xBD); 451: snprintf(zhongguoGbk, sizeof(zhongguoGbk), "%c%c%c%c", 0xD6, 0xD0, 0xB9, 0xFA);	
	污染路径: 无	
383	缺陷发生位置: /source/util/src/tdecompressavx.c	行号: 483
	步骤描述: 不使用返回值的函数'uError'应使用'void'说明.	
	代码片段: 481: return opos; 482: #else 483: uError("unable run %s without avx512 instructions", __func__);	

	484: return -1; 485: #endif	
	污染路径: 函数'uError'的定义点.	
384	缺陷发生位置: /source/libs/index/src/index.c	行号: 221
	步骤描述: 不使用返回值的函数'indexError'应使用'void'说明.	
	代码片段: 219: int32_t code = 0; 220: if (taosThreadMutexLock(&index->mtx) != 0) { 221: indexError("failed to lock index mutex"); 222: } 223:	
	污染路径: 函数'indexError'的定义点.	
385	缺陷发生位置: /source/client/src/clientHb.c	行号: 1470
	步骤描述: 不使用返回值的函数'setThreadName'应使用'void'说明.	
	代码片段: 1468: 1469: static void *hbThreadFunc(void *param) { 1470: setThreadName("hb"); 1471: #ifdef WINDOWS 1472: if (taosCheckCurrentInDll()) {	
	污染路径: 函数'setThreadName'的定义点.	
386	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeCommit.c	行号: 347
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 345: } 346: 347: static void vnodeCommitCancel(void *arg) { taosMemoryFree(arg); } 348: 349: int vnodeAsyncCommitEx(SVnode *pVnode, bool forceTrim) {	

	污染路径: 函数'taosMemoryFree'的定义点.	
387	缺陷发生位置: /source/libs/planner/src/planSpliter.c	行号: 2531
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 2529: qDebugL("apply split %s rule, jsonPlan: %s", pRuleName, pStr); 2530: } 2531: taosMemoryFree(pStr); 2532: } 2533: return code;	
	污染路径: 函数'taosMemoryFree'的定义点.	
388	缺陷发生位置: /source/libs/decimal/src/decimal.c	行号: 733
	步骤描述: 不使用返回值的函数'TAOS_STRNCAT'应使用'void'说明.	
	代码片段: 731: TAOS_STRNCAT(buf2, buf, wholeLen); 732: } else { 733: TAOS_STRNCAT(buf2, "0", 2); 734: } 735: if (scale > 0) {	
	污染路径: 函数'TAOS_STRNCAT'的定义点.	
389	缺陷发生位置: /utils/test/c/tmq_blob.c	行号: 49
	步骤描述: 不使用返回值的函数'tmq_free_json_meta'应使用'void'说明.	
	代码片段: 47: char* result = tmq_get_json_meta(msg); 48: printf("meta result: %s\n", result); 49: tmq_free_json_meta(result); 50: } 51:	
	污染路径: 函数'tmq_free_json_meta'的定义点.	

390	缺陷发生位置: /source/dnode/mgmt/mgmt_mnode/src/mmHandle.c	行号: 21
	步骤描述: 不使用返回值的函数'dError'应使用'void'说明.	
	代码片段: 19: void mmGetMonitorInfo(SMnodeMgmt *pMgmt, SMonMmInfo *pInfo) { 20: if (mndGetMonitorInfo(pMgmt->pMnode, &pInfo->cluster, &pInfo->vgroup, &pInfo->stb, &pInfo->grant) != 0) { 21: dError("failed to get monitor info"); 22: } 23: }	
	污染路径: 函数'dError'的定义点.	
391	缺陷发生位置: /source/dnode/mgmt/mgmt_snode/src/smHandle.c	行号: 235
	步骤描述: 不使用返回值的函数'streamDeleteAllCheckpoints'应使用'void'说明.	
	代码片段: 233: snprintf(path, TSDB_FILENAME_LEN, "%s%ssnode%d", pInfo->path, TD_DIRSEP, dropReq.dnodeId); 234: 235: streamDeleteAllCheckpoints(); 236: 237: bool deployed = false;	
	污染路径: 函数'streamDeleteAllCheckpoints'的定义点.	
392	缺陷发生位置: /source/dnode/snode/src/snode.c	行号: 74
	步骤描述: 不使用返回值的函数'streamReleaseTask'应使用'void'说明.	
	代码片段: 72: } 73: 74: streamReleaseTask(taskAddr); 75: 76: if(code == TSDB_CODE_MND_STREAM_TABLE_NOT_CREATE) {	
	污染路径: 函数'streamReleaseTask'的定义点.	

393	缺陷发生位置: /test/new_test_framework/script/api/stmtQuery.c	行号: 112
	步骤描述: 不使用返回值的函数'taos_stmt_add_batch'应使用'void'说明.	
	代码片段: 110: 111: taos_stmt_bind_param(stmt, params); 112: taos_stmt_add_batch(stmt); 113: 114: if (taos_stmt_execute(stmt) != 0) {	
	污染路径: 函数'taos_stmt_add_batch'的定义点.	
394	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 93
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 91: 92: char command[1024] = {0}; 93: sprintf(command, "%s", "create table meters(ts timestamp, f float, n int, " 94: "bin1 binary(20), c nchar(20), bin2 binary(20)) tags(area int, " 95: "city binary(20), dist nchar(20), street binary(20));");	
	污染路径: 无	
395	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 979
	步骤描述: 不使用返回值的函数'buffer_skip_whitespace'应使用'void'说明.	
	代码片段: 977: if (require_null_terminated) 978: { 979: buffer_skip_whitespace(&buffer); 980: if ((buffer.offset >= buffer.length) buffer_at_offset(&buffer)[0] != '\0') 981: {	
	污染路径: 函数'buffer_skip_whitespace'的定义点.	

396	缺陷发生位置: /docs/examples/c/asyncdemo.c	行号: 109
	步骤描述: 不使用返回值的函数'sprintf'应使用'void'说明.	
	代码片段: 107: printf("success to connect to server\n"); 108: 109: sprintf(sql, "drop database if exists %s", db); 110: queryDB(taos, sql); 111:	
	污染路径: 无	
397	缺陷发生位置: /source/util/src/thash.c	行号: 512
	步骤描述: 不使用返回值的函数'taosMemoryFreeClear'应使用'void'说明.	
	代码片段: 510: pe->num--; 511: (void)atomic_sub_fetch_64(&pHashObj->size, 1); 512: FREE_HASH_NODE(pHashObj->freeFp, pNode); 513: } 514: } else {	
	污染路径: expanded from macro 'FREE_HASH_NODE' 函数'taosMemoryFreeClear'的定义点. expanded from macro 'FREE_HASH_NODE'	
398	缺陷发生位置: /source/libs/catalog/src/ctgAsync.c	行号: 2587
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: 2585: tagVal.nData = strlen(pJson); 2586: if (NULL == taosArrayPush(pTagVals, &tagVal)) { 2587: taosMemoryFree(pJson); 2588: taosArrayDestroy(pTagVals); 2589: CTG_ERR_JRET(terrno);	
	污染路径: 函数'taosMemoryFree'的定义点.	
399	缺陷发生位置: /source/libs/command/src/command.c	行号: 757

	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 755: return terrno; 756: } 757: taosMemoryFree(pjson); 758: 759: return TSDB_CODE_SUCCESS; </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
400	缺陷发生位置: /source/libs/sync/test/sync_test_lib/src/syncRaftLogDebug.c	行号: 160
	步骤描述: 不使用返回值的函数'taosMemoryFree'应使用'void'说明.	
	代码片段: <pre> 158: printf("logStoreSimplePrint len:%d %s \n", (int32_t)strlen(serialized), serialized); 159: fflush(NULL); 160: taosMemoryFree(serialized); 161: } 162: </pre>	
	污染路径: 函数'taosMemoryFree'的定义点.	
401	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqHandleConnect.c	行号: 71
	步骤描述: 不使用返回值的函数'DL_DELETE'应使用'void'说明.	
	代码片段: <pre> 69: if (tmqttAcICheck(context, msg_tail->store->topic, msg_tail->store->payloadlen, msg_tail->store->payload, 70: msg_tail->store->qos, msg_tail->store- >retain, access) != TTQ_ERR_SUCCESS) { 71: DL_DELETE((*head), msg_tail); 72: ttqDbMsgStoreRefDec(&msg_tail->store); 73: tmqtt_property_free_all(&msg_tail->properties); </pre>	
	污染路径: 函数'DL_DELETE'的定义点.	

17 C/C++缺陷类型-不可达代码

17.1 分项汇总表

序号	C/C++缺陷类型-不可达代码	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4740	禁止不可达分支	1	严重

17.2 缺陷/安全漏洞详解

17.2.1 严重类

17.2.1.1 WK4740 禁止不可达分支

缺陷数量：1 个		
缺陷描述： 禁止不可达分支。		
修复意见： 无		
1	缺陷发生位置： /contrib/lemon/lemon.c	行号： 91
	步骤描述： 禁止不可达分支	
	代码片段： 89: static void *lemon_malloc(size_t nByte){ 90: MemChunk *p; 91: if(nByte<0) return 0; 92: p = malloc(nByte + sizeof(MemChunk)); 93: if(p==0){	
	污染路径：	

	无
--	---

18 C/C++缺陷类型-代码质量-头文件问题

18.1 分项汇总表

序号	C/C++缺陷类型-代码质量-头文件问题	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4680	禁止在#include头文件前有可执行代码	27	严重
2	WK4676	禁止头文件重复包含	187	严重

18.2 缺陷/安全漏洞详解

18.2.1 严重类

18.2.1.1 WK4680 禁止在#include头文件前有可执行代码

缺陷数量：27 个		
缺陷描述： 禁止在#include头文件前有可执行代码。		
修复意见： 无		
1	缺陷发生位置： /contrib/TSZ/sz/src/sz.c	行号： 25
	步骤描述： 禁止在#include头文件前有可执行代码	

	代码片段: 23: // #include "CurveFillingCompressStorage.h" 24: 25: unsigned char versionNumber = DATA_FORMAT_VER1; 26: int SZ_SIZE_TYPE_DEFAULT = 4; 27:	
	污染路径: 无	
2	缺陷发生位置: /source/common/test/SClientHbBatchReq.cpp	行号: 19
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 17: 18: // 旧版 SClientHbReq 编码: 写到 sVer/cInfo 为止, 不含 user/tokenName 19: static int32_t serializeOldSClientHbReq(SEncoder *pEncoder, const SClientHbReq *pReq) { 20: TAOS_CHECK_RETURN(tStartEncode(pEncoder)); 21: TAOS_CHECK_RETURN(tEncodeSClientHbKey(pEncoder, &pReq->connKey)); 22: }	
	污染路径: 无	
3	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttLogging.c	行号: 38
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 36: #include "ttqUtil.h" 37: 38: static char log_fptr_buffer[BUFSIZ]; 39: 40: /* Options for logging should be:	
	污染路径: 无	
4	缺陷发生位置: /contrib/lemon/lempar.c	行号: 222
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段:	

	220: ** is the value of the token */ 221: }; 222: typedef struct yyStackEntry yyStackEntry; 223: 224: /* The state of the parser is completely contained in an instance of	
	污染路径: 无	
5	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nohole.c	行号: 10
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 8: #include <unistd.h> 9: #include "taos.h" 10: static int64_t count = 10000; 11: 12: int64_t genReqid() {	
	污染路径: 无	
6	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeProductionTest.cpp	行号: 15
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 13: #include <cstring> 14: 15: extern "C" { 16: #include "mndDef.h" 17: #include "mndInt.h"	
	污染路径: 无	
7	缺陷发生位置: /test/new_test_framework/script/api/stmt2-nchar.c	行号: 9
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 7: #include <unistd.h> 8: #include "taos.h" 9: static int64_t count = 10000;	

	10: 11: int64_t genReqid() {	
	污染路径: 无	
8	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 43
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 41: 42: #define FSE_GENERATE_ENUM(ENUM) ENUM, 43: typedef enum { FSE_LIST_ERRORS(FSE_GENERATE_ENUM) } FSE_errorCodes; /* enum is exposed, to detect & handle specific errors; compare function result to -enum value */ 44: 45:	
	污染路径: 无	
9	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/bug-argp1.c	行号: 8
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 6: 7: 8: static const struct argp_option test_options[] = 9: { 10: { NULL, 'a', NULL, OPTION_DOC, NULL },	
	污染路径: 无	
10	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeAgentTest.cpp	行号: 15
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 13: #include <cstring> 14: 15: extern "C" {	

	16: #include "mndDef.h" 17: #include "mndInt.h"	
	污染路径: 无	
11	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 93
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 91: #if defined (__cplusplus) (defined (__STDC_VERSION__) && (__STDC_VERSION__ >= 199901L) /* C99 */) 92: # include <stdint.h> 93: typedef uint8_t BYTE; 94: typedef uint16_t U16; 95: typedef int16_t S16;	
	污染路径: 无	
12	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp1.c	行号: 32
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 30: 31: 32: static const struct argp_option test_options[] = 33: { 34: { NULL, 0, NULL, 0, "\	
	污染路径: 无	
13	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 94
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 92: #if defined (__cplusplus) (defined (__STDC_VERSION__) && (__STDC_VERSION__ >= 199901L) /* C99 */) 93: # include <stdint.h> 94: typedef uint8_t BYTE; 95: typedef uint16_t U16;	

	96: typedef int16_t S16;	
	污染路径: 无	
14	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 84
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 82: #if defined (__cplusplus) (defined (__STDC_VERSION__) && (__STDC_VERSION__ >= 199901L) /* C99 */) 83: # include <stdint.h> 84: typedef uint8_t BYTE; 85: typedef uint16_t U16; 86: typedef int16_t S16;	
	污染路径: 无	
15	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 102
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 100: # if !HAVE_DECL_STRERROR 101: #ifndef _WIN32 // { 102: char *strerror (int errnum); 103: #elif !defined(_MSC_VER) // }{ 104: char *strerror (int errnum);	
	污染路径: 无	
16	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 59
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 57: 58: #define ZSTDv07_FRAMEHEADERSIZE_MAX 18 /* for static allocation */ 59: static const size_t ZSTDv07_frameHeaderSize_min = 5; 60: static const size_t ZSTDv07_frameHeaderSize_max = ZSTDv07_FRAMEHEADERSIZE_MAX;	

	61: static const size_t ZSTDv07_skippableHeaderSize = 8; /* magic number + skippable frame length */	
	污染路径: 无	
17	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/tst-argp2.c	行号: 22
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 20: #include <argp.h> 21: 22: static const struct argp_option opt1[] = 23: { 24: { "opt1", '1', "NUMBER", 0, "Option 1" },	
	污染路径: 无	
18	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 24
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 22: #define UUIDLEN37 37 23: 24: typedef struct { 25: uint64_t user; 26: uint64_t nice;	
	污染路径: 无	
19	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeMemoryTest.cpp	行号: 13
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 11: #include <vector> 12: 13: extern "C" { 14: #include "mndDef.h" 15: }	

	污染路径: 无	
20	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSdbTest.cpp	行号: 33
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 31: #include <cstring> 32: 33: extern "C" { 34: #include "mndXnode.h" 35: }	
	污染路径: 无	
21	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 19
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 17: #include <algorithm> 18: 19: extern "C" { 20: #include "os.h" 21: #include "libs/metrics/metrics.h"	
	污染路径: 无	
22	缺陷发生位置: /source/dnode/vnode/test/tqTest.cpp	行号: 32
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 30: #pragma GCC diagnostic ignored "-Wsign-compare" 31: 32: SDmNotifyHandle dmNotifyHdl = {.state = 0}; 33: 34: #include "tq.h"	
	污染路径: 无	
23	缺陷发生位置: /test/new_test_framework/script/api/stmt2.c	行号: 9

	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 7: #include <unistd.h> 8: #include "taos.h" 9: static int64_t count = 10000; 10: 11: int64_t genReqid() {	
	污染路径: 无	
24	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchData.c	28
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 26: 27: 28: const char charset[] = 29: "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ1234567890"; 30:	
25	污染路径: 无	
	缺陷发生位置:	行号:
	/source/dnode/vnode/test/tsdbRepairTest.cpp	23
	步骤描述: 禁止在#include 头文件前有可执行代码	
26	代码片段: 21: #include <cstring> 22: 23: extern "C" { 24: #include "dmRepair.h" 25: #include "vnodeInt.h"	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/new-stream/test/streamTriggerTaskTest.cpp	19
	步骤描述: 禁止在#include 头文件前有可执行代码	

	代码片段: 17: #include <time.h> 18: 19: extern "C" { 20: #include "streamTriggerTask.h" 21: #include "taosdef.h"	
	污染路径: 无	
27	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 56
	步骤描述: 禁止在#include 头文件前有可执行代码	
	代码片段: 54: #if defined (__cplusplus) (defined (__STDC_VERSION__) && (__STDC_VERSION__ >= 199901L) /* C99 */) 55: # include <stdint.h> 56: typedef uint8_t BYTE; 57: typedef uint16_t U16; 58: typedef int16_t S16;	
	污染路径: 无	

18.2.1.2 WK4676 禁止头文件重复包含

缺陷数量：187 个		
缺陷描述: 禁止头文件重复包含。		
修复意见: 无		
1	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1490
	步骤描述: 禁止头文件重复包含	
	代码片段: 1488: ***** */ 1489: #include <stdlib.h> /* malloc, free, qsort */	

	1490: #include <string.h> /* memcpy, memset */ 1491: #include <stdio.h> /* printf (debug) */ 1492:	
	污染路径: 原头文件定义点	
2	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 83
	步骤描述: 禁止头文件重复包含	
	代码片段: 81: #include <string.h> 82: #include <time.h> 83: #include <stdarg.h> 84: 85: #include <taos.h>	
	污染路径: 原头文件定义点	
3	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshot.c	行号: 21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "tsdbFSetRW.h" 20: #include "tsdbIter.h" 21: #include "tsdbSttFileRW.h" 22: 23: extern int32_t tsdbUpdateTableSchema(SMeta* pMeta, int64_t suid, int64_t uid, SSkmlInfo* pSkmlInfo);	
	污染路径: 原头文件定义点	
4	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRetention.c	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdb.h" 18: #include "tsdbFS2.h" 19: #include "tsdbFSet2.h" 20: #include "vnd.h" 21: #include "tsdbInt.h"	

	污染路径: 原头文件定义点	
5	缺陷发生位置: /source/libs/executor/src/scanoperator.c	行号: 41
	步骤描述: 禁止头文件重复包含	
	代码片段: 39: #include "ttypes.h" 40: 41: #include "function.h" 42: #include "storageapi.h" 43: #include "wal.h"	
	污染路径: 原头文件定义点	
6	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.h	行号: 22
	步骤描述: 禁止头文件重复包含	
	代码片段: 20: #include "bseCache.h" 21: #include "bseTable.h" 22: #include "bseUtil.h" 23: 24: #ifdef __cplusplus	
	污染路径: 原头文件定义点	
7	缺陷发生位置: /test/new_test_framework/script/api/last-query-ws.cpp	行号: 7
	步骤描述: 禁止头文件重复包含	
	代码片段: 5: #include <condition_variable> 6: #include <cstdint> 7: #include <cstdint> 8: #include <iostream> 9: #include <mutex>	
	污染路径: 原头文件定义点	
8	缺陷发生位置:	行号:

	/source/dnode/vnode/src/tsdb/tsdbReadUtil.c	18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include "tsdbReadUtil.h" 17: #include "tsdb.h" 18: #include "tsdbDataFileRW.h" 19: #include "tsdbFS2.h" 20: #include "tsdbMerge.h"	
	污染路径: 原头文件定义点	
9	缺陷发生位置: /source/client/inc/clientSml.h	行号: 28
	步骤描述: 禁止头文件重复包含	
	代码片段: 26: #include "taoserror.h" 27: #include "tcommon.h" 28: #include "tdef.h" 29: #include "tglobal.h" 30: #include "tlog.h"	
	污染路径: 原头文件定义点	
10	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1656
	步骤描述: 禁止头文件重复包含	
	代码片段: 1654: ***** */ 1655: #include <stdlib.h> /* malloc, free, qsort */ 1656: #include <string.h> /* memcpy, memset */ 1657: #include <stdio.h> /* printf (debug) */ 1658:	
	污染路径: 原头文件定义点	
11	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet.c	行号: 11
	步骤描述:	

	禁止头文件重复包含	
	代码片段: 9: #include <netdb.h> 10: #include <netinet/tcp.h> 11: #include <sys/socket.h> 12: #include <unistd.h> 13:	
	污染路径: 原头文件定义点	
12	缺陷发生位置: /source/common/src/msg/tmsg.c	行号: 35
	步骤描述: 禁止头文件重复包含	
	代码片段: 33: #define TD_MSG_DICT_ 34: #undef TD_MSG_SEG_CODE_ 35: #include "tmsgdef.h" 36: 37: #undef TD_MSG_NUMBER_	
	污染路径: 原头文件定义点	
13	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMerge.h	行号: 17
	步骤描述: 禁止头文件重复包含	
	代码片段: 15: 16: #include "tsdbDataFileRW.h" 17: #include "tsdbFS2.h" 18: #include "tsdbFSetRW.h" 19: #include "tsdbIter.h"	
	污染路径: 原头文件定义点	
14	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include "dump.h" 17: #include "dumpUtil.h"	

	18: #include "tdef.h" 19: 20: static char **g_tsDumpInDebugFiles = NULL;	
	污染路径: 原头文件定义点	
15	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 72
	步骤描述: 禁止头文件重复包含	
	代码片段: 70: #include <string.h> 71: #include <assert.h> 72: #include <stdarg.h> 73: #include <ctype.h> 74: #include <limits.h>	
	污染路径: 原头文件定义点	
16	缺陷发生位置: /tools/lastqps-test/taos_query_a_simple.c	行号: 9
	步骤描述: 禁止头文件重复包含	
	代码片段: 7: #include <string.h> 8: #include <sys/time.h> 9: #include <time.h> 10: #include <unistd.h> 11:	
	污染路径: 原头文件定义点	
17	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1489
	步骤描述: 禁止头文件重复包含	
	代码片段: 1487: * Includes 1488: ***** */ 1489: #include <stdlib.h> /* malloc, free, qsort */	

	1490: #include <string.h> /* memcpy, memset */ 1491: #include <stdio.h> /* printf (debug) */	
	污染路径: 原头文件定义点	
18	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 79
	步骤描述: 禁止头文件重复包含	
	代码片段: 77: #include <errno.h> 78: #include <pthread.h> 79: #include <stdint.h> 80: #include <stdlib.h> 81: #include <string.h>	
	污染路径: 原头文件定义点	
19	缺陷发生位置: /source/common/test/SClientHbBatchReq.cpp	行号: 1
	步骤描述: 禁止头文件重复包含	
	代码片段: -1: #include <gtest/gtest.h> 0: #include <vector> 1: #include <cstring> 2: #include "tmsg.h" 3:	
	污染路径: 原头文件定义点	
20	缺陷发生位置: /source/client/inc/clientSml.h	行号: 24
	步骤描述: 禁止头文件重复包含	
	代码片段: 22: #include "clientInt.h" 23: #include "osThread.h" 24: #include "query.h" 25: #include "taos.h" 26: #include "taoserror.h"	
	污染路径:	

	原头文件定义点	
21	缺陷发生位置: /tools/taos-tools/inc/dump.h	行号: 35
	步骤描述: 禁止头文件重复包含	
	代码片段: 33: 34: #include <inttypes.h> 35: #include <limits.h> 36: 37: #include <avro.h>	
	污染路径: 原头文件定义点	
22	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 17
	步骤描述: 禁止头文件重复包含	
	代码片段: 15: 16: #include "bseSnapshot.h" 17: #include "bseInc.h" 18: #include "bseTable.h" 19: #include "bseTableMgt.h"	
	污染路径: 原头文件定义点	
23	缺陷发生位置: /contrib/lemon/lemon.c	行号: 10
	步骤描述: 禁止头文件重复包含	
	代码片段: 8: */ 9: #include <stdio.h> 10: #include <stdarg.h> 11: #include <string.h> 12: #include <ctype.h>	
	污染路径: 原头文件定义点	
24	缺陷发生位置: /source/libs/function/src/tppercentile.c	行号: 22
	步骤描述:	

	禁止头文件重复包含	
	代码片段: 20: 21: #include "taosdef.h" 22: #include "tcompare.h" 23: #include "tpagedbuf.h" 24: #include "tpercentile.h"	
	污染路径: 原头文件定义点	
25	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 22
	步骤描述: 禁止头文件重复包含	
	代码片段: 20: #include "bseTableMgt.h" 21: #include "bseUtil.h" 22: #include "cJSON.h" 23: #include "tencrypt.h" 24:	
	污染路径: 原头文件定义点	
26	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCommit2.h	行号: 17
	步骤描述: 禁止头文件重复包含	
	代码片段: 15: 16: #include "tsdbDataFileRW.h" 17: #include "tsdbFS2.h" 18: #include "tsdbFSetRW.h" 19: #include "tsdbIter.h"	
	污染路径: 原头文件定义点	
27	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbScan.c	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdbDataFileRW.h" 18: #include "tsdbFS2.h"	

	19: #include "tsdbSttFileRW.h" 20: 21: typedef struct {	
	污染路径: 原头文件定义点	
28	缺陷发生位置: /source/libs/executor/src/tfill.c	行号: 26
	步骤描述: 禁止头文件重复包含	
	代码片段: 24: #include "ttime.h" 25: 26: #include "executorInt.h" 27: #include "function.h" 28: #include "querynodes.h"	
	污染路径: 原头文件定义点	
29	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 17
	步骤描述: 禁止头文件重复包含	
	代码片段: 15: #include "benchLog.h" 16: #include "decimal.h" 17: #include "pub.h" 18: #if defined(_TD_DARWIN_64) 19: #include <sys/sysctl.h>	
	污染路径: 原头文件定义点	
30	缺陷发生位置: /source/libs/index/test/indexBench.cc	行号: 18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include <algorithm> 17: #include <iostream> 18: #include <string> 19: #include <thread> 20: #include "index.h"	

	污染路径: 原头文件定义点	
31	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 850
	步骤描述: 禁止头文件重复包含	
	代码片段: 848: * Includes 849: *****/ 850: #include <stddef.h> /* size_t */ 851: 852:	
	污染路径: 原头文件定义点	
32	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 16
	步骤描述: 禁止头文件重复包含	
	代码片段: 14: #include <bench.h> 15: #include "benchLog.h" 16: #include "decimal.h" 17: #include "pub.h" 18: #if defined(_TD_DARWIN_64)	
	污染路径: 原头文件定义点	
33	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbScan.c	行号: 18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include "tsdb.h" 17: #include "tsdbDataFileRW.h" 18: #include "tsdbFS2.h" 19: #include "tsdbSttFileRW.h" 20:	
	污染路径: 原头文件定义点	
34	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp	行号: 8

	步骤描述: 禁止头文件重复包含	
	代码片段: 6: 7: #include <shared_mutex> 8: #include <string> 9: #include <thread> 10: #include <vector>	
	污染路径: 原头文件定义点	
35	缺陷发生位置:	行号:
	/source/libs/decimal/src/detail/intx/int128.hpp	14
	步骤描述: 禁止头文件重复包含	
	代码片段: 12: #include <string> 13: #include <tuple> 14: #include <type_traits> 15: 16: #ifdef _MSC_VER	
36	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/tools/shell/test/shellTest.cpp	19
	步骤描述: 禁止头文件重复包含	
37	代码片段: 17: #include <iostream> 18: #include <cstring> 19: #include <cctype> 20: #include "shellAuto.h" 21: #include "shellInt.h"	
	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/source/dnode/vnode/src/bse/bseTableMgt.c	18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include "bseTableMgt.h"	

	17: #include "bse.h" 18: #include "bseTable.h" 19: #include "bseUtil.h" 20: #include "tglobal.h"	
	污染路径: 原头文件定义点	
38	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSecureErase.c	行号: 95
	步骤描述: 禁止头文件重复包含	
	代码片段: 93: #include "tsdbDef.h" 94: #include "tsdbFS2.h" 95: #include "tsdbFSet2.h" 96: #include "tsdbSttFileRW.h" 97: #include "tsdbUtil2.h"	
	污染路径: 原头文件定义点	
39	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbMerge.h	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include "tsdbFSetRW.h" 19: #include "tsdbIter.h" 20: #include "tsdbSttFileRW.h" 21: #include "tsdbUtil2.h" 22:	
	污染路径: 原头文件定义点	
40	缺陷发生位置: /source/dnode/mnode/impl/inc/mndInt.h	行号: 22
	步骤描述: 禁止头文件重复包含	
	代码片段: 20: 21: #include "sdb.h" 22: #include "sync.h" 23: #include "tcache.h" 24: #include "tdatablock.h"	

	污染路径: 原头文件定义点	
41	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSecureErase.c	行号: 96
	步骤描述: 禁止头文件重复包含	
	代码片段: 94: #include "tsdbFS2.h" 95: #include "tsdbFSet2.h" 96: #include "tsdbSttFileRW.h" 97: #include "tsdbUtil2.h" 98: #include "ttypes.h"	
	污染路径: 原头文件定义点	
42	缺陷发生位置: /source/os/src/osSocket.c	行号: 37
	步骤描述: 禁止头文件重复包含	
	代码片段: 35: #include <netinet/tcp.h> 36: #include <netinet/udp.h> 37: #include <sys/socket.h> 38: #include <unistd.h> 39:	
	污染路径: 原头文件定义点	
43	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 593
	步骤描述: 禁止头文件重复包含	
	代码片段: 591: * Includes 592: *****/ 593: #include <stddef.h> /* size_t, ptrdiff_t */ 594: 595:	
	污染路径: 原头文件定义点	
44	缺陷发生位置:	行号:

	/source/libs/wal/test/walMetaTest.cpp	6
	步骤描述: 禁止头文件重复包含	
	代码片段: 4: #include <queue> 5: #include <thread> 6: #include <vector> 7: #include <atomic> 8:	
	污染路径: 原头文件定义点	
45	缺陷发生位置: /docs/examples/c/tmq_demo.c	行号: 23
	步骤描述: 禁止头文件重复包含	
	代码片段: 21: #include <stdlib.h> 22: #include <string.h> 23: #include <time.h> 24: #include <unistd.h> 25: #include "taos.h"	
	污染路径: 原头文件定义点	
46	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.h	行号: 23
	步骤描述: 禁止头文件重复包含	
	代码片段: 21: #include "bseUtil.h" 22: #include "cJSON.h" 23: #include "os.h" 24: #include "tchecksum.h" 25: #include "tcompare.h"	
	污染路径: 原头文件定义点	
47	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 82
	步骤描述: 禁止头文件重复包含	
	代码片段:	

	80: #include <stdlib.h> 81: #include <string.h> 82: #include <time.h> 83: #include <stdarg.h> 84:	
	污染路径: 原头文件定义点	
48	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 25
	步骤描述: 禁止头文件重复包含	
	代码片段: 23: #include <math.h> 24: #include <benchData.h> 25: #include "decimal.h" 26: 27:	
	污染路径: 原头文件定义点	
49	缺陷发生位置: /source/libs/index/test/fstTest.cc	行号: 5
	步骤描述: 禁止头文件重复包含	
	代码片段: 3: #include <algorithm> 4: #include <iostream> 5: #include <string> 6: #include <thread> 7: #include <vector>	
	污染路径: 原头文件定义点	
50	缺陷发生位置: /source/libs/tmqtt/tools/src/topic-producer.c	行号: 23
	步骤描述: 禁止头文件重复包含	
	代码片段: 21: #include <stdlib.h> 22: #include <string.h> 23: #include <time.h> 24: #ifndef WINDOWS	

	25: #include <unistd.h>	
	污染路径: 原头文件定义点	
51	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1250
	步骤描述: 禁止头文件重复包含	
	代码片段: 1248: <pre> *****/ 1249: #include <stdlib.h> /* calloc */ 1250: #include <string.h> /* memcpy, memmove */ 1251: #include <stdio.h> /* debug : printf */ 1252:</pre>	
	污染路径: 原头文件定义点	
52	缺陷发生位置: /source/common/src/tvariant.c	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: <pre> 18: #include "ttime.h" 19: #include "ttokendef.h" 20: #include "tvariant.h" 21: 22: static int32_t parseBinaryUInteger(const char *z, int32_t n, uint64_t *value) {</pre>	
	污染路径: 原头文件定义点	
53	缺陷发生位置: /source/client/wrapper/jni/windows/win32/bridge/AccessBridgeCalls.h	行号: 33
	步骤描述: 禁止头文件重复包含	
	代码片段: <pre> 31: #include <jni.h> 32: #include "AccessBridgeCallbacks.h" 33: #include "AccessBridgePackages.h" 34: 35: #ifdef __cplusplus</pre>	

	污染路径: 原头文件定义点	
54	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.h	行号: 21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "bse.h" 20: #include "bseCache.h" 21: #include "bseTable.h" 22: #include "bseUtil.h" 23:	
	污染路径: 原头文件定义点	
55	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSecureErase.c	行号: 97
	步骤描述: 禁止头文件重复包含	
	代码片段: 95: #include "tsdbFSet2.h" 96: #include "tsdbSttFileRW.h" 97: #include "tsdbUtil2.h" 98: #include "ttypes.h" 99:	
	污染路径: 原头文件定义点	
56	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1136
	步骤描述: 禁止头文件重复包含	
	代码片段: 1134: ***** */ 1135: #include <stdlib.h> /* malloc, free, qsort */ 1136: #include <string.h> /* memcpy, memset */ 1137: #include <stdio.h> /* printf (debug) */ 1138:	
	污染路径: 原头文件定义点	

57	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCache.c	行号: 21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "tsdbReadUtil.h" 20: #include "tss.h" 21: #include "vnd.h" 22: 23: #define ROCKS_BATCH_SIZE (4096)	
	污染路径: 原头文件定义点	
58	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSetRW.h	行号: 17
	步骤描述: 禁止头文件重复包含	
	代码片段: 15: 16: #include "tsdbDataFileRW.h" 17: #include "tsdbSttFileRW.h" 18: 19: #ifndef _TSDB_FSET_RW_H	
	污染路径: 原头文件定义点	
59	缺陷发生位置: /source/os/test/osTimeTests.cpp	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include <thread> 19: #include <vector> 20: #include <chrono> 21: 22: #pragma GCC diagnostic push	
	污染路径: 原头文件定义点	
60	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 21
	步骤描述: 禁止头文件重复包含	

	代码片段: 19: #include <string.h> 20: #include <sys/stat.h> 21: #include <sys/types.h> 22: #include <time.h> 23: // #include <unistd.h>	
	污染路径: 原头文件定义点	
61	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 12
	步骤描述: 禁止头文件重复包含	
	代码片段: 10: #include <limits> 11: #include <stdexcept> 12: #include <string> 13: #include <tuple> 14: #include <type_traits>	
	污染路径: 原头文件定义点	
62	缺陷发生位置: /source/libs/tdb/test/tdbExOVFLTest.cpp	行号: 8
	步骤描述: 禁止头文件重复包含	
	代码片段: 6: 7: #include <shared_mutex> 8: #include <string> 9: #include <thread> 10: #include <vector>	
	污染路径: 原头文件定义点	
63	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 424
	步骤描述: 禁止头文件重复包含	
	代码片段: 422: * Includes 423: *****/ 424: #include <stddef.h> /* size_t, ptrdiff_t */	

	425: 426:	
	污染路径: 原头文件定义点	
64	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 25
	步骤描述: 禁止头文件重复包含	
	代码片段: 23: #include "tsdbMerge.h" 24: #include "tsdbReadUtil.h" 25: #include "tsdbUtil2.h" 26: #include "tsimplehash.h" 27: #include "vnode.h"	
	污染路径: 原头文件定义点	
65	缺陷发生位置: /source/libs/new-stream/src/streamReader.c	行号: 10
	步骤描述: 禁止头文件重复包含	
	代码片段: 8: #include "tarray.h" 9: #include "tdatablock.h" 10: #include "tdef.h" 11: #include "thash.h" 12: #include "tsimplehash.h"	
	污染路径: 原头文件定义点	
66	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1492
	步骤描述: 禁止头文件重复包含	
	代码片段: 1490: * Includes 1491: ***** */ 1492: #include <stdlib.h> /* malloc, free, qsort */ 1493: #include <string.h> /* memcpy, memset */ 1494: #include <stdio.h> /* printf (debug) */	

	污染路径: 原头文件定义点	
67	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 45
	步骤描述: 禁止头文件重复包含	
	代码片段: 43: #include "vnode.h" 44: #include "vnodeInt.h" 45: #include "executor.h" 46: 47: int32_t cacheTag(SVnode* pVnode, SHashObj* metaCache, SExprInfo* pExprInfo, int32_t numOfExpr, SStorageAPI* api, uint64_t uid, col_id_t colId, SRWLatch* lock);	
	污染路径: 原头文件定义点	
68	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 849
	步骤描述: 禁止头文件重复包含	
	代码片段: 847: * Includes 848: *****/ 849: #include <stddef.h> /* size_t */ 850: 851:	
	污染路径: 原头文件定义点	
69	缺陷发生位置: /source/common/test/tmsgTest.cpp	行号: 4
	步骤描述: 禁止头文件重复包含	
	代码片段: 2: #include <iomanip> 3: #include <fstream> 4: #include <sstream> 5: #include <string> 6: #include <vector>	
	污染路径:	

	原头文件定义点	
70	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1753
	步骤描述: 禁止头文件重复包含	
	代码片段: 1751: ***** */ 1752: #include <stdlib.h> /* malloc, free, qsort */ 1753: #include <string.h> /* memcpy, memset */ 1754: #include <stdio.h> /* printf (debug) */ 1755:	
	污染路径: 原头文件定义点	
71	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 32
	步骤描述: 禁止头文件重复包含	
	代码片段: 30: #include "filter.h" 31: #include "functionMgt.h" 32: #include "parUtil.h" 33: #include "planner.h" 34: #include "scalar.h"	
	污染路径: 原头文件定义点	
72	缺陷发生位置: /utils/test/c/tmqSim.c	行号: 22
	步骤描述: 禁止头文件重复包含	
	代码片段: 20: #include <string.h> 21: #include <sys/stat.h> 22: #include <sys/types.h> 23: #include <time.h> 24:	
	污染路径: 原头文件定义点	
73	缺陷发生位置:	行号:

	/source/dnode/vnode/src/tsdb/tsdbMergeTree.c	21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "tsdbMerge.h" 20: #include "tsdbReadUtil.h" 21: #include "tsdbSttFileRW.h" 22: #include "ttypes.h" 23:	
	污染路径: 原头文件定义点	
74	缺陷发生位置: /source/libs/executor/src/forecastoperator.c	行号: 26
	步骤描述: 禁止头文件重复包含	
	代码片段: 24: #include "tdatablock.h" 25: #include "tmsg.h" 26: #include "tmsg.h" 27: 28: #ifdef USE_ANALYTICS	
	污染路径: 原头文件定义点	
75	缺陷发生位置: /source/util/test/timerTest.cpp	行号: 58
	步骤描述: 禁止头文件重复包含	
	代码片段: 56: #include <mutex> 57: #include <condition_variable> 58: #include <chrono> 59: #include <thread> 60:	
	污染路径: 原头文件定义点	
76	缺陷发生位置: /source/common/test/tmsgTest.cpp	行号: 5
	步骤描述: 禁止头文件重复包含	
	代码片段:	

	3: #include <fstream> 4: #include <sstream> 5: #include <string> 6: #include <vector> 7: #include <cstring>	
	污染路径: 原头文件定义点	
77	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 9
	步骤描述: 禁止头文件重复包含	
	代码片段: 7: #include <time.h> 8: #include <stdlib.h> 9: #include <getopt.h> 10: #include <raft.h> 11: #include <raft/uv.h>	
	污染路径: 原头文件定义点	
78	缺陷发生位置: /source/dnode/vnode/src/bse/bseMgt.c	行号: 21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "bseTable.h" 20: #include "bseTableMgt.h" 21: #include "bseUtil.h" 22: #include "cJSON.h" 23: #include "tencrypt.h"	
	污染路径: 原头文件定义点	
79	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 17
	步骤描述: 禁止头文件重复包含	
	代码片段: 15: 16: #include "tsdbReadUtil.h" 17: #include "tsdb.h" 18: #include "tsdbDataFileRW.h"	

	19: #include "tsdbFS2.h"	
	污染路径: 原头文件定义点	
80	缺陷发生位置: /source/client/test/tmqTest.cpp	行号: 29
	步骤描述: 禁止头文件重复包含	
	代码片段: 27: 28: #include "../inc/clientInt.h" 29: #include "taos.h" 30: 31: int main(int argc, char** argv) {	
	污染路径: 原头文件定义点	
81	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRetention.c	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include "tsdbFS2.h" 19: #include "tsdbFSet2.h" 20: #include "vnd.h" 21: #include "tsdbInt.h" 22:	
	污染路径: 原头文件定义点	
82	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbIter.h	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdbDataFileRW.h" 18: #include "tsdbDef.h" 19: #include "tsdbSttFileRW.h" 20: 21: #ifndef _TSDB_ITER_H_	
	污染路径: 原头文件定义点	

83	缺陷发生位置： /source/libs/tdb/test/tdbBtreeToStackTest.cpp	行号： 9
	步骤描述： 禁止头文件重复包含	
	代码片段： 7: 8: #include <shared_mutex> 9: #include <string> 10: #include <thread> 11: #include <vector>	
	污染路径： 原头文件定义点	
84	缺陷发生位置： /source/libs/index/test/jsonUT.cc	行号： 4
	步骤描述： 禁止头文件重复包含	
	代码片段： 2: #include <algorithm> 3: #include <iostream> 4: #include <string> 5: #include <thread> 6: #include <vector>	
	污染路径： 原头文件定义点	
85	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号： 20
	步骤描述： 禁止头文件重复包含	
	代码片段： 18: #include "tsdbDataFileRW.h" 19: #include "tsdbFS2.h" 20: #include "tsdbSttFileRW.h" 21: 22: bool tsdbShouldForceRepair(STFileSystem *fs) {	
	污染路径： 原头文件定义点	
86	缺陷发生位置： /tools/taos-tools/inc/bench.h	行号： 51
	步骤描述： 禁止头文件重复包含	

	代码片段: 49: #include <sys/prctl.h> 50: #include <sys/socket.h> 51: #include <netinet/in.h> 52: #include <arpa/inet.h> 53: #include <sys/time.h>	
	污染路径: 原头文件定义点	
87	缺陷发生位置: /source/dnode/vnode/src/bse/bseCache.h	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: 19: #include "bseTable.h" 20: #include "bseUtil.h" 21: #include "tlockfree.h" 22:	
	污染路径: 原头文件定义点	
88	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: 19: #include <regex.h> 20: #include <stddef.h> 21: #include <stdio.h> 22: #include <string.h>	
	污染路径: 原头文件定义点	
89	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 45
	步骤描述: 禁止头文件重复包含	
	代码片段: 43: #define FSE_STATIC_LINKING_ONLY 44: #include "fse.h" 45: #include "error_private.h"	

	46: 47:	
	污染路径: 原头文件定义点	
90	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 61
	步骤描述: 禁止头文件重复包含	
	代码片段: 59: * Dependencies 60: *****/ 61: #include <stddef.h> /* size_t, ptrdiff_t */ 62: #include <string.h> /* memcpy */ 63:	
	污染路径: 原头文件定义点	
91	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 29
	步骤描述: 禁止头文件重复包含	
	代码片段: 27: 28: #include "../inc/clientSml.h" 29: #include "taos.h" 30: 31: int main(int argc, char **argv) {	
	污染路径: 原头文件定义点	
92	缺陷发生位置: /source/dnode/vnode/src/bse/bseTableMgt.c	行号: 17
	步骤描述: 禁止头文件重复包含	
	代码片段: 15: 16: #include "bseTableMgt.h" 17: #include "bse.h" 18: #include "bseTable.h" 19: #include "bseUtil.h"	
	污染路径:	

	原头文件定义点	
93	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.c	行号: 23
	步骤描述: 禁止头文件重复包含	
	代码片段: 21: 22: // new 23: #include "tsdbDataFileRW.h" 24: #include "tsdbFS2.h" 25: #include "tsdbSttFileRW.h"	
	污染路径: 原头文件定义点	
94	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.h	行号: 18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include "tsdb.h" 17: #include "tsdbDataFileRW.h" 18: #include "tsdbDef.h" 19: #include "tsdbFS2.h" 20: #include "tsdbUtil2.h"	
	污染路径: 原头文件定义点	
95	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSecureErase.c	行号: 93
	步骤描述: 禁止头文件重复包含	
	代码片段: 91: #include "tglobal.h" 92: #include "tsdbDataFileRW.h" 93: #include "tsdbDef.h" 94: #include "tsdbFS2.h" 95: #include "tsdbFSet2.h"	
	污染路径: 原头文件定义点	
96	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 23
	步骤描述:	

	禁止头文件重复包含	
	代码片段: 21: 22: #include <shared_mutex> 23: #include <string> 24: #include <thread> 25: #include <vector>	
	污染路径: 原头文件定义点	
97	缺陷发生位置: /tools/taos-tools/src/benchMain.c	行号: 16
	步骤描述: 禁止头文件重复包含	
	代码片段: 14: #include "benchLog.h" 15: #include <benchCsv.h> 16: #include <toolsdef.h> 17: 18: SArguments* g_arguments;	
	污染路径: 原头文件定义点	
98	缺陷发生位置: /source/libs/index/test/indexTests.cc	行号: 18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include <algorithm> 17: #include <iostream> 18: #include <string> 19: #include <thread> 20: #include "index.h"	
	污染路径: 原头文件定义点	
99	缺陷发生位置: /source/dnode/vnode/test/tsdbSmaTest.cpp	行号: 24
	步骤描述: 禁止头文件重复包含	
	代码片段: 22: 23: #include <tmsg.h>	

	24: #include <vnodeInt.h> 25: 26: #pragma GCC diagnostic push	
	污染路径: 原头文件定义点	
100	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 7
	步骤描述: 禁止头文件重复包含	
	代码片段: 5: #include <assert.h> 6: #include <getopt.h> 7: #include <time.h> 8: #include <stdlib.h> 9: #include <getopt.h>	
	污染路径: 原头文件定义点	
101	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdbDataFileRAW.h" 18: #include "tsdbDataFileRW.h" 19: #include "tsdbFS2.h" 20: #include "tsdbSttFileRW.h" 21:	
	污染路径: 原头文件定义点	
102	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1251
	步骤描述: 禁止头文件重复包含	
	代码片段: 1249: #include <stdlib.h> /* calloc */ 1250: #include <string.h> /* memcpy, memmove */ 1251: #include <stdio.h> /* debug : printf */ 1252: 1253:	

	污染路径: 原头文件定义点	
103	缺陷发生位置: /source/libs/executor/src/executorInt.c	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include "functionMgt.h" 19: #include "../function/inc/functionResInfoInt.h" 20: #include "os.h" 21: #include "querynodes.h" 22: #include "tfill.h"	
	污染路径: 原头文件定义点	
104	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: 18: #include "tdef.h" 19: #include "decimal.h" 20: 21: extern char g_configDir[MAX_PATH_LEN];	
	污染路径: 原头文件定义点	
105	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 13
	步骤描述: 禁止头文件重复包含	
	代码片段: 11: #include <cstring> 12: #include <limits> 13: #include <type_traits> 14: 15: namespace intx	
	污染路径: 原头文件定义点	
106	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1026

	步骤描述: 禁止头文件重复包含	
	代码片段: 1024: <pre>***** */ 1025: #include <stdlib.h> /* malloc, free, qsort */ 1026: #include <string.h> /* memcpy, memset */ 1027: #include <stdio.h> /* printf (debug) */ 1028:</pre>	
	污染路径: 原头文件定义点	
107	缺陷发生位置:	行号:
	/tools/taos-tools/inc/dump.h	28
	步骤描述: 禁止头文件重复包含	
	代码片段: <pre>26: #endif 27: 28: #include <stdio.h> 29: #include <stdlib.h> 30: #include <pthread.h></pre>	
108	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbSecureErase.c	94
	步骤描述: 禁止头文件重复包含	
109	代码片段:	
	<pre>92: #include "tsdbDataFileRW.h" 93: #include "tsdbDef.h" 94: #include "tsdbFS2.h" 95: #include "tsdbFSet2.h" 96: #include "tsdbSttFileRW.h"</pre>	
	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/source/os/src/osFile.c	44
	步骤描述: 禁止头文件重复包含	

	代码片段: 42: #include <sys/stat.h> 43: #include <sys/uio.h> 44: #include <unistd.h> 45: #define LINUX_FILE_NO_TEXT_OPTION 0 46: #define O_TEXT LINUX_FILE_NO_TEXT_OPTION	
	污染路径: 原头文件定义点	
110	缺陷发生位置: /source/client/inc/clientSml.h	行号: 31
	步骤描述: 禁止头文件重复包含	
	代码片段: 29: #include "tglobal.h" 30: #include "tlog.h" 31: #include "tmsg.h" 32: #include "tname.h" 33: #include "ttime.h"	
	污染路径: 原头文件定义点	
111	缺陷发生位置: /source/libs/new-stream/test/dataSinkTest.cpp	行号: 23
	步骤描述: 禁止头文件重复包含	
	代码片段: 21: #include <mutex> 22: #include <condition_variable> 23: #include <vector> 24: #include <random> 25:	
	污染路径: 原头文件定义点	
112	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 8
	步骤描述: 禁止头文件重复包含	
	代码片段: 6: 7: #include "int128.hpp" 8: #include <algorithm>	

	9: #include <array> 10: #include <cstdint>	
	污染路径: 原头文件定义点	
113	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.c	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include "bseTable.h" 19: #include "bseTableMgt.h" 20: #include "bseUtil.h" 21: #include "vnodeInt.h" 22:	
	污染路径: 原头文件定义点	
114	缺陷发生位置: /source/dnode/mnode/impl/src/mndScan.c	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include "mndDnode.h" 19: #include "mndPrivilege.h" 20: #include "mndScan.h" 21: #include "mndScanDetail.h" 22: #include "mndShow.h"	
	污染路径: 原头文件定义点	
115	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRead2.c	行号: 22
	步骤描述: 禁止头文件重复包含	
	代码片段: 20: #include "tsdb.h" 21: #include "tsdbDataFileRW.h" 22: #include "tsdbFS2.h" 23: #include "tsdbMerge.h" 24: #include "tsdbReadUtil.h"	
	污染路径:	

	原头文件定义点	
116	缺陷发生位置： /test/new_test_framework/script/api/insert_stb.c	行号： 24
	步骤描述： 禁止头文件重复包含	
	代码片段： 22: #include <string.h> 23: #include <sys/time.h> 24: #include <stdint.h> 25: 26: #include "taos.h" // TAOS header file	
	污染路径： 原头文件定义点	
117	缺陷发生位置： /source/libs/index/test/utilUT.cc	行号： 4
	步骤描述： 禁止头文件重复包含	
	代码片段： 2: #include <algorithm> 3: #include <iostream> 4: #include <string> 5: #include <thread> 6: #include <vector>	
	污染路径： 原头文件定义点	
118	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbUpgrade.c	行号： 19
	步骤描述： 禁止头文件重复包含	
	代码片段： 17: 18: // old 19: #include "tsdb.h" 20: // extern void tsdbGetCurrentFName(STsdb *pTsdb, char *current, char *current_t); 21:	
	污染路径： 原头文件定义点	
119	缺陷发生位置： /source/libs/decimal/src/detail/intx/intx.hpp	行号： 12

	步骤描述: 禁止头文件重复包含	
	代码片段: 10: #include <cstdint> 11: #include <cstring> 12: #include <limits> 13: #include <type_traits> 14:	
	污染路径: 原头文件定义点	
120	缺陷发生位置:	行号:
	/source/libs/tdb/test/tdbPageDefragmentTest.cpp	8
	步骤描述: 禁止头文件重复包含	
	代码片段: 6: 7: #include <shared_mutex> 8: #include <string> 9: #include <thread> 10: #include <vector>	
121	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/source/libs/index/test/fstUT.cc	5
	步骤描述: 禁止头文件重复包含	
122	代码片段: 3: #include <algorithm> 4: #include <iostream> 5: #include <string> 6: #include <thread> 7: #include <vector>	
	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/source/libs/planner/test/planTestUtil.cpp	27
	步骤描述: 禁止头文件重复包含	
	代码片段: 25: #include "mockCatalogService.h"	

	26: #include "parser.h" 27: #include "planInt.h" 28: #include "tglobal.h" 29:	
	污染路径: 原头文件定义点	
123	缺陷发生位置: /source/common/test/SClientHbBatchReq.cpp	行号: 4
	步骤描述: 禁止头文件重复包含	
	代码片段: 2: #include <vector> 3: #include <cstring> 4: #include "tmsg.h" 5: 6: // ===== ===== ===	
	污染路径: 原头文件定义点	
124	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1655
	步骤描述: 禁止头文件重复包含	
	代码片段: 1653: * Includes 1654: ***** */ 1655: #include <stdlib.h> /* malloc, free, qsort */ 1656: #include <string.h> /* memcpy, memset */ 1657: #include <stdio.h> /* printf (debug) */	
	污染路径: 原头文件定义点	
125	缺陷发生位置: /source/common/test/SClientHbBatchReq.cpp	行号: 3
	步骤描述: 禁止头文件重复包含	
	代码片段:	

	1: #include <gtest/gtest.h> 2: #include <vector> 3: #include <cstring> 4: #include "tmsg.h" 5:	
	污染路径: 原头文件定义点	
126	缺陷发生位置: /source/util/test/memPoolTest.cpp	行号: 41
	步骤描述: 禁止头文件重复包含	
	代码片段: 39: #include "stub.h" 40: #include "../inc/tmempoolInt.h" 41: #include "tglobal.h" 42: 43: namespace {	
	污染路径: 原头文件定义点	
127	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 1249
	步骤描述: 禁止头文件重复包含	
	代码片段: 1247: * Includes 1248: *****/ 1249: #include <stdlib.h> /* calloc */ 1250: #include <string.h> /* memcpy, memmove */ 1251: #include <stdio.h> /* debug : printf */	
	污染路径: 原头文件定义点	
128	缺陷发生位置: /contrib/test/craft/simulate_vnode.c	行号: 7
	步骤描述: 禁止头文件重复包含	
	代码片段: 5: #include <pthread.h> 6: #include <string.h> 7: #include <time.h>	

	8: #include <stdlib.h> 9: #include <getopt.h>	
	污染路径: 原头文件定义点	
129	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbCommit2.h	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include "tsdbFSetRW.h" 19: #include "tsdbIter.h" 20: #include "tsdbSttFileRW.h" 21: 22: #ifndef _TSDB_COMMIT_H_	
	污染路径: 原头文件定义点	
130	缺陷发生位置: /source/libs/metrics/test/metricsTest.cpp	行号: 14
	步骤描述: 禁止头文件重复包含	
	代码片段: 12: #include <gtest/gtest.h> 13: #include <thread> 14: #include <chrono> 15: #include <vector> 16: #include <random>	
	污染路径: 原头文件定义点	
131	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdb.h" 18: #include "tsdbDataFileRW.h" 19: #include "tsdbFS2.h" 20: #include "tsdbMerge.h" 21: #include "tsdbUtil2.h"	
	污染路径:	

	原头文件定义点	
132	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.h	行号: 25
	步骤描述: 禁止头文件重复包含	
	代码片段: 23: #include "storageapi.h" 24: #include "tsdbDataFileRW.h" 25: #include "tsdbUtil2.h" 26: 27: #define ASCENDING_TRAVERSE(o) (o == TSDB_ORDER_ASC)	
	污染路径: 原头文件定义点	
133	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.c	行号: 24
	步骤描述: 禁止头文件重复包含	
	代码片段: 22: // new 23: #include "tsdbDataFileRW.h" 24: #include "tsdbFS2.h" 25: #include "tsdbSttFileRW.h" 26:	
	污染路径: 原头文件定义点	
134	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRAW.h	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdbDef.h" 18: #include "tsdbFSet2.h" 19: #include "tsdbFile2.h" 20: #include "tsdbUtil2.h" 21:	
	污染路径: 原头文件定义点	
135	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/	行号: 34

	test/argp-test.c	
	步骤描述: 禁止头文件重复包含	
	代码片段: 32: #include <string.h> 33: #include <argp.h> 34: #include <stdarg.h> 35: 36: #ifdef _WIN32	
	污染路径: 原头文件定义点	
136	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 70
	步骤描述: 禁止头文件重复包含	
	代码片段: 68: * Includes 69: *****/ 70: #include <stddef.h> /* size_t, ptrdiff_t */ 71: #include <string.h> /* memcpy */ 72:	
	污染路径: 原头文件定义点	
137	缺陷发生位置: /source/util/test/terrorTest.cpp	行号: 8
	步骤描述: 禁止头文件重复包含	
	代码片段: 6: #include <fstream> 7: #include <sstream> 8: #include <string> 9: #include <vector> 10: #include <unistd.h>	
	污染路径: 原头文件定义点	
138	缺陷发生位置: /source/common/test/SClientHbBatchReq.cpp	行号: 2
	步骤描述: 禁止头文件重复包含	
	代码片段:	

	0: #include <gtest/gtest.h> 1: #include <vector> 2: #include <cstring> 3: #include "tmsg.h" 4:	
	污染路径: 原头文件定义点	
139	缺陷发生位置: /docs/examples/c-ws/tmq_demo.c	行号: 23
	步骤描述: 禁止头文件重复包含	
	代码片段: 21: #include <stdlib.h> 22: #include <string.h> 23: #include <time.h> 24: #include <unistd.h> 25: #include "taosws.h"	
	污染路径: 原头文件定义点	
140	缺陷发生位置: /source/dnode/vnode/src/bse/bseSnapshot.h	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: 19: #include "bseInc.h" 20: #include "bseUtil.h" 21: 22: #ifdef __cplusplus	
	污染路径: 原头文件定义点	
141	缺陷发生位置: /tools/inc/pub.h	行号: 27
	步骤描述: 禁止头文件重复包含	
	代码片段: 25: #include <ctype.h> 26: #include <inttypes.h> 27: #include <stdint.h> 28: #include <stdlib.h>	

	29: #include <string.h>	
	污染路径: 原头文件定义点	
142	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 14
	步骤描述: 禁止头文件重复包含	
	代码片段: 12: /*- Dependencies -*/ 13: #include "zstd_v06.h" 14: #include <stddef.h> /* size_t, ptrdiff_t */ 15: #include <string.h> /* memcpy */ 16: #include <stdlib.h> /* malloc, free, qsort */	
	污染路径: 原头文件定义点	
143	缺陷发生位置: /source/common/src/msg/tmsg.c	行号: 43
	步骤描述: 禁止头文件重复包含	
	代码片段: 41: #undef TD_MSG_SEG_CODE_ 42: #define TD_MSG_RANGE_CODE_ 43: #include "tmsgdef.h" 44: #include "tversion.h" 45:	
	污染路径: 原头文件定义点	
144	缺陷发生位置: /source/libs/index/test/fstUtilUT.cc	行号: 5
	步骤描述: 禁止头文件重复包含	
	代码片段: 3: #include <algorithm> 4: #include <iostream> 5: #include <string> 6: #include <thread> 7: #include <vector>	
	污染路径: 原头文件定义点	

145	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1493
	步骤描述: 禁止头文件重复包含	
	代码片段: 1491: ***** */ 1492: #include <stdlib.h> /* malloc, free, qsort */ 1493: #include <string.h> /* memcpy, memset */ 1494: #include <stdio.h> /* printf (debug) */ 1495:	
	污染路径: 原头文件定义点	
146	缺陷发生位置: /source/dnode/mgmt/node_mgmt/src/dmMgmt.c	行号: 25
	步骤描述: 禁止头文件重复包含	
	代码片段: 23: #include "tglobal.h" 24: #include "tgrant.h" 25: #include "tconv.h" 26: #include "stream.h" 27: #if defined(TD_ENTERPRISE) && defined(TD_HAS_TAOSK)	
	污染路径: 原头文件定义点	
147	缺陷发生位置: /source/client/inc/clientSml.h	行号: 27
	步骤描述: 禁止头文件重复包含	
	代码片段: 25: #include "taos.h" 26: #include "taoserror.h" 27: #include "tcommon.h" 28: #include "tdef.h" 29: #include "tglobal.h"	
	污染路径: 原头文件定义点	
148	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/	行号: 68

	argp/argp-help.c	
	步骤描述: 禁止头文件重复包含	
	代码片段: 66: 67: 68: #include <stddef.h> 69: #include <stdlib.h> 70: #include <string.h>	
	污染路径: 原头文件定义点	
149	缺陷发生位置:	行号:
	/source/os/src/osSocket.c	33
	步骤描述: 禁止头文件重复包含	
	代码片段: 31: #include <net/if.h> 32: #include <netdb.h> 33: #include <netinet/in.h> 34: #include <netinet/ip.h> 35: #include <netinet/tcp.h>	
150	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/source/dnode/vnode/src/tsdb/tsdbMigrate.c	19
	步骤描述: 禁止头文件重复包含	
151	代码片段: 17: #include "tsdb.h" 18: #include "tsdbFS2.h" 19: #include "tsdbFSet2.h" 20: #include "vnd.h" 21: #include "tsdbInt.h"	
	污染路径: 原头文件定义点	
	缺陷发生位置:	行号:
	/tools/taos-tools/src/toolstime.c	90
	步骤描述: 禁止头文件重复包含	
	代码片段:	

	88: 89: #else 90: #include <sys/time.h> 91: #endif 92:	
	污染路径: 原头文件定义点	
152	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.h	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdbDataFileRW.h" 18: #include "tsdbDef.h" 19: #include "tsdbFS2.h" 20: #include "tsdbUtil2.h" 21:	
	污染路径: 原头文件定义点	
153	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.h	行号: 21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "bse.h" 20: #include "bseInc.h" 21: #include "bseUtil.h" 22: #include "cJSON.h" 23: #include "os.h"	
	污染路径: 原头文件定义点	
154	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.h	行号: 20
	步骤描述: 禁止头文件重复包含	
	代码片段: 18: #include "tsdbDef.h" 19: #include "tsdbFS2.h" 20: #include "tsdbUtil2.h" 21:	

	22: #ifndef _TSDB_UPGRADE_H_	
	污染路径: 原头文件定义点	
155	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbUpgrade.c	行号: 25
	步骤描述: 禁止头文件重复包含	
	代码片段: 23: #include "tsdbDataFileRW.h" 24: #include "tsdbFS2.h" 25: #include "tsdbSttFileRW.h" 26: 27: static int32_t tsdbUpgradeHead(STsdb *tsdb, SDFileSet *pDFileSet, SDataFReader *reader, STFileSet *fset) {	
	污染路径: 原头文件定义点	
156	缺陷发生位置: /tools/taos-tools/inc/bench.h	行号: 50
	步骤描述: 禁止头文件重复包含	
	代码片段: 48: #include <netdb.h> 49: #include <sys/prctl.h> 50: #include <sys/socket.h> 51: #include <netinet/in.h> 52: #include <arpa/inet.h>	
	污染路径: 原头文件定义点	
157	缺陷发生位置: /source/client/inc/clientSml.h	行号: 25
	步骤描述: 禁止头文件重复包含	
	代码片段: 23: #include "osThread.h" 24: #include "query.h" 25: #include "taos.h" 26: #include "taoserror.h" 27: #include "tcommon.h"	
	污染路径:	

	原头文件定义点	
158	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.h	行号: 27
	步骤描述: 禁止头文件重复包含	
	代码片段: 25: #include "tcompare.h" 26: #include "tlog.h" 27: #include "tmsg.h" 28: #include "tutil.h" 29:	
	污染路径: 原头文件定义点	
159	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 10
	步骤描述: 禁止头文件重复包含	
	代码片段: 8: #include <algorithm> 9: #include <array> 10: #include <cstdlib> 11: #include <cstring> 12: #include <limits>	
	污染路径: 原头文件定义点	
160	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.c	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "bseCache.h" 18: #include "bseSnapshot.h" 19: #include "bseTable.h" 20: #include "bseTableMgt.h" 21: #include "crypt.h"	
	污染路径: 原头文件定义点	
161	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 9
	步骤描述:	

	禁止头文件重复包含	
	代码片段: 7: #include "int128.hpp" 8: #include <algorithm> 9: #include <array> 10: #include <cstdint> 11: #include <cstring>	
	污染路径: 原头文件定义点	
162	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbSnapshot.c	行号: 18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include "tsdb.h" 17: #include "tsdbDataFileRW.h" 18: #include "tsdbFS2.h" 19: #include "tsdbFSetRW.h" 20: #include "tsdbIter.h"	
	污染路径: 原头文件定义点	
163	缺陷发生位置: /source/dnode/vnode/src/bse/bseTable.h	行号: 26
	步骤描述: 禁止头文件重复包含	
	代码片段: 24: #include "tchecksum.h" 25: #include "tcompare.h" 26: #include "tlog.h" 27: #include "tmsg.h" 28: #include "tutil.h"	
	污染路径: 原头文件定义点	
164	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1020
	步骤描述: 禁止头文件重复包含	
	代码片段: 1018: *****	

	<pre> */ 1019: #include <stdlib.h> /* malloc, free, qsort */ 1020: #include <string.h> /* memcpy, memset */ 1021: #include <stdio.h> /* printf (debug) */ 1022: </pre>	
	污染路径: 原头文件定义点	
165	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 7
	步骤描述: 禁止头文件重复包含	
	代码片段: <pre> 5: #include <inttypes.h> 6: #include <stddef.h> 7: #include <stdint.h> 8: #include <stdio.h> 9: #include <stdlib.h> </pre>	
	污染路径: 原头文件定义点	
166	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 33
	步骤描述: 禁止头文件重复包含	
	代码片段: <pre> 31: #include "tdatablock.h" 32: #include "tdb.h" 33: #include "tdef.h" 34: #include "tencode.h" 35: #include "tglobal.h" </pre>	
	污染路径: 原头文件定义点	
167	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqNet2.c	行号: 6
	步骤描述: 禁止头文件重复包含	
	代码片段: <pre> 4: #include <netinet/tcp.h> 5: #include <strings.h> 6: #include <sys/socket.h> 7: #include <unistd.h> </pre>	

	8:	
	污染路径: 原头文件定义点	
168	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1752
	步骤描述: 禁止头文件重复包含	
	代码片段: 1750: * Includes 1751: ***** */ 1752: #include <stdlib.h> /* malloc, free, qsort */ 1753: #include <string.h> /* memcpy, memset */ 1754: #include <stdio.h> /* printf (debug) */	
	污染路径: 原头文件定义点	
169	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbDataFileRW.h	行号: 19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "tsdbFSet2.h" 18: #include "tsdbSttFileRW.h" 19: #include "tsdbUtil2.h" 20: 21: #ifndef _TSDB_DATA_FILE_RW_H	
	污染路径: 原头文件定义点	
170	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1025
	步骤描述: 禁止头文件重复包含	
	代码片段: 1023: ***** */ 1024: #include <stdlib.h> /* malloc, free, qsort */ 1025: #include <string.h> /* memcpy, memset */ 1026: #include <stdio.h> /* printf (debug) */	

	1027:	
	污染路径: 原头文件定义点	
171	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbReadUtil.c	行号: 21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "tsdbFS2.h" 20: #include "tsdbMerge.h" 21: #include "tsdbUtil2.h" 22: #include "tsimplehash.h" 23:	
	污染路径: 原头文件定义点	
172	缺陷发生位置: /source/libs/tdb/test/tdbTest.cpp	行号: 8
	步骤描述: 禁止头文件重复包含	
	代码片段: 6: 7: #include <shared_mutex> 8: #include <string> 9: #include <thread> 10: #include <vector>	
	污染路径: 原头文件定义点	
173	缺陷发生位置: /contrib/test/craft/raftMain.c	行号: 13
	步骤描述: 禁止头文件重复包含	
	代码片段: 11: #include <raft/uv.h> 12: #include "raftServer.h" 13: #include "tcommon.h" 14: 15: const char *exe_name;	
	污染路径: 原头文件定义点	

174	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbMigrate.c	行号： 20
	步骤描述： 禁止头文件重复包含	
	代码片段： 18: #include "tsdbFS2.h" 19: #include "tsdbFSet2.h" 20: #include "vnd.h" 21: #include "tsdbInt.h" 22:	
	污染路径： 原头文件定义点	
175	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号： 26
	步骤描述： 禁止头文件重复包含	
	代码片段： 24: #include <search.h> 25: #include <signal.h> 26: #include <stdio.h> 27: #include <stdlib.h> 28: #include <string.h>	
	污染路径： 原头文件定义点	
176	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号： 20
	步骤描述： 禁止头文件重复包含	
	代码片段： 18: #include "tsdbFS2.h" 19: #include "tsdbUpgrade.h" 20: #include "vnd.h" 21: 22: #define BLOCK_COMMIT_FACTOR 3	
	污染路径： 原头文件定义点	
177	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbMerge.h	行号： 21
	步骤描述：	

	禁止头文件重复包含	
	代码片段: 19: #include "tsdbIter.h" 20: #include "tsdbSttFileRW.h" 21: #include "tsdbUtil2.h" 22: 23: #ifndef _TD_TSDB_MERGE_H_	
	污染路径: 原头文件定义点	
178	缺陷发生位置:	行号:
	/source/dnode/vnode/src/bse/bseTableMgt.c	19
	步骤描述: 禁止头文件重复包含	
	代码片段: 17: #include "bse.h" 18: #include "bseTable.h" 19: #include "bseUtil.h" 20: #include "tglobal.h" 21: #include "thash.h"	
	污染路径: 原头文件定义点	
179	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v03.c	71
	步骤描述: 禁止头文件重复包含	
	代码片段: 69: * Includes 70: *****/ 71: #include <stddef.h> /* size_t, ptrdiff_t */ 72: #include <string.h> /* memcpy */ 73:	
	污染路径: 原头文件定义点	
180	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchCommandOpt.c	16
	步骤描述: 禁止头文件重复包含	
	代码片段: 14: #include <bench.h> 15: #include "benchLog.h"	

	16: #include <toolsdef.h> 17: 18: extern char g_configDir[MAX_PATH_LEN];	
	污染路径: 原头文件定义点	
181	缺陷发生位置: /tools/taos-tools/src/toolstime.c	行号: 36
	步骤描述: 禁止头文件重复包含	
	代码片段: 34: 35: #include "bench.h" 36: #include "toolsdef.h" 37: 38: #if defined(WINDOWS)	
	污染路径: 原头文件定义点	
182	缺陷发生位置: /docs/examples/c-ws-new/tmq_demo.c	行号: 8
	步骤描述: 禁止头文件重复包含	
	代码片段: 6: #include <stdlib.h> 7: #include <string.h> 8: #include <time.h> 9: #include <unistd.h> 10: #include "taos.h"	
	污染路径: 原头文件定义点	
183	缺陷发生位置: /source/util/test/terrorTest.cpp	行号: 7
	步骤描述: 禁止头文件重复包含	
	代码片段: 5: #include <iomanip> 6: #include <fstream> 7: #include <sstream> 8: #include <string> 9: #include <vector>	

	污染路径: 原头文件定义点	
184	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 21
	步骤描述: 禁止头文件重复包含	
	代码片段: 19: #include "tglobal.h" 20: #include "thash.h" 21: #include <time.h> 22: #include <osSleep.h> 23:	
	污染路径: 原头文件定义点	
185	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdblter.h	行号: 18
	步骤描述: 禁止头文件重复包含	
	代码片段: 16: #include "trbtree.h" 17: #include "tsdbDataFileRW.h" 18: #include "tsdbDef.h" 19: #include "tsdbSttFileRW.h" 20:	
	污染路径: 原头文件定义点	
186	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1481
	步骤描述: 禁止头文件重复包含	
	代码片段: 1479: * Dependency 1480: *****/ 1481: #include <stddef.h> /* size_t */ 1482: 1483:	
	污染路径: 原头文件定义点	
187	缺陷发生位置: /source/dnode/vnode/test/tqTest.cpp	行号: 24

	步骤描述: 禁止头文件重复包含
	代码片段: 22: 23: #include <tmsg.h> 24: #include <vnodeInt.h> 25: 26: #pragma GCC diagnostic push
	污染路径: 原头文件定义点

19 C/C++ 缺陷类型-算术运算

19.1 分项汇总表

序号	C/C++缺陷类型-算术运算	缺陷/安全漏洞名称	数量	缺陷级别
1	WK4707	禁止对逻辑表达式进行位运算	9	严重
2	WK4102	在用于控制程序流程的条件表达式中使用位运算符"&"或" "	5	严重
3	WK4682	逻辑判别表达式中的运算项必须要使用括号	413	严重
4	WK5661	禁止对有符号类型进行移位运算	122	严重

19.2 缺陷/安全漏洞详解

19.2.1 严重类

19.2.1.1 WK4707 禁止对逻辑表达式进行位运算

缺陷数量：9 个		
缺陷描述： 禁止对逻辑表达式进行位运算。		
修复意见： 无		
1	缺陷发生位置： /contrib/TSZ/zstd/compress/zstd_compress.c	行号： 1599
	步骤描述： 禁止对逻辑表达式进行位运算.	
	代码片段： 1597: } 1598: 1599: if ((cLitSize==0) (cLitSize >= srcSize - minGain) ERR_isError(cLitSize)) { 1600: memcpy(nextHuf, prevHuf, sizeof(*prevHuf)); 1601: return ZSTD_noCompressLiterals(dst, dstCapacity, src, srcSize);	
	污染路径： 无	
2	缺陷发生位置： /contrib/TSZ/zstd/legacy/zstd_v07.c	行号： 3550
	步骤描述： 禁止对逻辑表达式进行位运算.	
	代码片段： 3548: 3549: if (ofCode <= 1) { 3550: if ((llCode == 0) & (offset <= 1)) offset = 1- offset; 3551: if (offset) { 3552: size_t const temp = seqState->prevOffset[offset];	

	污染路径: 无	
3	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 234
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 232: { 233: // Analogous to ==, but == not used directly, because that confuses GCC 8-9. 234: return (x.lo != y.lo) (x.hi != y.hi); 235: } 236:	
	污染路径: 无	
4	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 2841
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 2839: if (MEM_32bits()) BIT_reloadDStream(&(seqState->DStream)); 2840: if (offsetCode==0) offset = prevOffset; /* cmove */ 2841: if (offsetCode !litLength) seqState->prevOffset = seq->offset; /* cmove */ 2842: } 2843:	
	污染路径: 无	
5	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 228
	步骤描述: 禁止对逻辑表达式进行位运算.	
	代码片段: 226: // GCC8: generates a bit worse cmp based code 227: // although it generates the xor based one for __int128. 228: return (x.lo == y.lo) & (x.hi == y.hi); 229: }	

	318: int32_t len = sprintf(buf, str, i); 319: STR_TO_VARSTR(varbuf, buf)	
	污染路径: 无	
5	缺陷发生位置: /source/common/test/commonTests.cpp	行号: 336
	步骤描述: 在用于控制流程的条件表达式中使用位运算符	
	代码片段: <pre> 334: SColumnInfoData* p1 = (SColumnInfoData*)taosArrayGet(b->pDataBlock, 1); 335: for (int32_t i = 0; i < 40; ++i) { 336: if (i & 0x01) { 337: ASSERT_EQ(colDataIsNull_f(p0, i), false); 338: ASSERT_EQ(colDataIsNull(p1, b->info.rows, i, nullptr), false); </pre>	
	污染路径: 无	

19.2.1.3 WK4682 逻辑判别表达式中的运算项必须要使用括号

缺陷数量：413 个		
缺陷描述: 逻辑判别表达式中的运算项必须要使用括号。		
修复意见: 无		
1	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 1564
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段: <pre> 1562: { 1563: size_t const minGain = ZSTD_minGain(srcSize, strategy); 1564: size_t const lhSize = 3 + (srcSize >= 1 KB) + (srcSize >= 16 KB); 1565: BYTE* const ostart = (BYTE*)dst; 1566: U32 singleStream = srcSize < 256; </pre>	

	污染路径: 无	
2	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1861
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1859: } 1860: 1861: for (int i = len; i < len + len1; i++) { 1862: void *rightData = prepareNchar(inputNchar[i - len]); 1863: code = makeCalculate(row, key, TSDB_DATA_TYPE_NCHAR, rightData, eRes3[i], op[i], false);	
	污染路径: 无	
3	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 1246
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1244: argp_fmtstream_set_wmargin (stream, uparams.opt_doc_col); 1245: 1246: if (col > (unsigned int) (uparams.opt_doc_col + 3)) 1247: argp_fmtstream_putc (stream, '\n'); 1248: else if (col >= (unsigned int) uparams.opt_doc_col)	
	污染路径: 无	
4	缺陷发生位置: /contrib/lemon/lemon.c	行号: 835
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 833: if(j<p->nLookahead) continue; 834: for(j=0; j<p->nAction; j++){ 835: if(p->aAction[j].lookahead==j+p->mnLookahead- i) break; 836: } 837: if(j==p->nAction){	

	污染路径: 无	
5	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 590
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 588: 589: ip += (bitCount+7)>>3; 590: if ((size_t)(ip-istart) > hbSize) return (size_t)-FSE_ERROR_srcSize_wrong; 591: return ip-istart; 592: }	
	污染路径: 无	
6	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 685
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 683: return; 684: } 685: if (fsize <= 0 fsize > 4 * 1024 * 1024) { 686: TAOS_UNUSED(taosCloseFile(&fp)); 687: tscWarn("sql security: invalid rule file size:%" PRId64 ", using default rules", fsize);	
	污染路径: 无	
7	缺陷发生位置: /source/libs/index/test/fstUtilUT.cc	行号: 106
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 104: sparSetClear(set); 105: 106: for (int i = 'a'; i < 'a' + 20; i++) { 107: EXPECT_EQ(sparSetGet(set, i, NULL), false); 108: }	
	污染路径:	

	无	
8	缺陷发生位置： /contrib/libmqtt/ttq/src/ttqSubs.c	行号： 860
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 858: } 859: 860: for (i = 1; i < count - 1; i++) { 861: (*topics)[i] = (*topics)[i + 1]; 862: }	
	污染路径： 无	
9	缺陷发生位置： /tools/taos-tools/src/dumpUtil.c	行号： 1101
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 1099: } 1100: 1101: if (strcmp(dbPath + len - sizeof(NTABLE_FOLDER) + 1, NTABLE_FOLDER) == 0) { 1102: return true; 1103: }	
	污染路径： 无	
10	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号： 792
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 790: non_opt = (**name != '-'); 791: /* Skip until part of name used for sorting. */ 792: while (**name && !isalnum (**name)) 793: (*name)++; 794: return non_opt;	
	污染路径： 无	
11	缺陷发生位置：	行号：

	/source/libs/index/test/indexTests.cc		914
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 912: SIndexMultiTerm* terms = indexMultiTermCreate(); 913: indexMultiTermAdd(terms, term); 914: for (size_t i = size * 3; i < size * 4; i++) { 915: int tableId = i; 916: int ret = index->Put(terms, tableId);		
	污染路径: 无		
12	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c		行号: 639
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 637: } 638: if (bitD->ptr == bitD->start) { 639: if (bitD->bitsConsumed < sizeof(bitD->bitContainer)*8) return BITv07_DStream_endOfBuffer; 640: return BITv07_DStream_completed; 641: }		
	污染路径: 无		
13	缺陷发生位置: /source/libs/catalog/src/ctgCache.c		行号: 117
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 115: int32_t ctgAcquireDBCachImpl(SCatalog *pCtg, const char *dbFName, SCtgDBCach **pCache, bool acquire) { 116: char *p = strchr(dbFName, '.'); 117: if (p && IS_SYS_DBNAME(p + 1)) { 118: dbFName = p + 1; 119: }		
	污染路径: 无		
14	缺陷发生位置: /tools/taos-tools/src/benchInsert.c		行号: 358
	步骤描述:		

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 356: n = snprintf(tagsBuf + len, tag_buffer_len - len, 357: "%s json", tag->name); 358: if (n < 0 n >= ((int)tag_buffer_len - len)) { 359: errorPrint("%s() LN%d snprintf overflow on %d\ n", 360: __func__, __LINE__, tagIndex); </pre>	
	污染路径: 无	
15	缺陷发生位置: /source/util/test/lrucacheTest.cpp	行号: 299
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 297: LRUHandle* handle = taosLRUCacheLookup(cache, key, strlen(key)); 298: 299: if (i % 2 == 0) { 300: // Should be erased 301: EXPECT_EQ(handle, nullptr); </pre>	
	污染路径: 无	
16	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 974
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 972: { U32 nbBytes = bitD->bitsConsumed >> 3; 973: BITv06_DStream_status result = BITv06_DStream_unfinished; 974: if (bitD->ptr - nbBytes < bitD->start) { 975: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 976: result = BITv06_DStream_endOfBuffer; </pre>	
	污染路径: 无	
17	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 830
	步骤描述:	

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 828: } 829: 830: if (benchArrayEnsureCap(pArray, pArray->size + elems) != 0) { 831: if (free) { 832: tmfree(pData);	
	污染路径: 无	
18	缺陷发生位置: /source/libs/parser/src/parlInsertSql.c	行号: 4147
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 4145: char* tbNameAfterDbName = strchr(pTbName->z, '.', pTbName->n, true); 4146: if (tbNameAfterDbName != NULL) { 4147: if (*(tbNameAfterDbName + 1) == '?') { 4148: pCxt->stmtTbNameFlag &= ~IS_FIXED_VALUE; 4149: char* tbName = NULL;	
	污染路径: 无	
19	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 1052
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1050: start += strlen(buf); 1051: char* end = strchr(start, ""); 1052: if (end && end - start > 0) { 1053: tstrncpy(val, start, end - start + 1); 1054: } else {	
	污染路径: 无	
20	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 1737
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	1735: #endif 1736: } 1737: if (iRow == nRow - 1) { 1738: #ifndef NO_UNALIGNED_ACCESS 1739: walMeta.ekey = pRow->ts; 	
	污染路径: 无	
21	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 969
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 967: } 968: if (bitD->ptr == bitD->start) { 969: if (bitD->bitsConsumed < sizeof(bitD->bitContainer)*8) return BITv06_DStream_endOfBuffer; 970: return BITv06_DStream_completed; 971: }	
	污染路径: 无	
22	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 431
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 429: if (qwtTestSinkBlockNum == 0) { 430: pOutput->bufStatus = DS_BUF_EMPTY; 431: } else if (qwtTestSinkBlockNum <= qwtTestSinkMaxBlockNum * 0.5) { 432: pOutput->bufStatus = DS_BUF_LOW; 433: } else {	
	污染路径: 无	
23	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 945
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 943: } 944:	

	945: if (hSize + dictContentSize < dictBufferCapacity) 946: memmove((char*)dictBuffer + hSize, (char*)dictBuffer + dictBufferCapacity - dictContentSize, dictContentSize); 947: return MIN(dictBufferCapacity, hSize+dictContentSize);	
	污染路径: 无	
24	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 1221
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1219: taosUsleep(taosRand() % 5); 1220: } 1221: if (++n % ctgTestPrintNum == 0) { 1222: (void)printf("Set:%d\n", n); 1223: }	
	污染路径: 无	
25	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 388
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 386: break; 387: } 388: } while (pStr - pSql < length); 389: return false; 390: }	
	污染路径: 无	
26	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 1043
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1041: 1042: char *p = strchr(dbFName, '.'); 1043: if (p && IS_SYS_DBNAME(p + 1)) {	

	1044: dbFName = p + 1; 1045: }	
	污染路径: 无	
27	缺陷发生位置: /utlis/test/c/tmqDemo.c	行号: 279
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 277: sqlLen += sprintf(sqlStr + sqlLen, "create stable if not exists %s (ts timestamp, ", g_stConflInfo.stbName); 278: for (int32_t i = 0; i < g_stConflInfo.numOfColumn; i++) { 279: if (i == g_stConflInfo.numOfColumn - 1) { 280: sqlLen += sprintf(sqlStr + sqlLen, "c%d int) ", i); 281: memcpy(g_pRowValue + dataLen, "66778899", strlen("66778899")); }	
	污染路径: 无	
28	缺陷发生位置: /source/client/test/clientMonitorTests.cpp	行号: 122
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 120: 121: if (len == 0) { // one item is empty 122: if (*(buf - 1) == '[') { // data is "\0" 123: // no data read 124: *buf = ']';	
	污染路径: 无	
29	缺陷发生位置: /source/libs/azure/test/azExceptionTest.cpp	行号: 147
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 145: GTEST_ASSERT_EQ(code, 0); 146: 147: for (int i = 0; i < size / 2; ++i) { 148: GTEST_ASSERT_EQ(pBlock[i * 2], 0);	

	149: GTEST_ASSERT_EQ(pBlock[i * 2 + 1], 1);	
	污染路径: 无	
30	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 86
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 84: 85: int sdbGetRawInt64(SSdbRaw* pRaw, int pos, int64_t* val) { 86: if (pos + 8 <= pRaw->dataLen) { 87: memcpy(val, pRaw->data + pos, 8); 88: return 0;	
	污染路径: 无	
31	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1337
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1335: }; 1336: 1337: for (int i = 0; i < sizeof(ops) / sizeof(ops[0]); i++) { 1338: SMetaTableOp *op = &ops[i]; 1339:	
	污染路径: 无	
32	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 1400
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1398: 1399: fieldName = "last_compact_time"; 1400: if (strncmp(field, fieldName, strlen(fieldName) + 1) == 0) { 1401: *(int64_t *)value = pReader->lastCompactTime; 1402: return TSDB_CODE_SUCCESS;	

	污染路径: 无	
33	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 379
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 377: } 378: 379: if (n < 0 n >= ((int)tag_buffer_len - len)) { 380: errorPrint("%s() LN%d snprintf overflow on %d\n", 381: __func__, __LINE__, tagIndex);	
	污染路径: 无	
34	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 96
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 94: break; 95: } 96: } while (pStr - pSql < length); 97: return false; 98: }	
	污染路径: 无	
35	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 1394
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1392: 1393: fieldName = "total_size"; 1394: if (strncmp(field, fieldName, strlen(fieldName) + 1) == 1395: 0) { 1396: *(int64_t *)value = pReader->totalSize; 1397: return TSDB_CODE_SUCCESS;	
	污染路径: 无	
36	缺陷发生位置:	行号:

	/source/common/test/commonTests.cpp	465
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 463: printf("the %d split, from: %d to %d\n", count++, startIndex, stopIndex); 464: 465: if (stopIndex == numOfRows - 1) { 466: break; 467: } </pre>	
37	缺陷发生位置: /utils/test/c/tmqSim.c	
	行号: 294	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
38	代码片段: <pre> 292: } else if (strcmp(argv[i], "-n") == 0) { 293: g_stConflInfo.payloadLen = atol(argv[++i]); 294: if (g_stConflInfo.payloadLen <= 0 g_stConflInfo.payloadLen > 1024 * 1024 * 1024) { 295: pError("%s calloc size is too large: %s %s", GREEN, argv[++i], NC); 296: exit(-1); </pre>	
	污染路径: 无	
39	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	
	行号: 715	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
38	代码片段: <pre> 713: 714: auto& n = na.num_divisor_words; 715: for (n = num_words; n > 0 && v[n - 1] == 0; --n) 716: ; 717: </pre>	
	污染路径: 无	
39	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	
	行号: 257	

	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 255: lb[i] = lenOfBinaryAct; 256: no_null[i] = 0; 257: is_null[i] = (i % 10 == 2) ? 1 : 0; 258: v->b[i] = (int8_t)(i % 2); 259: v->v1[i] = (int8_t)((i+1) % 2);	
	污染路径: 无	
40	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 99
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 97: { 98: size_t __old; 99: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) 100: __argp_fmtstream_update (__fs); 101: __old = __fs->rmargin;	
	污染路径: 无	
41	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 275
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 273: } 274: } else if (strcmp(argv[i], "-p") == 0) { 275: if (i + 1 < argc) { 276: i++; 277: strncpy(cachePath, argv[i], sizeof(cachePath) - 1);	
	污染路径: 无	
42	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 748
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	<pre> 746: if (bitD->ptr == bitD->start) 747: { 748: if (bitD->bitsConsumed < sizeof(bitD->bitContainer)*8) return BIT_DStream_endOfBuffer; 749: return BIT_DStream_completed; 750: }</pre>	
	污染路径: 无	
43	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 789
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 787: 788: SPgno pgno; 789: if (sldx + i == nCells) { 790: if (TDB_BTREE_PAGE_IS_LEAF(pParent)) { 791: return TSDB_CODE_INTERNAL_ERROR;</pre>	
	污染路径: 无	
44	缺陷发生位置: /tools/rocks-reader/rreader.cpp	行号: 251
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 249: return 0; 250: } else if (strcmp(argv[i], "-t") == 0) { 251: if (i + 1 < argc) { 252: i++; 253: if (strcmp(argv[i], "last") == 0) {</pre>	
	污染路径: 无	
45	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 589
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 587: char * format = "%d %d,"; 588: for(int32_t i = 0; i < cnt; i++) { 589: if (i == cnt - 1) {</pre>	

	590: format = "%d %d"; 591: }	
	污染路径: 无	
46	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 331
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 329: int n; 330: n = snprintf(tagsBuf + len, tag_buffer_len - len, "("); 331: if (n < 0 n >= ((int)tag_buffer_len - len)) { 332: errorPrint("%s() LN%d snprintf overflow\n", 333: __func__, __LINE__);	
	污染路径: 无	
47	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 990
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 988: MEM_STATIC unsigned BITv06_endOfDStream(const BITv06_DStream_t* DStream) 989: { 990: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 991: } 992:	
	污染路径: 无	
48	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1341
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1339: 1340: char* str = NULL; 1341: if (word1->type == WT_VAR_ALLTABLE) { 1342: // ALL_TABLE	

	1343: str = tireSearchWord(WT_VAR_STABLE, pre);	
	污染路径: 无	
49	缺陷发生位置: /contrib/lemon/lemon.c	行号: 768
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 766: */ 767: n = p->nsymbol + 1; 768: if(p->nAction + n >= p->nActionAlloc){ 769: int oldAlloc = p->nActionAlloc; 770: p->nActionAlloc = p->nAction + n + p->nActionAlloc + 20;	
	污染路径: 无	
50	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 631
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 629: } 630: 631: if (pos + bytes > length) { 632: break; 633: }	
	污染路径: 无	
51	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1799
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1797: ASSERT_EQ(code, TSDB_CODE_SUCCESS); 1798: } 1799: for (int i = len; i < len + len1; i++) { 1800: void *rightData = prepareNchar(inputNchar[i - len]); 1801: code = makeCalculate(row, key, TSDB_DATA_TYPE_NCHAR, rightData, eRes[i], op[i], false);	

	污染路径: 无	
52	缺陷发生位置: /source/client/src/clientMonitor.c	行号: 476
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 474: 475: if (len == 0) { // one item is empty 476: if (*(buf - 1) == '[') { // data is "\0" 477: // no data read 478: *buf = ']';	
	污染路径: 无	
53	缺陷发生位置: /contrib/lemon/lemon.c	行号: 581
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 579: exit(1); 580: } 581: for(i=0; i<amt-1; i++) actionfreelist[i].next = &actionfreelist[i+1]; 582: actionfreelist[amt-1].next = 0; 583: }	
	污染路径: 无	
54	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqMisc.c	行号: 70
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 68: } 69: 70: if (!S_ISREG(statbuf.st_mode) && ! S_ISLNK(statbuf.st_mode)) { 71: ttq_log(NULL, TTQ_LOG_ERR, "Error: %s is not a file.", path); 72: fclose(fp);	
	污染路径:	

	无	
55	缺陷发生位置： /source/libs/decimal/src/detail/intx/intx.hpp	行号： 333
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 331: constexpr uint<N> operator<<(const uint<N>& x, const T& shift) noexcept 332: { 333: if (shift < T{sizeof(x) * 8}) 334: return x << static_cast<unsigned>(shift); 335: return 0;	
	污染路径： 无	
56	缺陷发生位置： /contrib/TSZ/zstd/compress/fse_compress.c	行号： 235
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 233: unsigned start = charnum; 234: while (!normalizedCounter[charnum]) charnum+ +; 235: while (charnum >= start+24) { 236: start+=24; 237: bitStream += 0xFFFFU << bitCount;	
	污染路径： 无	
57	缺陷发生位置： /tools/taos-tools/src/benchInsertMix.c	行号： 100
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 98: 99: // malloc buffer 100: for (int32_t i = 0; i < MCNT - 1; i++) { 101: // max 102: if (mix->genCnt[i] > 0) {	
	污染路径： 无	
58	缺陷发生位置：	行号：

	/tools/shell/src/shellTire.c		300
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 298: 299: // previous items already matched 300: if (i == len - 1) { 301: // prefix is match to end char 302: if (c->d) enumAllWords(c->d, prefix, match);		
	污染路径: 无		
59	缺陷发生位置: /source/libs/tdb/test/tdbPageRecycleTest.cpp		行号: 791
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 789: char key[1024] = {0}; 790: int count = sizeof(key) / sizeof(key[0]); 791: for (int i = 0; i < count - 1; ++i) { 792: key[i] = 'a'; 793: }		
	污染路径: 无		
60	缺陷发生位置: /contrib/test/tdev/src/main.c		行号: 83
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 81: break; 82: case D: 83: for (size_t i = 0; i < 10 * 1024 * 1024 * 1024; i++) { 84: tPutD(pBuf, val); 85: if (POINTER_DISTANCE(buf, pBuf) == 1024) {		
	污染路径: 无		
61	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c		行号: 742
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段:		

	<pre> 740: if (fset == NULL) return; 741: 742: for (tsdb_fstype_t ftype = TSDB_FSTYPE_MIN; ftype < TSDB_FSTYPE_MAX; ++ftype) { 743: if (fset->farr[ftype] != NULL) { 744: int32_t code = tsdbTFileObjRemove(fset- >farr[ftype]); </pre>	
	污染路径: 无	
62	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 111
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 109: { 110: size_t __old; 111: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) 112: _argp_fmtstream_update (__fs); 113: __old = __fs->wmargin; </pre>	
	污染路径: 无	
63	缺陷发生位置: /contrib/lemon/lemon.c	行号: 808
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 806: for(j=0; j<p->nAction; j++){ 807: if(p->aAction[j].lookahead<0) continue; 808: if(p->aAction[j].lookahead==j+p->mnLookahead- i) n++; 809: } 810: if(n==p->nLookahead){ </pre>	
	污染路径: 无	
64	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 178
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	176: } else { 177: // not null 178: if (i == len - 1) { 179: // this is last, only set end false , not free node 180: nodes[m]->end = false;	
	污染路径: 无	
65	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 486
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 484: 485: a = first + (b - a), c = last - (d - c); 486: b = (v <= Td[PA[*a] - 1]) ? a : ss_partition(PA, a, c, depth); 487: 488: if((a - first) <= (last - c)) {	
	污染路径: 无	
66	缺陷发生位置: /source/util/src/tcrc32c.c	行号: 937
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 935: Westmere, Sandy Bridge, and Ivy Bridge architectures, which have a 936: throughput of one crc per cycle, but a latency of three cycles */ 937: while (len >= 3 * LONG_SHIFT) { 938: crc1 = 0; 939: crc2 = 0;	
	污染路径: 无	
67	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 284
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 282: item->valueint = LLONG_MAX;	

	<pre> 283: } 284: else if (number <= LLONG_MIN) 285: { 286: item->valueint = LLONG_MIN; </pre>	
	污染路径: 无	
68	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 13354
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 13352: if (ncols < TSDB_MIN_COLUMNS) { 13353: return generateSyntaxErrMsg(&pCxt->msgBuf, TSDB_CODE_PAR_INVALID_COLUMNS_NUM); 13354: } else if (ncols + ntags > maxCols) { 13355: return generateSyntaxErrMsg(&pCxt->msgBuf, TSDB_CODE_PAR_TOO_MANY_COLUMNS); 13356: } </pre>	
	污染路径: 无	
69	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 122
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 120: argp_fmtstream_point (argp_fmtstream_t __fs) 121: { 122: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) 123: _argp_fmtstream_update (__fs); 124: return __fs->point_col >= 0 ? __fs->point_col : 0; </pre>	
	污染路径: 无	
70	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1820
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 1818: } 1819: </pre>	

	1820: for (int i = len; i < len + len1; i++) { 1821: void *rightData = prepareNchar(inputNchar[i - len]); 1822: code = makeCalculate(row, key, TSDB_DATA_TYPE_NCHAR, rightData, eRes1[i], op[i], false);	
	污染路径: 无	
71	缺陷发生位置: /source/libs/azure/test/azExceptionTest.cpp	行号: 110
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 108: const int size = 4096; 109: char data[size] = {0}; 110: for (int i = 0; i < size / 2; ++i) { 111: data[i * 2 + 1] = 1; 112: }	
	污染路径: 无	
72	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 739
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 737: return BIT_DStream_overflow; 738: 739: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) 740: { 741: bitD->ptr -= bitD->bitsConsumed >> 3;	
	污染路径: 无	
73	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 469
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 467: 468: /* sprintf failed or buffer overrun occurred */ 469: if ((length < 0) (length > (int)(sizeof(number_buffer) - 1)))	

	470: { 471: return false;	
	污染路径: 无	
74	缺陷发生位置: /contrib/TSZ/sz/src/TightDataPointStorageF.c	行号: 129
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 127: // calc leadNumArray_size 128: size_t logicLeadNumBitsNum = (*this)->exactDataNum * 2; 129: if (logicLeadNumBitsNum % 8 == 0) 130: { 131: (*this)->leadNumArray_size = logicLeadNumBitsNum >> 3;	
	污染路径: 无	
75	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 519
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 517: } else { 518: limit += 1; 519: if(Td[PA[*first] - 1] < v) { 520: first = ss_partition(PA, first, last, depth); 521: limit = ss_ilg(last - first);	
	污染路径: 无	
76	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 105
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 103: while ((bitStream & 0xFFFF) == 0xFFFF) { 104: n0 += 24; 105: if (ip < iend-5) { 106: ip += 2; 107: bitStream = MEM_readLE32(ip) >>	

	bitCount;	
	污染路径: 无	
77	缺陷发生位置: /tools/taos-tools/src/benchUtilDs.c	行号: 112
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 110: if (len > 128*1024*1024) 111: cap = len + 32*1024*1024; 112: else if (len > 4*1024*1024) 113: cap = (uint64_t)(len + fast_sqrt(len * 8) * 1024); 114: else if (len < 16)	
	污染路径: 无	
78	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 227
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 225: info->nBatCols = stb->partialColNum; 226: int j = 0; 227: for (int i = stb->partialColFrom; i < stb->partialColFrom + stb->partialColNum; i++) { 228: info->batCols[j++] = i; 229: }	
	污染路径: 无	
79	缺陷发生位置: /source/util/src/tutil.c	行号: 52
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 50: while (z[i] != 0) { 51: if (z[i] == quote) { 52: if (z[i + 1] == quote) { 53: z[j++] = (char)quote; 54: i++;	
	污染路径:	

	无	
80	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 121
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 119: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 120: while (charnum < n0) normalizedCounter[charnum++] = 0; 121: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend- 4)) { 122: assert((bitCount >> 3) <= 3); /* For first condition to work */ 123: ip += bitCount>>3; </pre>	
	污染路径: 无	
81	缺陷发生位置: /source/util/src/tcompression.c	行号: 621
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 619: 620: for (int32_t i = 0; i < nelements; i++) { 621: if (i % ele_per_byte == 0) { 622: ipos++; 623: } </pre>	
	污染路径: 无	
82	缺陷发生位置: /tools/shell/src/shellUtil.c	行号: 151
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 149: } 150: 151: if (new_len >= remove_len && strncmp(srcInfo + new_len - remove_len, removeStr, remove_len) == 0) { 152: new_len -= remove_len; 153: srcInfo[new_len] = '\0'; </pre>	

	污染路径: 无	
83	缺陷发生位置: /source/libs/decimal/src/decimal.c	行号: 1295
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1293: bool negative = unscaled < 0 ? true : false; 1294: double abs = TABS(unscaled); 1295: if (abs > ldexp(1.0, 63) - 1) { 1296: goto __OVERFLOW__; 1297: }	
	污染路径: 无	
84	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2977
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 2975: int32_t caseldx = (gCaseCtrl.caseldx >= 0) ? gCaseCtrl.caseldx : 0; 2976: 2977: for (int32_t i = caseldx; i < sizeof(gCase)/sizeof(gCase[0]); ++i) { 2978: if (gCaseCtrl.caseNum > 0 && caseNum >= gCaseCtrl.caseNum) { 2979: break;	
	污染路径: 无	
85	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 475
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 473: U32 nbBytes = bitD->bitsConsumed >> 3; 474: BIT_DStream_status result = BIT_DStream_unfinished; 475: if (bitD->ptr - nbBytes < bitD->start) 476: { 477: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */	

	污染路径: 无	
86	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1789
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1787: 1788: // if last char not ' ', then not end field, like 'select count(*), su' can fill sum(1789: if (sql[strlen(sql) - 1] != ' ' && cnt <= 1) { 1790: return false; 1791: }	
	污染路径: 无	
87	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 60
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 58: 59: int sdbSetRawBinary(SSdbRaw* pRaw, int pos, const void* data, int len) { 60: if (pos + len <= 4096) { 61: memcpy(pRaw->data + pos, data, len); 62: return 0;	
	污染路径: 无	
88	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 141
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 139: } else if (strcmp(argv[i], "-m") == 0 && i < argc - 1) { 140: msgSize = atoi(argv[++i]); 141: } else if (strcmp(argv[i], "-s") == 0 && i < argc - 1) { 142: rpclnit.sessions = atoi(argv[++i]); 143: } else if (strcmp(argv[i], "-o") == 0 && i < argc - 1) {	
	污染路径:	

	无	
89	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 367
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 365: 366: /* calculate new buffer size */ 367: if (needed > (LONG_MAX / 2)) 368: { 369: /* overflow of int, use LLONG_MAX if possible */	
	污染路径: 无	
90	缺陷发生位置: /source/util/test/trefTest.c	行号: 138
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 136: if (strcmp(argv[i], "-n") == 0 && i < argc - 1) { 137: refNum = atoi(argv[++i]); 138: } else if (strcmp(argv[i], "-s") == 0 && i < argc - 1) { 139: steps = atoi(argv[++i]); 140: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) {	
	污染路径: 无	
91	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 331
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 329: vg.hashBegin = i * hashUnit; 330: vg.hashEnd = hashUnit * (i + 1) - 1; 331: if (i == ctgTestVgNum - 1) { 332: vg.hashEnd = htonl(UINT32_MAX); 333: }	
	污染路径: 无	
92	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 454
	步骤描述:	

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 452: } 453: 454: for (tsdb_ftype_t ftype = TSDB_FTYPE_MIN; ftype < TSDB_FTYPE_MAX; ++ftype) { 455: if (!fset1->farr[ftype] && !fset2->farr[ftype]) continue; 456:	
	污染路径: 无	
93	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 130
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 128: // Start of number 129: inNumber = true; 130: if (outLen + 3 < patternLen) { 131: strcpy(out, "?"); 132: out += 1;	
	污染路径: 无	
94	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 754
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 752: U32 nbBytes = bitD->bitsConsumed >> 3; 753: BIT_DStream_status result = BIT_DStream_unfinished; 754: if (bitD->ptr - nbBytes < bitD->start) 755: { 756: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */	
	污染路径: 无	
95	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 1245
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	

	代码片段: 1243: if (params.fParams.checksumFlag) { 1244: U32 const checksum = (U32)XXH64_digest(&mtctx->serial.xxhState); 1245: if (dstPos + 4 > dstCapacity) { 1246: error = ERROR(dstSize_tooSmall); 1247: } else { 污染路径: 无	
96	缺陷发生位置: /source/common/src/tmisc.c	行号: 72
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 70: tstrncpy(pEp->fqdn, ep + 1, ipLen + 1); 71: 72: if (*(end + 1) == ':' && *(end + 2)) { 73: pEp->port = taosStr2UInt16(end + 2, NULL, 10); 74: }	
	污染路径: 无	
97	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 122
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 120: for (int i = 1; i < argc; ++i) { 121: if (strcmp(argv[i], "-p") == 0 && i < argc - 1) { 122: } else if (strcmp(argv[i], "-i") == 0 && i < argc - 1) { 123: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) { 124: rpclnit.numOfThreads = atoi(argv[++i]);	
	污染路径: 无	
98	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 708
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 706: BYTE* oend = op + dstBuff.capacity; 707: int chunkNb;	

	708: if (sizeof(size_t) > sizeof(int)) assert(job->src.size < ((size_t)INT_MAX) * chunkSize); /* check overflow */ 709: DEBUGLOG(5, "ZSTDMT_compressionJob: compress %u bytes in %i blocks", (U32)job->src.size, nbChunks); 710: assert(job->cSize == 0);	
	污染路径: 无	
99	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 255
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 253: const char *arg = var; 254: 255: while (isalnum (*arg) *arg == '-' *arg == '_') 256: arg++; 257: var_len = arg - var;	
	污染路径: 无	
100	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1103
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1101: /* Sanity Checks */ 1102: if (maxSymbolValue > FSE_MAX_SYMBOL_VALUE) return ERROR(maxSymbolValue_tooLarge); 1103: if (tableLog > FSE_MAX_TABLELOG) return ERROR(tableLog_tooLarge); 1104: 1105: /* Init, lay down lowprob symbols */	
	污染路径: 无	
101	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 363
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 361: int precision_len = comma - paren - 1;	

	<pre> 362: char precision_str[16] = {0}; 363: if (precision_len > sizeof(precision_str) - 1) { 364: precision_len = sizeof(precision_str) - 1; 365: } </pre>	
	污染路径: 无	
102	缺陷发生位置: /source/libs/azure/test/azTest.cpp	行号: 163
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 161: GTEST_ASSERT_EQ(code, 0); 162: 163: for (int i = 0; i < size / 2; ++i) { 164: GTEST_ASSERT_EQ(data[i * 2], 0); 165: GTEST_ASSERT_EQ(data[i * 2 + 1], 1); </pre>	
	污染路径: 无	
103	缺陷发生位置: /source/client/test/stmtTest.cpp	行号: 136
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 134: 135: } else { 136: if (i % 2 == 0) { 137: code = taos_stmt_set_tbname(stmt, table_name); 138: checkError(stmt, code); </pre>	
	污染路径: 无	
104	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-fmtstream.c	行号: 87
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 85: { 86: size_t __old; 87: if ((size_t) (__fs->p - __fs->buf) > __fs->point_offs) </pre>	

	88: _argp_fmtstream_update (__fs); 89: __old = __fs->lmargin;	
	污染路径: 无	
105	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 867
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 865: } 866: if (bitD->ptr == bitD->start) { 867: if (bitD->bitsConsumed < sizeof(bitD->bitContainer)*8) return BITv05_DStream_endOfBuffer; 868: return BITv05_DStream_completed; 869: }	
	污染路径: 无	
106	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 435
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 433: { U32 nbBytes = bitD->bitsConsumed >> 3; 434: BIT_DStream_status result = BIT_DStream_unfinished; 435: if (bitD->ptr - nbBytes < bitD->start) { 436: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 437: result = BIT_DStream_endOfBuffer;	
	污染路径: 无	
107	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 720
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 718: void tsdbTFileSetClear(STFileSet **fset) { 719: if (fset && *fset) { 720: for (tsdb_ftype_t ftype = TSDB_FTYPE_MIN; ftype < TSDB_FTYPE_MAX; ++ftype) {	

	721: if ((*fset)->farr[ftype] == NULL) continue; 722: int32_t code = tsdbTFileObjUnref((*fset)->farr[ftype]);	
	污染路径: 无	
108	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 217
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 215: size_t value, unsigned nbBits) 216: { 217: MEM_STATIC_ASSERT(BIT_MASK_SIZE == 32); 218: assert(nbBits < BIT_MASK_SIZE); 219: assert(nbBits + bitC->bitPos < sizeof(bitC->bitContainer) * 8);	
	污染路径: expanded from macro 'MEM_STATIC_ASSERT'	
109	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2403
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 2401: } 2402: 2403: if (rand() % 2 == 0) { 2404: if (st_stmt2_add_batch(stmt)) { 2405: printf("!!!st_stmt2_add_batch error:%s\n", st_stmt2_error(stmt));	
	污染路径: 无	
110	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 600
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 598: for (int k = 0; k < kcnt; k++) { 599: int klen = (int)strlen(keys[k]); 600: if (i + klen < n && strncasecmp(cmd + i, keys[k], klen) == 0) {	

	601: matched = k; 602: // copy keyword	
	污染路径: 无	
111	缺陷发生位置: /source/client/test/session.cpp	行号: 687
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 685: * 3. test timestamp with string format 686: */ 687: for (unsigned int i = 0; i < sizeof(params1) / sizeof(params1[0]); ++i) { 688: do_insert(params1[i]); 689: do_select(params1[i]);	
	污染路径: 无	
112	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 644
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 642: { U32 nbBytes = bitD->bitsConsumed >> 3; 643: BITv07_DStream_status result = BITv07_DStream_unfinished; 644: if (bitD->ptr - nbBytes < bitD->start) { 645: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 646: result = BITv07_DStream_endOfBuffer;	
	污染路径: 无	
113	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 263
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 261: { 262: bitSize -= lackBits; 263: byteSize = bitSize%8==0 ? bitSize/8 : bitSize/8+1;	

	264: byteSizep = bitSize/8; 265: p += byteSizep;	
	污染路径: 无	
114	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 201
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 199: else { /* header compressed with FSE (normal case) */ 200: FSE_DTable fseWorkspace[FSE_DTABLE_SIZE_U32(6)]; /* 6 is max possible tableLog for HUF header (maybe even 5, to be tested) */ 201: if (iSize+1 > srcSize) return ERROR(srcSize_wrong); 202: oSize = FSE_decompress_wksp(huffWeight, hwSize- 1, ip+1, iSize, fseWorkspace, 6); /* max (hwSize-1) values decoded, as last one is implied */ 203: if (FSE_isError(oSize)) return oSize;	
	污染路径: 无	
115	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 191
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 189: oSize = iSize - 127; 190: iSize = ((oSize+1)/2); 191: if (iSize+1 > srcSize) return ERROR(srcSize_wrong); 192: if (oSize >= hwSize) return ERROR(corruption_detected); 193: ip += 1;	
	污染路径: 无	
116	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 681
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 679: taosUsleep(taosRand() % 5);	

	<pre> 680: } 681: if (++n % qwtTestPrintNum == 0) { 682: printf("query:%d\n", n); 683: } </pre>	
	污染路径: 无	
117	缺陷发生位置: /source/util/test/trefTest.c	行号: 140
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 138: } else if (strcmp(argv[i], "-s") == 0 && i < argc - 1) { 139: steps = atoi(argv[++i]); 140: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) { 141: threads = atoi(argv[++i]); 142: } else if (strcmp(argv[i], "-l") == 0 && i < argc - 1) { </pre>	
	污染路径: 无	
118	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 536
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 534: if (n0 > *maxSVPtr) return (size_t)- FSE_ERROR_maxSymbolValue_tooSmall; 535: while (charnum < n0) normalizedCounter[charnum++] = 0; 536: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend- 4)) 537: { 538: ip += bitCount>>3; </pre>	
	污染路径: 无	
119	缺陷发生位置: /contrib/TSZ/zstd/compress/huf_compress.c	行号: 673
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 671: { CHECK_V_F(largest, HIST_count_wksp (table- >count, &maxSymbolValue, (const BYTE*)src, srcSize, table- </pre>	

	>count)); 672: if (largest == srcSize) { *ostart = ((const BYTE*)src) [0]; return 1; } /* single symbol, rle */ 673: if (largest <= (srcSize >> 7)+4) return 0; /* heuristic : probably not compressible enough */ 674: } 675:	
	污染路径: 无	
120	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 1197
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1195: taosUsleep(taosRand() % 5); 1196: } 1197: if (++n % ctgTestPrintNum == 0) { 1198: (void)printf("Get:%d\n", n); 1199: }	
	污染路径: 无	
121	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtilTopic.c	行号: 304
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 302: if (sub[spos] == '+') { 303: /* Check for bad "+foo" or "a/+foo" subscription */ 304: if (spos > 0 && sub[spos - 1] != '/') { 305: return TTQ_ERR_INVALID; 306: }	
	污染路径: 无	
122	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 236
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 234: op[2] = FSE_GETSYMBOL(&state1); 235:	

	236: if (FSE_MAX_TABLELOG*2+7 > sizeof(bitD.bitContainer)*8) /* This test must be static */ 237: BIT_reloadDStream(&bitD); 238:	
	污染路径: 无	
123	缺陷发生位置: /utls/test/c/tmq_taosx_ci.c	行号: 741
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 739: "meters_summary","\tagNum\:1,\tags\":[{\name\:\"group_id\", \"type\:14, \"value\:1.313555008277358e+\" 740: \"19}], \"createList\":[]}"; 741: for (int i = 0; i < sizeof(result) / sizeof(result[0]); i++) { 742: taosFprintfFile(pFile2, result[i]); 743: taosFprintfFile(pFile2, "\n");	
	污染路径: 无	
124	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-fmtstream.c	行号: 50
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 48: size_t __len) 49: { 50: if (__fs->p + __len <= __fs->end __argp_fmtstream_ensure (__fs, __len)) 51: { 52: memcpy (__fs->p, __str, __len);	
	污染路径: 无	
125	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 458
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	<pre> 456: return BIT_DStream_overflow; 457: 458: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) 459: { 460: bitD->ptr -= bitD->bitsConsumed >> 3; </pre>	
	污染路径: 无	
126	缺陷发生位置: /source/libs/index/test/fstUtilUT.cc	行号: 90
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 88: EXPECT_EQ(val, i + 'a'); 89: } 90: for (int8_t i = 'a'; i < 'a' + 20; i++) { 91: EXPECT_EQ(sparSetContains(set, i), true); 92: } </pre>	
	污染路径: 无	
127	缺陷发生位置: /source/libs/scheduler/test/schedulerTests.cpp	行号: 692
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 690: schedulerFreeJob(&job, 0); 691: if (freeThread) { 692: if (++freeNum % schtTestPrintNum == 0) { 693: (void)printf("FreeNum:%d\n", freeNum); 694: } </pre>	
	污染路径: 无	
128	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 123
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 121: 122: // s0 is false 123: for (int32_t i = 0; i < sizeof(s0) / sizeof(char *); i++) { </pre>	

	124: printf("s0 i=%d isCreateFieldsArea %s expect false. \n", i, s0[i]); 125: ASSERT(isCreateFieldsArea((char *)s0[i]) == false);	
	污染路径: 无	
129	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 1032
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1030: } 1031: /* Print status */ 1032: LOCALDISPLAYUPDATE(displayLevel, 2, "\r%u%%", ", 1033: (U32)((iteration * 100) / kIterations)); 1034: ++iteration;	
	污染路径: expanded from macro 'LOCALDISPLAYUPDATE'	
130	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1210
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1208: uint32_t step = UINT32_MAX / groupCnt; 1209: for (int32_t i = 0; i < groupCnt; i++) { 1210: if (hash < (i + 1) * step) 1211: { 1212: return i;	
	污染路径: 无	
131	缺陷发生位置: /tools/shell/src/shellAuto.c	行号: 1806
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1804: bool needInsertFrom(char* sql, int len) { 1805: // last is blank 1806: if (sql[len - 1] != ' ') { 1807: // insert from keyword 1808: return false;	

	污染路径: 无	
132	缺陷发生位置: /source/os/test/osSemaphoreTests.cpp	行号: 257
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 255: }); 256: 257: for (int i = 0; i < count * 2; ++i) { 258: (void)tsem_wait(&sem); 259: }	
	污染路径: 无	
133	缺陷发生位置: /source/libs/tcs/test/tcsTest.cpp	行号: 173
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 171: GTEST_ASSERT_EQ(code, 0); 172: 173: for (int i = 0; i < size / 2; ++i) { 174: GTEST_ASSERT_EQ(pBlock[i * 2], 0); 175: GTEST_ASSERT_EQ(pBlock[i * 2 + 1], 1);	
	污染路径: 无	
134	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1156
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1154: } 1155: } else if (superTable->iface == SML_IFACE) { 1156: if (g_arguments->reqPerReq > SML_MAX_BATCH) { 1157: errorPrint("reqPerReq (%u) larger than maximum (%d)\n", 1158: g_arguments->reqPerReq, SML_MAX_BATCH);	

	污染路径: 无	
135	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 968
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 966: tm.tm_sec = 15; 967: 968: for (unsigned int i = 0; i < sizeof (test_mk) / sizeof (test_mk[0]); ++i) 969: { 970: timezone_t tz = tzalloc(test_mk[i].env);	
	污染路径: 无	
136	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 248
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 246: info->protocol = TSDB_SML_LINE_PROTOCOL; 247: info->dataFormat = false; 248: for (int i = 0; i < sizeof(data) / sizeof(data[0]); i++) { 249: char msg[256] = {0}; 250: SSmlMsgBuf msgBuf;	
	污染路径: 无	
137	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 467
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 465: if (bitD->ptr == bitD->start) 466: { 467: if (bitD->bitsConsumed < sizeof(bitD->bitContainer)*8) return BIT_DStream_endOfBuffer; 468: return BIT_DStream_completed; 469: }	
	污染路径: 无	

138	缺陷发生位置: /utils/test/c/tmq_td37265.c	行号: 97
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 95: } 96: } 97: assert(total_affected == CTB_NUMS * ROW_NUMS * CYC_NUMS); 98: 99: taos_stmt_close(stmt);	
	污染路径: 无	
139	缺陷发生位置: /source/libs/executor/src/sysscanoperator.c	行号: 5443
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 5441: SOperatorNode* pOper = (SOperatorNode*)cond; 5442: SColumnNode* pCol = (SColumnNode*)pOper->pLeft; 5443: for (int i = 0; i < sizeof(SYSTABLE_SPECIAL_COL) / sizeof(SYSTABLE_SPECIAL_COL[0]); i++) { 5444: if (0 == strcmp(pCol->colName, SYSTABLE_SPECIAL_COL[i])) { 5445: return 1;	
	污染路径: 无	
140	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 67
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 65: break; 66: case B: 67: for (size_t i = 0; i < 10 * 1024l * 1024l * 1024l; i++) { 68: tPutB(pBuf, val); 69: if (POINTER_DISTANCE(buf, pBuf) == 1024) {	
	污染路径: 无	

141	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 2049
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 2047: } 2048: 2049: if (filterNum && filterCol[c] && ((*bool*)(row + c)) cbig <= fbig) { 2050: filterOut = true; 2051: break;	
	污染路径: 无	
142	缺陷发生位置: /source/client/test/session.cpp	行号: 943
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 941: char* users[] = {"test1", "test2", "test3", "test4", "test5"}; 942: 943: for (int32_t i = 0; i < sizeof(users) / sizeof(users[0]); i+ +) { 944: code = sessMgtUpdataLimit(users[i], SESSION_PER_USER, expect); 945: ASSERT(code == 0);	
	污染路径: 无	
143	缺陷发生位置: /source/libs/tcs/test/tcsTest.cpp	行号: 136
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 134: const int size = 4096; 135: char data[size] = {0}; 136: for (int i = 0; i < size / 2; ++i) { 137: data[i * 2 + 1] = 1; 138: }	
	污染路径: 无	

144	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1429
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1427: }; 1428: 1429: for (int i = 0; i < sizeof(ops) / sizeof(ops[0]); i++) { 1430: SMetaTableOp *op = &ops[i]; 1431:	
	污染路径: 无	
145	缺陷发生位置: /source/libs/parser/src/parAstCreator.c	行号: 745
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 743: if (trim && (pToken->z[0] == TS_ESCAPE_CHAR)) { 744: int32_t i = 1, j = 0; 745: for (; i < len - 1; ++i) { 746: buf[j++] = pToken->z[i]; 747: if (pToken->z[i] == TS_ESCAPE_CHAR) {	
	污染路径: 无	
146	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 727
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 725: /* last block */ 726: assert(chunkSize > 0); assert((chunkSize & (chunkSize - 1)) == 0); /* chunkSize must be power of 2 for mask==(chunkSize-1) to work */ 727: if ((nbChunks > 0) job->lastJob /*must output a "last block" flag*/) { 728: size_t const lastBlockSize1 = job->src.size & (chunkSize-1); 729: size_t const lastBlockSize = ((lastBlockSize1==0) & (job->src.size>=chunkSize)) ? chunkSize : lastBlockSize1;	
	污染路径:	

	无	
147	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 102
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 100: bool inNumber = false; 101: 102: while (*p && outLen < patternLen - 10) { 103: if (*p == '\') { 104: if (!inString) {	
	污染路径: 无	
148	缺陷发生位置: /test/new_test_framework/script/api/stmt2-performance.c	行号: 206
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 204: snprintf(tbs[i], sizeof(tbs[i]), "ctb_%d", i); 205: for (int j = 0; j < ROW_NUMS; j++) { 206: if (i == CTB_NUMS - 1 && j == ROW_NUMS - 1) { 207: sprintf(tsc_sql + strlen(tsc_sql), "('%s', %lld,'%c',0,'after')", tbs[i], ts[i][j] + r * 10000, b[i][j]); 208: } else {	
	污染路径: 无	
149	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1476
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1474: }; 1475: 1476: for (int i = 0; i < sizeof(ops) / sizeof(ops[0]); i++) { 1477: SMetaTableOp *op = &ops[i]; 1478:	
	污染路径: 无	
150	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 469

	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 467: if (bitD->ptr == bitD->start) 468: { 469: if (bitD->bitsConsumed < sizeof(bitD->bitContainer)*8) return BIT_DStream_endOfBuffer; 470: return BIT_DStream_completed; 471: }	
	污染路径: 无	
151	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchUtilDs.c	110
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 108: return s; 109: 110: if (len > 128*1024*1024) 111: cap = len + 32*1024*1024; 112: else if (len > 4*1024*1024)	
	污染路径: 无	
152	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v02.c	473
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 471: U32 nbBytes = bitD->bitsConsumed >> 3; 472: BIT_DStream_status result = BIT_DStream_unfinished; 473: if (bitD->ptr - nbBytes < bitD->start) 474: { 475: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */	
	污染路径: 无	
153	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	78
	步骤描述:	

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 76: 77: int sdbGetRawInt32(SSdbRaw* pRaw, int pos, int32_t* val) { 78: if (pos + 4 <= pRaw->dataLen) { 79: memcpy(val, pRaw->data + pos, 4); 80: return 0;	
	污染路径: 无	
154	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtilTopic.c	行号: 324
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 322: } else if (sub[spos] == '#') { 323: /* Check for bad "foo#" subscription */ 324: if (spos > 0 && sub[spos - 1] != '/') { 325: return TTQ_ERR_INVALID; 326: }	
	污染路径: 无	
155	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 667
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 665: 666: /* not a string */ 667: if (buffer_at_offset(input_buffer)[0] != '\\') 668: { 669: goto fail;	
	污染路径: 无	
156	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 977
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 975: if (gCurCase->duplicateValue) {	

	<pre> 976: data->tsData[i] = bpTs; 977: if (i % 2 == 1) { 978: bpTs++; 979: } </pre>	
	污染路径: 无	
157	缺陷发生位置: /source/client/test/session.cpp	行号: 960
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 958: SSessParam para = {.type = SESSION_PER_USER, .value = 1}; 959: for (int32_t i = 0; i <= 1000; i++) { 960: for (int32_t i = 0; i < sizeof(users) / sizeof(users[0]); i+ +) { 961: code = sessMgtUpdateUserMetric(users[i], &para); 962: ASSERT(code == 0); </pre>	
	污染路径: 无	
158	缺陷发生位置: /utils/test/c/sml_test.c	行号: 1184
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 1182: "stab,t1=1 c1\\=3,c2=\"32fw\" 16619439900000000000", 1183: }; 1184: for(int i = 0; i < sizeof(sql) / sizeof(sql[0]); i++){ 1185: pRes = taos_schemaless_insert(taos, (char**)&sql[i], 1, TSDB_SML_LINE_PROTOCOL, 0); 1186: int code = taos_errno(pRes); </pre>	
	污染路径: 无	
159	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 609
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 607: if (trimLeft) { </pre>	

	608: int32_t pos = 0; 609: for (int32_t i = 0; i < orgLen - remLen; i += remLen) { 610: if (memcmp(orgStr + i, remStr, remLen) == 0) { 611: if (isCharStart(orgStr[i + remLen]) isNchar) {		
	污染路径: 无		
160	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 105	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 103: /* Sanity Checks */ 104: if (maxSymbolValue > FSE_MAX_SYMBOL_VALUE) return ERROR(maxSymbolValue_tooLarge); 105: if (tableLog > FSE_MAX_TABLELOG) return ERROR(tableLog_tooLarge); 106: 107: /* Init, lay down lowprob symbols */		
	污染路径: 无		
161	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 618	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 616: // find field same in server 617: bool findFieldInServer(ColDes *colDes, TableDes * tableDes) { 618: for (int32_t i = 0; i < tableDes->columns + tableDes-> tags ; i++) { 619: if (strcmp(colDes->field, tableDes->cols[i].field) == 0 && 620: colDes->type == tableDes->cols[i].type) {		
	污染路径: 无		
162	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 228	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段:		

	226: // GCC8: generates a bit worse cmp based code 227: // although it generates the xor based one for _int128. 228: return (x.lo == y.lo) & (x.hi == y.hi); 229: } 230:	
	污染路径: 无	
163	缺陷发生位置: /source/util/test/talgoTest.cpp	行号: 100
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 98: printf("taosSort length:%d cost: %f s\n", arraySize, cpu_time_used); 99: 100: for (int i = 0; i < arraySize - 1; i++) { 101: EXPECT_EQ(data1[i], data2[i]); 102: ASSERT_LE(data1[i], data1[i+1]);	
	污染路径: 无	
164	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 994
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 992: if (gCurCase->duplicateValue) { 993: data->tsData[i] = bpTs; 994: if (i % 2 == 1) { 995: bpTs++; 996: }	
	污染路径: 无	
165	缺陷发生位置: /source/libs/transport/test/transUTIPv6.cpp	行号: 304
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 302: req.info.ahandle = (void *) (int64_t) i; 303:	

	304: if (i % 2 == 0) { 305: req.msgType = 8; 306: cli->SendAndRecv("127.0.0.1", &req, &resp);	
	污染路径: 无	
166	缺陷发生位置: /source/client/src/clientSml.c	行号: 160
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 158: char *endPtr = NULL; 159: int64_t tsInt64 = taosStr2Int64(value, &endPtr, 10); 160: if (unlikely(value + len != endPtr)) { 161: return -1; 162: }	
	污染路径: 无	
167	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 515
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 513: { 514: n0+=24; 515: if (ip < iend-5) 516: { 517: ip+=2;	
	污染路径: 无	
168	缺陷发生位置: /source/util/src/tcompression.c	行号: 862
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 860: // zigzag encode the value. 861: uint64_t zigzag_value = ZIGZAG_ENCODE(int64_t, delta_of_delta); 862: if (i % 2 == 0) { 863: flags = 0; 864: ddl = zigzag_value;	

	污染路径: 无	
169	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 460
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 458: return BIT_DStream_overflow; 459: 460: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) 461: { 462: bitD->ptr -= bitD->bitsConsumed >> 3;	
	污染路径: 无	
170	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 248
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 246: } 247: *outSize += byteSize; 248: lackBits = bitSize%8==0 ? 0 : 8 - bitSize%8; 249: } 250: else	
	污染路径: 无	
171	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 362
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 360: for(i=1; i<segmentSize;i++) 361: { 362: if(i*i>segmentSize) 363: break; 364: }	
	污染路径: 无	

172	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 567
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 565: if (used + need + 1 <= *cap) return 0; 566: int ncap = (*cap) * 2; 567: if (ncap < used + need + 1) ncap = used + need + 1; 568: char *tmp = (char*)realloc(*out, ncap); 569: if (!tmp) return -1;	
	污染路径: 无	
173	缺陷发生位置: /source/util/test/arrayTest.cpp	行号: 103
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 101: 102: int32_t* pv = NULL; 103: for (int32_t i = 0; i < cnt*2; i++) { 104: pv = (int32_t*)taosArrayGet(pa, i); 105: ASSERT_EQ(*pv, i);	
	污染路径: 无	
174	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 157
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 155: uint64_t arr1[] = {13, 14, 15}; 156: SArray * f = (SArray *)taosArrayGetP(src, 0); 157: for (int i = 0; i < sizeof(arr1) / sizeof(arr1[0]); i++) { 158: taosArrayPush(f, &arr1[i]); 159: }	
	污染路径: 无	
175	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 728
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	

	代码片段: 726: taosUsleep(taosRand() % 5); 727: } 728: if (++n % qwtTestPrintNum == 0) { 729: printf("drop:%d\n", n); 730: }	
	污染路径: 无	
176	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-help.c	行号: 705
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 703: 704: while (num_opts-- > 0) 705: if (opt->name && ovisible (opt) && strcmp (opt->name, name) == 0) 706: return entry; 707: else	
177	污染路径: 无	
	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 891
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
178	代码片段: 889: if (parser->state.next > 1 890: && strcmp (parser->state.argv[parser->state.next - 1], QUOTE) 891: == 0) 892: /* Not only is this the end of the options, but it's a 893: `quoted' region, which may have args that *look* like	
	污染路径: 无	
178	缺陷发生位置: /source/client/src/clientSml.c	行号: 1935
	步骤描述:	

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1933: char *tmp = lines; 1934: for (int i = 0; i < len; i++) { 1935: if (lines[i] == '\n' i == len - 1) { 1936: if (!IS_COMMENT(protocol, tmp[0])) { // ignore comment 1937: numLines++;	
	污染路径: 无	
179	缺陷发生位置:	行号:
	/source/libs/catalog/test/catalogTests.cpp	1245
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1243: taosUsleep(taosRand() % 5); 1244: } 1245: if (++n % ctgTestPrintNum == 0) { 1246: (void)printf("Set:%d\n", n); 1247: }	
180	污染路径: 无	
	缺陷发生位置:	行号:
	/test/new_test_framework/script/api/stmtBatchTest.c	91
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
181	代码片段: 89: 90: int i = 0; 91: for (int j = 0; j < bingNum * tableNum; j++) { 92: params[i+0].buffer_type = TSDB_DATA_TYPE_TIMESTAMP; 93: params[i+0].buffer_length = sizeof(int64_t);	
	污染路径: 无	
	缺陷发生位置:	行号:
182	/source/libs/scalar/test/scalar/scalarTests.cpp	1882
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	1880: } 1881: 1882: for (int i = len; i < len + len1; i++) { 1883: void *rightData = prepareNchar(inputNchar[i - len]); 1884: code = makeCalculate(row, key, TSDB_DATA_TYPE_NCHAR, rightData, eRes4[i], op[i], false);	
	污染路径: 无	
182	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 849
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 847: } 848: 849: if (i < nOlds - 1) { 850: int szDivCell; 851: SCell* pDivCell = tdbOsMalloc(divCells[i].kLen + divCells[i].vLen + sizeof(SIntHdr));	
	污染路径: 无	
183	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 138
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 136: uint64_t arr2[] = {1, 2, 3, 5}; 137: f = (SArray *)taosArrayGetP(src, 1); 138: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) { 139: taosArrayPush(f, &arr2[i]); 140: }	
	污染路径: 无	
184	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 234
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 232: { 233: // Analogous to ==, but == not used directly, because	

	that confuses GCC 8-9. 234: return (x.lo != y.lo) (x.hi != y.hi); 235: } 236:	
	污染路径: 无	
185	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 1382
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1380: 1381: fieldName = "start_time"; 1382: if (strncmp(field, fieldName, strlen(fieldName) + 1) == 0) { 1383: *(int64_t *)value = pReader->startTime; 1384: return TSDB_CODE_SUCCESS;	
	污染路径: 无	
186	缺陷发生位置: /contrib/libaes/src/taes.c	行号: 157
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 155: } 156: 157: if (i % Nk == 0) { 158: // This function shifts the 4 bytes in a word to the left once. 159: // [a0,a1,a2,a3] becomes [a1,a2,a3,a0]	
	污染路径: 无	
187	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 120
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 118: return -1; 119: } 120: if (g_arguments->output_path[len - 1] != '/') {	

	121: int n = snprintf(g_arguments->output_path_buf, sizeof(g_arguments->output_path_buf), "%s/", g_arguments->output_path); 122: if (n < 0 n >= sizeof(g_arguments->output_path_buf)) {	
	污染路径: 无	
188	缺陷发生位置: /source/util/src/tcompression.c	行号: 412
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 410: 411: if (elems + 1 <= selector_to_elems[(int32_t)selector] && 412: elems + 1 <= selector_to_elems[(int32_t)(bit_to_selector[(int32_t)tmp_bit])]) { 413: // If can hold another one. 414: selector = selector > bit_to_selector[(int32_t)tmp_bit] ? selector : bit_to_selector[(int32_t)tmp_bit];	
	污染路径: 无	
189	缺陷发生位置: /source/util/src/tcrc32c.c	行号: 956
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 954: less 955: than a LONG_SHIFT*3 block */ 956: while (len >= 3 * SHORT_SHIFT) { 957: crc1 = 0; 958: crc2 = 0;	
	污染路径: 无	
190	缺陷发生位置: /source/util/test/trefTest.c	行号: 142
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	140: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) { 141: threads = atoi(argv[++i]); 142: } else if (strcmp(argv[i], "-l") == 0 && i < argc - 1) { 143: loops = atoi(argv[++i]); 144: } else if (strcmp(argv[i], "-d") == 0 && i < argc - 1) {	
	污染路径: 无	
191	缺陷发生位置: /source/libs/transport/src/thttp.c	行号: 181
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 179: gzipStream.zfree = (free_func)0; 180: gzipStream.opaque = (voidpf)0; 181: if (deflateInit2(&gzipStream, Z_DEFAULT_COMPRESSION, Z_DEFLATED, MAX_WBITS + 16, 8, Z_DEFAULT_STRATEGY) != Z_OK) { 182: code = TSDB_CODE_OUT_OF_MEMORY; 183: goto _OVER;	
	污染路径: 无	
192	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 917
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 915: 916: // find in list 917: for (int32_t i = tableDes->columns ; i < tableDes->columns + tableDes->tags; i++) { 918: if (tableDes->cols[i].idx == idx) { 919: return true;	
	污染路径: 无	
193	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 572
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 570:	

	<pre> 571: { 572: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) 573: { 574: ip += bitCount>>3; </pre>	
	污染路径: 无	
194	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 139
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 137: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) { 138: rpcInit.numOfThreads = atoi(argv[++i]); 139: } else if (strcmp(argv[i], "-m") == 0 && i < argc - 1) { 140: msgSize = atoi(argv[++i]); 141: } else if (strcmp(argv[i], "-s") == 0 && i < argc - 1) { </pre>	
	污染路径: 无	
195	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 342
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 340: constexpr uint<N> operator>>(const uint<N>& x, const T& shift) noexcept 341: { 342: if (shift < T{sizeof(x) * 8}) 343: return x >> static_cast<unsigned>(shift); 344: return 0; </pre>	
	污染路径: 无	
196	缺陷发生位置: /source/libs/decimal/test/decimalTest.cpp	行号: 874
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 872: } 873: 874: bool possible(float ratio) { return randomInt(ratio_base) </pre>	

	<= ratio * ratio_base; } 875: }; 876:	
	污染路径: 无	
197	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 288
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 286: } 287: 288: if ((int64_t)ctb_end_idx > ctb_offset + ctb_count) { 289: ctb_end_idx = ctb_offset + ctb_count; 290: }	
	污染路径: 无	
198	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 660
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 658: MEM_STATIC unsigned BITv07_endOfDStream(const BITv07_DStream_t* DStream) 659: { 660: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 661: } 662:	
	污染路径: 无	
199	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 93
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 91: SPoolMem *pMem; 92: 93: if (pPool->size > 1024 * 1024 * 10) {	

	94: return NULL; 95: }	
	污染路径: 无	
200	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 267
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 265: p += byteSize; 266: (*outSize)+=byteSize; 267: lackBits = bitSize%8==0 ? 0 : 8 - bitSize%8; 268: } 269: else //bitSize > 64	
	污染路径: 无	
201	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 266
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 264: taos_free_result(pRes); 265: el += (taosGetTimestampUs() - st); 266: if (i % 1000 == 0 && i != 0) { 267: (void)printf("total:%d, avg time:%.2fms\n", i, el / (double)(i * 1000)); 268: }	
	污染路径: 无	
202	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 54
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 52: break; 53: } 54: } while (pStr - pSql < length); 55: return false; 56: }	

	污染路径: 无	
203	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 51
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 49: size_t byteLength = 0; 50: size_t i, j; 51: if(intArrayLength%8==0) 52: byteLength = intArrayLength/8; 53: else	
	污染路径: 无	
204	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2258
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 2256: } 2257: 2258: if (rand() % 2 == 0) { 2259: if (taos_stmt_add_batch(stmt)) { 2260: printf("!!!taos_stmt_add_batch error:%s\n", taos_stmt_errstr(stmt));	
	污染路径: 无	
205	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 21
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 19: size_t byteLength = 0; 20: size_t i, j; 21: if(intArrayLength%8==0) 22: byteLength = intArrayLength/8; 23: else	
	污染路径: 无	

206	缺陷发生位置： /tools/shell/src/shellAuto.c	行号： 868
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 866: 867: int wordType(const char* p, int32_t len) { 868: for (int i = 0; i < WT_VAR_CNT; i++) { 869: if (strncmp(p, varTypes[i], len) == 0) return i; 870: }	
	污染路径： 无	
207	缺陷发生位置： /source/util/test/talgoTest.cpp	行号： 30
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 28: taosMergeSort(arr, 6, sizeof(TestStruct), cmpFunc); 29: 30: for (int i = 0; i < sizeof(arr) / sizeof(TestStruct); i++) { 31: printf("%d: %d %f\n", i, arr[i].a, arr[i].b); 32: }	
	污染路径： 无	
208	缺陷发生位置： /source/libs/metrics/test/metricsTest.cpp	行号： 544
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 542: allVgIds.push_back(i); 543: // Keep every 5th vgroup as valid 544: if (i % 5 == 1) { 545: validVgIds.push_back(i); 546: }	
	污染路径： 无	
209	缺陷发生位置： /tools/taos-tools/src/benchJsonOpt.c	行号： 129
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	

	代码片段: 127: uint8_t funType = FUNTYPE_NONE; 128: char* key2 = NULL; 129: for (int i = 0; i < FUNTYPE_CNT - 1; i++) { 130: key2 = strstr(key1, funsName[i]); 131: if(key2) {	
	污染路径: 无	
210	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 613
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 611: TAOS* pConn = getConnWithGlobalOption("UTC-8"); 612: 613: for (unsigned int i = 0; i < sizeof (tz_test) / sizeof (tz_test[0]); ++i){ 614: char sql[1024] = {0}; 615: (void)snprintf(sql, sizeof(sql), "insert into db1.ctb1 values('%s', '%s', 1)", tz_test[i], tz_test[i]);	
	污染路径: 无	
211	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 452
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 450: } 451: 452: if (n < 0 n >= TOOLS_MAX_ALLOWED_SQL_LEN - length) { 453: errorPrint("%s() LN%d snprintf overflow on %d iterat\n", __func__, __LINE__, i); 454: break;	
	污染路径: 无	
212	缺陷发生位置: /tools/taos-tools/src/benchDataMix.c	行号: 153
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	

	代码片段: 151: switch (fd->type) { 152: case TSDB_DATA_TYPE_BOOL: 153: strcpy(val, (ts % 2 == 0) ? "true" : "false"); 154: break; 155: // timestamp	
	污染路径: 无	
213	缺陷发生位置: /source/libs/transport/test/svrBench.c	行号: 143
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 141: } else if (strcmp(argv[i], "-s") == 0 && i < argc - 1) { 142: rpclnit.sessions = atoi(argv[++i]); 143: } else if (strcmp(argv[i], "-o") == 0 && i < argc - 1) { 144: tsCompressMsgSize = atoi(argv[++i]); 145: } else if (strcmp(argv[i], "-w") == 0 && i < argc - 1) {	
	污染路径: 无	
214	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 114
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 112: { 113: size_t i, j, byteLength = 0; 114: if(timeStepTypeLength%4==0) 115: byteLength = timeStepTypeLength*2/8; 116: else	
	污染路径: 无	
215	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 738
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 736: 737: // Skip the highest word of numerator if not significant. 738: if (un[m] != 0 un[m - 1] >= vn[n - 1])	

	739: ++m; 740:	
	污染路径: 无	
216	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 492
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 490: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 491: { 492: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> >bitContainer)*8)); 493: } 494:	
	污染路径: 无	
217	缺陷发生位置: /contrib/libmqt/ttp/src/ttpUtilTopic.c	行号: 138
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 136: for (i = 0; i < len; i++) { 137: if (str[i] == '+') { 138: if ((c != '\0' && c != '/') (i < len - 1 && str[i + 1] != '/')) { 139: return TTQ_ERR_INVALID; 140: } 	
	污染路径: 无	
218	缺陷发生位置: /source/libs/scalar/test/scalar/scalarTests.cpp	行号: 1840
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1838: } 1839: 1840: for (int i = len; i < len + len1; i++) {	

	1841: void *rightData = prepareNchar(inputNchar[i - len]); 1842: code = makeCalculate(row, key, TSDB_DATA_TYPE_NCHAR, rightData, eRes2[i], op[i], false);	
	污染路径: 无	
219	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 573
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 571: } 572: 573: if (regcomp(&pDst->regex, pDst->pattern, REG_EXTENDED REG_NOSUB REG_ICASE) != 0) { 574: tscWarn("sql security: skip invalid regex rule:%s pattern:%s", pDst->name, pDst->pattern); 575: return false;	
	污染路径: 无	
220	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 664
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 662: const auto lsh = clz(y.lo); 663: const auto rsh = (64 - lsh) % 64; 664: const auto rsh_mask = uint64_t{lsh == 0} - 1; 665: 666: const auto yn = y.lo << lsh;	
	污染路径: 无	
221	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 594
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 592: 593: // filed & type 594: for (int32_t i = 0; i < localDes->columns + localDes->tags; i++) {	

	595: ColDes * local = &localDes->cols[i]; 596: ColDes * srv = &tableDesSrv->cols[i];	
	污染路径: 无	
222	缺陷发生位置: /contrib/libaes/src/taes.c	行号: 148
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 146: 147: // All other round keys are found from the previous round keys. 148: for (i = Nk; i < Nb * (Nr + 1); ++i) { 149: { 150: k = (i - 1) * 4;	
	污染路径: 无	
223	缺陷发生位置: /utls/test/c/tmqDemo.c	行号: 470
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 468: } 469: 470: if (data_len > MAX_SQL_STR_LEN - MAX_ROW_STR_LEN) { 471: break; 472: }	
	污染路径: 无	
224	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 129
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 127: 128: // s1 is true 129: for (int32_t i = 0; i < sizeof(s1) / sizeof(char *); i++) { 130: printf("s1 i=%d isCreateFieldsArea %s expect true. \n", i, s1[i]);	

	131: ASSERT(isCreateFieldsArea((char *)s1[i]) == true);	
	污染路径: 无	
225	缺陷发生位置: /source/util/test/objpoolTest.cpp	行号: 25
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 23: int32_t npop = taosRand() % 100; 24: npop = TMIN(npop, list.neles + nappend); 25: if (r % 100 == 0) { 26: npop = list.neles + nappend; 27: }	
	污染路径: 无	
226	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 1066
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1064: void* newBuff; 1065: size_t const sBuffSize = ZDICT_totalSampleSize(samplesSizes, nbSamples); 1066: if (sBuffSize < ZDICT_MIN_SAMPLES_SIZE) return 0; /* not enough content => no dictionary */ 1067: 1068: newBuff = malloc(sBuffSize + NOISELENGTH);	
	污染路径: 无	
227	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 972
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 970: if (!dictList) return ERROR(memory_allocation); 971: if (maxDictSize < ZDICT_DICTSIZE_MIN) { free(dictList); return ERROR(dstSize_tooSmall); } /* requested dictionary size is too small */ 972: if (samplesBuffSize < ZDICT_MIN_SAMPLES_SIZE) { free(dictList); return ERROR(dictionaryCreation_failed); } /*	

	not enough source to create dictionary */ 973: 974: /* init */	
	污染路径: 无	
228	缺陷发生位置: /source/client/test/session.cpp	行号: 607
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 605: TAOS* pConn = getConnWithGlobalOption("UTC-8"); 606: 607: for (unsigned int i = 0; i < sizeof(tz_test) / sizeof(tz_test[0]); ++i) { 608: char sql[1024] = {0}; 609: (void)snprintf(sql, sizeof(sql), "insert into db1.ctb1 values('%s', '%s', 1)", tz_test[i], tz_test[i]);	
	污染路径: 无	
229	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 121
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 119: rpcDebugFlag = 143; 120: for (int i = 1; i < argc; ++i) { 121: if (strcmp(argv[i], "-p") == 0 && i < argc - 1) { 122: } else if (strcmp(argv[i], "-i") == 0 && i < argc - 1) { 123: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) {	
	污染路径: 无	
230	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 451
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 449: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 450: { 451: return ((DStream->ptr == DStream->start) &&	

	(DStream->bitsConsumed == sizeof(DStream->bitContainer)*8)); 452: } 453:	
	污染路径: 无	
231	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 565
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 563: 564: static inline int ensure_cap(char **out, int *cap, int need, int used) { 565: if (used + need + 1 <= *cap) return 0; 566: int ncap = (*cap) * 2; 567: if (ncap < used + need + 1) ncap = used + need + 1;	
	污染路径: 无	
232	缺陷发生位置: /contrib/TSZ/zstd/decompress/huf_decompress.c	行号: 968
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 966: { 967: assert(dstSize > 0); 968: assert(dstSize <= 128*1024); 969: /* decoder timing evaluation */ 970: { U32 const Q = (cSrcSize >= dstSize) ? 15 : (U32) (cSrcSize * 16 / dstSize); /* Q < 16 */	
	污染路径: 无	
233	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 784
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 782: int64_t tsdbTFileSetMaxCid(const STFileSet *fset) { 783: int64_t maxCid = 0; 784: for (tsdb_ftype_t ftype = TSDB_FTYPE_MIN; ftype <	

	TSDB_FTYPE_MAX; ++ftype) { 785: if (fset->farr[ftype] == NULL) continue; 786: maxCid = TMAX(maxCid, fset->farr[ftype]->f->cid);	
	污染路径: 无	
234	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 276
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 274: 275: int i = 0; 276: for (int j = 0; j < bingNum * tableNum; j++) { 277: params[i+0].buffer_type = TSDB_DATA_TYPE_TIMESTAMP; 278: params[i+0].buffer_length = sizeof(int64_t);	
	污染路径: 无	
235	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 163
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 161: uint64_t arr2[] = {8, 10, 12, 13}; 162: f = (SArray *)taosArrayGetP(src, 1); 163: for (int i = 0; i < sizeof(arr2) / sizeof(arr2[0]); i++) { 164: taosArrayPush(f, &arr2[i]); 165: }	
	污染路径: 无	
236	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/cover.c	行号: 652
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 650: tail -= segmentSize; 651: memcpy(dict + tail, ctx->samples + segment.begin, segmentSize); 652: DISPLAYUPDATE(653: 2, "\r%u%% ",	

	654: (U32)((((dictBufferCapacity - tail) * 100) / dictBufferCapacity));	
	污染路径: expanded from macro 'DISPLAYUPDATE' expanded from macro 'LOCALDISPLAYUPDATE'	
237	缺陷发生位置: /source/libs/tcs/test/tcsTest.cpp	行号: 222
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 220: GTEST_ASSERT_EQ(code, 0); 221: 222: for (int i = 0; i < size / 2; ++i) { 223: GTEST_ASSERT_EQ(data[i * 2], 0); 224: GTEST_ASSERT_EQ(data[i * 2 + 1], 1);	
	污染路径: 无	
238	缺陷发生位置: /source/os/src/osFile.c	行号: 110
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 108: static uint64_t seqId = 0; 109: 110: if (tmpPath[len - 1] != '/') { 111: tmpPath[len++] = '/'; 112: }	
	污染路径: 无	
239	缺陷发生位置: /source/libs/catalog/test/catalogTests.cpp	行号: 1280
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1278: } 1279: 1280: if (++n % ctgTestPrintNum == 0) { 1281: (void)printf("Get:%d\n", n); 1282: }	

	污染路径: 无	
240	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 417
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 415: /* Sanity Checks */ 416: if (maxSymbolValue > FSE_MAX_SYMBOL_VALUE) return (size_t)-FSE_ERROR_maxSymbolValue_tooLarge; 417: if (tableLog > FSE_MAX_TABLELOG) return (size_t)- FSE_ERROR_tableLog_tooLarge; 418: 419: /* Init, lay down lowprob symbols */	
	污染路径: 无	
241	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 1193
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1191: 1192: char *p = strchr(dbFName, '.'); 1193: if (p && IS_SYS_DBNAME(p + 1)) { 1194: dbFName = p + 1; 1195: }	
	污染路径: 无	
242	缺陷发生位置: /contrib/TSZ/zstd/compress/huf_compress.c	行号: 706
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 704: size_t const oldSize = HUF_estimateCompressedSize(oldHufTable, table->count, maxSymbolValue); 705: size_t const newSize = HUF_estimateCompressedSize(table->CTable, table->count, maxSymbolValue); 706: if (oldSize <= hSize + newSize hSize + 12 >= srcSize) {	

	828: for(j=0; j<p->nLookahead; j++){	
	污染路径: 无	
246	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 72
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 70: lb[i] = lenOfBinaryAct; 71: no_null[i] = 0; 72: is_null[i] = (i % 10 == 2) ? 1 : 0; 73: v->b[i] = (int8_t)(i % 2); 74: v->v1[i] = (int8_t)((i+1) % 2);	
	污染路径: 无	
247	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 890
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 888: parser->try_getopt = 0; 889: if (parser->state.next > 1 890: && strcmp (parser->state.argv[parser->state.next - 1], QUOTE) 891: == 0) 892: /* Not only is this the end of the options, but it's a	
	污染路径: 无	
248	缺陷发生位置: /source/libs/decimal/src/detail/intx/int128.hpp	行号: 242
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 240: // but compilers (GCC8, Clang7) 241: // have problem with properly optimizing subtraction. 242: return (x.hi < y.hi) ((x.hi == y.hi) & (x.lo < y.lo)); 243: } 244:	

	污染路径: 无	
249	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 107
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 105: // Start of string literal 106: inString = true; 107: if (outLen + 3 < patternLen) { 108: strcpy(out, "?"); 109: out += 1;	
	污染路径: 无	
250	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 295
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 293: char now[32] = "now"; 294: // write future 1% fixed fill 295: if (stb->writeFuture && RD(100) == 0) { 296: int32_t min = RD(stb->durMinute); 297: if (min <= 0) min = 1;	
	污染路径: 无	
251	缺陷发生位置: /source/libs/decimal/src/detail/intx/intx.hpp	行号: 711
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 709: 710: auto& m = na.num_numerator_words; 711: for (m = num_words; m > 0 && u[m - 1] == 0; --m) 712: ; 713:	
	污染路径: 无	
252	缺陷发生位置:	行号:

	/source/libs/transport/test/cliBench.c		77
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 75: tDebug("thread:%d, send request, contLen:%d num:%d", pInfo->index, pInfo->msgSize, pInfo->num); 76: rpcSendRequest(pInfo->pRpc, &pInfo->epSet, &rpcMsg, NULL); 77: if (pInfo->num % 20000 == 0) tInfo("thread:%d, %d requests have been sent", pInfo->index, pInfo->num); 78: tsem_wait(&pInfo->rspSem); 79: }		
	污染路径: 无		
253	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c		行号: 1102
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 1100: /* Sanity Checks */ 1101: if (maxSymbolValue > FSE_MAX_SYMBOL_VALUE) return ERROR(maxSymbolValue_tooLarge); 1102: if (tableLog > FSE_MAX_TABLELOG) return ERROR(tableLog_tooLarge); 1103: 1104: /* Init, lay down lowprob symbols */		
	污染路径: 无		
254	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c		行号: 962
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: 960: return BITv06_DStream_overflow; 961: 962: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) { 963: bitD->ptr -= bitD->bitsConsumed >> 3; 964: bitD->bitsConsumed &= 7;		
	污染路径:		

	无	
255	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 853
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 851: "\"tags\": [{"name\":\"t1\",\"type\":4,\"value\":543}, {"name\":\"t3\",\"type\":10,\"value\":\" 852: "\"\\\\sttb4\\\\\""}, {"name\":\"t4\",\"type\":1,\"value\":1}],\"createList\":\"[]\"}"; 853: for (int i = 0; i < sizeof(result) / sizeof(result[0]); i++) { 854: taosFprintfFile(pFile2, result[i]); 855: taosFprintfFile(pFile2, "\\n"); </pre>	
	污染路径: 无	
256	缺陷发生位置: /source/common/test/cosTest.cpp	行号: 126
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 124: } 125: 126: for (int i = 0; i < objectSize / 2; ++i) { 127: data[i * 2 + 1] = 1; 128: } </pre>	
	污染路径: 无	
257	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtf8.c	行号: 47
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 45: 46: /* Reconstruct full code point */ 47: if (i == len - codelen + 1) { 48: /* Not enough data */ 49: return TTQ_ERR_MALFORMED_UTF8; </pre>	
	污染路径:	

	无	
258	缺陷发生位置： /source/common/src/ttime.c	行号： 1545
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： <pre> 1543: break; 1544: } 1545: if (*formatStr == '\\' && *(formatStr + 1)) { 1546: formatStr++; 1547: last = NULL; // stop expanding last format, create new format </pre>	
	污染路径： 无	
259	缺陷发生位置： /source/libs/scalar/src/scifunc.c	行号： 625
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： <pre> 623: int32_t pos = orgLen; 624: for (int32_t i = orgLen - remLen; i >= 0; i -= remLen) { 625: if (memcmp(orgStr + i, remStr, remLen) == 0) { 626: if (isCharStart(orgStr[i]) isNchar) { 627: pos = i; </pre>	
	污染路径： 无	
260	缺陷发生位置： /source/libs/tdb/src/db/tdbBtree.c	行号： 784
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： <pre> 782: } 783: for (int i = 0; i < nOlds; i++) { 784: if (!(sldx + i <= nCells)) { 785: return TSDB_CODE_FAILED; 786: } </pre>	
	污染路径： 无	
261	缺陷发生位置： /contrib/TSZ/sz/src/Huffman.c	行号： 233

	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 231: if(lackBits==0) 232: { 233: byteSize = bitSize%8==0 ? bitSize/8 : bitSize/8+1; //it's equal to the number of bytes involved (for *outSize) 234: byteSizep = bitSize/8; //it's used to move the pointer p for next data 235: if(byteSize<=8)	
	污染路径: 无	
262	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/dictBuilder/divsufsort.c	428
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号. 代码片段: 426: } 427: } 428: if(Td[PA[*first] - 1] < v) { 429: first = ss_partition(PA, first, a, depth); 430: }	
	污染路径: 无	
263	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v06.c	1293
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号. 代码片段: 1291: while ((bitStream & 0xFFFF) == 0xFFFF) { 1292: n0+=24; 1293: if (ip < iend-5) { 1294: ip+=2; 1295: bitStream = MEM_readLE32(ip) >> bitCount;	
	污染路径: 无	
264	缺陷发生位置:	行号:
	/source/libs/parser/src/parUtil.c	788

	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 786: *pNext = ++p; 787: } 788: if (*pStart == `` && *(pStart + 1) == ``) { 789: *pStr = 0; 790: }	
	污染路径: 无	
265	缺陷发生位置:	行号:
	/contrib/TSZ/sz/src/sz_float.c	185
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 183: 184: //double-check the prediction error in case of machine-epsilon impact 185: if(fabs(oriFloat - pred) > realPrecision) 186: { 187: type[i] = 0;	
266	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/sz/src/sz_double.c	384
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
267	代码片段: 382: 383: double * data_pos = oriData + 2; 384: while(data_pos - oriData < dataLength){ 385: totalSampleSize++; 386: pred_value = data_pos[-1];	
	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/dictBuilder/divsufsort.c	598
267	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	596: 597: for(;;) { 598: if(*(last - 1) < 0) { x = 1; p = PA + ~*(last - 1); } 599: else { x = 0; p = PA + *(last - 1); } 600: for(a = first, len = middle - first, half = len >> 1, r = -1; 	
	污染路径: 无	
268	缺陷发生位置: /source/libs/transport/test/strBench.c	行号: 135
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 133: 134: for (int i = 1; i < argc; ++i) { 135: if (strcmp(argv[i], "-p") == 0 && i < argc - 1) { 136: rpclnit.localPort = atoi(argv[++i]); 137: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) {	
	污染路径: 无	
269	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 59
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 57: switch (t) { 58: case A: 59: for (size_t i = 0; i < 10 * 1024 * 1024 * 1024; i++) { 60: tPutA(pBuf, val); 61: if (POINTER_DISTANCE(buf, pBuf) == 1024) {	
	污染路径: 无	
270	缺陷发生位置: /tools/shell/src/shellEngine.c	行号: 635
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 633: } 634: int w = 0; 635: if (*(str + pos) == '\t' *(str + pos) == '\n' *(str +	

	<pre>pos) == '\r') { 636: w = bytes; 637: } else {</pre>	
	污染路径: 无	
271	缺陷发生位置: /source/libs/decimal/src/decimal.c	行号: 1426
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre>1424: bool negative = unscaled < 0 ? true : false; 1425: double abs = TABS(unscaled); 1426: if (abs > ldexp(1.0, 127) - 1) { 1427: goto __OVERFLOW__; 1428: }</pre>	
	污染路径: 无	
272	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 2210
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre>2208: } 2209: 2210: if (rand() % 2 == 0) { 2211: if (taos_stmt_add_batch(stmt)) { 2212: printf("!!!taos_stmt_add_batch error:%s\n", taos_stmt_errstr(stmt));</pre>	
	污染路径: 无	
273	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 417
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre>415: if (qwtTestSinkBlockNum == 0) { 416: pOutput->bufStatus = DS_BUF_EMPTY; 417: } else if (qwtTestSinkBlockNum <= qwtTestSinkMaxBlockNum * 0.5) { 418: pOutput->bufStatus = DS_BUF_LOW;</pre>	

	419: } else {	
	污染路径: 无	
274	缺陷发生位置:	行号:
	/tools/taos-tools/src/benchTmq.c	267
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 265: 266: int64_t currentPrintTime = toolsGetTimestampMs(); 267: if (currentPrintTime - lastPrintTime > 10 * 1000) { 268: infoPrint("consumer id %d has poll total msgs: %" PRIu64 " , period rate: %.3f msgs/s, total rows: %" PRIu64 " , period rate: %.3f rows/s\n", 269: pthreadInfo->id, totalMsgs, (totalMsgs - lastTotalMsgs) * 1000.0 / (currentPrintTime - lastPrintTime), totalRows, (totalRows - lastTotalRows) * 1000.0 / (currentPrintTime - lastPrintTime));	
	污染路径: 无	
275	缺陷发生位置:	行号:
	/source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	44
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 42: 43: int sdbSetRawInt32(SSdbRaw* pRaw, int pos, int32_t val) { 44: if (pos + 4 <= 4096) { 45: memcpy(pRaw->data + pos, &val, 4); 46: return 0;	
	污染路径: 无	
276	缺陷发生位置:	行号:
	/utils/test/c/tmq_taosx_ci.c	983
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 981: "t4" , "sttb4" , {"name": "t4" , "type": 1 , "value": 1 } } } } }";	

	<pre> 982: 983: for (int i = 0; i < sizeof(result) / sizeof(result[0]); i+ +) { 984: taosFprintfFile(pFile2, result[i]); 985: taosFprintfFile(pFile2, "\n"); </pre>	
	污染路径: 无	
277	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 3108
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 3106: int32_t caseldx = (gCaseCtrl.caseldx >= 0) ? gCaseCtrl.caseldx : 0; 3107: 3108: for (int32_t i = caseldx; i < sizeof(gCase) / sizeof(gCase[0]); ++i) { 3109: if (gCaseCtrl.caseNum > 0 && caseNum >= gCaseCtrl.caseNum) { 3110: break; </pre>	
	污染路径: 无	
278	缺陷发生位置: /contrib/test/sqlite/sqliteTest.c	行号: 58
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 56: sqlite3_exec(db, "PRAGMA read_uncommitted=true;", 0, 0, &err_msg); 57: 58: for (int k = 0; k < nrows / batch; k++) { 59: sqlite3_exec(db, "begin;", 0, 0, &err_msg); 60: </pre>	
	污染路径: 无	
279	缺陷发生位置: /source/libs/monitorfw/src/taos_metric_formatter.c	行号: 143
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	141: if (r) return r; 142: 143: if (i == label_count - 1) { 144: r = taos_string_builder_add_char(self->string_builder, '}'); 145: if (r) return r;	
	污染路径: 无	
280	缺陷发生位置: /source/libs/transport/test/strBench.c	行号: 137
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 135: if (strcmp(argv[i], "-p") == 0 && i < argc - 1) { 136: rpclnit.localPort = atoi(argv[++i]); 137: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) { 138: rpclnit.numOfThreads = atoi(argv[++i]); 139: } else if (strcmp(argv[i], "-m") == 0 && i < argc - 1) {	
	污染路径: 无	
281	缺陷发生位置: /source/client/test/smlTest.cpp	行号: 541
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 539: "sys.procs.running 1479496100 42 host= web01", 540: }; 541: for (int i = 0; i < sizeof(sql) / sizeof(sql[0]); i++) { 542: SSmlLineInfo elements = {0}; 543: int ret = smlParseTelnetString(info, (char*)sql[i], (char*)(sql[i] + strlen(sql[i])), &elements);	
	污染路径: 无	
282	缺陷发生位置: /contrib/TSZ/sz/src/iniparser.c	行号: 600
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 598: /* Comment line */ 599: sta = LINE_COMMENT ;	

	600: } else if (line[0]=='[' && line[len-1]==']') { 601: /* Section name */ 602: sscanf(line, "[%[^]]", section);	
	污染路径: 无	
283	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 226
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 224: op[0] = FSE_GETSYMBOL(&state1); 225: 226: if (FSE_MAX_TABLELOG*2+7 > sizeof(bitD.bitContainer)*8) /* This test must be static */ 227: BIT_reloadDStream(&bitD); 228:	
	污染路径: 无	
284	缺陷发生位置: /contrib/libmqt/ttq/src/ttqUtilTopic.c	行号: 308
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 306: } 307: /* Check for bad "foo+" or "foo+/a" subscription */ 308: if (spos + 1 < sublen && sub[spos + 1] != '/') { 309: return TTQ_ERR_INVAL; 310: }	
	污染路径: 无	
285	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 86
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 84: int32_t autoSleep(uint64_t interval, uint64_t delay) { 85: int32_t msleep = 0; 86: if (delay < interval * 1000) { 87: msleep = (int32_t)((interval - delay/1000)); 88: infoPrint("do sleep %dms ...\n", msleep);	

	污染路径: 无	
286	缺陷发生位置: /source/common/src/tvariant.c	行号: 212
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 210: } 211: if (pure) { 212: if (endPtr + 1 < end && endPtr[1] > '4') { 213: const char *p = z; 214: while (*p == ' ') {	
	污染路径: 无	
287	缺陷发生位置: /source/libs/executor/test/joinTests.cpp	行号: 2175
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 2173: } 2174: 2175: if (filterNum && filterCol[c] && ((*bool*)(row + c)) cbig <= fbig)) { 2176: filterOut = true; 2177: break;	
	污染路径: 无	
288	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 1081
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1079: 1080: char *p = strchr(dbFName, '.'); 1081: if (p && IS_SYS_DBNAME(p + 1)) { 1082: dbFName = p + 1; 1083: }	
	污染路径: 无	

289	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 397
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 395: // insert the generated big data 396: // char const *key = "key1"; 397: for (int i = 0; i < 1024 * 4; ++i) { 398: // char const *key = "key123456789"; 399: char key[32] = {0};	
	污染路径: 无	
290	缺陷发生位置: /source/os/test/osSemaphoreTests.cpp	行号: 281
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 279: }); 280: 281: for (int i = 0; i < count * 2; ++i) { 282: (void)tsem2_wait(&sem); 283: }	
	污染路径: 无	
291	缺陷发生位置: /source/util/src/tcompression.c	行号: 590
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 588: 589: for (int32_t i = 0; i < nelements; i++) { 590: if (i % ele_per_byte == 0) { 591: pos++; 592: output[pos] = 0;	
	污染路径: 无	
292	缺陷发生位置: /source/libs/azure/test/azExceptionTest.cpp	行号: 196
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	

	代码片段: 194: GTEST_ASSERT_EQ(code, 0); 195: 196: for (int i = 0; i < size / 2; ++i) { 197: GTEST_ASSERT_EQ(data[i * 2], 0); 198: GTEST_ASSERT_EQ(data[i * 2 + 1], 1);	
	污染路径: 无	
293	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1162
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1160: } 1161: } else if (isRest(superTable->iface)) { 1162: if (g_arguments->reqPerReq > SML_MAX_BATCH) { 1163: errorPrint("reqPerReq (%u) larger than maximum (%d)\n", 1164: g_arguments->reqPerReq, SML_MAX_BATCH);	
	污染路径: 无	
294	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 158
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 156: uint16_t left, right, mid; 157: left = RD(cnt); 158: right = cnt + RD(max - cnt); 159: SWAP(cols, left, right) 160: }	
	污染路径: expanded from macro 'RD'	
295	缺陷发生位置: /source/util/test/trefTest.c	行号: 144
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	<pre> 142: } else if (strcmp(argv[i], "-l") == 0 && i < argc - 1) { 143: loops = atoi(argv[++i]); 144: } else if (strcmp(argv[i], "-d") == 0 && i < argc - 1) { 145: uDebugFlag = atoi(argv[i]); 146: } else { </pre>	
	污染路径: 无	
296	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1191
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 1189: { 1190: n0+=24; 1191: if (ip < iend-5) 1192: { 1193: ip+=2; </pre>	
	污染路径: 无	
297	缺陷发生位置: /contrib/TSZ/zstd/compress/zstd_compress.c	行号: 1599
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 1597: } 1598: 1599: if ((cLitSize==0) (cLitSize >= srcSize - minGain) ERR_isError(cLitSize)) { 1600: memcpy(nextHuf, prevHuf, sizeof(*prevHuf)); 1601: return ZSTD_noCompressLiterals(dst, dstCapacity, src, srcSize); </pre>	
	污染路径: 无	
298	缺陷发生位置: /source/libs/tcs/test/tcsTest.cpp	行号: 195
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 193: GTEST_ASSERT_EQ(code, 0); 194: </pre>	

	195: for (int i = 0; i < size / 2; ++i) { 196: GTEST_ASSERT_EQ(data[i * 2], 0); 197: GTEST_ASSERT_EQ(data[i * 2 + 1], 1);	
	污染路径: 无	
299	缺陷发生位置: /source/libs/parser/src/parser.c	行号: 123
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 121: break; 122: } 123: } while (pStr - pSql < length); 124: 125: return false;	
	污染路径: 无	
300	缺陷发生位置: /contrib/TSZ/sz/src/TightDataPointStorageD.c	行号: 127
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 125: 126: size_t logicLeadNumBitsNum = (*this)->exactDataNum * 2; 127: if (logicLeadNumBitsNum % 8 == 0) 128: { 129: (*this)->leadNumArray_size logicLeadNumBitsNum >> 3;	
	污染路径: 无	
301	缺陷发生位置: /source/libs/tmqtt/mqtt/src/tmqttLogging.c	行号: 263
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 261: log_line_pos = (size_t)snprintf(log_line, sizeof(log_line), "%" PRIu64, (uint64_t)db.now_real_s); 262: } 263: if (log_line_pos < sizeof(log_line) - 3) {	

	264: log_line[log_line_pos] = ':'; 265: log_line[log_line_pos + 1] = ' ';	
	污染路径: 无	
302	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-help.c	行号: 999
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 997: { 998: if (argp_fmtstream_point (stream) + ensure 999: >= argp_fmtstream_rmargin (stream)) 1000: argp_fmtstream_putc (stream, '\n'); 1001: else	
	污染路径: 无	
303	缺陷发生位置: /tools/taos-tools/src/benchQuery.c	行号: 115
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 113: int64_t showRealQPS(qThreadInfo* pThreadInfo, int64_t lastPrintTime, int64_t startTs) { 114: int64_t now = toolsGetTimestampMs(); 115: if (now - lastPrintTime > 10 * 1000) { 116: // real total 117: uint64_t totalQueried = pThreadInfo->nSucc;	
	污染路径: 无	
304	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtilTopic.c	行号: 142
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 140: } 141: } else if (str[i] == '#') { 142: if ((c != '\0' && c != '/') i < len - 1) { 143: return TTQ_ERR_INVALID; 144: }	

	污染路径: 无	
305	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 490
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 488: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 489: { 490: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 491: } 492:	
	污染路径: 无	
306	缺陷发生位置: /source/libs/qworker/test/qworkerTests.cpp	行号: 703
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 701: taosUsleep(taosRand() % 5); 702: } 703: if (++n % qwtTestPrintNum == 0) { 704: printf("fetch:%d\n", n); 705: }	
	污染路径: 无	
307	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 886
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 884: char * field_name_ptr = field->name; 885: int32_t field_name_len = strlen(field_name_ptr); 886: if (field_name_ptr[0] == '`' && field_name_ptr[field_name_len - 1] == `') { 887: // remove the back quote 888: field_name_ptr++;	

	污染路径: 无	
308	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 632
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 630: return BITv07_DStream_overflow; 631: 632: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) { 633: bitD->ptr -= bitD->bitsConsumed >> 3; 634: bitD->bitsConsumed &= 7;	
	污染路径: 无	
309	缺陷发生位置: /contrib/test/tdev/src/main.c	行号: 91
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 89: break; 90: case E: 91: for (size_t i = 0; i < 10 * 1024 * 1024 * 1024 ; i++) { 92: tPutE(&pBuf, val); 93: if (POINTER_DISTANCE(buf, pBuf) == 1024) {	
	污染路径: 无	
310	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1226
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1224: 1225: // check limit offset 1226: if(superTable->childTblOffset + superTable->childTblLimit > superTable->childTblCount) { 1227: errorPrint("json config invalid. childtable_offset(%"PRIu64") + childtable_limit(%"PRIu64") > childtable_count(%"PRIu64")", 1228: superTable->childTblOffset, superTable-	

	>childTblLimit, superTable->childTblCount);	
	污染路径: 无	
311	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 1473
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1471: int const endFrame = (endOp == ZSTD_e_end); 1472: 1473: if (mtctx->nextJobID > mtctx->doneJobID + mtctx->jobIDMask) { 1474: DEBUGLOG(5, "ZSTDMT_createCompressionJob: will not create new job : table is full"); 1475: assert((mtctx->nextJobID & mtctx->jobIDMask) == (mtctx->doneJobID & mtctx->jobIDMask));	
	污染路径: 无	
312	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 244
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 242: bitStream>>=16; 243: } 244: while (charnum >= start+3) { 245: start+=3; 246: bitStream += 3 << bitCount;	
	污染路径: 无	
313	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 1388
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1386: 1387: fieldName = "end_time"; 1388: if (strncmp(field, fieldName, strlen(fieldName) + 1) == 0) { 1389: *(int64_t *)value = pReader->endTime;	

	污染路径: 无	
317	缺陷发生位置: /source/client/test/clientMonitorTests.cpp	行号: 79
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 77: char* pCont = NULL; 78: int64_t totalSize = 0; 79: if (size - *offset >= SLOW_LOG_SEND_SIZE_MAX) { 80: pCont = (char*)taosMemoryCalloc(1, 4 + SLOW_LOG_SEND_SIZE_MAX); 81: totalSize = 4 + SLOW_LOG_SEND_SIZE_MAX;	
	污染路径: 无	
318	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1309
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1307: while ((bitStream & 0xFFFF) == 0xFFFF) { 1308: n0+=24; 1309: if (ip < iend-5) { 1310: ip+=2; 1311: bitStream = MEM_readLE32(ip) >> bitCount;	
	污染路径: 无	
319	缺陷发生位置: /tools/taos-tools/src/benchCsv.c	行号: 98
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 96: } 97: 98: if (*endptr == '\0' 99: *(endptr + 1) != '\0') { 100: return -1;	
	污染路径:	

	无	
320	缺陷发生位置： /source/libs/azure/test/azTest.cpp	行号： 190
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 188: GTEST_ASSERT_EQ(code, 0); 189: 190: for (int i = 0; i < size / 2; ++i) { 191: GTEST_ASSERT_EQ(data[i * 2], 0); 192: GTEST_ASSERT_EQ(data[i * 2 + 1], 1);	
	污染路径： 无	
321	缺陷发生位置： /source/dnode/vnode/src/meta/metaEntry2.c	行号： 1282
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 1280: }; 1281: 1282: for (int i = 0; i < sizeof(ops) / sizeof(ops[0]); i++) { 1283: SMetaTableOp *op = &ops[i]; 1284:	
	污染路径： 无	
322	缺陷发生位置： /contrib/TSZ/zstd/dictBuilder/zdict.c	行号： 907
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： 905: 906: /* copy elements in final buffer ; note : src and dst buffer can overlap */ 907: if (hSize + dictContentSize > dictBufferCapacity) dictContentSize = dictBufferCapacity - hSize; 908: { size_t const dictSize = hSize + dictContentSize; 909: char* dictEnd = (char*)dictBuffer + dictSize;	
	污染路径： 无	
323	缺陷发生位置：	行号：

	/tools/taos-tools/src/benchCsv.c		99
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 97: 98: if (*endptr == '\0' 99: *(endptr + 1) != '\0') { 100: return -1; 101: }</pre>		
	污染路径: 无		
324	缺陷发生位置: /source/libs/tdb/test/tdbPageDefragmentTest.cpp		行号: 672
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 670: tdbBegin(pEnv, &txn, poolMalloc, poolFree, pPool, TDB_TXN_WRITE TDB_TXN_READ_UNCOMMITTED); 671: 672: for (int iData = nData + 1; iData <= nData * 2; iData+ +) { 673: // if (iData % 2 == 0) continue; 674: sprintf(key, "key%d", iData);</pre>		
	污染路径: 无		
325	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp		行号: 267
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 265: // insert the generated big data 266: // char const *key = "key1"; 267: for (int i = 0; i < 1024 * 4; ++i) { 268: // char const *key = "key123456789"; 269: char key[32] = {0};</pre>		
	污染路径: 无		
326	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp		行号: 343
	步骤描述:		

	逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 341: tdbBegin(pEnv, &txn, poolMallocRestricted, poolFree, pPool, TDB_TXN_WRITE TDB_TXN_READ_UNCOMMITTED); 342: 343: for (int i = 1024 * 4; i < 1024 * 8; ++i) { 344: char key[32] = {0}; 345: sprintf(key, "key-%d", i);	
	污染路径: 无	
327	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 374
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 372: for(i=1; i<segmentSize;i++) 373: { 374: if(i*i>segmentSize) 375: break; 376: }	
	污染路径: 无	
328	缺陷发生位置: /tools/taos-tools/src/taosdump.c	行号: 467
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 465: g_args.renameHead = node; 466: for (int k = 0; k < j; k++) { 467: if(p[k] == 0 && k + 1 != j && k > 0) { 468: // string end and not last end 469: char* name = &p[k] + 1;	
	污染路径: 无	
329	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 125
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 123: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) {	

	124: rpcInit.numOfThreads = atoi(argv[++i]); 125: } else if (strcmp(argv[i], "-m") == 0 && i < argc - 1) { 126: msgSize = atoi(argv[++i]); 127: } else if (strcmp(argv[i], "-s") == 0 && i < argc - 1) {	
	污染路径: 无	
330	缺陷发生位置: /tools/taos-tools/src/benchUtil.c	行号: 1499
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1497: } 1498: 1499: for (int l = 0; l < mod_table[userpass_buf_len % 3]; l++) 1500: g_arguments->base64_buf[encoded_len - 1 - l] = '='; 1501: }	
	污染路径: 无	
331	缺陷发生位置: /source/libs/tdb/src/db/tdbBtree.c	行号: 972
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 970: } 971: 972: if (infoNews[iNew].size + szRCell >= infoNews[iNew - 1].size - szRCell) { 973: break; 974: }	
	污染路径: 无	
332	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 132
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 130: uint64_t arr1[] = {2, 3, 4, 5}; 131: SArray * f = (SArray *)taosArrayGetP(src, 0);	

	<pre> 608: int32_t pos = 0; 609: for (int32_t i = 0; i < orgLen - remLen; i += remLen) { 610: if (memcmp(orgStr + i, remStr, remLen) == 0) { 611: if (isCharStart(orgStr[i + remLen]) isNchar) { 612: pos = i + remLen; </pre>		
	污染路径: 无		
336	缺陷发生位置: /source/libs/azure/test/azExceptionTest.cpp	行号: 169	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 167: GTEST_ASSERT_EQ(code, 0); 168: 169: for (int i = 0; i < size / 2; ++i) { 170: GTEST_ASSERT_EQ(data[i * 2], 0); 171: GTEST_ASSERT_EQ(data[i * 2 + 1], 1); </pre>		
	污染路径: 无		
337	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/zdict.c	行号: 1008	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 1006: if (dictContentSize < targetDictSize/4) { 1007: DISPLAYLEVEL(2, "! warning : selected content significantly smaller than requested (%u < %u) \n", dictContentSize, (U32)maxDictSize); 1008: if (samplesBuffSize < 10 * targetDictSize) 1009: DISPLAYLEVEL(2, "! consider increasing the number of samples (total size : %u MB)\n", (U32) (samplesBuffSize>>20)); 1010: if (minRep > MINRATIO) { </pre>		
	污染路径: 无		
338	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 790	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段:		

	<pre> 788: int fieldsSize, Field *field) { 789: int n = snprintf(sampleDataBuf + pos, bufLen - pos, 790: "{"); 791: if (n < 0 n >= bufLen - pos) { 792: errorPrint("%s() LN%d snprintf overflow\n", 793: __func__, __LINE__); </pre>	
	污染路径: 无	
339	缺陷发生位置: /source/os/src/osSysinfo.c	行号: 1571
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 1569: if (n < 0) n = 0; 1570: 1571: if (n > 0 && cmdline[n - 1] == '\n') --n; 1572: 1573: cmdline[n] = 0; </pre>	
	污染路径: 无	
340	缺陷发生位置: /source/dnode/mnode/impl/test/xnode/xnodeSimpleTest.cpp	行号: 52
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 50: 51: int sdbSetRawInt64(SSdbRaw* pRaw, int pos, int64_t val) 52: { 53: if (pos + 8 <= 4096) { 54: memcpy(pRaw->data + pos, &val, 8); 55: return 0; 56: } 57: return -1; 58: } </pre>	
	污染路径: 无	
341	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFSet2.c	行号: 318
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 316: if (code) return code; 317: </pre>	

	<pre> 318: for (tsdb_ftype_t ftype = TSDB_FTYPE_MIN; ftype < TSDB_FTYPE_MAX; ++ftype) { 319: code = tsdbJsonToFile(json, ftype, &tf); 320: if (code == TSDB_CODE_NOT_FOUND) { </pre>		
	污染路径: 无		
342	缺陷发生位置: /contrib/TSZ/zstd/common/bitstream.h	行号: 429	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 427: } 428: if (bitD->ptr == bitD->start) { 429: if (bitD->bitsConsumed < sizeof(bitD- >bitContainer)*8) return BIT_DStream_endOfBuffer; 430: return BIT_DStream_completed; 431: } </pre>		
	污染路径: 无		
343	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 669	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 667: int intTmp; 668: 669: if (field->min < (-1 * (RAND_MAX >> 1))) { 670: field->min = -1 * (RAND_MAX >> 1); 671: } </pre>		
	污染路径: 无		
344	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 652	
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.		
	代码片段: <pre> 650: intTmp = 105 + taosRandom() % 10; 651: } else { 652: if (field->min < (-1 * (RAND_MAX >> 1))) { 653: field->min = -1 * (RAND_MAX >> 1); </pre>		

	654: }	
	污染路径: 无	
345	缺陷发生位置: /source/libs/scheduler/test/schedulerTests.cpp	行号: 865
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 863: schtFreeQueryDag(dag); 864: 865: if (++jobFinished % schtTestPrintNum == 0) { 866: (void)printf("jobFinished:%d\n", jobFinished); 867: }	
	污染路径: 无	
346	缺陷发生位置: /source/dnode/mnode/impl/src/mndVgroup.c	行号: 3923
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 3921: pDst->id, dstScore); 3922: 3923: if (srcScore > dstScore - 0.000001) { 3924: code = mndBalanceVgroupBetweenDnode(pMnode, pTrans, pSrc, pDst, pBalancedVgroups); 3925: if (code == 0) {	
	污染路径: 无	
347	缺陷发生位置: /source/dnode/mnode/impl/src/mndDb.c	行号: 1306
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1304: } 1305: char *realDbName = strchr(createReq.db, '.'); 1306: if (realDbName && strcmp(realDbName + 1, "log") == 0) { 1307: mError("db:%s, failed to create, db name not allowed for audit db, %s", createReq.db, realDbName + 1); 1308: TAOS_CHECK_GOTO(TSDB_CODE_MND_INVALID_DB,	

	&lino, _OVER);	
	污染路径: 无	
348	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 900
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 898: 899: TEST(timezoneCase, format_timezone_Test) { 900: for (unsigned int i = 0; i < sizeof (test_tz) / sizeof (test_tz[0]); ++i){ 901: timezone_str_test(test_tz[i].name, test_tz[i].t, test_tz[i].timezone); 902: timezone_str_test(test_tz[i].name, test_tz[i].t, test_tz[i].timezone);	
	污染路径: 无	
349	缺陷发生位置: /source/util/test/trefTest.c	行号: 136
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 134: 135: for (int i = 1; i < argc; ++i) { 136: if (strcmp(argv[i], "-n") == 0 && i < argc - 1) { 137: refNum = atoi(argv[++i]); 138: } else if (strcmp(argv[i], "-s") == 0 && i < argc - 1) {	
	污染路径: 无	
350	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbFS2.c	行号: 1376
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1374: 1375: fieldName = "fileset_id"; 1376: if (strncmp(field, fieldName, strlen(fieldName) + 1) == 0) { 1377: *(int32_t *)value = pReader->fid;	

	1378: return TSDB_CODE_SUCCESS;	
	污染路径: 无	
351	缺陷发生位置: /tools/taos-tools/src/dumpUtil.c	行号: 667
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 665: 666: // loop all 667: for (int i = 0; i < oldc + oldt; i++) { 668: ColDes * colDes = localDes->cols + i; 669: if (i < oldc) {	
	污染路径: 无	
352	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 284
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 282: int64ToBytes_bigEndian(p, newCode); 283: bitSize -= lackBits; 284: byteSize = bitSize%8==0 ? bitSize/8 : bitSize/8+1; 285: byteSizep = bitSize/8; 286: p += byteSizep;	
	污染路径: 无	
353	缺陷发生位置: /source/libs/transport/test/cliBench.c	行号: 123
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 121: if (strcmp(argv[i], "-p") == 0 && i < argc - 1) { 122: } else if (strcmp(argv[i], "-i") == 0 && i < argc - 1) { 123: } else if (strcmp(argv[i], "-t") == 0 && i < argc - 1) { 124: rpclnit.numOfThreads = atoi(argv[++i]); 125: } else if (strcmp(argv[i], "-m") == 0 && i < argc - 1) {	

	污染路径: 无	
354	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 1085
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1083: #endif 1084: } 1085: if (iRow == nRow - 1) { 1086: #ifndef NO_UNALIGNED_ACCESS 1087: walMeta.ekey = pRow->ts;	
	污染路径: 无	
355	缺陷发生位置: /contrib/libmqtt/ttq/src/ttqUtf8.c	行号: 81
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 79: return TTQ_ERR_MALFORMED_UTF8; 80: } 81: if ((codepoint & 0xFFFF) == 0xFFFE (codepoint & 0xFFFF) == 0xFFFF) { 82: return TTQ_ERR_MALFORMED_UTF8; 83: }	
	污染路径: 无	
356	缺陷发生位置: /contrib/TSZ/zstd/compress/zstdmt_compress.c	行号: 1202
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1200: mtctx->jobs[u].jobID = u; 1201: mtctx->jobs[u].firstJob = (u==0); 1202: mtctx->jobs[u].lastJob = (u==nbJobs-1); 1203: 1204: DEBUGLOG(5, "ZSTDMT_compress_advanced_internal: posting job %u (%u bytes)", u, (U32)jobSize);	
	污染路径:	

	无	
357	缺陷发生位置： /tools/taos-tools/src/benchDataMix.c	行号： 66
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： <pre> 64: int64_t nowts= 0; 65: char val[512] = {0}; 66: if(fd->fillNull && RD(inul) == 0) { 67: size = sprintf(pstr + len, "%s", nullVal); 68: return size; </pre>	
	污染路径： 无	
358	缺陷发生位置： /source/os/src/osTimezone.c	行号： 1258
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： <pre> 1256: goto END; 1257: } 1258: if (tz[len - 1] == '\n') { 1259: tz[len - 1] = '\0'; 1260: } </pre>	
	污染路径： 无	
359	缺陷发生位置： /source/util/src/tcompression.c	行号： 411
	步骤描述： 逻辑判别表达式中的运算项必须要使用括号。	
	代码片段： <pre> 409: } 410: 411: if (elems + 1 <= selector_to_elems[(int32_t)selector] && 412: elems + 1 <= selector_to_elems[(int32_t) (bit_to_selector[(int32_t)tmp_bit])]) { 413: // If can hold another one. </pre>	
	污染路径： 无	
360	缺陷发生位置：	行号：

	/contrib/TSZ/zstd/common/fse_decompress.c	231
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 229: op[1] = FSE_GETSYMBOL(&state2); 230: 231: if (FSE_MAX_TABLELOG*4+7 > sizeof(bitD.bitContainer)*8) /* This test must be static */ 232: { if (BIT_reloadDStream(&bitD) > BIT_DStream_unfinished) { op+=2; break; } } 233:	
	污染路径: 无	
361	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 72
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 70: 71: // s1 is true 72: for (int32_t i = 0; i < sizeof(s1) / sizeof(char *); i++) { 73: printf("s1 i=%d fieldOptionsArea %s expect true \n", i, s1[i]); 74: ASSERT(fieldOptionsArea((char *)s1[i]) == true);	
	污染路径: 无	
362	缺陷发生位置: /test/new_test_framework/script/api/demoapi.c	行号: 131
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 129: int64_t total = 0; 130: int64_t affected; 131: for (; j < g_num_of_rec -2; j++) { 132: sprintf(command, "insert into t%"PRId64" " 133: "values(%" PRId64 ", %f, %"PRId64", "	
	污染路径: 无	
363	缺陷发生位置: /test/new_test_framework/script/api/batchprepare.c	行号: 1472

	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1470: 1471: if (gCurCase->bindRowNum > 1) { 1472: if (0 == (n++%2) gCurCase->stbInsert) { 1473: if (taos_stmt_bind_param_batch(stmt, bind)) { 1474: if (expectFail) return 0;	
	污染路径: 无	
364	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/compress/huf_compress.c	713
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 711: 712: /* Use the new huffman table */ 713: if (hSize + 12ul >= srcSize) { return 0; } 714: op += hSize; 715: if (repeat) { *repeat = HUF_repeat_none; }	
	污染路径: 无	
365	缺陷发生位置:	行号:
	/source/os/src/osDir.c	469
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 467: char *pPoint = strrchr(name, '/'); 468: if (pPoint != NULL) { 469: if (*(pPoint + 1) == '\0') { 470: *pPoint = '\0'; 471: return taosDirEntryBaseName(name);	
	污染路径: 无	
366	缺陷发生位置:	行号:
	/utils/test/c/tmqDemo.c	454
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 452: int64_t time_counter = g_stConflInfo.startTimestamp;	

	453: for (int i = 0; i < g_stConflInfo.totalRowsOfPerTbl;) { 454: for (int tID = 0; tID <= g_stConflInfo.numOfTables - 1; tID++) { 455: int inserted = i; 456: int64_t tmp_time = time_counter; 	
	污染路径: 无	
367	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 889
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 887: MEM_STATIC unsigned BITv05_endOfDStream(const BITv05_DStream_t* DStream) 888: { 889: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 890: } 891:	
	污染路径: 无	
368	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 302
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 300: if((size % 2) == 0) { 301: m--; 302: if(Td[PA[SA[m / 2]]] < Td[PA[SA[m]]]) { SWAP(SA[m], SA[m / 2]); } 303: } 304:	
	污染路径: 无	
369	缺陷发生位置: /source/os/test/osAtomicTests.cpp	行号: 683
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	<pre>681: // Half threads add, half subtract 682: for (int i = 0; i < NUM_THREADS; i++) { 683: if (i % 2 == 0) { 684: threads.emplace_back([&value, OPERATIONS]() { 685: for (int j = 0; j < OPERATIONS; j++) {</pre>	
	污染路径: 无	
370	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 873
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 871: U32 nbBytes = bitD->bitsConsumed >> 3; 872: BITv05_DStream_status result = BITv05_DStream_unfinished; 873: if (bitD->ptr - nbBytes < bitD->start) { 874: nbBytes = (U32)(bitD->ptr - bitD->start); /* ptr > start */ 875: result = BITv05_DStream_endOfBuffer;	
	污染路径: 无	
371	缺陷发生位置: /tools/shell/test/shellTest.cpp	行号: 66
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 64: 65: // s0 is false 66: for (int32_t i = 0; i < sizeof(s0) / sizeof(char *); i++) { 67: printf("s0 i=%d fieldOptionsArea %s expect false \n", i, s0[i]); 68: ASSERT(fieldOptionsArea((char *)s0[i]) == false);	
	污染路径: 无	
372	缺陷发生位置: /source/client/test/connectOptionsTest.cpp	行号: 945
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 943: tm.tm_sec = 15;	

	944: 945: for (unsigned int i = 0; i < sizeof (test_mk) / sizeof (test_mk[0]); ++i) 946: { 947: timezone_t tzp = tzalloc(test_mk[i].env);	
	污染路径: 无	
373	缺陷发生位置: /source/util/src/tenv.c	行号: 64
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 62: if (p != NULL) { 63: char buf[CFG_NAME_MAX_LEN]; 64: if (*(p + 1) == '\\') { 65: *(p + 1) = ' '; 66: char *pEnd = &cfgStr[strlen(cfgStr) - 1];	
	污染路径: 无	
374	缺陷发生位置: /contrib/TSZ/sz/src/conf.c	行号: 205
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 203: } 204: 205: if(quantization_intervals%2!=0) 206: { 207: printf("Error: quantization_intervals must be an even number!\n");	
	污染路径: 无	
375	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 1625
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1623: 1624: if (gCurCase->bindRowNum > 1) { 1625: if (0 == (n++ % 2) gCurCase->stbInsert) {	

	1626: if (st_stmt2_bind_param_batch(stmt, bind)) { 1627: if (expectFail) return 0;	
	污染路径: 无	
376	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 310
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 308: object->valueint = LLONG_MAX; 309: } 310: else if (number <= LLONG_MIN) 311: { 312: object->valueint = LLONG_MIN;	
	污染路径: 无	
377	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号: 685
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 683: { 684: if (!(parser->state.flags & ARGP_NO_ERRS) 685: && parser->state.err_stream) 686: fprintf (parser->state.err_stream, 687: dgettext (parser->argp->argp_domain,	
	污染路径: 无	
378	缺陷发生位置: /tools/taos-tools/src/benchInsert.c	行号: 302
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 300: } 301: 302: if (n < 0 n >= ((int)col_buffer_len - len)) { 303: errorPrint("%s() LN%d, snprintf overflow on %d\ n", 304: __func__, __LINE__, colIndex);	

	污染路径: 无	
379	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 860
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 858: return BITv05_DStream_overflow; 859: 860: if (bitD->ptr >= bitD->start + sizeof(bitD->bitContainer)) { 861: bitD->ptr -= bitD->bitsConsumed >> 3; 862: bitD->bitsConsumed &= 7;	
	污染路径: 无	
380	缺陷发生位置: /contrib/libmqt/ttq/src/ttqUtf8.c	行号: 51
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 49: return TTQ_ERR_MALFORMED_UTF8; 50: } 51: for (j = 0; j < codelen - 1; j++) { 52: if ((ustr[++i] & 0xC0) != 0x80) { 53: /* Not a continuation byte */	
	污染路径: 无	
381	缺陷发生位置: /source/libs/index/test/utilUT.cc	行号: 144
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 142: uint64_t arr3[] = {3, 5, 10, 11}; 143: f = (SArray *)taosArrayGetP(src, 2); 144: for (int i = 0; i < sizeof(arr3) / sizeof(arr3[0]); i++) { 145: taosArrayPush(f, &arr3[i]); 146: }	
	污染路径: 无	

382	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 799
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 797: n += sprintf(sampleDataBuf + pos + n, bufLen - pos - n, 798: "\k%d\":" , j); 799: if (n < 0 n >= bufLen - pos) { 800: errorPrint("%s() LN%d sprintf overflow\n", 801: __func__, __LINE__);	
	污染路径: 无	
383	缺陷发生位置: /tools/taos-tools/src/benchInsertMix.c	行号: 400
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 398: uint32_t size = 0; 399: // remain need generate rows 400: bool forceDis = FORCE_TAKEOUT(MDIS); 401: bool forceUpd = FORCE_TAKEOUT(MUPD); 402:	
	污染路径: expanded from macro 'FORCE_TAKEOUT'	
384	缺陷发生位置: /source/client/test/session.cpp	行号: 966
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 964: } 965: 966: for (int32_t i = 0; i < sizeof(users) / sizeof(users[0]); i+ +) { 967: code = sessMgtUpdateUserMetric(users[i], ¶); 968: ASSERT(code != 0);	
	污染路径: 无	
385	缺陷发生位置: /source/libs/azure/test/azTest.cpp	行号: 231

	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 229: } 230: 231: for (int i = 0; i < size / 2; ++i) { 232: data[i * 2 + 1] = 1; 233: }	
	污染路径: 无	
386	缺陷发生位置:	行号:
	/source/client/test/connectOptionsTest.cpp	687
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 685: * 3. test timestamp with string format 686: */ 687: for (unsigned int i = 0; i < sizeof (params1) / sizeof (params1[0]); ++i){ 688: do_insert(params1[i]); 689: do_select(params1[i]);	
387	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/tdb/test/tdbPageDefragmentTest.cpp	590
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
388	代码片段: 588: 589: for (int iData = 1; iData <= nData; iData++) { 590: if (iData % 2 == 0) continue; 591: 592: sprintf(key, "key%d", iData);	
	污染路径: 无	
	缺陷发生位置:	行号:
	/source/libs/tdb/src/db/tdbBtree.c	968
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	<pre> 966: } 967: 968: if (!(infoNews[iNew - 1].cnt > 0)) { 969: return TSDB_CODE_FAILED; 970: } </pre>	
	污染路径: 无	
389	缺陷发生位置: /tools/shell/src/shellTire.c	行号: 107
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 105: memset(p, 0, sizeof(STireNode)); 106: nodes[m] = p; 107: if (i == len - 1) { 108: // is end 109: p->end = true; </pre>	
	污染路径: 无	
390	缺陷发生位置: /source/libs/azure/test/azTest.cpp	行号: 104
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 102: const int size = 4096; 103: char data[size] = {0}; 104: for (int i = 0; i < size / 2; ++i) { 105: data[i * 2 + 1] = 1; 106: } </pre>	
	污染路径: 无	
391	缺陷发生位置: /source/libs/tdb/test/tdbPageFlushTest.cpp	行号: 330
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: <pre> 328: // insert the generated big data 329: // char const *key = "key1"; 330: for (int i = 0; i < 1024 * 4; ++i) { 331: // char const *key = "key123456789"; </pre>	

	污染路径: 无	
395	缺陷发生位置: /source/libs/parser/src/parTranslator.c	行号: 4589
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 4587: 4588: static int32_t rewriteQueryTimeFunc(STranslateContext* pCxt, int64_t val, SNode** pNode) { 4589: if (INT64_MIN == val INT64_MAX == val) { 4590: return rewriteFuncToValue(pCxt, NULL, pNode); 4591: }	
	污染路径: 无	
396	缺陷发生位置: /source/dnode/mnode/impl/src/mndXnode.c	行号: 4652
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 4650: // remove padding char '=' 4651: size_t len = strlen(base64_str); 4652: while (len > 0 && base64_str[len - 1] == '=') { 4653: base64_str[len - 1] = '\0'; 4654: len--;	
	污染路径: 无	
397	缺陷发生位置: /utils/test/c/tmqDemo.c	行号: 484
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 482: totalMsgs++; 483: 484: if (tID == g_stConfInfo.numOfTables - 1) { 485: i = inserted; 486: time_counter = tmp_time;	
	污染路径: 无	
398	缺陷发生位置:	行号:

	/contrib/TSZ/sz/src/TypeManager.c	76
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 74: void convertByteArray2IntArray_fast_2b(size_t stepLength, unsigned char* byteArray, size_t byteArrayLength, unsigned char **intArray) 75: { 76: if(stepLength > byteArrayLength*4) 77: { 78: printf("Error: stepLength > byteArray.length*4\n");	
	污染路径: 无	
399	缺陷发生位置: /source/libs/catalog/src/ctgCache.c	行号: 1243
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1241: 1242: char *p = strchr(dbFName, '.'); 1243: if (p && IS_SYS_DBNAME(p + 1)) { 1244: dbFName = p + 1; 1245: }	
	污染路径: 无	
400	缺陷发生位置: /source/client/test/stmtTest.cpp	行号: 131
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 129: 130: if (!isCreateTable) { 131: if (k % 2 == 0) { 132: code = taos_stmt_set_tbname_tags(stmt, table_name, tags); 133: checkError(stmt, code);	
	污染路径: 无	
401	缺陷发生位置: /test/new_test_framework/script/api/stmtBatchTest.c	行号: 173

	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 171: 172: //int64_t tts = 1591060628000; 173: for (int i = 0; i < totalRowsPerTbl * tableNum; ++i) { 174: v->ts[i] = tts + i; 175: }	
	污染路径: 无	
402	缺陷发生位置: /test/new_test_framework/script/api/insertBlob.c	行号: 88
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 86: char *buf = calloc(1, BLOB_LEN); 87: 88: for (i = 0; i < BLOB_LEN - 1000; ++i) { 89: buf[i] = 'a' + (i % 26); // fill with 'a' to 'z' 90: }	
	污染路径: 无	
403	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 1159
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1157: for (int32_t pos = 0; pos < orgLen; pos++) { 1158: if (delimStr) { 1159: if (pos + delimLen > orgLen) { 1160: return orgLen; 1161: } 1162: } 1163: }	
	污染路径: 无	
404	缺陷发生位置: /utils/test/c/tmq_taosx_ci.c	行号: 1007
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1005: "19]],\"createList\":[[]]\"}];	

	1006: 1007: for (int i = 0; i < sizeof(result) / sizeof(result[0]); i+ +) { 1008: taosFprintfFile(pFile2, result[i]); 1009: taosFprintfFile(pFile2, "\n");	
	污染路径: 无	
405	缺陷发生位置: /utlis/test/c/tmqDemo.c	行号: 530
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 528: } 529: 530: for (int tID = 0; tID <= g_stConfInfo.numOfTables - 1; tID++) { 531: if (0 == tID) { 532: tmp_time = tsOfT1;	
	污染路径: 无	
406	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 157
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 155: { 156: int __key = __opt->key; 157: return __key > 0 && __key <= UCHAR_MAX && isprint (__key); 158: } 159: }	
	污染路径: 无	
407	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1237
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1235: while ((bitStream & 0xFFFF) == 0xFFFF) {	

	1236: n0+=24; 1237: if (ip < iend-5) { 1238: ip+=2; 1239: bitStream = MEM_readLE32(ip) >> bitCount;	
	污染路径: 无	
408	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 771
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 769: MEM_STATIC unsigned BIT_endOfDStream(const BIT_DStream_t* DStream) 770: { 771: return ((DStream->ptr == DStream->start) && (DStream->bitsConsumed == sizeof(DStream-> bitContainer)*8)); 772: } 773:	
	污染路径: 无	
409	缺陷发生位置: /test/new_test_framework/script/api/stmt2-test.c	行号: 2355
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 2353: } 2354: 2355: if (rand() % 2 == 0) { 2356: if (st_stmt2_add_batch(stmt)) { 2357: printf("!!!st_stmt2_add_batch error:%s\n", st_stmt2_error(stmt));	
	污染路径: 无	
410	缺陷发生位置: /source/libs/catalog/src/ctgUtil.c	行号: 1332
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段:	

	1330: 1331: char* pSep = strchr(dbFName, '.'); 1332: if (pSep && IS_SYS_DBNAME(pSep + 1)) { 1333: SVgroupInfo mgmtInfo = {0}; 1334: mgmtInfo.vgId = MNODE_HANDLE;	
	污染路径: 无	
411	缺陷发生位置: /source/libs/azure/test/azTest.cpp	行号: 141
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 139: GTEST_ASSERT_EQ(code, 0); 140: 141: for (int i = 0; i < size / 2; ++i) { 142: GTEST_ASSERT_EQ(pBlock[i * 2], 0); 143: GTEST_ASSERT_EQ(pBlock[i * 2 + 1], 1);	
	污染路径: 无	
412	缺陷发生位置: /contrib/TSZ/zstd/common/mem.h	行号: 44
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 42: /* code only tested on 32 and 64 bits systems */ 43: #define MEM_STATIC_ASSERT(c) { enum { MEM_static_assert = 1/(int)(!!(c)) }; } 44: MEM_STATIC void MEM_check(void) { MEM_STATIC_ASSERT((sizeof(size_t)==4) (sizeof(size_t)==8)); } 45: 46:	
	污染路径: expanded from macro 'MEM_STATIC_ASSERT'	
413	缺陷发生位置: /source/dnode/vnode/src/meta/metaEntry2.c	行号: 1238
	步骤描述: 逻辑判别表达式中的运算项必须要使用括号.	
	代码片段: 1236: };	

	1237: 1238: for (int i = 0; i < sizeof(ops) / sizeof(ops[0]); i++) { 1239: SMetaTableOp *op = &ops[i]; 1240: const SMetaHandleParam param = {
	污染路径: 无

19.2.1.4 WK5661 禁止对有符号类型进行移位运算

缺陷数量： 122 个		
缺陷描述： 对有符号类型进行移位运算会导致不可预料的后果。		
修复意见： 无		
1	缺陷发生位置： /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号： 1180
	步骤描述： 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段： 1178: if (can_read(input_buffer, 4) && (strncmp((const char*)buffer_at_offset(input_buffer), "true", 4) == 0)) 1179: { 1180: item->type = tools_cJSON_True; 1181: item->valueint = 1; 1182: input_buffer->offset += 4;	
	污染路径： expanded from macro 'tools_cJSON_True'	
2	缺陷发生位置： /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/argp/argp-parse.c	行号： 424
	步骤描述： 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段： 422: however)... */ 423: cvt->long_end->val = 424: ((opt->key real->key) & USER_MASK) 425: + (((group - cvt->parser->groups) + 1) << USER BITS);	

	426:	
	污染路径: expanded from macro 'USER_MASK'	
3	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 97
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 95: *tableLogPtr = nbBits; 96: remaining = (1<<nbBits)+1; 97: threshold = 1<<nbBits; 98: nbBits++; 99:	
	污染路径: 无	
4	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1165
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1163: * Complex types 1164: ***** */ 1165: typedef U32 DTable_max_t[FSEv05_DTABLE_SIZE_U32(FSEv05_MAX_TABLEL OG)]; 1166: 1167:	
	污染路径: expanded from macro 'FSEv05_DTABLE_SIZE_U32'	
5	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 123
	步骤描述: 禁止对有符号数'int(bitCount)'进行二元位运算.	
	代码片段: 121: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend- 4)) { 122: assert((bitCount >> 3) <= 3); /* For first condition to work */	

	123: ip += bitCount>>3; 124: bitCount &= 7; 125: bitStream = MEM_readLE32(ip) >> bitCount;	
	污染路径: 无	
6	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1097
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1095: U32 position = 0; 1096: U32 highThreshold = tableSize-1; 1097: const S16 largeLimit= (S16)(1 << (tableLog-1)); 1098: U32 noLarge = 1; 1099: U32 s;	
	污染路径: 无	
7	缺陷发生位置: /source/dnode/mnode/impl/src/mndUser.c	行号: 4697
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 4695: // process remaining bits 4696: if (bits > 0) { 4697: int v = (buffer << (5 - bits)) & 0x1F; 4698: out[outLen++] = (v >= 26) ? (v - 26 + '2') : (v + 'A'); 4699: }	
	污染路径: 无	
8	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1096
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1094: U32 position = 0; 1095: U32 highThreshold = tableSize-1; 1096: const S16 largeLimit= (S16)(1 << (tableLog-1)); 1097: U32 noLarge = 1; 1098: U32 s;	

	污染路径: 无	
9	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 416
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 414: bits += 5; 415: if (bits >= 8) { 416: out[outLen++] = (buffer >> (bits - 8)) & 0xFF; 417: bits -= 8; 418: }	
	污染路径: 无	
10	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1253
	步骤描述: 禁止对有符号数'int(bitCount)'进行二元位运算.	
	代码片段: 1251: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 1252: while (charnum < n0) normalizedCounter[charnum++] = 0; 1253: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1254: ip += bitCount>>3; 1255: bitCount &= 7;	
	污染路径: 无	
11	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 822
	步骤描述: 禁止对有符号数'int(opt)'进行二元位运算.	
	代码片段: 820: err = 821: group_parse (&parser->groups[group_key - 1], &parser->state, 822: (opt << GROUP_BITS) >> GROUP_BITS, 823: parser->opt_data.optarg); 824:	

	污染路径: 无	
12	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1053
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1051: * Complex types 1052: ***** */ 1053: typedef U32 DTable_max_t[FSE_DTABLE_SIZE_U32(FSE_MAX_TABLELOG)]; 1054: 1055:	
	污染路径: expanded from macro 'FSE_DTABLE_SIZE_U32'	
13	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 765
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 763: *output_pointer = '\0'; 764: 765: item->type = tools_cJSON_String; 766: item->valuelstring = (char*)output; 767:	
	污染路径: expanded from macro 'tools_cJSON_String'	
14	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1180
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1178: *tableLogPtr = nbBits; 1179: remaining = (1<<nbBits)+1; 1180: threshold = 1<<nbBits; 1181: nbBits++; 1182:	

	污染路径: 无	
15	缺陷发生位置: /source/util/src/tlruccache.c	行号: 31
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 29: 30: enum { 31: TAOS_LRU_IN_CACHE = (1 << 0), // Whether this entry is referenced by the hash table. 32: 33: TAOS_LRU_IS_HIGH_PRI = (1 << 1), // Whether this entry is high priority entry.	
	污染路径: 无	
16	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 165
	步骤描述: 禁止对有符号数'int(n)'进行二元位运算.	
	代码片段: 163: return (n & 0xff00) ? 164: 8 + lg_table[(n >> 8) & 0xff] : 165: 0 + lg_table[(n >> 0) & 0xff]; 166: #endif 167: }	
	污染路径: 无	
17	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 259
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 257: TSDB_REPAIR_HEAD_OP_REMOVE_HEAD = 1 << 0, 258: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 259: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 260: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3, 261: TSDB_REPAIR_HEAD_OP_REMOVE_SMA = 1 << 4,	
	污染路径: 无	

18	缺陷发生位置: /source/util/src/tlruache.c	行号: 37
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 35: TAOS_LRU_IN_HIGH_PRI_POOL = (1 << 2), // Whether this entry is in high-pri pool. 36: 37: TAOS_LRU_HAS_HIT = (1 << 3), // Whether this entry has had any lookups (hits). 38: }; 39:	
	污染路径: 无	
19	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1199
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1197: { 1198: if (tableLog > FSEv05_TABLELOG_ABSOLUTE_MAX) tableLog = FSEv05_TABLELOG_ABSOLUTE_MAX; 1199: return (FSEv05_DTable*)malloc(FSEv05_DTABLE_SIZE_U32(tableLog) * sizeof (U32)); 1200: } 1201:	
	污染路径: expanded from macro 'FSEv05_DTABLE_SIZE_U32'	
20	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 257
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 255: 256: enum { 257: TSDB_REPAIR_HEAD_OP_REMOVE_HEAD = 1 << 0, 258: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 259: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2,	
	污染路径:	

	无	
21	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1229
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1227: *tableLogPtr = nbBits; 1228: remaining = (1<<nbBits)+1; 1229: threshold = 1<<nbBits; 1230: nbBits++; 1231:	
	污染路径: 无	
22	缺陷发生位置: /contrib/TSZ/sz/src/szd_double.c	行号: 188
	步骤描述: 禁止对有符号数'int(resiBits)'进行二元位运算.	
	代码片段: 186: curBytes[j] = tdps->exactMidBytes[curByteIndex++]; 187: if (resiBitsLength != 0) { 188: unsigned char resiByte = (unsigned char) (resiBits << (8 - resiBitsLength)); 189: curBytes[reqBytesLength] = resiByte; 190: }	
	污染路径: 无	
23	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 645
	步骤描述: 禁止对有符号数'int(resiBits)'进行二元位运算.	
	代码片段: 643: curBytes[j] = exactMidBytes[curByteIndex++]; 644: if (resiBitsLength != 0) { 645: unsigned char resiByte = (unsigned char) (resiBits << (8 - resiBitsLength)); 646: curBytes[reqBytesLength] = resiByte; 647: }	
	污染路径: 无	

24	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 1804
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 1802: 1803: while (bits >= 5) { 1804: int v = (buffer >> (bits - 5)) & 0x1F; 1805: out[outLen++] = (v >= 26) ? (v - 26 + '2') : (v + 'A'); 1806: bits -= 5;	
	污染路径: 无	
25	缺陷发生位置: /source/util/src/tlruache.c	行号: 35
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 33: TAOS_LRU_IS_HIGH_PRI = (1 << 1), // Whether this entry is high priority entry. 34: 35: TAOS_LRU_IN_HIGH_PRI_POOL = (1 << 2), // Whether this entry is in high-pri pool. 36: 37: TAOS_LRU_HAS_HIT = (1 << 3), // Whether this entry has had any lookups (hits).	
	污染路径: 无	
26	缺陷发生位置: /source/os/test/osAtomicTests.cpp	行号: 755
	步骤描述: 禁止对有符号数'long long(1)'进行二元位运算.	
	代码片段: 753: // Verify all bits are set 754: for (int i = 0; i < NUM_THREADS; i++) { 755: int64_t bit_pattern = 1LL << (i * 8); 756: EXPECT_NE(value & bit_pattern, 0); 757: }	
	污染路径: 无	

27	缺陷发生位置: /source/common/test/cosCpTest.cpp	行号: 39
	步骤描述: 禁止对有符号数'int(64)'进行二元位运算.	
	代码片段: 37: 38: int64_t contentLength = 1024; 39: const int64_t MULTIPART_CHUNK_SIZE = 64 << 20; // multipart is 768M 40: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 41: int totalSeq = (contentLength + chunk_size - 1) / chunk_size;	
	污染路径: 无	
28	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1627
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1625: // csv timestamp format 1626: tools_cJSON* csv_ti = tools_cJSON_GetObjectItem(stbInfo, "csv_ts_interval"); 1627: if (csv_ti && csv_ti->type == tools_cJSON_String && csv_ti->valuelstring != NULL) { 1628: superTable->csv_ts_interval = csv_ti- >valuelstring; 1629: } else {	
	污染路径: expanded from macro 'tools_cJSON_String'	
29	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 59
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 57: 58: enum { 59: TSDB_REPAIR_HEAD_OP_REMOVE_HEAD = 1 << 0, 60: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 61: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2,	
	污染路径:	

	无	
30	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1090
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1088: FSE_DECODE_TYPE* const tableDecode = (FSE_DECODE_TYPE*)ptr; 1089: FSE_DTableHeader DTableH; 1090: const U32 tableSize = 1 << tableLog; 1091: const U32 tableMask = tableSize-1; 1092: const U32 step = FSE_tableStep(tableSize);	
	污染路径: 无	
31	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 63
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 61: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 62: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3, 63: TSDB_REPAIR_HEAD_OP_REMOVE_SMA = 1 << 4, 64: TSDB_REPAIR_HEAD_OP_CREATE_SMA = 1 << 5, 65: };	
	污染路径: 无	
32	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 61
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 59: TSDB_REPAIR_HEAD_OP_REMOVE_HEAD = 1 << 0, 60: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 61: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 62: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3, 63: TSDB_REPAIR_HEAD_OP_REMOVE_SMA = 1 << 4,	
	污染路径: 无	
33	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 669

	步骤描述: 禁止对有符号数'int(2147483647)'进行二元位运算.	
	代码片段: 667: int intTmp; 668: 669: if (field->min < (-1 * (RAND_MAX >> 1))) { 670: field->min = -1 * (RAND_MAX >> 1); 671: }	
	污染路径: expanded from macro 'RAND_MAX'	
34	缺陷发生位置: /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号: 164
	步骤描述: 禁止对有符号数'int(n)'进行二元位运算.	
	代码片段: 162: #else 163: return (n & 0xff00) ? 164: 8 + lg_table[(n >> 8) & 0xff] : 165: 0 + lg_table[(n >> 0) & 0xff]; 166: #endif	
	污染路径: 无	
35	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1048
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1046: * Complex types 1047: ***** */ 1048: typedef U32 DTable_max_t[FSE_DTABLE_SIZE_U32(FSE_MAX_TABLELOG)]; 1049: 1050:	
	污染路径: expanded from macro 'FSE_DTABLE_SIZE_U32'	
36	缺陷发生位置: /contrib/libmqt/ttq/src/ttqUtf8.c	行号: 56
	步骤描述:	

	禁止对有符号数'int(codepoint)'进行二元位运算.	
	代码片段: 54: return TTQ_ERR_MALFORMED_UTF8; 55: } 56: codepoint = (codepoint << 6) (ustr[i] & 0x3F); 57: } 58:	
	污染路径: 无	
37	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 258
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 256: enum { 257: TSDB_REPAIR_HEAD_OP_REMOVE_HEAD = 1 << 0, 258: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 259: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 260: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3,	
	污染路径: 无	
38	缺陷发生位置: /tools/taos-tools/src/benchCommandOpt.c	行号: 67
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 65: 66: // DECIMAL 67: reti = regcomp(®ex, "^DECIMAL\\s*\\((\\s*(-?[0-9]+)\\)\\s*,\\s*(-?[0-9]+)\\)\\s*\\)\$", REG_ICASE REG_EXTENDED); 68: if (!reti) { 69: reti = regexec(®ex, token, 3, pmatch, 0);	
	污染路径: expanded from macro 'REG_ICASE'	
39	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 180
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 178: void* dPtr = dt + 1;	

	179: FSE_decode_t* const dinfo = (FSE_decode_t*)dPtr; 180: const unsigned tableSize = 1 << nbBits; 181: const unsigned tableMask = tableSize - 1; 182: const unsigned maxSV1 = tableMask+1;	
	污染路径: 无	
40	缺陷发生位置: /source/client/src/clientFirewall.c	行号: 573
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 571: } 572: 573: if (regcomp(&pDst->regex, pDst->pattern, REG_EXTENDED REG_NOSUB REG_ICASE) != 0) { 574: tscWarn("sql security: skip invalid regex rule:%s pattern:%s", pDst->name, pDst->pattern); 575: return false;	
	污染路径: expanded from macro 'REG_NOSUB'	
41	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 141
	步骤描述: 禁止对有符号数'int(3)'进行二元位运算.	
	代码片段: 139: break; 140: case 3: 141: tmp = (tmp (3 << (6-j*2))); 142: break; 143: default:	
	污染路径: 无	
42	缺陷发生位置: /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号: 62
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 60: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 61: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 62: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3,	

	63: TSDB_REPAIR_HEAD_OP_REMOVE_SMA = 1 << 4, 64: TSDB_REPAIR_HEAD_OP_CREATE_SMA = 1 << 5,	
	污染路径: 无	
43	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 39
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 37: type = intArray[n]; 38: if(type == 1) 39: tmp = (tmp (1 << (7-j))); 40: n++; 41: }	
	污染路径: 无	
44	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1189
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1187: bitCount = 4; 1188: *tableLogPtr = nbBits; 1189: remaining = (1<<nbBits)+1; 1190: threshold = 1<<nbBits; 1191: nbBits++;	
	污染路径: 无	
45	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 135
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 133: break; 134: case 1: 135: tmp = (tmp (1 << (6-j*2))); 136: break; 137: case 2:	
	污染路径:	

	无	
46	缺陷发生位置： /source/dnode/vnode/src/tsdb/tsdbRepair.c	行号： 60
	步骤描述： 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段： 58: enum { 59: TSDB_REPAIR_HEAD_OP_REMOVE_HEAD = 1 << 0, 60: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 61: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 62: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3,	
	污染路径： 无	
47	缺陷发生位置： /tools/taos-tools/src/benchCommandOpt.c	行号： 49
	步骤描述： 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段： 47: 48: // BINARY/NCHAR/VARCHAR/JSON/GEOMETRY/VARBINARY 49: reti = regcomp(®ex, "^(BINARY NCHAR VARCHAR JSON GEOMETRY VARBINARY)\\(((\\[1-9\\][0-9]*\\)\\)\\)\$", REG_ICASE REG_EXTENDED); 50: if (!reti) { 51: reti = regexec(®ex, token, 3, pmatch, 0);	
	污染路径： expanded from macro 'REG_ICASE'	
48	缺陷发生位置： /contrib/TSZ/zstd/dictBuilder/divsufsort.c	行号： 200
	步骤描述： 禁止对有符号数'int(x)'进行二元位运算.	
	代码片段： 198: e = (x & 0xffff0000) ? 199: ((x & 0xff000000) ? 200: 24 + lg_table[(x >> 24) & 0xff] : 201: 16 + lg_table[(x >> 16) & 0xff]) : 202: ((x & 0x0000ff00) ?	
	污染路径： 无	

49	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 147
	步骤描述: 禁止对有符号数'int(threshold)'进行二元位运算.	
	代码片段: 145: while (remaining < threshold) { 146: nbBits--; 147: threshold >>= 1; 148: } 149:	
	污染路径: 无	
50	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 79
	步骤描述: 禁止对有符号数'int(i)'进行二元位运算.	
	代码片段: 77: { 78: int j, i = huffmanTree->qend++; 79: while ((j = (i>>1))) //j=i/2 80: { 81: if (huffmanTree->qq[j]->freq <= n->freq) break;	
	污染路径: 无	
51	缺陷发生位置: /contrib/TSZ/sz/src/dataCompression.c	行号: 509
	步骤描述: 禁止对有符号数'int(resiBits)'进行二元位运算.	
	代码片段: 507: curBytes[j] = exactMidBytes[curByteIndex++]; 508: if (resiBitsLength != 0) { 509: unsigned char resiByte = (unsigned char) (resiBits << (8 - resiBitsLength)); 510: curBytes[reqBytesLength] = resiByte; 511: }	
	污染路径: 无	
52	缺陷发生位置: /source/common/test/cosCpTest.cpp	行号: 80
	步骤描述:	

	禁止对有符号数'const int64_t(MULTIPART_CHUNK_SIZE)'进行二元位运算.	
	代码片段: 78: int64_t contentLength = 1024; 79: const int64_t MULTIPART_CHUNK_SIZE = 64 << 20; // multipart is 768M 80: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 81: int totalSeq = (contentLength + chunk_size - 1) / chunk_size; 82: const int max_part_num = 10000;	
	污染路径: 无	
53	缺陷发生位置: /source/client/test/clientTests.cpp	行号: 413
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 411: return -1; // invalid character 412: } 413: buffer = (buffer << 5) c; 414: bits += 5; 415: if (bits >= 8) {	
54	污染路径: 无	
	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1190
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
55	代码片段: 1188: *tableLogPtr = nbBits; 1189: remaining = (1<<nbBits)+1; 1190: threshold = 1<<nbBits; 1191: nbBits++; 1192:	
	污染路径: 无	
	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1335
55	步骤描述: 禁止对有符号数'int(threshold)'进行二元位运算.	
	代码片段:	

	1333: while (remaining < threshold) { 1334: nbBits--; 1335: threshold >>= 1; 1336: } 1337:	
	污染路径: 无	
56	缺陷发生位置: /source/util/src/tlruCache.c	行号: 114
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 112: static int taosLRUEntryTableInit(SLRUEntryTable *table, int maxUpperHashBits) { 113: table->lengthBits = 16; 114: table->list = taosMemoryCalloc(1 << table->lengthBits, sizeof(SLRUEntry *)); 115: if (!table->list) { 116: TAOS_RETURN(terrno);	
	污染路径: 无	
57	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1619
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1617: // csv timestamp format 1618: tools_cJSON* csv_tf = tools_cJSON_GetObjectItem(stbInfo, "csv_ts_format"); 1619: if (csv_tf && csv_tf->type == tools_cJSON_String && csv_tf->valstring != NULL) { 1620: superTable->csv_ts_format = csv_tf->valstring; 1621: } else {	
	污染路径: expanded from macro 'tools_cJSON_String'	
58	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1179
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	

	代码片段: 1177: bitCount = 4; 1178: *tableLogPtr = nbBits; 1179: remaining = (1<<nbBits)+1; 1180: threshold = 1<<nbBits; 1181: nbBits++;	
	污染路径: 无	
59	缺陷发生位置: /source/util/test/hashTest.cpp	行号: 312
	步骤描述: 禁止对有符号数'int64_t(cap)'进行二元位运算.	
	代码片段: 310: } 311: int64_t cap = 4; 312: while (cap < 100000000) cap = (cap << 1u); 313: 314: _hash_fn_t hashFp = taosGetDefaultHashFunction(TSDB_DATA_TYPE_BINARY);	
	污染路径: 无	
60	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 1880
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 1878: return -1; // invalid character 1879: } 1880: buffer = (buffer << 5) c; 1881: bits += 5; 1882: if (bits >= 8) {	
	污染路径: 无	
61	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1081
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1079: void* const tdPtr = dt+1; /* because dt is unsigned, 32-bits aligned on 32-bits */	

	1080: FSE_DECODE_TYPE* const tableDecode = (FSE_DECODE_TYPE*) (tdPtr); 1081: const U32 tableSize = 1 << tableLog; 1082: const U32 tableMask = tableSize-1; 1083: const U32 step = FSE_tableStep(tableSize);	
	污染路径: 无	
62	缺陷发生位置: /source/util/src/tlruCache.c	行号: 33
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 31: TAOS_LRU_IN_CACHE = (1 << 0), // Whether this entry is referenced by the hash table. 32: 33: TAOS_LRU_IS_HIGH_PRI = (1 << 1), // Whether this entry is high priority entry. 34: 35: TAOS_LRU_IN_HIGH_PRI_POOL = (1 << 2), // Whether this entry is in high-pri pool.	
	污染路径: 无	
63	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v03.c	行号: 1091
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1089: FSE_DTableHeader DTableH; 1090: FSE_DECODE_TYPE* const tableDecode = (FSE_DECODE_TYPE*)ptr; 1091: const U32 tableSize = 1 << tableLog; 1092: const U32 tableMask = tableSize-1; 1093: const U32 step = FSE_tableStep(tableSize);	
	污染路径: 无	
64	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1189
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段:	

	1187: *tableLogPtr = nbBits; 1188: remaining = (1<<nbBits)+1; 1189: threshold = 1<<nbBits; 1190: nbBits++; 1191:	
	污染路径: 无	
65	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 300
	步骤描述: 禁止对有符号数'int(64)'进行二元位运算.	
	代码片段: 298: if (TEST_DATA_LIMIT == RLIM_INFINITY) 299: data_limit.rlim_cur = data_limit.rlim_max; 300: else if (data_limit.rlim_cur > (rlim_t) TEST_DATA_LIMIT) 301: data_limit.rlim_cur = MIN ((rlim_t) TEST_DATA_LIMIT, 302: data_limit.rlim_max);	
	污染路径: expanded from macro 'TEST_DATA_LIMIT'	
66	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 1173
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1171: if (can_read(input_buffer, 5) && (strncmp((const char*)buffer_at_offset(input_buffer), "false", 5) == 0)) 1172: { 1173: item->type = tools_cJSON_False; 1174: input_buffer->offset += 5; 1175: return true;	
	污染路径: expanded from macro 'tools_cJSON_False'	
67	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1636
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1634: superTable->csv_output_header = true;	

	1635: tools_cJSON* oph = tools_cJSON_GetObjectItem(stbInfo, "csv_output_header"); 1636: if (oph && oph->type == tools_cJSON_String && oph->valuelstring != NULL) { 1637: if (0 == strcasecmp(oph->valuelstring, "yes")) { 1638: superTable->csv_output_header = true;	
	污染路径: expanded from macro 'tools_cJSON_String'	
68	缺陷发生位置: /tools/taos-tools/deps/toolscJson/src/toolscJson.c	行号: 296
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 294: } 295: 296: item->type = tools_cJSON_Number; 297: 298: input_buffer->offset += (size_t)(after_end - number_c_string);	
	污染路径: expanded from macro 'tools_cJSON_Number'	
69	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 298
	步骤描述: 禁止对有符号数'int(64)'进行二元位运算.	
	代码片段: 296: if (getrlimit (RLIMIT_DATA, &data_limit) == 0) 297: { 298: if (TEST_DATA_LIMIT == RLIM_INFINITY) 299: data_limit.rlim_cur = data_limit.rlim_max; 300: else if (data_limit.rlim_cur > (rlim_t) TEST_DATA_LIMIT)	
	污染路径: expanded from macro 'TEST_DATA_LIMIT'	
70	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1309
	步骤描述: 禁止对有符号数'int(bitCount)'进行二元位运算.	
	代码片段:	

	1307: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 1308: while (charnum < n0) normalizedCounter[charnum++] = 0; 1309: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1310: ip += bitCount>>3; 1311: bitCount &= 7;	
	污染路径: 无	
71	缺陷发生位置: /source/common/test/cosCpTest.cpp	行号: 124
	步骤描述: 禁止对有符号数'int(64)'进行二元位运算.	
	代码片段: 122: 123: int64_t contentLength = 1024; 124: const int64_t MULTIPART_CHUNK_SIZE = 64 << 20; // multipart is 768M 125: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 126: int totalSeq = (contentLength + chunk_size - 1) / chunk_size;	
	污染路径: 无	
72	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1310
	步骤描述: 禁止对有符号数'int(bitCount)'进行二元位运算.	
	代码片段: 1308: while (charnum < n0) normalizedCounter[charnum++] = 0; 1309: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1310: ip += bitCount>>3; 1311: bitCount &= 7; 1312: bitStream = MEM_readLE32(ip) >> bitCount;	
	污染路径: 无	
73	缺陷发生位置:	行号:

	/source/common/test/cosCpTest.cpp	40
	步骤描述: 禁止对有符号数'const int64_t(MULTIPART_CHUNK_SIZE)'进行二元位运算.	
	代码片段: <pre> 38: int64_t contentLength = 1024; 39: const int64_t MULTIPART_CHUNK_SIZE = 64 << 20; // multipart is 768M 40: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 41: int totalSeq = (contentLength + chunk_size - 1) / chunk_size; 42: const int max_part_num = 10000; </pre>	
	污染路径: 无	
74	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 269
	步骤描述: 禁止对有符号数'int(threshold)'进行二元位运算.	
	代码片段: <pre> 267: previous0 = (count==1); 268: if (remaining<1) return ERROR(GENERIC); 269: while (remaining<threshold) { nbBits--; threshold>>=1; } 270: } 271: if (bitCount>16) { </pre>	
	污染路径: 无	
75	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 212
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: <pre> 210: BYTE* const oend = ostart + headerBufferSize; 211: int nbBits; 212: const int tableSize = 1 << tableLog; 213: int remaining; 214: int threshold; </pre>	
	污染路径: 无	
76	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1218

	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1216: U32 position = 0; 1217: U32 highThreshold = tableSize-1; 1218: const S16 largeLimit= (S16)(1 << (tableLog-1)); 1219: U32 noLarge = 1; 1220: U32 s;	
	污染路径: 无	
77	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v01.c	405
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 403: FSE_DTableHeader* const DTableH = (FSE_DTableHeader*)ptr; 404: FSE_DECODE_TYPE* const tableDecode = (FSE_DECODE_TYPE*)(ptr) + 1; /* because dt is unsigned, 32- bits aligned on 32-bits */ 405: const U32 tableSize = 1 << tableLog; 406: const U32 tableMask = tableSize-1; 407: const U32 step = FSE_tableStep(tableSize);	
78	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/sz/src/TypeManager.c	65
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
79	代码片段: 63: type = intArray[n]; 64: if(type == 1) 65: tmp = (tmp (1 << (7-j))); 66: n++; 67: }	
	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/legacy/zstd_v07.c	1254
	步骤描述:	

	禁止对有符号数'int(bitCount)'进行二元位运算.	
	代码片段: <pre> 1252: while (charnum < n0) normalizedCounter[charnum++] = 0; 1253: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend-4)) { 1254: ip += bitCount>>3; 1255: bitCount &= 7; 1256: bitStream = MEM_readLE32(ip) >> bitCount; </pre>	
	污染路径: 无	
80	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/compress/fse_compress.c	462
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: <pre> 460: U64 const step = ((U64)1<<62) / total; /* <== here, one division ! */ 461: U64 const vStep = 1ULL<<(scale-20); 462: int stillToDistribute = 1<<tableLog; 463: unsigned s; 464: unsigned largest=0; </pre>	
81	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/dictBuilder/divsufsort.c	203
	步骤描述: 禁止对有符号数'int(x)'进行二元位运算.	
	代码片段: <pre> 201: 16 + lg_table[(x >> 16) & 0xff] : 202: ((x & 0x0000ff00) ? 203: 8 + lg_table[(x >> 8) & 0xff] : 204: 0 + lg_table[(x >> 0) & 0xff]); 205: </pre>	
82	污染路径: 无	
	缺陷发生位置:	行号:
	/contrib/TSZ/zstd/common/fse_decompress.c	299
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	

	代码片段: 297: 298: 299: typedef FSE_DTable DTable_max_t[FSE_DTABLE_SIZE_U32(FSE_MAX_TABLELOG)]; 300: 301: size_t FSE_decompress(void* dst, size_t dstCapacity, const void* cSrc, size_t cSrcSize)	
	污染路径: expanded from macro 'FSE_DTABLE_SIZE_U32'	
83	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 44
	步骤描述: 禁止对有符号数'int(maxRangeRadius)'进行二元位运算.	
	代码片段: 42: { 43: int maxRangeRadius = 32768; 44: int stateNum = maxRangeRadius << 1; /*2 45: 46: return createHuffmanTree(stateNum);	
	污染路径: 无	
84	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1228
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1226: bitCount = 4; 1227: *tableLogPtr = nbBits; 1228: remaining = (1<<nbBits)+1; 1229: threshold = 1<<nbBits; 1230: nbBits++;	
	污染路径: 无	
85	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1052
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1050: * Complex types	

	1051: ***** */ 1052: typedef U32 DTable_max_t[FSE_DTABLE_SIZE_U32(FSE_MAX_TABLELOG)]; 1053: 1054:	
	污染路径: expanded from macro 'FSE_DTABLE_SIZE_U32'	
86	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 100
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 98: 99: U32 const maxSV1 = maxSymbolValue + 1; 100: U32 const tableSize = 1 << tableLog; 101: U32 highThreshold = tableSize-1; 102:	
	污染路径: 无	
87	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 653
	步骤描述: 禁止对有符号数'int(2147483647)'进行二元位运算.	
	代码片段: 651: } else { 652: if (field->min < (-1 * (RAND_MAX >> 1))) { 653: field->min = -1 * (RAND_MAX >> 1); 654: } 655: if (field->max > (RAND_MAX >> 1)) {	
	污染路径: expanded from macro 'RAND_MAX'	
88	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 655
	步骤描述: 禁止对有符号数'int(2147483647)'进行二元位运算.	
	代码片段: 653: field->min = -1 * (RAND_MAX >> 1); 654: }	

	<pre> 655: if (field->max > (RAND_MAX >> 1)) { 656: field->max = RAND_MAX >> 1; 657: } </pre>	
	污染路径: expanded from macro 'RAND_MAX'	
89	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 656
	步骤描述: 禁止对有符号数'int(2147483647)'进行二元位运算.	
	代码片段: <pre> 654: } 655: if (field->max > (RAND_MAX >> 1)) { 656: field->max = RAND_MAX >> 1; 657: } 658: intTmp = field->min; </pre>	
	污染路径: expanded from macro 'RAND_MAX'	
90	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ argp/argp-parse.c	行号: 797
	步骤描述: 禁止对有符号数'int(opt)'进行二元位运算.	
	代码片段: <pre> 795: /* The group key encoded in the high bits; 0 for short opts or 796: group_number + 1 for long opts. */ 797: int group_key = opt >> USER_BITS; 798: error_t err = EBADKEY; 799: </pre>	
	污染路径: 无	
91	缺陷发生位置: /contrib/TSZ/sz/src/TypeManager.c	行号: 138
	步骤描述: 禁止对有符号数'int(2)'进行二元位运算.	
	代码片段: <pre> 136: break; 137: case 2: 138: tmp = (tmp (2 << (6-j*2))); 139: break; </pre>	

	140: case 3:	
	污染路径: 无	
92	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 260
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 258: TSDB_REPAIR_HEAD_OP_CREATE_HEAD = 1 << 1, 259: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 260: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3, 261: TSDB_REPAIR_HEAD_OP_REMOVE_SMA = 1 << 4, 262: TSDB_REPAIR_HEAD_OP_CREATE_SMA = 1 << 5,	
	污染路径: 无	
93	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v02.c	行号: 1188
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1186: bitCount = 4; 1187: *tableLogPtr = nbBits; 1188: remaining = (1<<nbBits)+1; 1189: threshold = 1<<nbBits; 1190: nbBits++;	
	污染路径: 无	
94	缺陷发生位置: /source/dnode/vnode/test/tsdbRepairTest.cpp	行号: 261
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 259: TSDB_REPAIR_HEAD_OP_REMOVE_DATA = 1 << 2, 260: TSDB_REPAIR_HEAD_OP_CREATE_DATA = 1 << 3, 261: TSDB_REPAIR_HEAD_OP_REMOVE_SMA = 1 << 4, 262: TSDB_REPAIR_HEAD_OP_CREATE_SMA = 1 << 5, 263: };	
	污染路径: 无	

95	缺陷发生位置: /contrib/TSZ/zstd/compress/fse_compress.c	行号: 513
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 511: size_t FSE_buildCTable_raw (FSE_CTable* ct, unsigned nbBits) 512: { 513: const unsigned tableSize = 1 << nbBits; 514: const unsigned tableMask = tableSize - 1; 515: const unsigned maxSymbolValue = tableMask;	
	污染路径: 无	
96	缺陷发生位置: /contrib/TSZ/zstd/common/fse_decompress.c	行号: 111
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 109: DTableH.tableLog = (U16)tableLog; 110: DTableH.fastMode = 1; 111: { S16 const largeLimit= (S16)(1 << (tableLog-1)); 112: U32 s; 113: for (s=0; s<maxSV1; s++) {	
	污染路径: 无	
97	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 1812
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 1810: // process remaining bits 1811: if (bits > 0) { 1812: int v = (buffer << (5 - bits)) & 0x1F; 1813: out[outLen++] = (v >= 26) ? (v - 26 + '2') : (v + 'A'); 1814: }	
	污染路径: 无	
98	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v07.c	行号: 1279
	步骤描述:	

	禁止对有符号数'int(threshold)'进行二元位运算.	
	代码片段: 1277: while (remaining < threshold) { 1278: nbBits--; 1279: threshold >>= 1; 1280: } 1281:	
	污染路径: 无	
99	缺陷发生位置: /contrib/TSZ/sz/src/Huffman.c	行号: 142
	步骤描述: 禁止对有符号数'int(len)'进行二元位运算.	
	代码片段: 140: return; 141: } 142: int index = len >> 6; //=len/64 143: if(index == 0) 144: {	
	污染路径: 无	
100	缺陷发生位置: /source/libs/scalar/src/scfunc.c	行号: 1883
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 1881: bits += 5; 1882: if (bits >= 8) { 1883: out[outLen++] = (buffer >> (bits - 8)) & 0xFF; 1884: bits -= 8; 1885: }	
	污染路径: 无	
101	缺陷发生位置: /contrib/TSZ/sz/src/szd_float.c	行号: 197
	步骤描述: 禁止对有符号数'int(resiBits)'进行二元位运算.	
	代码片段: 195: curBytes[j] = tdps- >exactMidBytes[curByteIndex++];	

	196: if (resiBitsLength != 0) { 197: unsigned char resiByte = (unsigned char) (resiBits << (8 - resiBitsLength)); 198: curBytes[reqBytesLength] = resiByte; 199: } 	
	污染路径: 无	
102	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 411
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 409: U32 position = 0; 410: U32 highThreshold = tableSize-1; 411: const S16 largeLimit= (S16)(1 << (tableLog-1)); 412: U32 noLarge = 1; 413: U32 s;	
	污染路径: 无	
103	缺陷发生位置: /source/common/test/cosCpTest.cpp	行号: 79
	步骤描述: 禁止对有符号数'int(64)'进行二元位运算.	
	代码片段: 77: 78: int64_t contentLength = 1024; 79: const int64_t MULTIPART_CHUNK_SIZE = 64 << 20; // multipart is 768M 80: uint64_t chunk_size = MULTIPART_CHUNK_SIZE >> 3; 81: int totalSeq = (contentLength + chunk_size - 1) / chunk_size;	
	污染路径: 无	
104	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 504
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 502: *tableLogPtr = nbBits; 503: remaining = (1<<nbBits)+1;	

	504: threshold = 1<<nbBits; 505: nbBits++; 506:	
	污染路径: 无	
105	缺陷发生位置: /source/libs/scalar/src/scifunc.c	行号: 1800
	步骤描述: 禁止对有符号数'int(buffer)'进行二元位运算.	
	代码片段: 1798: // process all input bytes 1799: for (int i = 0; i < inLen; i++) { 1800: buffer = (buffer << 8) in[i]; 1801: bits += 8; 1802:	
	污染路径: 无	
106	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 337
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 335: } FSE_symbolCompressionTransform; /* total 8 bytes */ 336: 337: typedef U32 DTable_max_t[FSE_DTABLE_SIZE_U32(FSE_MAX_TABLELOG)]; 338: 339: /***** **	
	污染路径: expanded from macro 'FSE_DTABLE_SIZE_U32'	
107	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1212
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1210: void* const tdPtr = dt+1; /* because dt is unsigned, 32-bits aligned on 32-bits */ 1211: FSEv05_DECODE_TYPE* const tableDecode =	

	(FSEv05_DECODE_TYPE*) (tdPtr); 1212: const U32 tableSize = 1 << tableLog; 1213: const U32 tableMask = tableSize-1; 1214: const U32 step = FSEv05_tableStep(tableSize);	
	污染路径: 无	
108	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v04.c	行号: 1087
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1085: U32 position = 0; 1086: U32 highThreshold = tableSize-1; 1087: const S16 largeLimit= (S16)(1 << (tableLog-1)); 1088: U32 noLarge = 1; 1089: U32 s;	
	污染路径: 无	
109	缺陷发生位置: /tools/taos-tools/src/benchData.c	行号: 652
	步骤描述: 禁止对有符号数'int(2147483647)'进行二元位运算.	
	代码片段: 650: intTmp = 105 + taosRandom() % 10; 651: } else { 652: if (field->min < (-1 * (RAND_MAX >> 1))) { 653: field->min = -1 * (RAND_MAX >> 1); 654: } 	
	污染路径: expanded from macro 'RAND_MAX'	
110	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1611
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1609: // csv file prefix 1610: tools_cJSON* csv_fp = tools_cJSON_GetObjectItem(stbInfo, "csv_file_prefix"); 1611: if (csv_fp && csv_fp->type == tools_cJSON_String && csv_fp->valuelstring != NULL) {	

	1166: item->type = tools_cJSON_NULL; 1167: input_buffer->offset += 4; 1168: return true;	
	污染路径: expanded from macro 'tools_cJSON_NULL'	
114	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 96
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 94: bitCount = 4; 95: *tableLogPtr = nbBits; 96: remaining = (1<<nbBits)+1; 97: threshold = 1<<nbBits; 98: nbBits++;	
	污染路径: 无	
115	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1284
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1282: bitCount = 4; 1283: *tableLogPtr = nbBits; 1284: remaining = (1<<nbBits)+1; 1285: threshold = 1<<nbBits; 1286: nbBits++;	
	污染路径: 无	
116	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v01.c	行号: 503
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 501: bitCount = 4; 502: *tableLogPtr = nbBits; 503: remaining = (1<<nbBits)+1; 504: threshold = 1<<nbBits; 505: nbBits++;	

	污染路径: 无	
117	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v05.c	行号: 1300
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1298: bitCount = 4; 1299: *tableLogPtr = nbBits; 1300: remaining = (1<<nbBits)+1; 1301: threshold = 1<<nbBits; 1302: nbBits++;	
	污染路径: 无	
118	缺陷发生位置: /tools/taos-tools/deps/emfisis.physics.uiowa.edu/Software/C/libargp/ test/gnu-test-skeleton.c	行号: 301
	步骤描述: 禁止对有符号数'int(64)'进行二元位运算.	
	代码片段: 299: data_limit.rlim_cur = data_limit.rlim_max; 300: else if (data_limit.rlim_cur > (rlim_t) TEST_DATA_LIMIT) 301: data_limit.rlim_cur = MIN ((rlim_t) TEST_DATA_LIMIT, 302: data_limit.rlim_max); 303: if (setrlimit (RLIMIT_DATA, &data_limit) < 0)	
	污染路径: expanded from macro 'TEST_DATA_LIMIT'	
119	缺陷发生位置: /tools/taos-tools/src/benchJsonOpt.c	行号: 1646
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1644: // csv tbname alias 1645: tools_cJSON* tba = tools_cJSON_GetObjectItem(stbInfo, "csv_tbname_alias"); 1646: if (tba && tba->type == tools_cJSON_String && tba->valuestring != NULL) { 1647: superTable->csv_tbname_alias = tba-> valuestring; 1648: } else {	

	污染路径: expanded from macro 'tools_cJSON_String'	
120	缺陷发生位置: /contrib/TSZ/zstd/legacy/zstd_v06.c	行号: 1285
	步骤描述: 禁止对有符号数'int(1)'进行二元位运算.	
	代码片段: 1283: *tableLogPtr = nbBits; 1284: remaining = (1<<nbBits)+1; 1285: threshold = 1<<nbBits; 1286: nbBits++; 1287:	
	污染路径: 无	
121	缺陷发生位置: /contrib/TSZ/zstd/common/entropy_common.c	行号: 121
	步骤描述: 禁止对有符号数'int(bitCount)'进行二元位运算.	
	代码片段: 119: if (n0 > *maxSVPtr) return ERROR(maxSymbolValue_tooSmall); 120: while (charnum < n0) normalizedCounter[charnum++] = 0; 121: if ((ip <= iend-7) (ip + (bitCount>>3) <= iend- 4)) { 122: assert((bitCount >> 3) <= 3); /* For first condition to work */ 123: ip += bitCount>>3;	
	污染路径: 无	
122	缺陷发生位置: /source/dnode/vnode/src/vnd/vnodeStream.c	行号: 88
	步骤描述: 禁止对有符号数'int64_t(type)'进行二元位运算.	
	代码片段: 86: } 87: 88: static int64_t getSessionKey(int64_t session, int64_t type) { return (session (type << 32)); } 89:	

	90: int32_t sortCid(const void *lp, const void *rp) {
	污染路径: 无

20 CWE CSHARP

20.1 分项汇总表

序号	CWE CSHARP	缺陷/安全漏洞名称	数量	缺陷级别
1	CWE316	内存中敏感信息的明文存储	6	严重

20.2 缺陷/安全漏洞详解

20.2.1 严重类

20.2.1.1 CWE316 内存中敏感信息的明文存储

缺陷数量：6 个		
缺陷描述： 该程序将敏感信息以明文形式存储在内存中。 敏感内存可能会保存到磁盘、存储在核心转储中，或者在产品崩溃或程序员在释放内存之前没有正确清除内存时保持未清除状态。 可以说，这些问题通常只有具有管理员权限的人才能利用。然而，交换可能会导致内存被写入磁盘，并使其在之后容易受到物理攻击。核心转储文件可能具有不安全的权限，或者存储在不受信任的人可以访问的存档文件中。或者，由于另一个弱点，未清除的敏感内存可能会无意中暴露给攻击者。		
修复意见： 无		
1	缺陷发生位置： /examples/C#/taosdemo/TDEngineDriver.cs	行号： 107

	步骤描述: TDengineDriver.cs	
	代码片段: 无	
	污染路径: 无	
2	缺陷发生位置: /examples/C#/TDengineDriver.cs	行号: 107
	步骤描述: TDengineDriver.cs	
	代码片段: 无	
	污染路径: 无	
3	缺陷发生位置: /examples/C#/TDengineTest.cs	行号: 30
	步骤描述: TDengineTest.cs	
	代码片段: 28: private string configDir; 29: private string user; 30: private string password; 31: private short port = 0; 32:	
	污染路径: 无	
4	缺陷发生位置: /examples/C#/C#checker/TDengineDriver.cs	行号: 107
	步骤描述: TDengineDriver.cs	
	代码片段: 无	
	污染路径: 无	
5	缺陷发生位置: /examples/C#/C#checker/C#checker.cs	行号: 30
	步骤描述: C#checker.cs	
	代码片段: 28: private string configDir; 29: private string user; 30: private string password;	

	31: private short port = 0; 32:	
	污染路径: 无	
6	缺陷发生位置: /examples/C#/taosdemo/taosdemo.cs	行号: 32
	步骤描述: taosdemo.cs	
	代码片段: 30: private string configDir = "C:/TDengine/cfg"; 31: private string user = "root"; 32: private string password = "taosdata"; 33: private short port = 0; 34:	
	污染路径: 无	

21 CWE JAVA

21.1 分项汇总表

序号	CWE JAVA	缺陷/安全漏洞名称	数量	缺陷级别
1	CWE397	抛出通用异常	38	严重
2	CWE459	资源未关闭/释放	19	严重
3	CWE400	不受控制的资源消耗	2	严重
4	CWE375	将可变对象返回给不受信任的调用者	13	严重

21.2 缺陷/安全漏洞详解

21.2.1 严重类

21.2.1.1 CWE397 抛出通用异常

缺陷数量： 38 个		
缺陷描述： 抛出过于宽泛的异常会促进复杂的错误处理代码，这些代码更可能包含安全漏洞。 声明抛出 Exception 或 Throwable 的方法会使调用方难以执行正确的错误处理和错误恢复。例如，Java 的异常机制的设置使调用者能够轻松地预测可能出现的错误，并编写代码来处理每个特定的异常情况。声明一个方法抛出泛型形式的异常会使系统失败。		
修复意见： 将方法抛出异常的类型具体详细化。		
1	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号： 99
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 97: } 98: @Test 99: public void testJniSchemaless() throws Exception { 100: dropDB("power"); 101: SchemalessJniTest.main(args);	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopFull.java	行号： 26
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception {	

	27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120: } 121: @Test 122: public void testWsStmtExtend() throws Exception { 123: dropDB("power"); 124: WSParameterBindingExtendInterfaceDemo.main(args);	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 74
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 72: 73: @Test 74: public void testJNIconnect() throws Exception { 75: JNIconnectExample.main(args); 76: }	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 151
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 149: } 150: 151: static SourceSplitSql getTagSplit() throws Exception { 152: // ANCHOR: tag_split	

	153: SourceSplitSql splitSql = new SourceSplitSql();	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ GeometryDemo.java	行号: 165
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 163: e.printStackTrace(); 164: } catch (ParseException e) { 165: throw new RuntimeException(e); 166: } 167: }	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 289
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 287: 288: //ANCHOR: cdc_source 289: void testTDengineCdc() throws Exception { 290: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 291: env.setParallelism(3);	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 441
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 439: 440: //ANCHOR: CdcRowDataToSink 441: static void testCdcToSink() throws Exception { 442: System.out.println("testTDengineCdcToTdSink start! "); 443: StreamExecutionEnvironment env =	

	StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
9	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 372
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 370: 371: //ANCHOR: cdc_custom_type_test 372: static void testCustomTypeCdc() throws Exception { 373: System.out.println("testCustomTypeTDengineCdc start! "); 374: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 105
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 103: 104: @Test 105: public void testJniStmtBasic() throws Exception { 106: dropDB("power"); 107: ParameterBindingBasicDemo.main(args);	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 63
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: }	

	65: scanner.close();	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 80
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 78: 79: @Test 80: public void testWsConnect() throws Exception { 81: WSConnectExample.main(args); 82: }	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 527
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 525: 526: //ANCHOR: cdc_table 527: static void testCdcTableToSink() throws Exception { 528: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build(); 529: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 174
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 172: 173: //ANCHOR: source_test 174: static void testSource() throws Exception { 175: Properties connProps = new Properties(); 176:	

	329: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 330: env.setParallelism(3);	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 37
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 35: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 36: } catch (Exception e) { 37: throw new RuntimeException(e); 38: } 39: }	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/springbootdemo/src/main/java/com/taosdata/ example/springbootdemo/service/WeatherService.java	行号: 95
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 93: statement.executeUpdate("create table if not exists " + tableName + " using test.weather tags('" + weather.getLocation() + "', " + weather.getGroupId() + "')"); 94: } catch (SQLException e) { 95: throw new RuntimeException(e); 96: } 97: }	
	污染路径: 无	
20	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 134
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段:	

	132: 133: @Test 134: public void testConsumer() throws Exception { 135: dropDB("power"); 136: SubscribeDemo.main(args);	
	污染路径: 无	
21	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
22	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 60
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: } 62: scanner.close();	
	污染路径: 无	
23	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 49
	步骤描述:	

	Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 47: consumerLoop.pollData(); 48: } catch (SQLException e) { 49: throw new RuntimeException(e); 50: } 51: }); </pre>	
	污染路径: 无	
24	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/src/test/java/com/taosdata/example/mybatisplusdemo/provider/CachedBatchWriterTest.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 120:); 121: } catch (SQLException e) { 122: throw new RuntimeException("Batch write failed", e); 123: } 124: } </pre>	
	污染路径: 无	
25	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 111
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 109: 110: @Test 111: public void testJniStmtFull() throws Exception { 112: dropDB("power"); 113: ParameterBindingFullDemo.main(args); </pre>	
	污染路径: 无	
26	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 128
	步骤描述:	

	Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 126: 127: @Test 128: public void testWsStmtFull() throws Exception { 129: dropDB("power"); 130: WSParameterBindingFullDemo.main(args);	
	污染路径: 无	
27	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/data/SuperTableMetaGenerator.java	行号: 62
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 60: String tagPrefix) { 61: if (fieldSize < 2 tagSize < 1) { 62: throw new RuntimeException("create super table but fieldSize less than 2 or tagSize less than 1"); 63: } 64: SuperTableMeta tableMetadata = new SuperTableMeta();	
	污染路径: 无	
28	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
29	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/	行号: 85

	TestAll.java	
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 83: 84: @Test 85: public void testBase() throws Exception { 86: JdbcCreatDBDemo.main(args); 87: JdbcInsertDataDemo.main(args);	
	污染路径: 无	
30	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 46
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 44: consumerLoop.pollData(); 45: } catch (SQLException e) { 46: throw new RuntimeException(e); 47: } 48: });	
	污染路径: 无	
31	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 212
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 210: 211: //ANCHOR: source_custom_type_test 212: void testCustomTypeSource() throws Exception { 213: System.out.println("testTDengineSourceByTimeSplit start! "); 214: Properties connProps = new Properties();	
	污染路径: 无	
32	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98

	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
33	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 94
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 92: 93: @Test 94: public void testWsSchemaless() throws Exception { 95: dropDB("power"); 96: SchemalessWsTest.main(args);	
	污染路径: 无	
34	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/highvolume/Util.java	行号: 96
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e); 97: } 98: }	
	污染路径:	

	无	
35	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 254
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 252: 253: //ANCHOR: source_batch_test 254: void testBatchSource() throws Exception { 255: Properties connProps = new Properties(); 256: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
36	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 413
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 411: 412: //ANCHOR: RowDataToSink 413: static void testRowDataToSink() throws Exception { 414: Properties connProps = new Properties(); 415: connProps.setProperty(TDengineConfigParams.VALUE_DESERIALIZER, "RowData");	
	污染路径: 无	
37	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号: 34
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 32: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 33: } catch (Exception e) { 34: throw new RuntimeException(e); 35: } 36: }	

38	污染路径:	
	无	
	缺陷发生位置:	行号:
	/docs/examples/flink/source/Main.java	480
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
38	代码片段: 478: 479: //ANCHOR: source_table 480: static void testTableToSink() throws Exception { 481: System.out.println("testTableToSink start! "); 482: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build();	
	污染路径:	
	无	

21.2.1.2 CWE459 资源未关闭/释放

缺陷数量: 19 个		
缺陷描述: 软件在使用后没有正确“清理”并删除临时的或支持的资源。		
修复意见: 临时文件和其他支持资源应在不再需要后立即删除/发布。Java 应用程序中流资源应该在 finally 块中释放。		
1	缺陷发生位置:	行号:
	/docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	23
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 21: 22: // ANCHOR: create_topic 23: Connection connection = DriverManager.getConnection(url, properties); 24: Statement statement = connection.createStatement(); 25: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase,	

	groupid, location FROM meters");	
	污染路径: 无	
2	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ SubscribeDemo.java	行号: 35
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 33: Class.forName("com.taosdata.jdbc.TSDBDriver"); 34: String jdbcUrl = "jdbc:TAOS://127.0.0.1:6030/? user=root&password=taosdata"; 35: Connection connection = DriverManager.getConnection(jdbcUrl); 36: try (Statement statement = connection.createStatement()) { 37: statement.executeUpdate("drop topic if exists " + TOPIC);	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/src/main/java/com/ taosdata/example/mybatisplusdemo/provider/CachedBatchWriter.java	行号: 156
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 154: String sql = ms.getBoundSql(sample).getSql(); 155: 156: Connection conn = dataSource.getConnection(); 157: PreparedStatement ps = conn.prepareStatement(sql); 158:	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 112
	步骤描述: Use try-with-resources or close this "FileOutputStream" in a "finally" clause.	
	代码片段:	

	<pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); 113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close(); </pre>	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ GeometryDemo.java	行号: 89
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段: <pre> 87: 88: private void stmtInsert() throws SQLException { 89: TSDBPreparedStatement preparedStatement = (TSDBPreparedStatement) connection.prepareStatement("insert into " + dbName + "." + tbName + " values (?, ?, ?, ?)"); 90: 91: long current = System.currentTimeMillis(); </pre>	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号: 67
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: <pre> 65: // ANCHOR: query_data 66: // query data 67: ResultSet resultSet = stmt.executeQuery("SELECT * FROM meters"); 68: 69: Timestamp ts; </pre>	
	污染路径: 无	

7	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号： 21
	步骤描述： Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段： 19: properties.setProperty(TSDBDriver.PROPERTY_KEY_CHARSET, "UTF-8"); 20: 21: Connection connection = DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters");	
	污染路径： 无	
8	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号： 36
	步骤描述： Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段： 34: } 35: 36: Statement stmt = connection.createStatement(); 37: 38: // ANCHOR: create_db_and_table	
	污染路径： 无	
9	缺陷发生位置： /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/ example/ProxoolDemo.java	行号： 24
	步骤描述： Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段： 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23:	

	24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases");	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/ example/ProxoolDemo.java	行号: 26
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases"); 27: ResultSetMetaData metaData = rs.getMetaData(); 28: while (rs.next()) {	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ParameterBindingFullDemo.java	行号: 106
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段: 104: String sql = "INSERT INTO ? using stb tags(?,?,?,?,?,?) VALUES (?,?,?,?,?,?,?)"; 105: 106: TSDBPreparedStatement pstmt = conn.prepareStatement(sql).unwrap(TSDBPreparedStatement. class); 107: 108: for (int i = 1; i <= numOfSubTable; i++) {	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 112
	步骤描述:	

	Use try-with-resources or close this "FileOutputStream" in a "finally" clause.	
	代码片段: <pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); 113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close(); </pre>	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号: 82
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: <pre> 80: 81: // ANCHOR: with_reqid 82: AbstractStatement aStmt = (AbstractStatement) connection.createStatement(); 83: aStmt.execute("CREATE DATABASE IF NOT EXISTS power", 1L); 84: aStmt.executeUpdate("USE power", 2L); </pre>	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ ConsumerDemo.java	行号: 47
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: <pre> 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + "/* user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not </pre>	

	exists " + dbName + " WAL_RETENTION_PERIOD 3650"); 49: statement.executeUpdate("use " + dbName);	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ConsumerDemo.java	行号: 46
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 44: String tableName = "st"; 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650");	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	行号: 22
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 20: JAXPConfigurator.configure(xml, false); 21: Class.forName("org.logicalcobwebs.proxool.ProxoolDriver"); 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23: 24: Statement stmt = connection.createStatement();	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号: 22

	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 20: 21: Connection connection = DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters"); 24:	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 38
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 36: try (Connection conn = dataSource.getConnection()); Statement stmt = conn.createStatement()) { 37: stmt.execute("use " + dbName); 38: ResultSet rs = stmt.executeQuery("show stables"); 39: while (rs.next()) { 40: String name = rs.getString("stable_name");	
19	污染路径: 无	
	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 24
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 22: // ANCHOR: create_topic 23: Connection connection = DriverManager.getConnection(url, properties); 24: Statement statement = connection.createStatement(); 25: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase,	

	groupid, location FROM meters"); 26: // ANCHOR_END: create_topic
	污染路径: 无

21.2.1.3 CWE400 不受控制的资源消耗

缺陷数量： 2 个		
缺陷描述： 该软件不能正确控制有限资源的分配和维护，从而使参与者能够影响消耗的资源量，最终导致可用资源的耗尽。 扩展说明： <p _msthash="1499004" _msttexthash="4385650659">有限的资源包括内存、文件系统存储、数据库连接池项和 CPU。如果攻击者可以触发这些有限资源的分配，但资源的数量或大小不受控制，则攻击者可能导致拒绝服务，从而消耗所有可用资源。这将阻止有效用户访问该软件，并可能对周围环境产生影响。例如，针对应用程序的内存耗尽攻击可能会降低应用程序及其主机操作系统的速度。 <p _msthash="1499005" _msttexthash="129864475">至少有三种不同的情况通常会导致资源耗尽： <div><li _msthash="2281136" _msttexthash="50929060"> 1、分配的资源数量缺乏限制<li _msthash="2281137" _msttexthash="106624778"> 2、在进入关闭阶段之前丢失对资源的所有引用 <li _msthash="2281138" _msttexthash="44069870"> 3、处理后不关闭/返回资源<li _msthash="2281138" _msttexthash="44069870">
 </div><p _msthash="1499006" _msttexthash="197694913">资源耗尽问题通常是由于以下情况的不正确实现而导致的： <div><li _msthash="2281139" _msttexthash="46035431"> 1、错误条件和其他特殊情况。<li _msthash="2281140" _msttexthash="104840528"> 2、对程序的哪个部分负责释放资源感到困惑。</div>		
修复意见： 缓解资源耗尽攻击要求目标系统： 识别攻击并在给定的时间内拒绝该用户进一步访问，或者 统一限制所有请求，以便使消耗资源的速度比再次释放资源的速度更快。但是，这些解决方案中的第一个本身就是一个问题，因为它可能允许攻击者阻止特定有效用户使用系统。如果攻击者模拟有效用户，他们可能能够阻止用户访问相关服务器。第二种解决方案很难有效地制定 - 即使做得好，它也不能提供完整的解决方案。它只是使攻击需要攻击者提供更多资源。		
1	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/utils/DataGenerator.java	行号： 87
	步骤描述： Use a StringBuilder instead.	
	代码片段： <pre> 85: for (int i = 0; i < length; i++) { 86: char c = (char) (random.nextInt(top - bottom + 1) + bottom); 87: zh_cn += new String(new char[] {c}); 88: } </pre>	

	64: }	
	污染路径: 无	
2	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 111
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 109: try { 110: thread.join(); 111: } catch (InterruptedException e) { 112: e.printStackTrace(); 113: }	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/AbstractService.java	行号: 27
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 25: try { 26: count += future.get(); 27: } catch (InterruptedException e) { 28: e.printStackTrace(); 29: } catch (ExecutionException e) {	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/TaosDemoApplication.java	行号: 43
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 41: try { 42: task.join(); 43: } catch (InterruptedException e) { 44: e.printStackTrace(); 45: }	

	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/AbstractService.java	行号: 14
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 12: try { 13: count += future.get(); 14: } catch (InterruptedException e) { 15: e.printStackTrace(); 16: } catch (ExecutionException e) {	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 59
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 57: try { 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: }	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 40
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 38: try { 39: executor.awaitTermination(Long.MAX_VALUE, TimeUnit.NANOSECONDS); 40: } catch (InterruptedException e) { 41: e.printStackTrace(); 42: }	

	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 120
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 118: try { 119: affectedRows += task.get(); 120: } catch (InterruptedException e) { 121: e.printStackTrace(); 122: } catch (ExecutionException e) {	
	污染路径: 无	
9	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 83
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 81: try { 82: thread.join(); 83: } catch (InterruptedException e) { 84: e.printStackTrace(); 85: }	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/ example/ConnectionPoolDemo.java	行号: 105
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 103: if (sleep > 0) 104: TimeUnit.MILLISECONDS.sleep(sleep); 105: } catch (InterruptedException e) { 106: e.printStackTrace(); 107: }	

	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号: 94
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 92: } 93: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e);	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 74
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 72: } 73: 74: } catch (SQLException InterruptedException e) { 75: e.printStackTrace(); 76: }	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/StatTask.java	行号: 38
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 36: logger.info("numberOfTable={ } count={ } speed={ }", subTableNum, count, (count - lastCount) / 10); 37: lastCount = count; 38: } catch (InterruptedException e) { 39: logger.error("interrupted", e);	

	40: break;
	污染路径: 无

22 OWASP

22.1 分项汇总表

序号	OWASP	缺陷/安全漏洞名称	数量	缺陷级别
1	A10:2004	资源未关闭/释放	19	严重

22.2 缺陷/安全漏洞详解

22.2.1 严重类

22.2.1.1 A10:2004 资源未关闭/释放

缺陷数量：19 个		
缺陷描述： 软件在使用后没有正确“清理”并删除临时的或支持的资源。		
修复意见： 临时文件和其他支持资源应在不再需要后立即删除/发布。Java 应用程序中流资源应该在 finally 块中释放。		
1	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ParameterBindingFullDemo.java	行号： 106
	步骤描述： Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	

	代码片段: 104: String sql = "INSERT INTO ? using stb tags(?,?,?,?,?,?,?) VALUES (?,?,?,?,?,?,?)"; 105: 106: TSDBPreparedStatement pstmt = conn.prepareStatement(sql).unwrap(TSDBPreparedStatement. class); 107: 108: for (int i = 1; i <= numOfSubTable; i++) {	
	污染路径: 无	
2	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号: 36
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 34: } 35: 36: Statement stmt = connection.createStatement(); 37: 38: // ANCHOR: create_db_and_table	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 24
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 22: // ANCHOR: create_topic 23: Connection connection = DriverManager.getConnection(url, properties); 24: Statement statement = connection.createStatement(); 25: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters"); 26: // ANCHOR_END: create_topic	
	污染路径:	

	无	
4	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ConsumerDemo.java	行号: 47
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: <pre> 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + "/? user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650"); 49: statement.executeUpdate("use " + dbName); </pre>	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/SubscribeDemo.java	行号: 35
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: <pre> 33: Class.forName("com.taosdata.jdbc.TSDBDriver"); 34: String jdbcUrl = "jdbc:TAOS://127.0.0.1:6030/? user=root&password=taosdata"; 35: Connection connection = DriverManager.getConnection(jdbcUrl); 36: try (Statement statement = connection.createStatement()) { 37: statement.executeUpdate("drop topic if exists " + TOPIC); </pre>	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/MavenWrapperDownloader.java	行号: 112
	步骤描述: Use try-with-resources or close this "FileOutputStream" in a "finally" clause.	

	代码片段: <pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); 113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close(); </pre>	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ ConsumerDemo.java	行号: 46
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: <pre> 44: String tableName = "st"; 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + "/? user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650"); </pre>	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 38
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: <pre> 36: try (Connection conn = dataSource.getConnection(); Statement stmt = conn.createStatement()) { 37: stmt.execute("use " + dbName); 38: ResultSet rs = stmt.executeQuery("show stables"); 39: while (rs.next()) { 40: String name = rs.getString("stable_name"); </pre>	

	Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 154: String sql = ms.getBoundSql(sample).getSql(); 155: 156: Connection conn = dataSource.getConnection(); 157: PreparedStatement ps = conn.prepareStatement(sql); 158:	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 21
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 19: properties.setProperty(TSDBDriver.PROPERTY_KEY_CHARSET, "UTF-8"); 20: 21: Connection connection = DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters");	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/ example/ProxoolDemo.java	行号: 26
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases"); 27: ResultSetMetaData metaData = rs.getMetaData(); 28: while (rs.next()) {	

	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/JdbcBasicDemo.java	行号: 67
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 65: // ANCHOR: query_data 66: // query data 67: ResultSet resultSet = stmt.executeQuery("SELECT * FROM meters"); 68: 69: Timestamp ts;	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/JdbcBasicDemo.java	行号: 82
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 80: 81: // ANCHOR: with_reqid 82: AbstractStatement aStmt = (AbstractStatement) connection.createStatement(); 83: aStmt.execute("CREATE DATABASE IF NOT EXISTS power", 1L); 84: aStmt.executeUpdate("USE power", 2L);	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/GeometryDemo.java	行号: 89
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段: 87: 88: private void stmtInsert() throws SQLException { 89: TSDBPreparedStatement preparedStatement =	

	(TSDBPreparedStatement) connection.prepareStatement("insert into " + dbName + "." + tbName + " values (?, ?, ?, ?)"); 90: 91: long current = System.currentTimeMillis();	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	行号: 24
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23: 24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases");	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	行号: 22
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 20: JAXPConfigurator.configure(xml, false); 21: Class.forName("org.logicalcobwebs.proxool.ProxoolDriver"); 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23: 24: Statement stmt = connection.createStatement();	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 112

23.2 缺陷/安全漏洞详解

23.2.1 严重类

23.2.1.1 Guidelines07 抛出通用异常

缺陷数量：38 个		
缺陷描述： 方法如果抛出 RuntimeException，Exception 或 Throwable，处理这些异常需要捕获 RuntimeException，这是其他规则允许的“不要捕获 NullPointerException 或其任何祖先”。而且，抛一个 RuntimeException 会导致奇妙的错误；例如，调用者无法检查异常以确定它为什么被抛出，因此无法尝试恢复。使用诸如 Error、RuntimeException、Throwable 和 Exception 这样的通用异常可以防止调用方法来处理真实的、系统生成的异常，而不是应用程序生成的错误。		
修复意见： 不要抛出 RuntimeException、Exception 或 Throwable 的异常。		
1	缺陷发生位置： /docs/examples/flink/source/Main.java	行号： 212
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 210: 211: //ANCHOR: source_custom_type_test 212: void testCustomTypeSource() throws Exception { 213: System.out.println("testTDengineSourceByTimeSplit start! "); 214: Properties connProps = new Properties();	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号： 37
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 35: System.out.println("data: " + JsonUtil.getMapper().writeValueAsString(result)); 36: } catch (Exception e) {	

	37: throw new RuntimeException(e); 38: } 39: }	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 328
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 326: 327: //ANCHOR: cdc_batch_source 328: void testTDengineCdcBatch() throws Exception { 329: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 330: env.setParallelism(3);	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 289
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 287: 288: //ANCHOR: cdc_source 289: void testTDengineCdc() throws Exception { 290: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 291: env.setParallelism(3);	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 60
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e);	

	61: } 62: scanner.close();	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 85
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 83: 84: @Test 85: public void testBase() throws Exception { 86: JdbcCreatDBDemo.main(args); 87: JdbcInsertDataDemo.main(args);	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 480
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 478: 479: //ANCHOR: source_table 480: static void testTableToSink() throws Exception { 481: System.out.println("testTableToSink start! "); 482: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build();	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 46
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 44: consumerLoop.pollData()); 45: } catch (SQLException e) { 46: throw new RuntimeException(e); 47: }	

	48: });	
	污染路径: 无	
9	缺陷发生位置: /docs/examples/JDBC/springbootdemo/src/main/java/com/taosdata/ example/springbootdemo/service/WeatherService.java	行号: 95
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 93: statement.executeUpdate("create table if not exists " + tableName + " using test.weather tags('"' + weather.getLocation() + "',' " + weather.getGroupId() + "')"); 94: } catch (SQLException e) { 95: throw new RuntimeException(e); 96: } 97: }	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 63
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: } 65: scanner.close();	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/data/SuperTableMetaGenerator.java	行号: 62
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 60: String tagPrefix) { 61: if (fieldSize < 2 tagSize < 1) { 62: throw new RuntimeException("create super table	

	but fieldSize less than 2 or tagSize less than 1"); 63: } 64: SuperTableMeta tableMetadata = new SuperTableMeta();	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 441
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 439: 440: //ANCHOR: CdcRowDataToSink 441: static void testCdcToSink() throws Exception { 442: System.out.println("testTDengineCdcToTdSink start! "); 443: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号: 96
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e); 97: } 98: }	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	

	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 128
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 126: 127: @Test 128: public void testWsStmtFull() throws Exception { 129: dropDB("power"); 130: WSParameterBindingFullDemo.main(args);	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 372
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 370: 371: //ANCHOR: cdc_custom_type_test 372: static void testCustomTypeCdc() throws Exception { 373: System.out.println("testCustomTypeTDengineCdc start! "); 374: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/	行号: 99

	TestAll.java	
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 97: } 98: @Test 99: public void testJniSchemaless() throws Exception { 100: dropDB("power"); 101: SchemalessJniTest.main(args); </pre>	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 151
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 149: } 150: 151: static SourceSplitSql getTagSplit() throws Exception { 152: // ANCHOR: tag_split 153: SourceSplitSql splitSql = new SourceSplitSql(); </pre>	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 94
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 92: 93: @Test 94: public void testWsSchemaless() throws Exception { 95: dropDB("power"); 96: SchemalessWsTest.main(args); </pre>	
	污染路径: 无	
20	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 527
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	

	代码片段: 525: 526: //ANCHOR: cdc_table 527: static void testCdcTableToSink() throws Exception { 528: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build(); 529: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
21	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/data/SuperTableMetaGenerator.java	行号: 20
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 18: superTableSQL = superTableSQL.trim().toLowerCase(); 19: if (!superTableSQL.startsWith("create")) 20: throw new RuntimeException("invalid create super table SQL"); 21: 22: if (superTableSQL.contains("tags")) {	
	污染路径: 无	
22	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ GeometryDemo.java	行号: 165
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 163: e.printStackTrace(); 164: } catch (ParseException e) { 165: throw new RuntimeException(e); 166: } 167: }	
	污染路径: 无	
23	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/	行号: 122

	TestAll.java	
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120: } 121: @Test 122: public void testWsStmtExtend() throws Exception { 123: dropDB("power"); 124: WSParameterBindingExtendInterfaceDemo.main(args);	
	污染路径: 无	
24	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 80
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 78: 79: @Test 80: public void testWsConnect() throws Exception { 81: WSConnectExample.main(args); 82: }	
	污染路径: 无	
25	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 74
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 72: 73: @Test 74: public void testJNIConnect() throws Exception { 75: JNIConnectExample.main(args); 76: }	
	污染路径: 无	
26	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 254

	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 252: 253: //ANCHOR: source_batch_test 254: void testBatchSource() throws Exception { 255: Properties connProps = new Properties(); 256: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
27	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 117
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 115: 116: @Test 117: public void testWsStmtStd() throws Exception { 118: dropDB("power"); 119: WSPParameterBindingStdInterfaceDemo.main(args);	
	污染路径: 无	
28	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 111
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 109: 110: @Test 111: public void testJniStmtFull() throws Exception { 112: dropDB("power"); 113: ParameterBindingFullDemo.main(args);	
	污染路径: 无	
29	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/	行号: 26

	/docs/examples/flink/source/Main.java	174
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 172: 173: //ANCHOR: source_test 174: static void testSource() throws Exception { 175: Properties connProps = new Properties(); 176: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
33	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号: 34
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 32: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 33: } catch (Exception e) { 34: throw new RuntimeException(e); 35: } 36: }	
	污染路径: 无	
34	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径:	

	无	
35	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号: 49
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 47: consumerLoop.pollData(); 48: } catch (SQLException e) { 49: throw new RuntimeException(e); 50: } 51: });	
	污染路径: 无	
36	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 413
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 411: 412: //ANCHOR: RowDataToSink 413: static void testRowDataToSink() throws Exception { 414: Properties connProps = new Properties(); 415: connProps.setProperty(TDengineConfigParams.VALUE_DESERIALIZER, "RowData");	
	污染路径: 无	
37	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username =	

	System.getenv("MVNW_USERNAME");	
	污染路径: 无	
38	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 105
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 103: 104: @Test 105: public void testJniStmtBasic() throws Exception { 106: dropDB("power"); 107: ParameterBindingBasicDemo.main(args);	
	污染路径: 无	

23.2.1.2 Guidelines07 Guidelines13 资源未关闭/释放

缺陷数量： 19 个		
缺陷描述： 调用 Java 垃圾收集器以释放未引用但尚未释放的内存。但是，垃圾收集器无法释放非内存资源，例如打开的文件描述符和数据库连接。		
修复意见： 使用后应立即关闭输出流。		
1	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号: 67
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 65: // ANCHOR: query_data 66: // query data 67: ResultSet resultSet = stmt.executeQuery("SELECT * FROM meters"); 68: 69: Timestamp ts;	

	<pre> 33: Class.forName("com.taosdata.jdbc.TSDBDriver"); 34: String jdbcUrl = "jdbc:TAOS://127.0.0.1:6030/? user=root&password=taosdata"; 35: Connection connection = DriverManager.getConnection(jdbcUrl); 36: try (Statement statement = connection.createStatement()) { 37: statement.executeUpdate("drop topic if exists " + TOPIC); </pre> 污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 24
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 22: // ANCHOR: create_topic 23: Connection connection = DriverManager.getConnection(url, properties); 24: Statement statement = connection.createStatement(); 25: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters"); 26: // ANCHOR_END: create_topic	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ ConsumerDemo.java	行号: 47
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement();	

	无	
9	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/QueryService.java	行号: 38
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: <pre> 36: try (Connection conn = dataSource.getConnection()); Statement stmt = conn.createStatement()) { 37: stmt.execute("use " + dbName); 38: ResultSet rs = stmt.executeQuery("show stables"); 39: while (rs.next()) { 40: String name = rs.getString("stable_name"); </pre>	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/JdbcBasicDemo.java	行号: 82
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: <pre> 80: 81: // ANCHOR: with_reqid 82: AbstractStatement aStmt = (AbstractStatement) connection.createStatement(); 83: aStmt.execute("CREATE DATABASE IF NOT EXISTS power", 1L); 84: aStmt.executeUpdate("USE power", 2L); </pre>	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/MavenWrapperDownloader.java	行号: 112
	步骤描述: Use try-with-resources or close this "FileOutputStream" in a "finally" clause.	
	代码片段: <pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); </pre>	

	113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close();	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号: 23
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 21: 22: // ANCHOR: create_topic 23: Connection connection = DriverManager.getConnection(url, properties); 24: Statement statement = connection.createStatement(); 25: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters");	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	行号: 26
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases"); 27: ResultSetMetaData metaData = rs.getMetaData(); 28: while (rs.next()) {	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/src/main/java/com/taosdata/example/mybatisplusdemo/provider/CachedBatchWriter.java	行号: 156
	步骤描述:	

	113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close();
	污染路径: 无

23.2.1.3 Guidelines05 将可变对象返回给不受信任的调用者

缺陷数量： 13 个		
缺陷描述： 可变类允许类外部的代码更改其实例或类字段。提供用于创建可变类的副本的方法，以便可以将此类的一次性实例传递给不受信任的代码。当其他类中的方法必须创建特定类实例的副本时，此功能非常有用。		
修复意见： 可变类中缺少复制拷贝函数，可变类允许类的外部代码修改它们的实例或者类字段中数据成员，建议提供一个复制构造函数或一个返回实例副本的公用静态工厂方法或者通过覆盖 clone()方法来通告它们的复制功能，保证可变类方法直接从不可信的代码中调用。		
1	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号： 62
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 60: try { 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: } </pre>	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ConnectionPoolDemo.java	行号： 105
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 103: if (sleep > 0) 104: TimeUnit.MILLISECONDS.sleep(sleep); 105: } catch (InterruptedException e) { </pre>	

	106: e.printStackTrace(); 107: }	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/AbstractService.java	行号: 27
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 25: try { 26: count += future.get(); 27: } catch (InterruptedException e) { 28: e.printStackTrace(); 29: } catch (ExecutionException e) {	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/AbstractService.java	行号: 14
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 12: try { 13: count += future.get(); 14: } catch (InterruptedException e) { 15: e.printStackTrace(); 16: } catch (ExecutionException e) {	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/highvolume/StatTask.java	行号: 38
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 36: logger.info("numberOfTable={ } count={ } speed={ }", subTableNum, count, (count - lastCount) / 10); 37: lastCount = count; 38: } catch (InterruptedException e) {	

	39: logger.error("interrupted", e); 40: break;	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 59
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 57: try { 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: }	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 74
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 72: } 73: 74: } catch (SQLException InterruptedException e) { 75: e.printStackTrace(); 76: }	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 111
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 109: try { 110: thread.join(); 111: } catch (InterruptedException e) { 112: e.printStackTrace();	

	113: }	
	污染路径: 无	
9	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/TaosDemoApplication.java	行号: 43
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 41: try { 42: task.join(); 43: } catch (InterruptedException e) { 44: e.printStackTrace(); 45: }	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 40
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 38: try { 39: executor.awaitTermination(Long.MAX_VALUE, TimeUnit.NANOSECONDS); 40: } catch (InterruptedException e) { 41: e.printStackTrace(); 42: }	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号: 94
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 92: } 93: 94: } catch (InterruptedException ExecutionException e) {	

	95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e);	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 83
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 81: try { 82: thread.join(); 83: } catch (InterruptedException e) { 84: e.printStackTrace(); 85: }	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 120
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 118: try { 119: affectedRows += task.get(); 120: } catch (InterruptedException e) { 121: e.printStackTrace(); 122: } catch (ExecutionException e) {	
	污染路径: 无	

24 SJ/T 11683-2017

24.1 分项汇总表

序号	SJ/T 11683-2017	缺陷/安全漏洞名称	数量	缺陷级别
1	6.4.6	抛出通用异常	38	严重
2	6.6.16	循环内字符串拼接	2	严重

24.2 缺陷/安全漏洞详解

24.2.1 严重类

24.2.1.1 6.4.6 抛出通用异常

缺陷数量：38 个		
缺陷描述： 方法声明抛出的异常过于宽泛，如声明抛出 Exception 或 Throwable。		
修复意见： 一个方法应抛出适当的抽象水平的异常，以便于调用者根据不同的异常类型做不同的异常处理。		
1	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号： 99
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 97: } 98: @Test 99: public void testJniSchemaless() throws Exception { 100: dropDB("power"); 101: SchemalessJniTest.main(args);	

	污染路径: 无	
2	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 46
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 44: consumerLoop.pollData(); 45: } catch (SQLException e) { 46: throw new RuntimeException(e); 47: } 48: });	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 174
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 172: 173: //ANCHOR: source_test 174: static void testSource() throws Exception { 175: Properties connProps = new Properties();	
	176:	

	93: @Test 94: public void testWsSchemaless() throws Exception { 95: dropDB("power"); 96: SchemalessWsTest.main(args);	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 480
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 478: 479: //ANCHOR: source_table 480: static void testTableToSink() throws Exception { 481: System.out.println("testTableToSink start! "); 482: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build();	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/springbootdemo/src/main/java/com/taosdata/ example/springbootdemo/service/WeatherService.java	行号: 95
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 93: statement.executeUpdate("create table if not exists " + tableName + " using test.weather tags('" + weather.getLocation() + "', " + weather.getGroupId() + "')"); 94: } catch (SQLException e) { 95: throw new RuntimeException(e); 96: } 97: }	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段:	

	24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 111
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 109: 110: @Test 111: public void testJniStmtFull() throws Exception { 112: dropDB("power"); 113: ParameterBindingFullDemo.main(args);	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 151
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 149: } 150: 151: static SourceSplitSql getTagSplit() throws Exception { 152: // ANCHOR: tag_split 153: SourceSplitSql splitSql = new SourceSplitSql();	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: }	

	97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 328
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 326: 327: //ANCHOR: cdc_batch_source 328: void testTDengineCdcBatch() throws Exception { 329: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 330: env.setParallelism(3);	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 74
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 72: 73: @Test 74: public void testJNICConnect() throws Exception { 75: JNICConnectExample.main(args); 76: }	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 254
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段:	

	252: 253: //ANCHOR: source_batch_test 254: void testBatchSource() throws Exception { 255: Properties connProps = new Properties(); 256: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
20	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/data/SuperTableMetaGenerator.java	行号: 62
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 60: String tagPrefix) { 61: if (fieldSize < 2 tagSize < 1) { 62: throw new RuntimeException("create super table but fieldSize less than 2 or tagSize less than 1"); 63: } 64: SuperTableMeta tableMetadata = new SuperTableMeta();	
	污染路径: 无	
21	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 413
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 411: 412: //ANCHOR: RowDataToSink 413: static void testRowDataToSink() throws Exception { 414: Properties connProps = new Properties(); 415: connProps.setProperty(TDengineConfigParams.VALUE_DESERIALIZER, "RowData");	
	污染路径: 无	
22	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 212

	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 210: 211: //ANCHOR: source_custom_type_test 212: void testCustomTypeSource() throws Exception { 213: System.out.println("testTDengineSourceByTimeSplit start! "); 214: Properties connProps = new Properties();	
	污染路径: 无	
23	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ GeometryDemo.java	行号: 165
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 163: e.printStackTrace(); 164: } catch (ParseException e) { 165: throw new RuntimeException(e); 166: } 167: }	
	污染路径: 无	
24	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号: 96
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e); 97: } 98: }	
	污染路径: 无	
25	缺陷发生位置:	行号:

	/docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	80
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 78: 79: @Test 80: public void testWsConnect() throws Exception { 81: WSCConnectExample.main(args); 82: }	
	污染路径: 无	
26	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120: } 121: @Test 122: public void testWsStmtExtend() throws Exception { 123: dropDB("power"); 124: WSParameterBindingExtendInterfaceDemo.main(args);	
	污染路径: 无	
27	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	

28	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 441
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 439: 440: //ANCHOR: CdcRowDataToSink 441: static void testCdcToSink() throws Exception { 442: System.out.println("testTDengineCdcToTdSink start! "); 443: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
29	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 60
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: } 62: scanner.close();	
	污染路径: 无	
30	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 527
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 525: 526: //ANCHOR: cdc_table 527: static void testCdcTableToSink() throws Exception { 528: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build(); 529: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径:	

	无	
31	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号： 63
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: } 65: scanner.close();	
	污染路径： 无	
32	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号： 128
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 126: 127: @Test 128: public void testWsStmtFull() throws Exception { 129: dropDB("power"); 130: WSParameterBindingFullDemo.main(args);	
	污染路径： 无	
33	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号： 85
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 83: 84: @Test 85: public void testBase() throws Exception { 86: JdbcCreatDBDemo.main(args); 87: JdbcInsertDataDemo.main(args);	
	污染路径： 无	

34	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号： 34
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： <pre> 32: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 33: } catch (Exception e) { 34: throw new RuntimeException(e); 35: } 36: }</pre>	
	污染路径： 无	
35	缺陷发生位置： /docs/examples/flink/source/Main.java	行号： 372
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： <pre> 370: 371: //ANCHOR: cdc_custom_type_test 372: static void testCustomTypeCdc() throws Exception { 373: System.out.println("testCustomTypeTDengineCdc start! "); 374: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();</pre>	
	污染路径： 无	
36	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号： 134
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： <pre> 132: 133: @Test 134: public void testConsumer() throws Exception { 135: dropDB("power"); 136: SubscribeDemo.main(args);</pre>	
	污染路径：	

	无	
37	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号： 117
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 115: 116: @Test 117: public void testWsStmtStd() throws Exception { 118: dropDB("power"); 119: WSPparameterBindingStdInterfaceDemo.main(args);	
	污染路径： 无	
38	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号： 105
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 103: 104: @Test 105: public void testJniStmtBasic() throws Exception { 106: dropDB("power"); 107: ParameterBindingBasicDemo.main(args);	
	污染路径： 无	

24.2.1.2 6.6.16 循环内字符串拼接

缺陷数量： 2 个		
缺陷描述： 在循环体中使用 “+”或 “+=”运算符拼接字符串。		
修复意见： 使用 StringBuffer 类完成字符串拼接后再转换为 String 类型字符串。		
1	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/Utils/DataGenerator.java	行号： 75

25.2 缺陷/安全漏洞详解

25.2.1 严重类

25.2.1.1 7.4 抛出通用异常

缺陷数量：38 个		
缺陷描述： 审计指标：宜恰当进行异常处理。 本规则主要检查：代码中声明抛出 Exception 或 Throwable 的方法。		
修复意见： 将方法抛出异常的类型具体详细化。		
1	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号： 94
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 92: 93: @Test 94: public void testWsSchemaless() throws Exception { 95: dropDB("power"); 96: SchemalessWsTest.main(args);	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/flink/source/Main.java	行号： 372
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 370: 371: //ANCHOR: cdc_custom_type_test 372: static void testCustomTypeCdc() throws Exception { 373: System.out.println("testCustomTypeTDengineCdc start! "); 374: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	

	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 37
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 35: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 36: } catch (Exception e) { 37: throw new RuntimeException(e); 38: } 39: }	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/springbootdemo/src/main/java/com/taosdata/ example/springbootdemo/service/WeatherService.java	行号: 95
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 93: statement.executeUpdate("create table if not exists " + tableName + " using test.weather tags('" + weather.getLocation() + "', " + weather.getGroupId() + "');	

	<pre> 94: } catch (SQLException e) { 95: throw new RuntimeException(e); 96: } 97: }</pre>	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 254
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 252: 253: //ANCHOR: source_batch_test 254: void testBatchSource() throws Exception { 255: Properties connProps = new Properties(); 256: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true"); </pre>	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 441
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 439: 440: //ANCHOR: CdcRowDataToSink 441: static void testCdcToSink() throws Exception { 442: System.out.println("testTDengineCdcToTdSink start! "); 443: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); </pre>	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 151
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 149: }</pre>	

	<pre> superTableSQL.trim().toLowerCase(); 19: if (!superTableSQL.startsWith("create")) 20: throw new RuntimeException("invalid create super table SQL"); 21: 22: if (superTableSQL.contains("tags")) { </pre>	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ GeometryDemo.java	行号: 165
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 163: e.printStackTrace(); 164: } catch (ParseException e) { 165: throw new RuntimeException(e); 166: } 167: } </pre>	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 480
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 478: 479: //ANCHOR: source_table 480: static void testTableToSink() throws Exception { 481: System.out.println("testTableToSink start! "); 482: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build(); </pre>	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 46
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段:	

	<pre> 44: consumerLoop.pollData(); 45: } catch (SQLException e) { 46: throw new RuntimeException(e); 47: } 48: }); </pre>	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 74
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 72: 73: @Test 74: public void testJNICConnect() throws Exception { 75: JNICConnectExample.main(args); 76: } </pre>	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 128
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 126: 127: @Test 128: public void testWsStmtFull() throws Exception { 129: dropDB("power"); 130: WSParameterBindingFullDemo.main(args); </pre>	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 99
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 97: } </pre>	

	98: @Test 99: public void testJniSchemaless() throws Exception { 100: dropDB("power"); 101: SchemalessJniTest.main(args);	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 49
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 47: consumerLoop.pollData(); 48: } catch (SQLException e) { 49: throw new RuntimeException(e); 50: } 51: });	
	污染路径: 无	
20	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/data/SuperTableMetaGenerator.java	行号: 62
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	

	代码片段: <pre> 60: String tagPrefix) { 61: if (fieldSize < 2 tagSize < 1) { 62: throw new RuntimeException("create super table but fieldSize less than 2 or tagSize less than 1"); 63: } 64: SuperTableMeta tableMetadata = new SuperTableMeta(); </pre>	
	污染路径: 无	
21	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 80
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 78: 79: @Test 80: public void testWsConnect() throws Exception { 81: WsConnectExample.main(args); 82: } </pre>	
	污染路径: 无	
22	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号: 96
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e); 97: } 98: } </pre>	
	污染路径: 无	
23	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/	行号: 26

	WsConsumerLoopFull.java	
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
24	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 85
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 83: 84: @Test 85: public void testBase() throws Exception { 86: JdbcCreatDBDemo.main(args); 87: JdbcInsertDataDemo.main(args);	
	污染路径: 无	
25	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 328
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 326: 327: //ANCHOR: cdc_batch_source 328: void testTDengineCdcBatch() throws Exception { 329: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 330: env.setParallelism(3);	
	污染路径: 无	
26	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/	行号: 63

	ConsumerLoopImp.java	
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: } 65: scanner.close(); </pre>	
	污染路径: 无	
27	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties(); </pre>	
	污染路径: 无	
28	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 60
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: } 62: scanner.close(); </pre>	
	污染路径: 无	
29	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/	行号: 111

	TestAll.java	
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 109: 110: @Test 111: public void testJniStmtFull() throws Exception { 112: dropDB("power"); 113: ParameterBindingFullDemo.main(args);	
	污染路径: 无	
30	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120: } 121: @Test 122: public void testWsStmtExtend() throws Exception { 123: dropDB("power"); 124: WSPParameterBindingExtendInterfaceDemo.main(args);	
	污染路径: 无	
31	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 289
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 287: 288: //ANCHOR: cdc_source 289: void testTDengineCdc() throws Exception { 290: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 291: env.setParallelism(3);	
	污染路径: 无	
32	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 174

	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 172: 173: //ANCHOR: source_test 174: static void testSource() throws Exception { 175: Properties connProps = new Properties(); 176: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
33	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/src/test/java/com/taosdata/example/mybatisplusdemo/provider/CachedBatchWriterTest.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120:); 121: } catch (SQLException e) { 122: throw new RuntimeException("Batch write failed", e); 123: } 124: }	
	污染路径: 无	
34	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 413
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 411: 412: //ANCHOR: RowDataToSink 413: static void testRowDataToSink() throws Exception { 414: Properties connProps = new Properties(); 415: connProps.setProperty(TDengineConfigParams.VALUE_DESERIALIZER, "RowData");	
	污染路径: 无	

35	缺陷发生位置： /docs/examples/flink/source/Main.java	行号： 212
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 210: 211: //ANCHOR: source_custom_type_test 212: void testCustomTypeSource() throws Exception { 213: System.out.println("testTDengineSourceByTimeSplit start! "); 214: Properties connProps = new Properties();	
	污染路径： 无	
36	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号： 134
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 132: 133: @Test 134: public void testConsumer() throws Exception { 135: dropDB("power"); 136: SubscribeDemo.main(args);	
	污染路径： 无	
37	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号： 105
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 103: 104: @Test 105: public void testJniStmtBasic() throws Exception { 106: dropDB("power"); 107: ParameterBindingBasicDemo.main(args);	
	污染路径： 无	
38	缺陷发生位置：	行号：

	/docs/examples/flink/source/Main.java	527
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 525: 526: //ANCHOR: cdc_table 527: static void testCdcTableToSink() throws Exception { 528: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build(); 529: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	

25.2.1.2 8.1.5 (8.3.1、8.4.5) 资源未关闭/释放

缺陷数量: 19 个		
缺陷描述: 审计指标: 宜避免不安全的资源清理。 本规则主要检查: 软件在使用后没有正确“清理”并删除临时的或支持的资源。		
修复意见: 临时文件和其他支持资源应在不再需要后立即删除/发布。Java 应用程序中流资源应该在 finally 块中释放。		
1	缺陷发生位置:	行号:
	/docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/JdbcBasicDemo.java	36
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 34: } 35: 36: Statement stmt = connection.createStatement(); 37: 38: // ANCHOR: create_db_and_table	
	污染路径: 无	
2	缺陷发生位置:	行号:
	/docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/	47

	ConsumerDemo.java	
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + "/*/?user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650"); 49: statement.executeUpdate("use " + dbName);	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号: 21
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 19: properties.setProperty(TSDBDriver.PROPERTY_KEY_CHARSET, "UTF-8"); 20: 21: Connection connection = DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters");	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ParameterBindingFullDemo.java	行号: 106
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段:	

	<pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); 113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close(); </pre>	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	行号: 22
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: <pre> 20: JAXPConfigurator.configure(xml, false); 21: Class.forName("org.logicalcobwebs.proxool.ProxoolDriver"); 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23: 24: Statement stmt = connection.createStatement(); </pre>	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/GeometryDemo.java	行号: 89
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段: <pre> 87: 88: private void stmtInsert() throws SQLException { 89: TSDBPreparedStatement preparedStatement = (TSDBPreparedStatement) connection.prepareStatement("insert into " + dbName + "." + tbName + " values (?, ?, ?, ?)"); 90: 91: long current = System.currentTimeMillis(); </pre>	
	污染路径:	

	无	
12	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	行号: 26
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases"); 27: ResultSetMetaData metaData = rs.getMetaData(); 28: while (rs.next()) {	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/src/main/java/com/taosdata/example/mybatisplusdemo/provider/CachedBatchWriter.java	行号: 156
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 154: String sql = ms.getBoundSql(sample).getSql(); 155: 156: Connection conn = dataSource.getConnection(); 157: PreparedStatement ps = conn.prepareStatement(sql); 158:	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/JdbcBasicDemo.java	行号: 82
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 80: 81: // ANCHOR: with_reqid 82: AbstractStatement aStmt = (AbstractStatement) connection.createStatement(); 83: aStmt.execute("CREATE DATABASE IF NOT EXISTS power", 1L);	

	84: aStmt.executeUpdate("USE power", 2L);	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 22
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 20: 21: Connection connection = DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters"); 24:	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 23
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 21: 22: // ANCHOR: create_topic 23: Connection connection = DriverManager.getConnection(url, properties); 24: Statement statement = connection.createStatement(); 25: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters");	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号: 67

	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 65: // ANCHOR: query_data 66: // query data 67: ResultSet resultSet = stmt.executeQuery("SELECT * FROM meters"); 68: 69: Timestamp ts;	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ SubscribeDemo.java	行号: 35
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 33: Class.forName("com.taosdata.jdbc.TSDBDriver"); 34: String jdbcUrl = "jdbc:TAOS://127.0.0.1:6030/? user=root&password=taosdata"; 35: Connection connection = DriverManager.getConnection(jdbcUrl); 36: try (Statement statement = connection.createStatement()) { 37: statement.executeUpdate("drop topic if exists " + TOPIC);	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/ example/ProxoolDemo.java	行号: 24
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23: 24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases");	

	污染路径: 无

26 GB/T 38674-2020 JAVA

26.1 分项汇总表

序号	GB/T 38674-2020 JAVA	缺陷/安全漏洞名称	数量	缺陷级别
1	6.4	抛出通用异常	38	严重
2	7.1	需确保流得到释放	19	严重
3	6.1	将可变对象返回给不受信任的调用者	13	严重

26.2 缺陷/安全漏洞详解

26.2.1 严重类

26.2.1.1 6.4 抛出通用异常

缺陷数量： 38 个
缺陷描述： 抛出过于宽泛的异常会促进复杂的错误处理代码，这些代码更可能包含安全漏洞。 声明抛出 Exception 或 Throwable 的方法会使调用方难以执行正确的错误处理和错误恢复。例如，Java 的异常机制的设置使调用者能够轻松地预测可能出现的错误，并编写代码来处理每个特定的异常情况。声明一个方法抛出泛型形式的异常会使系统失败。
修复意见： 将方法抛出异常的类型具体细化。

1	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ GeometryDemo.java	行号: 165
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 163: e.printStackTrace(); 164: } catch (ParseException e) { 165: throw new RuntimeException(e); 166: } 167: }	
	污染路径: 无	
2	缺陷发生位置: /docs/examples/JDBC/springbootdemo/src/main/java/com/taosdata/ example/springbootdemo/service/WeatherService.java	行号: 95
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 93: statement.executeUpdate("create table if not exists " + tableName + " using test.weather tags('" + weather.getLocation() + "', " + weather.getGroupId() + ""); 94: } catch (SQLException e) { 95: throw new RuntimeException(e); 96: } 97: }	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 289
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 287: 288: //ANCHOR: cdc_source 289: void testTDengineCdc() throws Exception { 290: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 291: env.setParallelism(3);	
	污染路径:	

	无	
4	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 151
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 149: } 150: 151: static SourceSplitSql getTagSplit() throws Exception { 152: // ANCHOR: tag_split 153: SourceSplitSql splitSql = new SourceSplitSql();	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 74
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 72: 73: @Test 74: public void testJNIconnect() throws Exception { 75: JNIconnectExample.main(args); 76: }	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	

	污染路径: 无	
7	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 94
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 92: 93: @Test 94: public void testWsSchemaless() throws Exception { 95: dropDB("power"); 96: SchemalessWsTest.main(args);	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 46
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 44: consumerLoop.pollData(); 45: } catch (SQLException e) { 46: throw new RuntimeException(e); 47: } 48: });	
	污染路径: 无	
9	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 174
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 172: 173: //ANCHOR: source_test 174: static void testSource() throws Exception { 175: Properties connProps = new Properties(); 176: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	

	污染路径: 无	
10	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 254
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 252: 253: //ANCHOR: source_batch_test 254: void testBatchSource() throws Exception { 255: Properties connProps = new Properties(); 256: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 85
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 83: 84: @Test 85: public void testBase() throws Exception { 86: JdbcCreatDBDemo.main(args); 87: JdbcInsertDataDemo.main(args);	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号: 37
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 35: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 36: } catch (Exception e) { 37: throw new RuntimeException(e); 38: } 39: }	

	污染路径: 无	
13	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 413
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 411: 412: //ANCHOR: RowDataToSink 413: static void testRowDataToSink() throws Exception { 414: Properties connProps = new Properties(); 415: connProps.setProperty(TDengineConfigParams.VALUE_DESERIALIZER, "RowData");	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 212
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 210: 211: //ANCHOR: source_custom_type_test 212: void testCustomTypeSource() throws Exception { 213: System.out.println("testTDengineSourceByTimeSplit start! "); 214: Properties connProps = new Properties();	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 134
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 132: 133: @Test 134: public void testConsumer() throws Exception { 135: dropDB("power"); 136: SubscribeDemo.main(args);	

	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 60
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: } 62: scanner.close();	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 328
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 326: 327: //ANCHOR: cdc_batch_source 328: void testTDengineCdcBatch() throws Exception { 329: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 330: env.setParallelism(3);	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 527
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 525: 526: //ANCHOR: cdc_table 527: static void testCdcTableToSink() throws Exception { 528: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build(); 529: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	

	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 99
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 97: } 98: @Test 99: public void testJniSchemaless() throws Exception { 100: dropDB("power"); 101: SchemalessJniTest.main(args);	
	污染路径: 无	
20	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 372
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 370: 371: //ANCHOR: cdc_custom_type_test 372: static void testCustomTypeCdc() throws Exception { 373: System.out.println("testCustomTypeTDengineCdc start! "); 374: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
21	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120: } 121: @Test 122: public void testWsStmtExtend() throws Exception { 123: dropDB("power"); 124:	

	WSParameterBindingExtendInterfaceDemo.main(args);	
	污染路径: 无	
22	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
23	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 80
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 78: 79: @Test 80: public void testWsConnect() throws Exception { 81: WSConnectExample.main(args); 82: }	
	污染路径: 无	
24	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 441
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 439: 440: //ANCHOR: CdcRowDataToSink 441: static void testCdcToSink() throws Exception {	

	442: System.out.println("testTDengineCdcToTdSink start! "); 443: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
25	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 480
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 478: 479: //ANCHOR: source_table 480: static void testTableToSink() throws Exception { 481: System.out.println("testTableToSink start! "); 482: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build();	
	污染路径: 无	
26	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 63
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: } 65: scanner.close();	
	污染路径: 无	
27	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 49
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 47: consumerLoop.pollData(); 48: } catch (SQLException e) {	

	49: throw new RuntimeException(e); 50: } 51: });	
	污染路径: 无	
28	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
29	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/src/test/java/com/taosdata/ example/mybatisplusdemo/provider/CachedBatchWriterTest.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120:); 121: } catch (SQLException e) { 122: throw new RuntimeException("Batch write failed", e); 123: } 124: }	
	污染路径: 无	
30	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 111
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 109:	

	110: @Test 111: public void testJniStmtFull() throws Exception { 112: dropDB("power"); 113: ParameterBindingFullDemo.main(args);	
	污染路径: 无	
31	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 34
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 32: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 33: } catch (Exception e) { 34: throw new RuntimeException(e); 35: } 36: }	
	污染路径: 无	
32	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 117
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 115: 116: @Test 117: public void testWsStmtStd() throws Exception { 118: dropDB("power"); 119: WSParameterBindingStdInterfaceDemo.main(args);	
	污染路径: 无	
33	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1";	

	25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
34	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/data/SuperTableMetaGenerator.java	行号: 62
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 60: String tagPrefix) { 61: if (fieldSize < 2 tagSize < 1) { 62: throw new RuntimeException("create super table but fieldSize less than 2 or tagSize less than 1"); 63: } 64: SuperTableMeta tableMetadata = new SuperTableMeta();	
	污染路径: 无	
35	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号: 96
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e); 97: } 98: }	
	污染路径: 无	
36	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 105

26.2.1.2 7.1 需确保流得到释放

缺陷数量： 19 个		
缺陷描述： 程序创建或分配流资源后，不进行合理释放，将会降低系统性能。攻击者可能会通过耗尽资源池的方式发起拒绝服务攻击。		
修复意见： 暂无		
1	缺陷发生位置： /docs/examples/JDBC/mybatisplus-demo/src/main/java/com/taosdata/example/mybatisplusdemo/provider/CachedBatchWriter.java	行号： 156
	步骤描述： Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段： 154: String sql = ms.getBoundSql(sample).getSql(); 155: 156: Connection conn = dataSource.getConnection(); 157: PreparedStatement ps = conn.prepareStatement(sql); 158:	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ConsumerDemo.java	行号： 47
	步骤描述： Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段： 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + "/? user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650"); 49: statement.executeUpdate("use " + dbName);	
	污染路径：	

	无	
3	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ParameterBindingFullDemo.java	行号: 106
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段: 104: String sql = "INSERT INTO ? using stb tags(?,?,?,?,?,?,?) VALUES (?,?,?,?,?,?,?)"; 105: 106: TSDBPreparedStatement pstmt = conn.prepareStatement(sql).unwrap(TSDBPreparedStatement. class); 107: 108: for (int i = 1; i <= numOfSubTable; i++) {	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/JdbcBasicDemo.java	行号: 67
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 65: // ANCHOR: query_data 66: // query data 67: ResultSet resultSet = stmt.executeQuery("SELECT * FROM meters"); 68: 69: Timestamp ts;	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	行号: 24
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23: 24: Statement stmt = connection.createStatement();	

	25: 26: ResultSet rs = stmt.executeQuery("show databases");	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号: 21
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 19: properties.setProperty(TSDBDriver.PROPERTY_KEY_CHARSET, "UTF-8"); 20: 21: Connection connection = DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters");	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/JdbcBasicDemo.java	行号: 36
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 34: } 35: 36: Statement stmt = connection.createStatement(); 37: 38: // ANCHOR: create_db_and_table	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/GeometryDemo.java	行号: 89

	SubscribeDemo.java	
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: <pre> 33: Class.forName("com.taosdata.jdbc.TSDBDriver"); 34: String jdbcUrl = "jdbc:TAOS://127.0.0.1:6030/? user=root&password=taosdata"; 35: Connection connection = DriverManager.getConnection(jdbcUrl); 36: try (Statement statement = connection.createStatement()) { 37: statement.executeUpdate("drop topic if exists " + TOPIC); </pre>	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ ConsumerDemo.java	行号: 46
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: <pre> 44: String tableName = "st"; 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + "/? user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650"); </pre>	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号: 82
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: <pre> 80: 81: // ANCHOR: with_reqid </pre>	

	82: AbstractStatement aStmt = (AbstractStatement) connection.createStatement(); 83: aStmt.execute("CREATE DATABASE IF NOT EXISTS power", 1L); 84: aStmt.executeUpdate("USE power", 2L);	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 112
	步骤描述: Use try-with-resources or close this "FileOutputStream" in a "finally" clause.	
	代码片段: <pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); 113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close(); </pre>	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 112
	步骤描述: Use try-with-resources or close this "FileOutputStream" in a "finally" clause.	
	代码片段: <pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); 113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close(); </pre>	
	污染路径: 无	
18	缺陷发生位置:	行号:

	/docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ProxoolDemo.java	26
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: <pre> 24: Statement stmt = connection.createStatement(); 25: 26: ResultSet rs = stmt.executeQuery("show databases"); 27: ResultSetMetaData metaData = rs.getMetaData(); 28: while (rs.next()) { </pre>	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/QueryService.java	行号: 38
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: <pre> 36: try (Connection conn = dataSource.getConnection(); Statement stmt = conn.createStatement()) { 37: stmt.execute("use " + dbName); 38: ResultSet rs = stmt.executeQuery("show stables"); 39: while (rs.next()) { 40: String name = rs.getString("stable_name"); </pre>	
	污染路径: 无	

26.2.1.3 6.1 将可变对象返回给不受信任的调用者

缺陷数量: 13 个
缺陷描述: 将未克隆的可变数据作为返回值发送可能会导致调用函数更改或删除该数据。 扩展说明: 在函数返回对可变数据的引用的情况下,调用该函数的外部代码可能会对发送的数据进行更改。如果以前未克隆此数据,则该类将使用可能违反其内部状态假设的已修改数据。
修复意见: 1.将不应更改的返回数据声明为常量或不可变。2.在返回对可变数据的引用之前克隆所有可变数据。这是首选的缓解措施。这样,无论对数据进行哪些更改,都会保留一个有效副本以供类使用。

1	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号： 74
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： 72: } 73: 74: } catch (SQLException InterruptedException e) { 75: e.printStackTrace(); 76: }	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/TaosDemoApplication.java	行号： 43
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： 41: try { 42: task.join(); 43: } catch (InterruptedException e) { 44: e.printStackTrace(); 45: }	
	污染路径： 无	
3	缺陷发生位置： /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号： 94
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： 92: } 93: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e);	
	污染路径：	

	无	
4	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/AbstractService.java	行号： 14
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 12: try { 13: count += future.get(); 14: } catch (InterruptedException e) { 15: e.printStackTrace(); 16: } catch (ExecutionException e) { </pre>	
	污染路径： 无	
5	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/AbstractService.java	行号： 27
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 25: try { 26: count += future.get(); 27: } catch (InterruptedException e) { 28: e.printStackTrace(); 29: } catch (ExecutionException e) { </pre>	
	污染路径： 无	
6	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号： 59
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 57: try { 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: } </pre>	
	污染路径： 无	

7	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/SubTableService.java	行号： 40
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 38: try { 39: executor.awaitTermination(Long.MAX_VALUE, TimeUnit.NANOSECONDS); 40: } catch (InterruptedException e) { 41: e.printStackTrace(); 42: } </pre>	
	污染路径： 无	
8	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号： 62
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 60: try { 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: } </pre>	
	污染路径： 无	
9	缺陷发生位置： /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/example/ConnectionPoolDemo.java	行号： 105
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 103: if (sleep > 0) 104: TimeUnit.MILLISECONDS.sleep(sleep); 105: } catch (InterruptedException e) { 106: e.printStackTrace(); 107: } </pre>	
	污染路径： 无	

10	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/SubTableService.java	行号： 111
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 109: try { 110: thread.join(); 111: } catch (InterruptedException e) { 112: e.printStackTrace(); 113: } </pre>	
	污染路径： 无	
11	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/SubTableService.java	行号： 120
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 118: try { 119: affectedRows += task.get(); 120: } catch (InterruptedException e) { 121: e.printStackTrace(); 122: } catch (ExecutionException e) { </pre>	
	污染路径： 无	
12	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/QueryService.java	行号： 83
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： <pre> 81: try { 82: thread.join(); 83: } catch (InterruptedException e) { 84: e.printStackTrace(); 85: } </pre>	
	污染路径： 无	
13	缺陷发生位置：	行号：

	/docs/examples/JDBC/highvolume/src/main/java/com/taos/example/highvolume/StatTask.java	38
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: <pre> 36: logger.info("numberOfTable={ } count={ } speed={ }", subTableNum, count, (count - lastCount) / 10); 37: lastCount = count; 38: } catch (InterruptedException e) { 39: logger.error("interrupted", e); 40: break; </pre>	
	污染路径: 无	

27 Java 缺陷类型-异常情况检查或处理不当

27.1 分项汇总表

序号	Java 缺陷类型-异常情况检查或处理不当	缺陷/安全漏洞名称	数量	缺陷级别
1	WK1126	抛出通用异常	38	严重

27.2 缺陷/安全漏洞详解

27.2.1 严重类

27.2.1.1 WK1126 抛出通用异常

缺陷数量：38 个

缺陷描述： 抛出过于宽泛的异常会促进复杂的错误处理代码，这些代码更可能包含安全漏洞。 声明抛出 Exception 或 Throwable 的方法会使调用方难以执行正确的错误处理和错误恢复。例如，Java 的异常机制的设置使调用者能够轻松地预测可能出现的错误，并编写代码来处理每个特定的异常情况。声明一个方法抛出泛型形式的异常会使系统失败。		
修复意见： 将方法抛出异常的类型具体细化。		
1	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号： 105
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 103: 104: @Test 105: public void testJniStmtBasic() throws Exception { 106: dropDB("power"); 107: ParameterBindingBasicDemo.main(args);	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号： 134
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 132: 133: @Test 134: public void testConsumer() throws Exception { 135: dropDB("power"); 136: SubscribeDemo.main(args);	
	污染路径： 无	
3	缺陷发生位置： /docs/examples/JDBC/mybatisplus-demo/src/test/java/com/taosdata/example/mybatisplusdemo/provider/CachedBatchWriterTest.java	行号： 122
	步骤描述： Define and throw a dedicated exception instead of using a generic one.	
	代码片段： 120:); 121: } catch (SQLException e) {	

	<pre> 122: throw new RuntimeException("Batch write failed", e); 123: } 124: } </pre>	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 94
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 92: 93: @Test 94: public void testWsSchemaless() throws Exception { 95: dropDB("power"); 96: SchemalessWsTest.main(args); </pre>	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号: 60
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: } 62: scanner.close(); </pre>	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 174
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 172: 173: //ANCHOR: source_test 174: static void testSource() throws Exception { </pre>	

	175: Properties connProps = new Properties(); 176: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/JDBC/springbootdemo/src/main/java/com/taosdata/example/springbootdemo/service/WeatherService.java	行号: 95
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 93: statement.executeUpdate("create table if not exists " + tableName + " using test.weather tags(" + weather.getLocation() + ",", " + weather.getGroupId() + ")"); 94: } catch (SQLException e) { 95: throw new RuntimeException(e); 96: } 97: }	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 128
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 126: 127: @Test 128: public void testWsStmtFull() throws Exception { 129: dropDB("power"); 130: WSParameterBindingFullDemo.main(args);	
	污染路径: 无	
9	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/TestAll.java	行号: 111
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段:	

	<pre> 109: 110: @Test 111: public void testJniStmtFull() throws Exception { 112: dropDB("power"); 113: ParameterBindingFullDemo.main(args); </pre>	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME"); </pre>	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 441
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 439: 440: //ANCHOR: CdcRowDataToSink 441: static void testCdcToSink() throws Exception { 442: System.out.println("testTDengineCdcToTdSink start! "); 443: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); </pre>	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 289
	步骤描述:	

	Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 287: 288: //ANCHOR: cdc_source 289: void testTDengineCdc() throws Exception { 290: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 291: env.setParallelism(3);	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 63
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: } 65: scanner.close();	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 34
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 32: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 33: } catch (Exception e) { 34: throw new RuntimeException(e); 35: } 36: }	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ GeometryDemo.java	行号: 165

	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 163: e.printStackTrace(); 164: } catch (ParseException e) { 165: throw new RuntimeException(e); 166: } 167: }	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 99
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 97: } 98: @Test 99: public void testJniSchemaless() throws Exception { 100: dropDB("power"); 101: SchemalessJniTest.main(args);	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 413
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 411: 412: //ANCHOR: RowDataToSink 413: static void testRowDataToSink() throws Exception { 414: Properties connProps = new Properties(); 415: connProps.setProperty(TDengineConfigParams.VALUE_DESERIALIZER, "RowData");	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 37

	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 35: System.out.println("data: " + JsonUtil.getObjectMapper().writeValueAsString(result)); 36: } catch (Exception e) { 37: throw new RuntimeException(e); 38: } 39: } </pre>	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 328
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 326: 327: //ANCHOR: cdc_batch_source 328: void testTDengineCdcBatch() throws Exception { 329: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment(); 330: env.setParallelism(3); </pre>	
	污染路径: 无	
20	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/data/SuperTableMetaGenerator.java	行号: 20
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: <pre> 18: superTableSQL = superTableSQL.trim().toLowerCase(); 19: if (!superTableSQL.startsWith("create")) 20: throw new RuntimeException("invalid create super table SQL"); 21: 22: if (superTableSQL.contains("tags")) { </pre>	
	污染路径: 无	
21	缺陷发生位置:	行号:

	/docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	74
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 72: 73: @Test 74: public void testJNIConnect() throws Exception { 75: JNIConnectExample.main(args); 76: }	
	污染路径: 无	
22	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/data/SuperTableMetaGenerator.java	行号: 62
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 60: String tagPrefix) { 61: if (fieldSize < 2 tagSize < 1) { 62: throw new RuntimeException("create super table but fieldSize less than 2 or tagSize less than 1"); 63: } 64: SuperTableMeta tableMetadata = new SuperTableMeta();	
	污染路径: 无	
23	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/Util.java	行号: 96
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage()); 96: throw new RuntimeException(e); 97: } 98: }	

	污染路径: 无	
24	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 372
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 370: 371: //ANCHOR: cdc_custom_type_test 372: static void testCustomTypeCdc() throws Exception { 373: System.out.println("testCustomTypeTDengineCdc start! "); 374: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
25	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
26	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 117
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 115: 116: @Test 117: public void testWsStmtStd() throws Exception { 118: dropDB("power"); 119: WSParameterBindingStdInterfaceDemo.main(args);	

	污染路径: 无	
27	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 480
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 478: 479: //ANCHOR: source_table 480: static void testTableToSink() throws Exception { 481: System.out.println("testTableToSink start! "); 482: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build();	
	污染路径: 无	
28	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopFull.java	行号: 26
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 24: static private String clientId = "clinet1"; 25: 26: public static TaosConsumer<ResultBean> getConsumer() throws Exception { 27: // ANCHOR: create_consumer 28: Properties config = new Properties();	
	污染路径: 无	
29	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 46
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 44: consumerLoop.pollData(); 45: } catch (SQLException e) { 46: throw new RuntimeException(e); 47: } 48: });	

	污染路径: 无	
30	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 254
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 252: 253: //ANCHOR: source_batch_test 254: void testBatchSource() throws Exception { 255: Properties connProps = new Properties(); 256: connProps.setProperty(TDengineConfigParams.PROPERTY_KEY_ENABLE_AUTO_RECONNECT, "true");	
	污染路径: 无	
31	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/ MavenWrapperDownloader.java	行号: 98
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 96: } 97: 98: private static void downloadFileFromURL(String urlString, File destination) throws Exception { 99: if (System.getenv("MVNW_USERNAME") != null && System.getenv("MVNW_PASSWORD") != null) { 100: String username = System.getenv("MVNW_USERNAME");	
	污染路径: 无	
32	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 122
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 120: } 121: @Test	

	122: public void testWsStmtExtend() throws Exception { 123: dropDB("power"); 124: WSParameterBindingExtendInterfaceDemo.main(args);	
	污染路径: 无	
33	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 527
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 525: 526: //ANCHOR: cdc_table 527: static void testCdcTableToSink() throws Exception { 528: EnvironmentSettings fsSettings = EnvironmentSettings.newInstance().inStreamingMode().build(); 529: StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();	
	污染路径: 无	
34	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 85
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 83: 84: @Test 85: public void testBase() throws Exception { 86: JdbcCreatDBDemo.main(args); 87: JdbcInsertDataDemo.main(args);	
	污染路径: 无	
35	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ConsumerLoopImp.java	行号: 49
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 47: consumerLoop.pollData();	

	48: } catch (SQLException e) { 49: throw new RuntimeException(e); 50: } 51: });	
	污染路径: 无	
36	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 151
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 149: } 150: 151: static SourceSplitSql getTagSplit() throws Exception { 152: // ANCHOR: tag_split 153: SourceSplitSql splitSql = new SourceSplitSql();	
	污染路径: 无	
37	缺陷发生位置: /docs/examples/flink/source/Main.java	行号: 212
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 210: 211: //ANCHOR: source_custom_type_test 212: void testCustomTypeSource() throws Exception { 213: System.out.println("testTDengineSourceByTimeSplit start! "); 214: Properties connProps = new Properties();	
	污染路径: 无	
38	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/test/java/com/taos/test/ TestAll.java	行号: 80
	步骤描述: Define and throw a dedicated exception instead of using a generic one.	
	代码片段: 78: 79: @Test 80: public void testWsConnect() throws Exception {	

	81: WSConnectExample.main(args); 82: }
	污染路径: 无

28 Java 缺陷类型-资源泄露

28.1 分项汇总表

序号	Java 缺陷类型-资源泄露	缺陷/安全漏洞名称	数量	缺陷级别
1	WK1129	资源未关闭/释放	19	严重

28.2 缺陷/安全漏洞详解

28.2.1 严重类

28.2.1.1 WK1129 资源未关闭/释放

缺陷数量： 19 个		
缺陷描述： 软件在使用后没有正确“清理”并删除临时的或支持的资源。		
修复意见： 临时文件和其他支持资源应在不再需要后立即删除/发布。Java 应用程序中流资源应该在 finally 块中释放。		
1	缺陷发生位置： /docs/examples/JDBC/mybatisplus-demo/.mvn/wrapper/ MavenWrapperDownloader.java	行号： 112
步骤描述：		

	Use try-with-resources or close this "FileOutputStream" in a "finally" clause.	
	代码片段: <pre> 110: ReadableByteChannel rbc; 111: rbc = Channels.newChannel(website.openStream()); 112: FileOutputStream fos = new FileOutputStream(destination); 113: fos.getChannel().transferFrom(rbc, 0, Long.MAX_VALUE); 114: fos.close(); </pre>	
	污染路径: 无	
2	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/ example/ProxoolDemo.java	行号: 22
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: <pre> 20: JAXPConfigurator.configure(xml, false); 21: Class.forName("org.logicalcobwebs.proxool.ProxoolDriver"); 22: Connection connection = DriverManager.getConnection("proxool.ds"); 23: 24: Statement stmt = connection.createStatement(); </pre>	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ JdbcBasicDemo.java	行号: 67
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: <pre> 65: // ANCHOR: query_data 66: // query data 67: ResultSet resultSet = stmt.executeQuery("SELECT * FROM meters"); 68: 69: Timestamp ts; </pre>	
	污染路径:	

	<pre> 33: Class.forName("com.taosdata.jdbc.TSDBDriver"); 34: String jdbcUrl = "jdbc:TAOS://127.0.0.1:6030/? user=root&password=taosdata"; 35: Connection connection = DriverManager.getConnection(jdbcUrl); 36: try (Statement statement = connection.createStatement()) { 37: statement.executeUpdate("drop topic if exists " + TOPIC); </pre>	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 22
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: <pre> 20: 21: Connection connection = DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters"); 24: </pre>	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ ParameterBindingFullDemo.java	行号: 106
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段: <pre> 104: String sql = "INSERT INTO ? using stb tags(?,?,?,?,?,?) VALUES (?,?,?,?,?,?)"; 105: 106: TSDBPreparedStatement pstmt = conn.prepareStatement(sql).unwrap(TSDBPreparedStatement. class); 107: </pre>	

	108: for (int i = 1; i <= numOfSubTable; i++) {	
	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ConsumerDemo.java	行号: 46
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: <pre> 44: String tableName = "st"; 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650"); </pre>	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号: 24
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: <pre> 22: // ANCHOR: create_topic 23: Connection connection = DriverManager.getConnection(url, properties); 24: Statement statement = connection.createStatement(); 25: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters"); 26: // ANCHOR_END: create_topic </pre>	
	污染路径: 无	
14	缺陷发生位置: /docs/examples/JDBC/mybatisplus-demo/src/main/java/com/	行号: 156

	taosdata/example/mybatisplusdemo/provider/CachedBatchWriter.java	
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 154: String sql = ms.getBoundSql(sample).getSql(); 155: 156: Connection conn = dataSource.getConnection(); 157: PreparedStatement ps = conn.prepareStatement(sql); 158:	
	污染路径: 无	
15	缺陷发生位置: /docs/examples/JDBC/consumer-demo/src/main/java/com/taosdata/ ConsumerDemo.java	行号: 47
	步骤描述: Use try-with-resources or close this "Statement" in a "finally" clause.	
	代码片段: 45: String url = "jdbc:TAOS-WS://" + TAOS_HOST + ":" + TAOS_PORT + "/? user=root&password=taosdata&varcharAsString=true"; 46: Connection connection = DriverManager.getConnection(url); 47: Statement statement = connection.createStatement(); 48: statement.executeUpdate("create database if not exists " + dbName + " WAL_RETENTION_PERIOD 3650"); 49: statement.executeUpdate("use " + dbName);	
	污染路径: 无	
16	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ WsConsumerLoopImp.java	行号: 21
	步骤描述: Use try-with-resources or close this "Connection" in a "finally" clause.	
	代码片段: 19: properties.setProperty(TSDBDriver.PROPERTY_KEY_CHARSET, "UTF-8"); 20: 21: Connection connection =	

	DriverManager.getConnection(url, properties); 22: Statement statement = connection.createStatement(); 23: statement.executeUpdate("CREATE TOPIC IF NOT EXISTS topic_meters AS SELECT ts, current, voltage, phase, groupid, location FROM meters");	
	污染路径: 无	
17	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/QueryService.java	行号: 38
	步骤描述: Use try-with-resources or close this "ResultSet" in a "finally" clause.	
	代码片段: 36: try (Connection conn = dataSource.getConnection()); Statement stmt = conn.createStatement()) { 37: stmt.execute("use " + dbName); 38: ResultSet rs = stmt.executeQuery("show stables"); 39: while (rs.next()) { 40: String name = rs.getString("stable_name");	
	污染路径: 无	
18	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/GeometryDemo.java	行号: 89
	步骤描述: Use try-with-resources or close this "PreparedStatement" in a "finally" clause.	
	代码片段: 87: 88: private void stmtInsert() throws SQLException { 89: TSDBPreparedStatement preparedStatement = (TSDBPreparedStatement) connection.prepareStatement("insert into " + dbName + "." + tableName + " values (?, ?, ?, ?)"); 90: 91: long current = System.currentTimeMillis();	
	污染路径: 无	
19	缺陷发生位置: /docs/examples/JDBC/taosdemo/.mvn/wrapper/	行号: 112

29.2 缺陷/安全漏洞详解

29.2.1 严重类

29.2.1.1 WK1315 循环内字符串拼接

缺陷数量： 2 个		
缺陷描述： 在循环体中使用 “+”或 “+=”运算符拼接字符串。		
修复意见： 使用 StringBuffer 类完成字符串拼接后再转换为 String 类型字符串。		
1	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/utills/DataGenerator.java	行号： 75
	步骤描述： Use a StringBuilder instead.	
	代码片段： 73: String zh_en = ""; 74: for (int i = 0; i < length; i++) { 75: zh_en += alphabet.charAt(random.nextInt(alphabet.length())); 76: } 77: return zh_en;	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/utills/DataGenerator.java	行号： 87
	步骤描述： Use a StringBuilder instead.	
	代码片段： 85: for (int i = 0; i < length; i++) { 86: char c = (char) (random.nextInt(top - bottom + 1) + bottom); 87: zh_cn += new String(new char[]{c}); 88: } 89: return zh_cn;	

	污染路径: 无

29.2.1.2 WK1120 将可变对象返回给不受信任的调用者

缺陷数量： 13 个		
缺陷描述： 将未克隆的可变数据作为返回值发送可能会导致调用函数更改或删除该数据。 扩展说明： 在函数返回对可变数据的引用的情况下，调用该函数的外部代码可能会对发送的数据进行更改。如果以前未克隆此数据，则该类将使用可能违反其内部状态假设的已修改数据。		
修复意见： 1.将不应更改的返回数据声明为常量或不可变。2.在返回对可变数据的引用之前克隆所有可变数据。这是首选的缓解措施。这样，无论对数据进行哪些更改，都会保留一个有效副本以供类使用。		
1	缺陷发生位置： /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/WsConsumerLoopImp.java	行号： 59
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： 57: try { 58: consumerLoop.shutdown(); 59: } catch (InterruptedException e) { 60: throw new RuntimeException(e); 61: }	
	污染路径： 无	
2	缺陷发生位置： /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/highvolume/Util.java	行号： 94
	步骤描述： Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段： 92: } 93: 94: } catch (InterruptedException ExecutionException e) { 95: logger.error("Failed to delete/create topic: " + e.getMessage());	

	96: throw new RuntimeException(e);	
	污染路径: 无	
3	缺陷发生位置: /docs/examples/JDBC/highvolume/src/main/java/com/taos/example/ highvolume/StatTask.java	行号: 38
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 36: logger.info("numberOfTable={ } count={ } speed={ }", subTableNum, count, (count - lastCount) / 10); 37: lastCount = count; 38: } catch (InterruptedException e) { 39: logger.error("interrupted", e); 40: break;	
	污染路径: 无	
4	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/AbstractService.java	行号: 14
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 12: try { 13: count += future.get(); 14: } catch (InterruptedException e) { 15: e.printStackTrace(); 16: } catch (ExecutionException e) {	
	污染路径: 无	
5	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 111
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 109: try { 110: thread.join(); 111: } catch (InterruptedException e) { 112: e.printStackTrace();	

	113: }	
	污染路径: 无	
6	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 74
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 72: } 73: 74: } catch (SQLException InterruptedException e) { 75: e.printStackTrace(); 76: }	
	污染路径: 无	
7	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/SubTableService.java	行号: 120
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 118: try { 119: affectedRows += task.get(); 120: } catch (InterruptedException e) { 121: e.printStackTrace(); 122: } catch (ExecutionException e) {	
	污染路径: 无	
8	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/AbstractService.java	行号: 27
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 25: try { 26: count += future.get(); 27: } catch (InterruptedException e) { 28: e.printStackTrace(); 29: } catch (ExecutionException e) {	

	污染路径: 无	
9	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/TaosDemoApplication.java	行号: 43
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 41: try { 42: task.join(); 43: } catch (InterruptedException e) { 44: e.printStackTrace(); 45: }	
	污染路径: 无	
10	缺陷发生位置: /docs/examples/JDBC/JDBCDemo/src/main/java/com/taos/example/ConsumerLoopImp.java	行号: 62
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 60: try { 61: consumerLoop.shutdown(); 62: } catch (InterruptedException e) { 63: throw new RuntimeException(e); 64: }	
	污染路径: 无	
11	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/taosdemo/service/SubTableService.java	行号: 40
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 38: try { 39: executor.awaitTermination(Long.MAX_VALUE, TimeUnit.NANOSECONDS); 40: } catch (InterruptedException e) { 41: e.printStackTrace(); 42: }	

	污染路径: 无	
12	缺陷发生位置: /docs/examples/JDBC/taosdemo/src/main/java/com/taosdata/ taosdemo/service/QueryService.java	行号: 83
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 81: try { 82: thread.join(); 83: } catch (InterruptedException e) { 84: e.printStackTrace(); 85: }	
	污染路径: 无	
13	缺陷发生位置: /docs/examples/JDBC/connectionPools/src/main/java/com/taosdata/ example/ConnectionPoolDemo.java	行号: 105
	步骤描述: Either re-interrupt this method or rethrow the "InterruptedException".	
	代码片段: 103: if (sleep > 0) 104: TimeUnit.MILLISECONDS.sleep(sleep); 105: } catch (InterruptedException e) { 106: e.printStackTrace(); 107: }	
	污染路径: 无	

30 Python 通用缺陷

30.1 分项汇总表

序号	Python 通用缺陷	缺陷/安全漏洞名称	数量	缺陷级别
1	Python_1047	方法和字段名不应该只在大小写上有所不同	2	致命

30.2 缺陷/安全漏洞详解

30.2.1 致命类

30.2.1.1 Python_1047 方法和字段名不应该只在大小写上有所不同

缺陷数量： 2 个		
缺陷描述： 查看一个类中的一组方法和字段，并找到两个仅在大小写方面不同的方法和字段，这会让用户感到困惑，这种情况可能只是表明命名不当。 方法名称应该是面向动作的，因此包含一个动词，这在方法和字段名称相同（有或没有大小写差异）的情况下不太可能。但是，重命名公共方法可能会对调用者造成干扰，因此重命名成员是推荐的操作。		
修复意见： 暂无		
1	缺陷发生位置： /examples/python/PYTHONConnectorChecker/PythonChecker.py	行号： 18
	步骤描述： Rename field "dbname" to prevent any misunderstanding/clash with field "dbName"	

	defined on line 8	
	代码片段: 16: self.host=FQDN 17: if(dbname): 18: self.dbname=dbname 19: if(tdbname): 20: self.tbName	
	污染路径: 无	
2	缺陷发生位置:	行号:
	/test/new_test_framework/utils/log.py	47
	步骤描述: Rename method "format" to prevent any misunderstanding/clash with field "FORMAT" defined on line 32	
	代码片段: 45: } 46: 47: def format(self, record): 48: if hasattr(record, 'color') and record.color in self.COLOR_MAP: 49: color = self.COLOR_MAP[record.color]	
	污染路径: 无	